Technical Bulletins

창원대학교 데이타베이스 연구실

■ 제목차례 ■

▷ EXPORT
EXPORT/IMPORT Q & A ····· 1

▷ IMPORT
IMPORT시 TABLE과 INDEX를 분리하여 저장하는 방법 ····· 4

▷ LOADER
SAMPLE: SQL*LOADER SAMPLES PART I ····· 5
SAMPLE: SQL*LOADER SAMPLE USING TRUNC FUNCTION ····· 16
SAMPLE: SQL*LOADER SAMPLE PART II ····· 17

▷ ODBC
ODBC DRIVER를 이용하여 ORACLE DB에 접속하려는 경우 ····· 18
"ODBC DRIVER CALL FAILED"에러 메시지 ····· 20
VB,GLUE에서 STORED PROCEDURE 사용방법 ····· 21
ODBC & SQL*NET SUPPORT MATRIX ····· 22

▷ ORACLE GRAPHICS
UNIX 에서 사용되는 ORACLE 환경 변수 ····· 23

▷ RDBMS
(V7.3) PARTITION VIEW 테스트 REPORT ····· 31
(V7.3) PARTITION VIEWS IN RELEASE 7.3 ····· 45
(V7.3) STANDBY DATABASE IN ORACLE 7.3 ····· 47
ARCHIVE LOG MODE 운영방법 ····· 48
AUDITING 이란 무엇인가? ····· 50
BITMAP INDEX 생성과 SPACE 비교 ····· 52
BITMAPPED INDEX ····· 53
CONNET INTERNAL에 대해서 PASSWORD 설정하는 방법 ····· 58
CONTEXT SIZE & CURSORS(ORA-1040, 1050, 1051) ····· 59
CONTROLFILE 재생성하는 방법 ····· 60
CORE DUMP 에 대한 설명 ····· 62
CORRUPTED BLOCK을 가진 OBJECT 찾는법 ····· 65
COST BASED OPTIMIZER 와 RULE BASED OPTIMIZER ····· 66
DATA BLOCK CORRUPTION : ORA 1578 해결방법 ····· 69
DATABASE INTERNAL RECOVERY ····· 72
DATABASE LINK 사용 방법 ····· 74
DATAFILE을 RENAME 하는 방법 ····· 75
DBA 정보와 CONVERT SCRIPT ····· 77
DBMS PACKAGE : DBMS_ALERT을 이용하여 SESSION간 통신하는 방법 ····· 85
DBMS PACKAGE : DBMS_PIPE NOT DECLARED ERROR 발생시 ····· 90
DBMS PACKAGE : DBMS_PIPE사용(SESSION간의 MESSAGE SENDING) ····· 91
DB_FILE_MULTIBLOCK_READ_COUNT와 퍼포먼스의 관계 ····· 94
DB_NAME과 SID 변경하는 방법 ····· 95
DB를 COLD BACKUP(FULL BACKUP) 받는 방법 ····· 97
DDB 기본기능 및 분산 트랜잭션 문제의 해결 ····· 98
DBMS PACKAGE : DEBUGGING PL/SQL PROGRAMS USING DBMS_OUTPUT ····· 102
DISK STRIPING ····· 108
DOWN TIME WITH OPS ····· 109
DYNAMIC SQL(DBMS_SQL PACKAGE) ····· 110
ENTERPRIZE BACKUP UTILITY ····· 119
E7XP/IMP 를 이용하여 SCOTT의 데이타를 LARRY옮기는 방법 ····· 124
EXPLAIN PLAN 사용법 ····· 125
EXPLAIN PLAN 을 자동으로 수행하는 방법 ····· 126
EXPORT 받은 화일의 CHARCTER SET 변경 프로그램 ····· 128
EXPORT받은 DUMP파일의 CHARACTER SET 확인 방법 ····· 130
EXPORT실행시 ORA-1403 조치방법 ····· 131
EXTENT된 영역의 FREESPACE ····· 132
FOREIGN KEY CONSTRAINT 의 MASTER TABLE NAME의 확인 ····· 133
FREE LIST와 FREE SPACE관리 ····· 134

FULL TABLE SCAN을 보다 빠르게 하는 방법 ····· 136
HIERARCHICAL QUERIES ····· 137
HP-UX: ASYNCH I/O USING FS_ASYNC KERNEL PARAMETER ····· 138
INDEX 생성시 필요한 TEMP SEGMENT 크기 추정 ····· 139
INDEX_DESC HINT 사용법 ····· 140
INSTR 함수의 의미와 특정 CHARACTER를 지우는 방법 ····· 141
LAST ORA-1578 RECOVERY METHOD ····· 142
LOCK 문제 확인과 해결 방법 ····· 144
LOCKING CONFLICTS를 DETECTING/RESOLVING ····· 153
MESSAGE CONVERSION PROGRAM ····· 164
METHOD 4를 이용한 SAM FILE생성 PRORAM ····· 167
NETWARE LISTENER 활용(MAX USERS) ····· 174
NETWARE SUPPORT MATRIX ····· 175
NLS CHARACTERSET 변경방법 (DB REBUILD 없이) ····· 176
OPEN VMS 시스템에서의 ORACLE AUTO STARTUP ····· 178
OPS 에서 새로운 NODE 추가 방법 ····· 180
ORA-54 WHEN DROP A TABLE(SESSION KILL) ····· 181
데이타베이스 STARTUP 시 ORA-312,313 에러 ····· 182
ORA-600[3339] DATA BLOCK CORRUPTION DETECTION ····· 184
ORA-604 조치 방법 ····· 186
ORA-1118 조치 방법(MAXDATAFILES와 DB_FILES) ····· 187
ORA-1547 조치 방법 ····· 190
ORA-1552 조치 방법 ····· 192
ORA-1555 조치 방법 ····· 193
ORA-1562 처리 방법 ····· 195
ORA-1628,1630,1631,1630 조치 방법 ····· 196
ORA-1652 조치 방법 ····· 198
ORA-4031 조치 방법 ····· 199
ORACLE DATABASE HOME DIRECTORY 변경 ····· 200
ORACLE DATABASE의 AUTOMATIC STARTUP/SHUTDOWN ····· 202
Oracle Enterprise Manager Install Procedure ····· 207
ORACLE MEDIA SERVER ····· 208
ORACLE SERVER AND UNIX TRANSACTION PROCESSING(TP) MONITOR-1 ····· 211
ORACLE7 BACKUP 및 RECOVERY 방법 ····· 222
ORACLE7 INSTALL 후 데이타베이스 FILE의 구성 ····· 231
ORACLE7 에서 USER 등록 ····· 236
ORACLE의 MEMORY 사용량에 대한 테스트 ····· 237
OS 명령으로 DATA FILE 을 삭제한 경우 - ORA-1157, 1110 ····· 239
OUTER JOIN의 UIDE LINE ····· 241
PARALLEL LOAD - SPACE USAGE ····· 244
(V7.3) PARTITION VIEWS IN ORACLE7 RELEASE 7.3 ····· 245
PRIMARY KEY 재생성 방법 ····· 247
PRIMARY KEY를 REFERENCE 하는 FOREIGN KEY 찾는 SQL ····· 248
PROC 의 실행화일 크기를 줄이는 방법 ····· 249
ORACLE SERVER AND UNIX TRANSACTION ····· 250
RECOVERY 의 모든 것(DATABASE,TABLESPACE,DATAFILE) ····· 261
REDO LOGFILE SIZE의 변경 또는 추가 ····· 264
ROLLBACK SEGMENT 관리 ····· 267
ROLLBACK SEGMENT 크기를 증가하여 재생성 하는 방법 ····· 271
ROLLBACK SEGMENT의 상태 해결 ····· 273
ROW CHAINING 과 ROW MIGRATION ····· 275
ROWID에 대한 이해 ····· 276
IMPLEMENTING DATA REPLICATION ····· 281
SHARED MEMORY 문제 ····· 287
SHARED POOL SIZE의 계산 ····· 291
SNAPSHOT 생성방법 ····· 293
SNAPSHOT 을 자동적으로 REFRESH 하는 방법 ····· 295
SNAPSHOT의 기본 개념과 예제 ····· 296
TABLE SIZE 와 INDEX SIZE 계산 ····· 309
TABLE 크기 추정 방법 ····· 310
TABLESPACE 의 내용을 다른 TABLESPACE로 옮기는 방법 ····· 314
TABLESPACE 크기를 확장하는 방법 ····· 316
TAPE 으로 ARCHIVE 를 직접 받는 방법 ····· 317

TKPROF 사용시 CORE DUMP 조치 사항 ····· 322
TPC Q&A ····· 323
TRANSACTION ENTRY란 무엇인가? ····· 325
TRANSACTION을 ROLLBACK SEGMENT에 할당하는 방법 ····· 326
TRIGGER 에서 ERROR 확인(SET SERVER OUTPT ON 사용) ····· 327
TWO-PHASE COMMIT ····· 328
Using Oracle Database Manager 7.2 & 7.3 ····· 329
WINDOWS NT와 ORACLE7 TUNING GUIDE ····· 333
새로운 DATABASE 생성 ····· 339
인포믹스 STATIC PARTITIONING KNOCKOFFS ····· 341

▷ SQL FORMS

(V3.0) BASE TABLES FOR MENU 5.0 BASE TABLES ····· 342
(V3.0) FIELD를 이동할 때 자동으로 한글/영문 MODE를 변경하려면? ····· 344
(V3.0) FORMS3.0에서 USER EXIT이란? ····· 345
(V3.0) FORMS30에서의 LOCKING ····· 347
(V4.5,3.0) FRM-40735 ERROR에 관하여 ····· 349
(V4.5,5.X) SUPPORT FOR VBX AND OCX CONTROLS ····· 356
(V4.5.7 LATER) SQL*NET V2 SUPPORT ····· 358
(V4.5) CDE2 AND RDBMS ····· 359
(V4.5) FORMS4.5 NEW FEATURE ····· 361
(V4.5) CDE2 의 ON-LOCK 기능 ····· 366
(V4.5) DEVELOPER/2000 REGENERATE FORM ····· 367
(V4.5) FORMS4.5간의 PARAMETER PASSING ····· 368
(V4.5) FORMS4.5에서 REPORT2.5로의 DATA PARAMETER PASSING ····· 371
(V4.5) DDE(DYNAMIC DATA EXCHANGE) ····· 374
(V4.5) FORMS45와 TEXT FILE간의 INTERFACE ····· 383
(V4.5) DEVELOPER2000으로 작성한 MODULE을 DATABASE에 저장하려면? ····· 385
(V4.5) FORMS4.5 RUNTIME INSTALL 방법 ····· 386
(V4.5) FORMS4.5에서 FILE DIALOG 처리 방법 ····· 388
(V4.5) HOW TO IMPLEMENT MENU SECUTIRY IN FORMS4.5? ····· 389
(V4.5) DEV/2000에서 DOCUMENTATION 생성방법 ····· 391
(V4.5) EXITING MULTIPLE FORM APPLICATIONS ····· 393
(V3.0,4.5) PRE-QUERY TRIGGER 사용방법 ····· 394
(V4.5) ALERT을 이용하여 MESSAGE를 처리하는 방법 (ADVANCED) ····· 396
(V4.5) ALERT을 이용하여 MESSAGE를 처리하는 방법 (BASIC) ····· 401
(V3.0,4.5) USER가 아무 ACTION이 없이 일정 시간이 지나면 자동으로 LOGOUT시키는 방법 ····· 403
(V4.5) CUSTOMIZING LOGON BEHAVIOR ····· 404
(V4.5) BASE TABLES FOR DEVELOPER 2000 TOOLS ····· 411
(V4.5) APPLICATION을 DB에 저장하는 방법 ····· 420
(V4.5) F45에서 특정 FIELD에서 한/영키 자동 변환방법 ····· 421
(V4.5) FORMS4.5에서 GRAPHIC2.5로의 DATA PARAMETER PASSING ····· 423
(V4.5) FORMS4.5에서 SERVER쪽 APPLICATION을 실행하는 방법 ····· 426
(V3.0, MENU V5.0) FORM과 MENU의 연결 ····· 429
(V3.0) BUFFER AND ARRAY SIZE WITHIN SQL*FORMS 3.0 ····· 430
(V3.0) FORMS사용시 ORA-942 ERROR 처리 ····· 431
FORM에서의 2차원 TABLE집계 처리 ····· 432
(V4.5) HOW TO IMPLEMENT MENU SECUTIRY IN FORMS4.5? ····· 434
(V3.0) FORMS30 에서 LEFT,RIGHT KEY 를 사용하여 MENU DESIGN ····· 436
(V4.5) MENU UPGRADE ····· 438
(V4.5) MULTI FORM GENERATION (UPDATE) ····· 443
(V4.5) NEW FEATURES IN 4.5.7 (DEV/2000 R1.3) ····· 449
(V3.0) ORATERM 을 이용한 FORMS3.0 FUNCTION KEY 수정 ····· 454
(V3.0) OS의 환경변수를 받아들이는 방법 ····· 457
(V3.0) SQL*FORMS V3에서 SQL*REPORTWRITER로 PARAMETER전송하는 방법 ····· 458
(V3.0) FORMS30 및 STORED PROCEDURE 에서 ROLLBACK SEGMENT 사용 ····· 459
RUNNING SQL_TRACE FROM ORACLE TOOLS ····· 460
STORED PROCEDURE & DATABASE TRIGGER ····· 461
(V4.5) USING DATABASE ROLES WITH ORCLE FORMS 4.5 ····· 464
(V3.0) VT100에서 KEY-F0 ~ KEY-F9사용법 ····· 467
(V4.5) WINDOWS PERFORMANCE TUNING FOR FORMS4 ····· 468
(V4.5) ORACLE FORMS 4.5 에서 WINDOWS APPLICATION을 호출하는 방법 ····· 470

▷ SQL NET

PRO*C 2.0의 새로운 특징 ····· 472
AUTOMATIC STARTUP AND SHUTDOWN OF SQL*NET LISTENERS ····· 474
CLINET/SERVER 환경에서의 PERFORMANCE TUNING ····· 482
SAMPLE: DB에 IMAGE FILE을 저장 ····· 489
DIGITAL UNIX 4.0/4.0A 사용시 PRO*C PRECOMPILE 주의 사항 ····· 491
DOS, WINDOWS에서의 PRO*C와 C COMPILER와의 관계 ····· 492
DYNAMIC SQL METHOD1(CREATE TABLE) ····· 493
SAMPLE: DYNAMIC SQL METHOD2 SAMPLE(INSERT) ····· 495
SAMPLE: DYNAMIC SQL METHOD2 SAMPLE(UPDATE) ····· 497
SAMPLE: DYNAMIC SQL METHOD3 SAMPLE(SELECT) ····· 499
FUTURE TCP 와 SQL*NET V2 사용시 PERFORMANCE ····· 501
PRO*C 에서 HOLD_CURSOR,RELEASE CURSOR 의 의미 ····· 502
SQL-2101(ORA-2101): INCONSISTENT CURSOR CACHE (UCE/CUC MISMATCH) ····· 506
PRO*C ON MS-DOS AND MS-WINDOWS PLATFORMS ····· 508
ORA-1002 FETCH OUT OF SEQUENCE IN PRO*C ····· 510
ORA-1405 처리 방법 ····· 512
ORA-3113,3114 의 발생원인 및 진단방법 ····· 513
ORA-6401 ERROR 발생시 처리 방법 ····· 514
ORACLE의 TWO-TASK DRIVERS ····· 515
PC CLIENT TOOL 과 ORACLE DB 와의 INTERFACE 방법 ····· 517
PRECOMPILE에서 MODE OPTION으로 저장하는 방법 ····· 518
PROC 2.X 에서 INCLUDE FILE 사용방법 ····· 520
SAMPLE: PROC 에서 SESSION 을 KILL 하는 방법 ····· 521
PROC 에서 외부 환경변수를 받아 들이는 방법 ····· 522
PROC의 MAKE FILE을 이용하여 SHELL프로그램을 만드는 방법 ····· 524
SQL*NET SESSION의 DEAD CONNECTION 처리 방법 ····· 526
SQL*NET과 관련된 기본적인 점검사항 ····· 527
SQL*NET V1 FOR APPC/LU6.2 TEST ····· 530
SQL*NET V1, V2의 STATUS를 확인하는 방법 ····· 532
SQL*NET V1, V2의 차이점 ····· 535
SQL*NET V1사용시 ORA-6107 조치 방법 ····· 537
SQL*NET V2 SETUP 방법 ····· 539
SQL*NET V2 사용시 ORA-12154 조치 방법 ····· 542
SQL*NET V2 에서 BUFFER SIZE 지정 방법 ····· 546
SQL*NET V2에서 DEDICATED로 접속하는 방법 ····· 547
WINDOWS PERFORMANCE TUNING ····· 548
WINDOWS용 SQL*NET V1 사용할 때 ORA-6136 조치방법 ····· 550
WINDOWS용 SQL*NET V1,V2 사용시 ORA-9241, ORA-9301 조치방법 ····· 551
WINDOWS용 SQL*NET 사용시 ORA-6106 ····· 552
WINDOWS용 SQL*NET 사용시 ORA-6108 ····· 556
WINDOWS용 SQL*NET 사용시 ORA-6110 ····· 560
WINDOWS용 SQL*NET 사용시 ORA-6120 ····· 562
WINDOWS용 SQL*NET 사용시 ORA-6122 ····· 566
WINDOWS용 SQL*NET 사용시 TNS-12203 ····· 567
WINDOWS용 SQL*NET 사용시 TNS-12533 ····· 569
WINDOWS용 SQL*NET 사용시 TNS-12545 ····· 570
WINDOW용 SQL*NET 를 사용시 ORA-3121 ····· 571
SAMPLE: XA PROGRAM ····· 574

▷ SQL PLUS

NUMBER DATA TYPE ····· 582
DATE TYPE 데이타를 효과적으로 조회하는 방법 ····· 586
ORACLE7 OPTIMIZER 에 대하여 ····· 587
한글이 ??? 로 DISPLAY 되는 경우 ····· 589
SQLPLUS 기동시 WARNING MESSAGE 처리 ····· 591
SQL,SQL*PLUS 를 사용하여 SQL SCRIPT를 동적으로 생성,수행 ····· 592
SQL TIPS(1): SORT ····· 594
SQL TIPS(2): 특정조건의 순서대로 번호를 부여하는 방법 ····· 595
SQL TRACE FACILITY (TKPROF UTILITY 사용) ····· 597
LONG COLUMN을 가진 TABLE의 COPY ····· 600
SQL TIPS(3): DATE의 기간을 계산하는 방법 ····· 601
SQL TIPS(4): 중복된 RECORD 삭제 방법 ····· 602

외부에서 값을 받아서 SQLPLUS의 SCRIRT 실행하는 방법 ····· 603
SQL TIPS(5): DECODE이용사례 (1) : 월별로 집계하는 방법 ····· 604
SQL TIPS(6): 해당 COLUMN에 걸려 있는 CONSTRAINT확인 ····· 605
SQL TIPS(7): 한 CONSTRAINT에서 REFERENCE하는 COLUMN확인 ····· 606
SQL TIPS(8): PARENT-CHILD관계를 갖는 TABLE의 PRIMARY,FOREIGN KEY의 COLUMN명과 POSITION 확인 ····· 607
SQL TIPS(9): 해당 TABLE에 걸려있는 CONSTRAINT의 확인 ····· 609
SQL TIPS(10): CHECK CONSTRAINT의 조건확인 ····· 611
SQL TIPS(11) :TABLE에 대하여 EXTENT 발생량 산정방법 ····· 612
SQL TIPS(12): PRINTING A HIERARCHICAL LIST OF REFERRING AND REFERENCED ····· 613
SQL TIPS(13): DROPPING DATABASE OBJECTS FOR A USER ····· 622
SQL TIPS(14): PROVIDING USERS WITH SESSION INFORMATION ····· 624
SQL TIPS(15): FORECASTING ON NEXT EXTENTS ····· 627
SQL TIPS(16): CREATING SCRIPTS TO RECREATE A TABLE STRUCTURE ····· 629
SQL TIPS(17): AVAILABLE SPACE, FRAGMENTATION, AND EXTENTS ····· 640
SQL TIPS(18): DBMS_OUTPUT을 이용하여 SQL*PLUS에서 REPORT 작성하기 ····· 643
SQL TIPS(19): MAKE A MAP OF YOUR ORACLE FILES ····· 645
SQL TIPS(20): HIERARCHICAL DATA의 SORT와 ORDERING ····· 648
SQL TIPS(21): DBA가 가지고 있어야 할 10개의 SQL ····· 651
SQL TIPS(22): DETAILED LOOK AT DATABASE INDEX ····· 656
SQL TIPS(23): EXPORTING TABLES BASED ON TABLESPACE NAME ····· 660
SQL TIPS(24): CALCULATING HIT RATIO ····· 662
SQL TIPS(25): ANALYZING TABLESPACE USAGE WITH AN OS-INDEPENDENT SQL SCRIPT ····· 663
SQL TIPS(26): CREATING A DATABASE LAYOUT ····· 666
SQL TIPS(27): DECODE 이용사례 (2) ····· 668
SQL TIPS(28): DECODE 이용사례 (3) ····· 669

▷ SQL REPORT

(V1.1) REPORTWRITER 에서 LOCAL PRINTER로 출력하는 방법 ····· 670
(V1.1) 출력물을 현 단말기에 연결된 PRINTER로 출력하는 방법 ····· 671
(V1.1) PAGE HEIGHT가 66보다 작은 SPECIAL FORM지에 출력하는 방법 ····· 672
(V2.5) ASCII DRIVER로 WIDE FORMAT 출력 ····· 673
(V1.1) SQL*REPORTWRITER 1.1에서 프로그램의 BACK-UP 및 RESTORE 방법 ····· 674
(V1.1) SQL*REPORTWRITER 1.1에서의 TERMINAL FUNCTION KEY 수정방법 ····· 675
(V1.1) SQL*REPORTWRITER 1.1에서 PRINTER DRIVER DEFINITION 수정방법 ····· 677
(V1.1) 고정된 위치에 합계를 PRINT 하는 방법 ····· 680
(V1.1) PRINTER로 찍을때 확대축소하려면? ····· 681
(V2.5) ORACLE REPORT SERVER에 관하여 ····· 682
(V2.5) ORACLE REPORTS PERFORMANCE TIPS ····· 684
(V1.1) REPORT SOURCE LISTING GENERATION ····· 688
(V1.1) SQL*REPORTWRITER에서 SOURCE를 자동으로 BACKUP받는 방법 ····· 689
(V2.5) REPORT에서 자주 사용되는 SRW PACKAGED PROCEDURE ····· 691
(V2.5.X) 조건에 따라 HIGHLIGHTING, COLORING 처리하는 방법 ····· 695
(V2.5) ORACLE REPORTS에서의 FORMAT TRIGGER사용에 관하여 ····· 698
(V2.5) TABLE에 IMAGE FILE에 대한 PATH만 가지고 있는 경우의 처리 ····· 701
(V2.5) REPORT2.5에서 ORACLE GRAPHICS를 띄우려면? ····· 702

▷ UNIX

SOLARIS 2.X SETUP PROBLEM ····· 703
UNIX 에서 RAW DEVICE 사용하는 방법 ····· 705
ORACLE 과 UNIX SEMAPHORE 와의 관계 ····· 707
SUN SOLARIS C COMPILE 시 고려 사항 ····· 710
ORACLE DATABASE 의 AUTOMATIC STARTUP/SHUTDOWN ····· 711
ORACLE 에서 CRON 사용 ····· 719
UNIX 에서 ORACLE PERMISSION 문제 - ORA-1034, ORA-7320,7429 ····· 723
UNIX 에서 SHARED MEMORY REQUIREMENT ····· 726
UNIX 에서 SHARED MEMORY/SEMAPHORE 의 이해 ····· 730
RAID 와 ORACLE의 관계 소개 ····· 738
UNIX 에서 사용되는 ORACLE 환경 변수 ····· 740
CORE DUMP TRACING 방법 ····· 749
UNIX SVR4 에서 O/S의 REBUILD ····· 751
SYSTEM RESOURCE FOR TUXEDO ····· 754
SCRIPT FOR HOT BACKUP ····· 757

EXPORT/IMPORT Q & A

Bulletin no : 10296

[질문1] RDBMS와 Export, Import의 연관 관계는 (catexp.sql 이란) ?

Export, Import시 이미 생성된 오브젝트의 정보를 데이타 딕셔너리에서 쿼리를 하는데 이러한 오브젝트의 정보가 데이타 딕셔너리내의 여러 테이블에 나누어져 있다. 필요한 데이타 딕셔너리 정보를 편리하게 이용하기 위하여 여러 가지의 뷰를 catexp.sql에 Script되어 있다. 이 스크립트 화일은 $ORACLE_HOME/rdbms/admin에 있으며 Install시 수행되도록 되어 있다.

[질문2] Export시 오브젝트의 백업 순서는 있는가 ?

Export하는 오브젝트의 순서는 존재 며 이는 Oracle의 Version Up등에 의한 새로운 오브젝트가 만들어지거나 하면 Export되는 오브젝트의 순서는 변할 수 있다. 오브젝트의 Export순서는 다음과 같다.

1. Tablespaces
2. Profiles
3. Users
4. Roles
5. System Privilege Grants
6. Role Grants
7. Default Roles
8. Tablespace Quotas
9. Resource Costs
10. Rollback Segments
11. Database Links
12. Sequences(includes Grants)
13. Snapshots (includes grants, auditing)
14. Snapshot logs
15. Job Queues
16. Refresh Groups
17. Cluster Definitions
18. Tables(includes grants,column grants, comments,indexes,constraints,auditing)
19. Referential Integrity
20. POSTTABLES actions
21. Synonyms
22. Views
23. Stored Procedures
24. Triggers
25. Default and System Auditing

[질문3] Export BUFFER와 RECORDLENGTH는 무엇인가?

-BUFFER

Export시 오브젝트 내에 있는 여러 개의 Row가 한꺼번에 Fetch된다. 디스크에서 Fetch된 정보는 화일에 Write하기 전에 메모리를 거치게 되며, 이때 할당되는 메모리의 양이 Buffer 파라미터의 값이다.

-RECORDLENGTH

메모리에 있는 Export할 자료를 화일에 Write하기 위해 한번에 운반되는 양을 결정하는 파라미터이다.

[주의] 위의 BUFFER와 RECORDLENGTH는 O/S의 Block Size의 배수가 되도록하는 것이 효율적이다.

[질문4] 다량의 Row를 Export, Import시 어느 정도의 Row가 처리되었는지 알 수 있는가?

알 수 있다. V 7.1까지는 다량의 Row를 Export, Import시 처리된 정도를 알 수가 없어 현재 작업중인지 시스템이 Hang인지 파악되지 않았으나 V 7.2부터는 FEEDBACK이라는 옵션을 이용하여 체크가 가능하다.

[질문5] Export시 한번에 몇 개의 Row가 Fetch되는가?

한번에 Fetch되는 Row의 수는 Buffer Size와 연관 관계가 있다. 하나의 Row가 Export시 차지하는 양은 각 Column Size의 합 + 4 * (Column의 수)로 구할 수 있다. 한번 Fetch되는 Row의 수는 Buffer Size / 한 Row의 Export시 Size이다. 이를 이용하면 Export된 Output File의 Size는 대략 한 Row의 Export시 Size * Row 수 이다.

[질문6] Export, Import의 호환성은 어떻게 되는가?

Export, Import의 호환성은 Oracle의 버젼과 직접적인 연관관계를 갖고 있다. 호환성은 4가지로 나누어 설명할 수 있으며 이를 아래의 가정을 이용해 설명하겠다. 가령 A라는 기계에 Oracle V 7.0, B라는 기계에 Oracle V 7.1이 설치되어 운영 중이라 가정하자. Oracle V7.0을 X라 하고 Oracle V7.1을 Y라고 하자.

- **Base Compatibility** : X의 exp를 이용해 X DB를 export하여 X의 imp를 이용해 X DB에 import하는 것을 말한다. 이는 당연히 지원한다.
- **Upward Compatibility** : X의 exp를 이용해 X DB를 export하여 Y DB에 Y의 imp를 이용해 import하는 것을 말한다. 이도 Oracle에서는 지원한다.
- **Downward Compatibility** : Y exp를 이용해 Y DB를 export하여 X DB에 X의 imp로 import하는 것을 말한다. 이는 지원될 수도 안될 수도 있다.
- **Cross Compatibility** : X exp를 이용해 Y DB를 export(SQL*Net 이용)하여 X 또는 Y DB에 import(imp는 적정한 것을 활용)하는 것을 말한다. 이는 지원이 안될 수도 있다.

[질문7] 어떤 경우에 Downward Compatibility가 실패하는가?

V7.2에 hash cluster expressions라는 옵션이 있는데, 이를 이용해서 클러스터를 생성하여 사용후 export한 것을 V7.0 또는 V7.1로 downward시 create cluster문에 옵션이 맞지않아 실패하게 된다.

[질문8] EXP-37 에러(export views not compatible with database version)발생의 원인은 무엇인가 ?

이 에러는 Cross Compatibility에서 발생하는 문제로 이는 Export가 이용하는 View(Catexp.sql에 의해 생성된)가 Oracle Version내 일치하지 않아 발생한 문제로 이를 해결하기 위해 Exp에서 이용가능한 View를 설치한다.

[질문9] Full Export는 DBA 권한을 갖고 있는 유저만 할 수 있는가 ?

Version 6 에서는 DBA권한을 갖고 있는 유저만 Full Export를 할 수 있으며, V7에서는 DBA가 아니더라도 EXP_FULL_DATABASE Role이 Grant되면 Full Export가 가능하다.

[질문10] 테이블 Import 시에 디폴트 테이블 스페이스가 아닌 곳으로 들어가는 경우는 왜 발생하는가?

예를 들어서 scott 유저의 디폴트 테이블 스페이스가 users 인데 임포트를 해보면 tools 테이블 스페이스에 테이블이 만들어졌다고 하자. 그 이유는 다음과 같다. 즉, 임포트 하는 테이블이 원래 tools 테이블 스페이스에 있었고 scott가 현재 tools 테이블스페이스에 대한 Quota 를 가지고 있거나 아니면 Unlimited Tablespace 권한(Resource Role에 포)을 부여 받았기 때문이다.

Import시에 테이블을 디폴트 테이블 스페이스에 만들려면 디폴트 테이블스페이스 외의 테이블 스페이스에 대한 모든 Quota 를 0 로 만들고 Unlimited Tablespace 권한을 Revoke 시킨 다음에 임포트를 수행해야 한다. 그리고,디폴트 테이블 스페이스에 대한 Quota만 Unlimited 로 한다.
예를 들면 다음과 같다.

```
$ sqlplus system/manager

SQL>  alter user scott
        quota 0 on system
        quota 0 on tools
        .......
        quota 0 on data
        quota unlimited on users;

SQL>revoke unlimited tablespace from scott;
```

이렇게 한 다음 Import 를 수행하면 된다. 물론 유저를 만들 때 quota를 주지 않은 테이블 스페이스는 상관 없으며 Unlimited Tablespace 권한(또는 Resource Role) 을 주지 않았다면 Revoke 명령도 사용할 필요가 없다.

[질문11] Import 시에 Core Dump/Segmentation Fault 가 발생하는 경우

오라클에는 Character Set이 있다. 국내에서는 US7ASCII 또는 KO16KSC5601을 주로 사용하는데 Export 받은 곳과 Import 하는 곳의 Character Set이 다르면 Import시에 Core Dump 가 발생하거나 원인 불명의 에러가 발생하면서 임포트가 중단되는 경우가 발생한다.
이 경우에는 Export 받은 dump file 을 convert 프로그램을 이용하여 Import하는 곳의 Character Set 으로 변환 시킨 다음 임포트를 하는 방법이 있고,아니면 어느 한 쪽 DB 의 Character Set 자체를 바꿔서 동일하게 맞춘 다음 Export/Import 하는 방법이 있다. 이중에서 Convert 프로그램을 이용하는 방법이 간단한데 이 프로그램은 Unix상에서 cc로 컴파일 하여서 사용하면 된다.프로그램 소스는 천리안에서 go oracle 하여 정보 보 창고에서 구할 수 있다.

IMPORT시 TABLE과 INDEX를 분리하여 저장하는 방법

Bulletin no : 10338

Export/Import를 이용하여 Data를 옮기거나 Space를 정리할 때 Table과 Index를 각각 다른 Tablespace에 분리하여 저장하기도 한다. 이러한 방법은 Oracle의 속도를 향상시키는 데 많은 도움을 준다.

Import할 때 Table과 Index를 생성시켰던 Script를 가지고 있 면 편리하게 이 작업을 할 수 있다.
먼저 Import할 때 Indexes=N option을 사용하여 Index를 제외한 나머지 Object를 Import하고, Index는 Script에서 Tablespace를 지정한 후 SQL file을 실행시켜서 생성하면 된다. 그러나 Index Script를 가지고 있지 않을 때는 문 가 복잡해진다.

그러한 경우에 Import Option중에서 Indexfile Option을 사용하여 Index script를 만들어 낼 수가 있다. 이 방법은 Indexfile만 만들기 때문에 속도가 빠르고, Index 뿐만이 아니라 Table Script도 만들어 지기 때문에 결과적으로 Table Script도 활용할 수 있다. 다음과 같이 만들 수 있다.
다음 작업은 Scott User의 모든 Data를 Test User로 옮기기 위해서 Export/Import를 이용하려고 한다. Import시에 Table과 Index는 Tablespace를 다르게하여 저장한다.

1. 다음과 같이 Scott User를 Export 받는다.

```
$ exp scott/tiger file=scott.dmp
```

2. Export받은 File에서 Index를 제외한 나머지만 Import한다.

```
$ imp  system/manager  file=scott.dmp  fromuser=scott  touser=test indexes=n  commit=y
```

위와 같이 실행하면 Test User에는 Index를 제외한 나머지 Object만 Import된다.

3. 그리고 다음과 같이 Indexfile Option을 이용하여 Index Script를 만든다.

```
$ imp  system/manager  file=scott.dmp indexfile=index.sql
```

위와 같이 명령을 실행하면 실제로 Data는 Import되지 않고, index.sql이라는 File만 만들어진다. 그리고 화일을 VI 등으로 열어서 확인해 보면 create table 문장과 create index문장이 있고 Create Table 문장은 REM으로 막혀 있어서 결과적으로 Create Index문만 실행할 수 있도록 되어 있다.
이 화일에서 Create Index문의 Tablespace만 바꾸어서 다음과 같이 SQL*Plus에서 실행시킨다.

```
$  sqlplus test/test
SQL>  @index
```

이렇게 하여 Table과 Index를 다른 Tablespace로 분리하여 저장할 수 있다.
이 Indexfile option은 Index만이 아니라 Table 생성문장이 필요할 때도 유용하게 사용하여 활용할 수 있다.

SAMPLE: SQL*LOADER SAMPLES PART I

Bulletin no : 10194

---

1. SQL*LOADER SAMPLES

SQL*Loader는 외부 화일의 데이타를 ORACLE 데이타베이스의 테이블에 넣기 위한 유틸리티입니다. SQL*Loader를 사용하려면 외부 데이타 화일과 콘트롤화일이 필요합니다. 콘트롤화일이라고 하는 것은 로드하는 데이타의 정보를 저장한 화일입니다. 간단한 샘플 콘트롤화일을 설명하겠습니다.

```
load data                 제어화일의 선두에는 반드시 이 말이 필요합니다.
infile sample.dat         외부화일을 지정합니다.
replace                   테이블에 데이타 넣는 방법 지정
into table TABLE_NAME     데이타를 로드하는 테이블을 지정
field terminated by ',' 데이타 필드의 종결문자 지정
(a integer external,      테이블의 열, 외부데이타 화일의 데이타 형을 지정
 b char)
```

참고로 replace 외에 다음의 옵션이 가능합니다.

```
replace                   테이블의 기존 행을 모두 삭제(delete)하고 인서트
append                    새로운 행을 기존의 데이타에 추가
insert                    비어 있는 테이블에 넣을 때
truncate                  테이블의 기존 데이타를 모두 truncate 하고 인서트
```

SQL*Loader를 실행하면 아래의 화일이 작성됩니다.

* 로드작업중 동작에 관한 통계 등을 포함한 로그화일(확장자는 log)
* 데이타 에러 때문에 로드가 안된 레코드를 저장한 화일(확장자는 bad)
* 사용자의 선택기준에 적합하지 않은 레코드를 저장한 화일(discard 화일)
  이것은 discardfile 옵션으로 별도로 지정해야 생성됩니다.

실행방법은 다음과 같습니다.

```
$sqlload scott/tiger control=sample.ctl data=sample.dat
```

1.1 임의의 열에 변화없는 고정 문자열(값)을 입력한 경우

***** 테이블 구조 *****

```
create table cons_test
(a number,
 b number,
 c number,
 d varchar(10))
```

***** 콘트롤 화일 *****

```
load data
infile cons.dat
replace
into table cons_test
fields terminated by ','
(a integer external, b integer external,
 c CONSTANT '100',   d char)
```

***** 외부 데이타 화일 *****

```
1,2,DATA
2,4,DATA2
```

***** 검색결과 *****

```
SQL>select * from cons_test;

      A       B       C D
---------------------------------
      1       2     100 DATA
      2       4     100 DATA2
```

[주의] 이 예에서 C열은 데이타 화일에 넣어서는 안됩니다. COSNTANT는 그것으로 완결된 열 지정의 하나가 됩니다. integer external 데이타형은 수치데이타를 문자형식(ASCII CODE)로 나타낸 것입니다.

1.2 로드한 때의 날짜를 데이타로 로드하고 싶은 경우

***** 테이블 구조 *****

```
create table sysdatetb
(a number,
 b date,
 c varchar(10))
```

***** 콘트롤 화일 *****

```
load data
infile sysdate2.dat
replace
into table sysdatetb
fields terminated by ','
(a integer external,
 b sysdate,  c char(10))
```

***** 외부 데이타 화일 *****

```
111,STRINGS
222,STRINGS2
```

***** 검색결과 *****

```
SQL>select * from sysdatetb;

      A      B       C
-----------------------------
     111 13-MAY-94 STRING
     222 13-MAY=94 STRING2
```

[주의] 이 예에서는 B열은 데이타 화일에 넣어서 안됩니다. SYSDATE는 그것으로 완결된 열지정의 하나가 됩니다. 새로운 시스템 날짜매김은 컨벤셔널 패스에서는 실행시에 삽입된 각각의 레코드 배열마다 또, 다이렉트 패스의 경우는 로드된 각각의 레코드의 블럭마다 사용됩니다.

1.3 임의의 수치열에 연속번호를 붙이고 싶은 경우

***** 테이블 구조 *****

```
create table rectb
(a varchar(10),
 b number, c varchar(10))
```

```
**** 콘트롤 화일 ****
load data
infile rec.dat
replace
into table rectb
fields terminated by ','
(a char,
 b recnum,
 c char)
```

```
**** 외부 데이타 화일 ****
A,a
B,b
C,c

**** 검색결과 ****
SQL>select * from rectb;

A       B C
--------------------
A       1 a
B       2 b
C       3 c
```

[주의] 이 예에서는 B열은 데이타 화일에 넣어서는 안됩니다. RECNUM는 그것으로 완전한 열지정의 하나가 됩니다. 연속번호는 1부터 차례대로 1씩 더해져서 번호가 매겨집니다. 가산된 번호를 둘씩 건너뛰거나 하는 것은 불가능 합니다.

1.4 임의의 수치열에 연속번호를 임의의 간격으로 붙이고 싶은 경우

```
**** 테이블 구조 ****
create table seqtb
(a varchar(10),
 b number,
 c varchar(10))
```

```
**** 외부 데이타 화일 ****
1,a
2,b
3,c
```

```
**** 콘트롤 화일 ****
load data
infile seq.dat
replace
into table seqtb
fields terminated by ','
(a char,
 b sequence(100,5),
 c char)
```

```
**** 검색결과 ****
SQL>select * from seqtb;

A         B C
-------------------------
1       100 a
2       105 b
3       110 c
```

[주의] 이 예에서는 B열은 데이타 화일에 넣어서는 안됩니다. SEQUENCE는 그것으로 완결된 열지정의 하나가 됩니다. 초기 값 100과 늘인 값 5는 다른 수치로 변경 가능합니다.

1.5 로드하는 논리 레코드를 구성 는 물리 레코드가 복수열로 구성된 경우

(물리 레코드의 1바이트째로 판단되는 경우)

```
**** 테이블 구조 ****
create table conti_test
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

***** 콘트롤 화일 *****

```
load data
infile conti.dat
replace
continueif this
(1) = '%'
into table conti_test
fields terminated by ','
(a char,
 b char,
 c char)
```

***** 외부 데이타 화일 *****

```
%1,
%2,
3
%A,B
,C
%a,b
%c
%d
,ef
```

***** 검색결과 *****

```
SQL>select * from conti_test;

A       B       C
----------------------
1       2       3
A       B       C
a       bcd     ef
```

[주의] 이 예의 경우 1바이트 째가 계속 행의 체크를 위해서 사용되기 때문에, 실 데이타를 1 바이트 째부터 시작해서는 안됩니다. 상기 예의 경우, 레코드의 선두 바이트가 '%'일 때 다음의 레코드가 연결됩니다.

1.6 외부데이타 화 의 물리 레코드가 복수 레코드로 구성된 경우

(구성하는 물리 레코드 수가 모두 일정한 경우)

****** 테이블 구조 ******

```
create table con_test
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

****** 외부 데이타 화일 ******

```
1,2,
3
a,b,
c
A,
B,C
```

****** 검색결과 ******

```
SQl) select * from con_test;
A       B       C
-------------------------
1       2       3
a       b       c
A       B       C
```

****** 콘트롤 화일 ******

```
load data
infile conti.dat
replace
--일례로 모든 논리레코드가 그 레코드로 구성됩니다.
concatenate 2
into table con_test
fields terminated by ','
(a char,
 b char,
 c char)
```

[주의] 하나의 논리레코드가 모두 일정한 갯수의 물리 레코드로부터 성립되는 것 같은 단순한 경우에 한합니다.

1.7 데이타의 잘린 문자를 데이타로 로드하고 싶은 경우

***** 테이블 구조 *****
```
create table enc
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

***** 콘트롤 화일 *****
```
load data
infile enc.dat
replace
into table enc
fields terminated by ',' optionally enclosed by ' " ' and ' " '
(a char,
 b char,
 c char)
```

***** 외부 데이타 화일 *****
```
"abc,d",2,3
"a,,d",4,5
```

***** 검색결과 *****
```
SQL>select * from enc;
A       B       C
-------------------------
abc,d   2       3
a,,d    4       5
```

1.8 포지션 지정시 char형 데이타 전후의 블랭크도 로드하고 싶은 경우

***** 테이블 구조 *****
```
create table pretb
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

***** 콘트롤 화일 *****
```
load data
infile pre.dat
replace
preserve blanks
into table pretb
(a position(01:05) char,
 b position(06:10) char,
 c position(11:20) char)
```

***** 외부 데이타 화일 *****
```
12 4 67890 ab def hi
 2   67890 ab def hi
```

**** 검색결과 ****
```
SQL>select * from pretb;
 A       B       C
-------------------------
12 4    67890   ab def hi
 2      67890   ab def hi
```

**** 결과확인 ****
```
SQL>select length(a), length(c) from pretb;
 LENGTH(A) LENGTH(C)
 ------------------
      5        10
      5        10
```

1.9 어떤 데이타열의 데이타 유무와 상관없이 데이타가 없는 경우 NULL 데이타를 넣도록 하고 싶다

```
**** 테이블 구조 ****
create table tratb
(a varchar(10),
 b varchar(10),
 c varchar(10))

**** 콘트롤 화일 ****
load data
infile tra.dat
replace
into table tratb
fields termintated by ','
trailing nullcols
(a char,
 b char,
 c char)
```

```
 **** 외부 데이타 화일 ****
1,aa,
2,bb,FF
3,cc,

**** 검색결과 ****
SQL>select * from tratbl

A       B       C
------------------------
1       aa
2       bb      FF
3       cc
```

[주의] trailing nullcols를 사용하지 않으면 1 레코드째와 3 레코드째가 데이타 에러가 됩니다. 데이타가 들어있기도 하고 없기도 한 열의 데이타는 데이타 화일의 최후로 가져갑니다.

1.10 CHAR형 필드가 BLANK로 채워져 있을 때 NULL을 삽입하고 싶은 경우

```
**** 테이블 구조 ****
create table nulltb
(a varchar(10),
 b varchar(10),
 c varchar(10))

**** 콘트롤 화일 ****
load data
infile null.dat
replace
into table nulltb
fields terminated by ','
(a char,
 b char,
 c char(10) nullif c = blanks)
```

```
**** 외부 데이타 화일 ****
aa,bb, ,
11,22, ,
99,88,AA
00,00,BB

**** 검색결과 ****
SQL>select * from nulltb;

A       B       C
-------------------------
aa      bb
11      22
99      88      AA
00      00      BB
```

[주의] 검색결과의 1 행째, 2 행째의 열 C는 블랭크가 아니라 NULL 입니다.

1.11 DATE필드가 BLANK로 채워져 있을 때 NULL을 삽입하고 싶은 경우

***** 테이블 구조 *****

```
create table nulltb2
(a varchar(10),
 b varchar(10),
 c date)
```

***** 외부 데이타 화일 *****

```
aa,bb, ,
11,22, ,
99,88,92/11/11,
00,00,94/12/12,
```

***** 콘트롤 화일 *****

```
load data
infile null2.dat
replace
into table nulltb2
fields terminated by ','
(a char,
 b char,
 c date "YY/MM/DD" nullif c = blanks)
```

***** 검색결과 *****

```
SQL>select * from nulltb2;

A       B       C
-------------------------
aa      bb
11      22
99      88      92/11/11
00      00      94/12/12
```

[주의] 검색결과의 1 행째, 2 행째의 열 C는 블랭크가 아니라 NULL 입니다.

1.12 POSITION 지정시 BLANK를 그대로 로드하고 싶은 경우

***** 테이블 구조 *****

```
cr eate table nulltb2
(a varchar(10),
 b varchar(10),
 c date)
```

***** 외부 데이타 화일 *****

```
  998892/11/11
  94/12/12
```

***** 콘트롤 화일 *****

```
--- position 지정으로 블랭크를 그대로 입력 원하는 경우
--- preserve blanks를 지정한다.
load data
infile null3.dat
replace
preserve blanks
into table nulltb2
(a position(1:2) char,
 b position(3:4) char nullif b = blanks,
 c position(5:13) date "YY/MM/DD")
```

***** 검색결과 *****

```
SQL>select * from nulltb2;

A       B       C
-------------------------
99      88      92/11/11
                94/12/12
```

```
SQL>select length(a), length(b) from nulltb2;

LENGTH(A) LENGTH(B)
------------------
        2         2
        2
```

[주의] 이 경우 2 레코드째는 A에 블랭크가 들어가고 B에 NULL이 들어갑니다.

1.13 데이타 화일의 수치 데이타 열에 BLANK가 들어가 있을 때 0을 입력하고 싶은 경우

***** 테이블 구조 *****
```
create table def2
(a varchar(10),
 b varchar(10),
 c number)
```

***** 외부 데이타 화일 ****
```
11,11,123
22,22, ,
33,33, ,
44,44, ,
```

***** 콘트롤 화일 *****
```
load data
infile def2.dat
replace
into table def2
fields terminated by ','
(a char,
 b char,
 c integer external defaultif c = blanks)
```

***** 검색결과 *****
```
SQL>select * from deft;

A       B           C
-----------------------
 11      11        123
 22      22          0
 33      33          0
 44      44          0
```

1.14 데이타 NULL일 때 NULL이라고 하는 문자열을 넣고 싶은 경우

**** 테이블 구조 ****
```
create table ifnulltb
(a varchar(10),
 b varchar(10),
 c varchar(10))
```

**** 콘트롤 화일 ****
```
load data
infile ifnull.dat
replace
into table ifnulltb
fields terminated by ','
(a char,
 b char "nvl(:b,'NULL')",
 c char)
```

**** 외부 데이타 화일 ****
```
1,2,3,
A,,B
a,b,c
```

**** 검색결과 ****
```
SQL>select * from ifnulltb;

A       B       C
------------------------
1       2       3
A       NULL    B
a       b       c
```

[주의] NVL과 같은 SQL 함수는 DIRECT LOAD의 경우 SQL 인터페이스를 경유하지 않기 때문에 사용할 수 없습니다.

1.15 어떤 열을 모두 대문자(소문자)로 변환하여 로드하고 싶은 경우

***** 테이블 구조 *****
```
create table uptb
(a varchar(10),
 b varchar(10))
```

***** 외부 데이타 화일 *****
```
aBcDeFg,AbCdEf
ccDD11,ffGG22
```

***** 콘트롤 화일 *****

```
load data
infile upper.dat
replace
into table uptb
fields terminated by ','
(a char "lower(:a)",
 b char "upper(:b)")
```

***** 검색결과 *****

```
SQL>select * from uptb;

  A          B
--------------------
 abcdefg    ABCDEF
 cdd11      FFGG22
```

1.16 ZONE형 데이타를 로드하고 싶은 경우

ZONE의 데이타 형식은 1문자 1바이트로 나타납니다.

숫자 +123이라면

bit	1234	5678	1234	5678	1234	5678
	1111	0001	1111	0010	1100	0011
10진		1		2	+	3

최후의 8비트의 상위 4비트(부호비트)로 정부가 지정됩니다.

1100(정) 1101(부)

가 됩니다.

원래는 EBCDIC코드에서

Hex F0 to F9 → +0 to +9
Hex C0 to C9 → +0 to +9(부호비트)
Hex D0 to D9 → +0 to -9(부호비트)

이것을 아스키로 변환했기 때문에

```
부호비트        {ABCDEFGHI}JKLMNOPQR
-----------------------------------
부호            ++++++++++----------
수              01234567890123456789
```

가 됩니다.

****** 테이블 구조 ******

```
create table z
(a varchar(10),
 i number,
 j number)
```

****** 외부 데이타 화일 ******

```
A12345{12345}
```

```
**** 콘트롤 화일 ****
load data
infile filename.dat
replace
into table z
(a position (01) char,
 i position(02:07) zoned(6),
 j position(08:13) zoned(6))
```

```
**** 검색결과 ****
 SQL>select * from z;

A           I        J
----------------------
A       123450  -123450
```

1.17 decimal형 데이타를 로드하고 싶은 경우

decimal 데이타는 1바이트에 2개의 숫자가 지정됩니다.

숫자 +123 이라면

```
bit    1234 5678 1234 5678
       0001 0010 0011 1100
10진      1    2    3    +
```

끝의 4비트는 부호비트로 불리고

```
    1100(정)        1101(부)
```

가 됩니다.

```
**** 테이블 구조 ****
create table dec
(a varchar(10),
 col1 number,
 col2 number)
```

```
**** 콘트롤 화일 ****
load data
infile filename.dat
replace
into table dec
(a position (01:01) char,
 col1 position(02:05) decimal(7,4),
 col2 position(06:09) decimal(7,4))
```

```
**** 외부 데이타 화일 ****
```
데이타화일은 바이너리이기 때문에 more 등으로 볼 수 없습니다.

```
4112 3456 7c12 3456 7d0a
```

```
**** 검색결과 ****
SQL>select * from dec;

A              COL1        COL2
--------------------------------
A          123.4567   -123.4567
```

[주의]

콘트롤화일에서 decimal형을 사용하는 것에 따라 (length, scale)을 지정합니다.

1.18 고정길이 레코드 데이타를 로드하고 싶은 경우

```
**** 테이블 구조 ****
create table fixtb
(a varchar(10),
 b varchar(10))

**** 콘트롤 화일 ****
load data
infile fix.dat
"FIX 6"
replace
into table fixtb
(a char(3),
 b char(3))
```

```
**** 외부 데이타 화일 ****
123456abcdef

**** 외부 데이타 화일(16진) ****
3132 3334 3536 6162 6364 6566

**** 검색결과 ****
SQL>select * from fixtb;

A      B
---------------
123    456
abc    def
```

[주의] 데이타 화일의 끝에는 0x0a(CARRIAGE RETURN)가 들어가면 안됩니다. 0x0a도 1문자로 카운트 되기 때문입니다.

SAMPLE: SQL*LOADER SAMPLE
USING TRUNC FUNCTION

Bulletin no : 10197

TRUNC FUNCTION 을 사용한 LOADER 예

```
---------------------------------
<< TABLE DESCRIPTION >>

 Name                           Null?    Type
 ------------------------------ -------- ----
 EMPNO                          NOT NULL NUMBER(4)
 ENAME                                   VARCHAR2(10)
 JOB                                     VARCHAR2(9)
 MGR                                     NUMBER(4)
 HIREDATE                                DATE
 SAL                                     NUMBER(7,1)   <---
 COMM                                    NUMBER(7,2)
 DEPTNO                                  NUMBER(2)

<< CONTROL FILE >>

LOAD DATA
INFILE 't.dat'
INTO TABLE EMP2

( EMPNO    POSITION(01:04) INTEGER EXTERNAL,
  ENAME    POSITION(06:15) CHAR,
  JOB      POSITION(17:25) CHAR,
  MGR      POSITION(27:30) INTEGER EXTERNAL,
  SAL      POSITION(32:39) DECIMAL EXTERNAL "TRUNC(:SAL,1)",  <---
  COMM     POSITION(41:48) DECIMAL EXTERNAL,
  DEPTNO   POSITION(50:51) INTEGER EXTERNAL)

<< DATA FILE - 't.dat' >>
                              ******
7934 MILLER     CLERK     7782  920.00          10
7566 JONES      MANAGER   7839  3123.75         20
7499 ALLEN      SALESMAN  7698  1600.00   300.00 30
7654 MARTIN     SALESMAN  7698  1312.50  1400.00 30
7658 CHAN       ANALYST   7566  3450.00         20

<< LODER 실행 >>

$ sqlldr scott/tiger t.ctl
```

SAMPLE: SQL*LOADER SAMPLE PART II

Bulletin no : 10222

NUMBER 와 DATE TYPE 데이타 LOAD

---

1. DATA FILE

```
7566 31237 961010 20
```

2. CONTROL FILE

```
LOAD DATA
INFILE 'ul2.dat'
INTO TABLE EMP
append
( EMPNO    POSITION(01:04) INTEGER EXTERNAL,
  SAL      POSITION(06:10) DECIMAL EXTERNAL ":SAL/100",
  HIREDATE POSITION(12:17) DATE "YYMMDD",
  DEPTNO   POSITION(19:20) INTEGER EXTERNAL)
```

3. LOADRER 실행

```
$ sqlldr scott/tiger control=ul2.ctl
```

4. TABLE 구조

```
 Name                           Null?    Type
 ------------------------------ -------- ----
 EMPNO                          NOT NULL NUMBER(4)
 SAL                                     NUMBER(7,2)
 HIREDATE                                DATE
 DEPTNO                                  NUMBER(2)
```

5. 실행결과

```
     EMPNO        SAL HIREDATE               DEPTNO
---------- ---------- ------------------ ----------
      7566     312.37 10-OCT-96                  20
```

ODBC DRIVER를 이용하여 ORACLE DB에 접속하려는 경우

Bulletin no : 10130

---

* Open Connectivity

Oracle7 Workgroup Server는 사용자의 MS Windows응용프로그램과 데이타베이스사이에 Open Connectivity를 제공한다. Open Connectivity는 사용자가 데이타베이스에 접근할 때 가장 최적의 방법으로 접근할 수 있는 기술을 선택하기 위한 방법이다.

ODBC를 사용하기 위하여 다음과 같은 단계로 설정을 한다

[단계 1] ORACLE7 ODBC 프로그램 그룹을 연다.
[단계 2] ORACLE7 ODBC 윈도우가 나타난다.
(Oracle7 ODBC 를 설치하지 않았다면 기본프로그램 그룹에 있는 제어판을 열면 ODBC Administrator 이라는 ICON이 있다.)
[단계 3] ODBC Administrator ICON을 Double-Click하면 다음과 같이 Data Source 윈도우가 나타난다. 만일 Data Source에 새로운 것을 추가 하려면 Add 버튼을 클릭한다.
[단계 4] 현재 사용자의 컴퓨터에 설치되어 있는 ODBC Driver가 나타나고 사용자는 이용하고 싶은 ODBC Driver를 선택한후 OK 버튼을 누른다.
[단계 5] Oracle7 ODBC Setup 또는 ODBC Oracle Setup 다이얼로그 박스가 나타나며 그림 밑에 있는 [Section]은 ODBC Data Source 설치가 끝난 후 windows 디렉토리의 ODBC.INI에 저 되는 내용을 보여주고 있다.

```
[ORACLE_ODBC]
Driver=C:\WINDOWS\SYSTEM\sqora71.dll
Description=ORACLE 7 ODBC DRIVER TEST
Server=T:krhp2:RC
```

또는

```
[Oracle7]
Driver=C:\WINDOWS\SYSTEM\sqora.dll
Description=Testing YC&C
Server=T:krncr1:PRE
UserId=scott
```

. Data Source Name : Data Source에서 참조할때 사용하기 위한 이름을 기술한다.
. Description : Data Source의 설명을 기술한다. (Option)
. SQL*NET Connect String (Network Address)
  - SQL*Net Connect String을 기술한다.
      SQL*NET V1 - ABBREV: Host name:Oracle_SID
      SQL*NET V2 - TNS: db_alias_name 또는 db_alias_name

[참 조]

1. ABBREVIATION

. A : Asynchronous, Dial-up Modem을 통하여 비동기 접속시
. AT : Appletalk Driver, Apple사의 Appletalk 프로토콜
. B : NETBIOS, Windows NT에서 로컬 접속과 WFW(Windows For Workgroup) 에서 사용하는 대부분의 NetBIOS와 NetBEUI를 위해
. D : DECnet, Digital Equipment사의 프로토콜
. P : Named Pipes, WFW(Windows For Workgroup)
. S : Single Task, MS-DOS상의 Oracle 6 DBMS
. T : TCP/IP, MAN 과 WAN에서 사용
. TT : Unix machine상에서 local server를 위한 Two Task driver
. X : SPX/IPX, Novell사의 LAN 프로토콜

2. DB_ALIAS_NAME

SQL*NET V2를 이용할 때, tnsnames.ora 화일내에 기술되는 기술자 서
DB_ALIAS_NAME은 사용자정의 필드로 ORACLE_SID와 동일하게 부여해서는 안된다.
. HOST_NAME : hosts 파일에 기술되어 있는 HOST_ALIAS /etc directory의
hosts 파일에 등록이 다음과 같이 되어있어야 한다.

HOST_ALIAS xxx.xxx.xxx.xxx IP Address

. PORT NUMBER : orasrv, listener는 다음의port를 통해 서비스 받는다.
NET V1일 경우에는 1525를 사용
NET V2일 경우에는 1521를 사용.
. /etc directory의 services 파일에 등록이 다음과 같이 되어 있어야 한다.
orasrv 1525/TCP
listener 1521/TCP

[Example] 다음은 SQL*NET V2의 tnsnames.ora에서 접속하려는 서버에 대한 내용이다.

```
DB_ALIAS_NAME =
   (DESCRIPTION =
     (ADDRESS_LIST =
        (ADDRESS =
           (PROTOCOL = TCP)
           (HOST = HOST_ALIAS 또는 HOST_IP_ADDRESS)
           (PORT = PORT _NUMBER)
        )
     )
    (CONNECT_DATA =
        (SID = ORACLE_SID)
    )
  )
```

"ODBC DRIVER CALL FAILED"에러 메시지

Bulletin no : 10131

Visual Basic, Excel, Access,.....등 ODBC Driver를 이용하여 ORACLE 데이타베이스에 접속시 ORACLE connect string(Data Source윈도우의 SETUP버튼 또는 ADD...버튼을 통하여 새로운 Data Source를 추가시 설정하는 SQL*NET connect string)이 제대로 부여되었다는 가정하에서 다음과 같이 에러가 난다면 다음과 같은 5가지 접근을 통하여 고칠수 있다.

Error Message : ODBC Driver call failed.

1. Client에서 ORACLE ODBC Driver를 1.10.0000 ~ 1.11.0002를 사용한다고 가정하면 c:\windows\system 디렉토리에 sqorast7.dll과 sqora7.dll 파일이 있는지 확인한다.

[주의] ODBC version 1.11.0002이상을 사용하면 sqlora71.dll과 sqoras71.dll 파일을 확인한다.

2. ODBC.INI 파일의 ["data source name"] 드라이버 section에 다음과 같이 지정이 되어 있는지 확인한다.

Driver=C:\WINDOWS\SYSTEM\sqora7.dll

[주의] ODBC version 1.11.0002이상인 경우

Driver=C:\WINDOWS\SYSTEM\sqora71.dll

3. ORACLE HOME 디렉토리가 path에 설정이 되어 있는지 확인한다.
4. ORA7WIN.DLL 파일이 중복되어 있지 않았나 확인합니다. (C:\WINDOWS\SYSTEM은 반드시 확인한다.)
5. Application을 실행하지 않은 상태에서 새로운 항목으로V7.0 RSFs를 재설치 한다.

[주의] ODBC version 1.11.0002이상을 사용하면 V7.1 RSFs를 재설치한다.

<<< 참고 >>>

ODBC API Conformance에는 CORE, LEVEL 1, LEVEL 2의 3가지가 LEVEL 정의되어 있으며 이를 이용하여 Application을 개발할 경우 현재까지는 ODBC Driver에서 제공되는 Function은 CORE, LEVEL 1 Function Call 만을 지원합니다.

<< ODBC API CORE FUNCTION CALLS >>

SQLAllocConnect, SQLAllocEnv, SQLAllocStmt, SQLBindCol, SQLCancel, SQLColAttributes, SQLConnect, SQLDescribeCol, SQLDisconnect, SQLError, SQLExecDirect, SQLExecute, SQLFetch, SQLFreeConnect, SQLFreeEnv, SQLFreeStmt, SQLGetCursorName, SQLNumResultCols, SQLPrepare, SQLRowCount, SQLSetCursorName, SQLSetParam, SQLTransact

<< ODBC API LEVEL 1 FUNCTION CALLS>>

SQLColumns, SQLDriverConnect, SQLGetConnectOption, SQLGetData, SQLGetFunctions, SQLGetInfo, SQLGetStmtOption, SQLGetTypeInfo, SQLParamData, SQLPutData, SQLSetConnectOption, SQLSetStmtOption, SQLSpecialColumns, SQLStatistics, SQLTables

<< ODBC API LEVEL 2 FUNCTION CALLS>>

SQLBrowseConnect, SQLColumnPrivileges, SQLDataSources, SQLDescribeParam, SQLExtendedFetch, SQLForeignKeys, SQLMoreResults, SQLNativeSql, SQLNumParams, SQLParamOptions, SQLPrimaryKeys, SQLProcedureColumns, SQLProcedures, SQLSetPos, SQLSetScrollOptions, SQLTablePrivileges

VB,GLUE에서 STORED PROCEDURE 사용방법

Bulletin no : 10235

Visual Basic에서 ORACLE에 지정되어 있는 Stored Procedure를 수행하는 방법은 다음과 같습니다.

1) Button(test)에 입력된 내용

```
Private Sub Command3_Click()
     Dim Serno As String, cmd As String, iRC As Integer

     cmd = "begin odbc_stp.odbc_test('RESPONSE CENTER');  end;"
     iRC = ExecuteSQL(hstmt, cmd)
End Sub
```

2) Stored Procedure의 이름 및 내용

```
create or replace package odbc_stp is
     g_deptno    number;
procedure odbc_test(v_name In dept.dname%type);
end odbc_stp;
/
create or replace package body odbc_stp is
procedure odbc_test(v_name In dept.dname%type) is
  begin
       select max(deptno) + 1 inoto g_deptno from dept;
       insert into dept(deptno, dname, loc) values
         (g_deptno, v_dname, 'ORACLE KOREA');
       end odbc_test;
end odbc_stp;
```

3) 수행 결과

```
SQL> select * from dept;

          DEPTNO  DNAME           LOC
          ------- --------------- ---------------
          10      ACCOUNTING      NEWYORK
          20      RESEARCH        DALLAS
          30      SALES           CHICAGO
          40      OPERATIONS      BOSTON
          41      RESPONSE CENTER ORACLE KOREA
          41      RESPONSE CENTER ORACLE KOREA
          43      RESPONSE CENTER ORACLE KOREA
          44      RESPONSE CENTER ORACLE KOREA
```

ODBC & SQL*NET SUPPORT MATRIX

Bulletin no : 10317

모든 버젼의 ODBC DRIVER와 SQL*NET은 특정 버전의 Required Support Files(RSF)를 요구하는데 구체적인 사항은 다음과 같다.

ODBC driver version	16/32 bit	Surpported Platforms	Supported SQL*NET ver.	RSF version	R D B M S version
1.11.00.05	16 bit	Windows 3.1	2.0.X for Windows	7.0.X	7.0 or later
1.11.01.05	16 bit	Windows 3.1	2.1.X for Windows	7.1.X	7.1 or later
1.14.01.00	16 bit	Windows 3.1	2.2.X for Windows	7.2.X	7.2 or later
1.15.3.0.1A	16 bit	Windows 3.1	2.3.X for Windows	7.3.X	7.3 or later
1.13.05.00	32 bit	Windows NT and Windows95	2.2.X (32 bit)~~	7.2.X	7.2 or later
1.13.05.04	32 bit	Windows NT and Windows95	2.2.X (32 bit)~~	7.2.X	7.2 or later
1.15.3.0.1A	32 bit	Windows NT and Windows95	2.3.X (32 bit)~~	7.3.X	7.3 or later

UNIX 에서 사용되는 ORACLE 환경 변수

Bulletin no : 10161

The document describes each Oracle environment variable with its name, it use, general sample values, and specific examples in the following manner:

ENVIRONMENT VARIABLE NAME
    Use: What is this environment variable?
    General: What are some sample values?
    Example: Specific example

In addition, UNIX environment variables that affect Oracle are briefly described.

General Notes
=============
1. $O_H == $ORACLE_HOME
2. $O_S == $ORACLE_SID
3. An environment variable followed by a "*" applies to Oracle7.
4. An environment variable followed by a "#" is used during database creation for Oracle Version 6 only.
5. For more information, see your IUG, ICG, or the "ORACLE7 Server for UNIX Administrator's Reference Guide"
6. Note that not all UNIX platforms use all of these environment variables.

Oracle Environment Variables
==========

APIPATH*
    Use: version 7.0.12 only: directory containing Tool Kit II *.res files
    General:
    Example: $O_H/orainst:$O_H/tk2/admin

BOOK_LOCALPREFERENCE*
    Use: Oracle*Book env var
    General: $ORACLE_HOME/book/admin
    Example: $ORACLE_HOME/book/admin

BOOK_RESOURCE*
    Use: directory for Oracle*Book resource files
    General: $O_H/book/admin/resource/US, $O_H/book/admin/resource/JA
    Example: $ORACLE_HOME/book/admin/resource/US

BOOK_GLOBALPREFERENCE*

Use: Oracle*Book env var

General: $ORACLE_HOME/book/admin

Example: $ORACLE_HOME/book/admin

BOOK_HELP*

Use: directory for Oracle*Book help files

General: $ORACLE_HOME/book/admin/help/US

Example: $ORACLE_HOME/book/admin/help/US

CASE_HP_CMD

Use: Command to plot a file from CASE*Designer

General: Actual print command or name of a script

Example: lpr -Pplot

CASE_PS_CMD

Use: Command to print postscript file from CASE*Designer

General: Actual print command or name of a script

Example: lpr -P

CASE_RESOURCE

Use: Tool Kit I terminal type for CASE

General: file:device (file.r from $O_H/dict50/admin/etc) and (device from Oraterm)

Example: case_hpx:xterm

CASE_SDPRINT

Use: to print CASE*Designer screen prints to line or PostScript printers

General: printer name

Example: lw

CASE_XTERM

Use: Tool Kit I GUI window type for CASE

General: xterm, hpterm, aixterm, dxterm, etc.

Example: xterm

CGEN_HOME

Use: home directory of CASE Generator

General:

Example: $O_H/cgen20

DEBUG_SLFIND*

Use: Debug for TK2 tools

General: 0,1,?

Example: 1

DBS_FILE#

Use: initial database file name

General: $O_H/dbs/dbs$O_S.dbf will hard code db file (you will not
be able to move $O_H); setting to dbs$O_S.dbf will allow you to move $O_H

Example: dbsoracle.dbf

```
DBS_SIZE#
       Use:      initial database file size
       General:  numberK, numberM
       Example:  5000K or 5M minimum, up to partition size

FORMS_DEVICE*
       Use:      devicename for TK2 tools (CDE tools)
       General:  vt220, vt100, hp, sun(see $ORACLE_HOME/tk2/admin/terminal)
       Example:  hp

FORMS30PATH
       Use:      directory containing Forms 3.0 (TK I) resource files
       General:
       Example:  $ORACLE_HOME/forms30/admin/resource

LOG
       Use:      file to record the install procedure
       General:  $O_H/install/install.log
       Example:  $O_H/install/install.log

LOG_FILE1#
       Use:      first redo log file name
       General:  $O_H/dbs/log1$O_S.dbf will hard code log file (you will
                 not be able to move $O_H); setting to log1$O_S.dbf will allow you to move $O_H
       Example:  log1oracle.dbf

LOG_FILE2#
       Use:      first redo log file name
       General:  $O_H/dbs/log2$O_S.dbf will hard code log file (you will
                 not be able to move $O_H); setting to log2$O_S.dbf will allow you to move $O_H
       Example:  log2oracle.dbf

LOG_SIZE#
       Use:      initial log file size
       General:  numberK, numberM
       Example:  no minimum, 500K default, no maximum (tune to your database needs)

MENU5PATH
       Use:      directory containing Menu 5.0 (TK I) resource files
       General:
       Example:  $ORACLE_HOME/menu5/admin/resource

MM_RESOURCE*
       Use:      ?
       General:  $ORACLE_HOME/mm/admin/resource/US
       Example:  $ORACLE_HOME/mm/admin/resource/US
```

NLS_LANG

Use: National Language Support language, territory, and character set

General: language_territory.characterset

Example: american_american.us7ascii

NO_MAKE

Use: set to true if your system doesn't have C dev tools (cc,make, ld, etc.)

General:

Example: false

ORACLE_BASE*

Use: new directory structure for Oracle 7 to follow Oracle Flexible Architecture

General: see ICG "Preparing to install Oracle Products"

Example: can be any directory

ORACLE_HELP

Use: directory containing help file?

General: $ORACLE_HOME/help/admin/resource

Example: $ORACLE_HOME/help/admin/resource

ORACLE_HOME

Use: dir containing top level Oracle directories

General: any directory with enough space

Example: /usr/oracle

ORACLE_ICON*

Use: dir containing icons for CDE tools

General: $ORACLE_HOME/guicommon/tk2/admin/icon

Example: $ORACLE_HOME/guicommon/tk2/admin/icon

ORACLE_LPARGS

Use: which lp arguments to use for Easy*SQL,SQL*Calc,SQL*Forms,
SQL*Menu, SQL*Report, or SQL*ReportWriter

General:

Example: -c -s

ORACLE_LPPROG

Use: which lp command to use for Easy*SQL,SQL*Calc, SQL*Forms,
SQL*Menu, SQL*Report, or SQL*ReportWriter

General: lp, lpr, print

Example: lp

ORACLE_LPSTAT

Use: which lp status command to use

General: lpstat, lpq

Example: lpstat

ORACLE_OWNER
Use: UNIX Operating System usercode who owns the Oracle files
General: can be any user, default is oracle
Example: oracle

ORACLE_PAGER
Use: which UNIX pager to use (more, less, pg) in Oracle Products like SQL*Menu
General:
Example: more

ORACLE_PATH
Use: directory for reading and writing to and from SQL*Forms, SQL*Menu 5.0, SQL*Plus
General: can be any directory (if not set, read and write from current directory)
Example: $HOME/oracle

ORACLE_SERVER#
Use: For client only installs
General: T if client only, F if there will be a local database
Example: T

ORACLE_SID
Use: Oracle System Identifier
General: must begin with a letter, followed by a number or character; limited by some OS's to 4 chars
Example: v712

ORACLE_TERM*
Use: Tool Kit II env var pointing to the tk2c${ORACLE_TERM}.res file under $ORACLE_HOME/rdbms/admin/terminal (tk2c stands for character toolkit2)
General: any file like tk2c${ORACLE_TERM}.res
Example: vt100

ORACLE_TERMINAL
Use: directory where the Tool Kit II .res files reside
General: $O_H/tk2/admin/terminal
Example: $O_H/tk2/admin/terminal

ORACLE_TRACE
Use: allow verification/trace to be turned on and off during install (echoes everything done by sh)
General: T does set -x; anything else is off
Example: T

ORACLE_VERIFY
Use: if true, runs the .verify scripts during install
General: T; anything else is off
Example: T

ORACLE_DOC*

Use: points to directory containing on-line doc starting v7.0.16

General: to any directory containing online doc (file.obd)

Example: $ORACLE_HOME/doc

ORAENV_ASK

Use: if set to anything, prompts for $O_S or $O_H when (c)oraenv is invoked

General: NO or anything

Example: NO

ORAKITPATH

Use: set to directory containing the v1 orakit resource file

General: $ORACLE_HOME//admin/resource

Example: $ORACLE_HOME/forms30/admin/resource

ORAMAIL_EDITOR

Use: which editor to use for Oracle*Mail

General: vi, emacs, ed

Example: vi

ORANSEMS#

Use: number of semaphores that make up a set

General: 1-UNIX maximum

Example: 60

ORAPIPES*

Use: sets the default pipe driver to v1 or v2

General: v1, v2

Example: v2

ORATERMPATH

Use: directory containing Oraterm (TK I) resource files

General:

Example: $ORACLE_HOME/oraterm/admin/resource

PATCH_HOME

Use: directory containing patches and patch.list

General: any directory,$O_H/patch,$O_H/install/patch

Example: $ORACLE_HOME/install/patch

RT_STATUS

Use: directory for SQL*TextRetrieval V2.0 to find forms

General: can be any directory (if not set, read and write from current directory)

Example: $HOME/oracle

SDD_HOME

Use: directory containing top level CASE Dictionary directories

General: can be any directory containing top level CASE directories

Example: $O_H/dict50/admin

SDD_PRINT

Use: for CASE: set to UNIX print command to use

General: lp, lpr

Example: lp

SDD_WPRINT

Use: for CASE: set to UNIX print command to use for wide

General: lp, lpr

Example: lp

SQLPATH

Use: now superceded by $ORACLE_PATH

General: can be any directory (if not set, read and write from current directory)

Example: $HOME/oracle

SQLTR_STATUS

Use: directory for SQL*TextRetrieval V1.1 to find forms

General: can be any directory (if not set, read and write from current directory)

Example: $HOME/oracle

SRW_TMP

Use: directory to use for SQL*ReportWriter temporary files

General: any directory with >10M of disk space

Example: /usr/tmp

TK2DEV*

Use: version 7.0.12 only: Tool Kit II terminal resource file

General: vt100 portion from tk2_vt100.res

Example: vt100

TNS_ADMIN*

Use: directory containing SQL*Net v2 files

General: any dir (see manual for order followed--looks at /etc, $HOME, and $TNS_ADMIN

Example: $O_H/network/admin

TWO_TASK

Use: hoststring to use for SQL*Net v1

General: p:, f:, a:host:$O_S, d:host:$O_S, star:host:$O_S,
t:host:$O_S, tt:host:$O_S, x:host:$O_S

Example: t:host:oracle

UNIX Environment Variables Affecting Oracle

==========

```
ADA_PATH        set to directory containing Ada compiler

ALSYCOMP_DIR    set to directory containing Ada compiler

DISPLAY
       Use:      Tells X-based tools which Display to use
       General:  `hostname`:0.0
       Example:  fubar:0.0

HOME            $HOME

LANG            en_US, De_DE

LANGUAGE        american_america

LDOPTS          ld options for compiling

LDPATH          directory containing shared object libraries

LD_LIBRARY_PATH   directory containing shared object libraries

LOG             when set to a filename, logs information from install session

LOGNAME         `logname` or `whoami`

MALLOCTYPE      3.1, 3.2 for telling AIX which malloc type to use

NLSPATH         OS directory containing language message files

NONAMESERVER
       Use:      For tcp/ip networks without a named server
       General:  1 for no named server, 0 for named server
       Example:  1

PATH            $PATH

PRINTER

SHELL           /bin/csh or /bin/sh

TERM            any valid term: ansi, hft, hp, mac2, sun, xterm, vtxxx

TMPDIR          some UNIX boxes allow /tmp to be renamed to $TMPDIR (any directory)

TZ              sets local time zone

XENVIRONMENT    set to file Orakit, or other X11 resource files

XAPPLRESDIR     set to dir containing resource specifications file
                Orakit, or other X11 resource files
```

(V7.3) PARTITION VIEW 테스트 REPORT

Bulletin no : 10386

---

1. 개요

대용량 Database를 구축하고 관리하는 데 있어서 필연적으로 나타날 수 있는 문제는
시간적 증가 또 지역적으로 확대 됨에 따라 어느 TABLE의 SIZE가 증가함에 따라
이에 따른 관리 방법의 효율화 일 것이다.
이는 주기적인 재편성(Re-Organization)뿐만 아니라 Backup, Index의 비대화 등에 따른
Query의 효율저하 등 여러가지 문제를 야기 시킬 수 있다.
이에 대한 해결책인 Partition View(Ver.7.3 이상)를 실무적용사례를 통해서 구축하는
방법, Partition View의 효율적 사용 방법, 주의 할 점등을 알아보고자 한다.

2. 구축 과정

1) Partition View를 위한 Parameter

init$ORACLE_SID.ora File에 다음의 Parameter Setting

partition_view_enabled=TRUE

2) Partition View 대상 Table 및 Partition Column을 선정

이는 물리적 설계 시 결정
Partition Column의 주요 후보는 일시, 지역 등 명확히 구분되는 Column이어야 함

3) Table Create->Partition Column에 반드시 해당 범위의 Check Constraint를 반드시주어야 함.
(주의 사항)

Partition Column의 DATATYPE을 CHAR로 하지 말 것 (BUG) --> VARCHAR2 사용 (BUG로 등록되어 있음)
Partition View를 구성하는 모든 Table Scheme는 완전히 동일 해야 함
(Column의 갯수, Datatype, Null허용 여부 등)

실제로 비씨카드에서 TYPE의 불일치로 문제가 발생했었음.
1996.01월 Table(TBCARDUSE_199601)의 한 Column의 Type이 다른 Table은
모두 VARCHAR(9) 는데 이 Table만 VARCHAR2(10)로 되어있어서 Partition
View를 JOIN하면 VIEW 내의 모든 TABLE이 FULL TABLE SCAN을 함
DATATYPE을 동일하게 해준 후 정상적으로 동작

4) Index Create --> 모든 Table이 반드시 동일한 Index를 가져야 함.

Partition Column에 Index가 반드시 필요한 것은 아님.

5) 각 Table에 ANALYZE TABLE을 수행 (가능한 한 COMPUTE STATISTICS로)

6) Partition 내의 모든 Table을 UNION ALL 한 Partition View를 생성

3. 구축 사례

TBCARDUSE_199507 - TBCARDUSE_199609 (15개 Table)

각 Table의 예상 Row 수는 최소 1,200 만 이상으로 15개월을 보관함을 원칙으로 하여 총 Row는 2억건 정도이다.

```
CREATE TABLE TBCARDUSE_199507
      ( USE_NO              varchar2(8)    not null,
        CARD_NO             varchar2(16)   not null,
        REQ_REJ_CLSS        char(1)        not null,
        LO_AB_CLSS          char(1)        not null,
        MER_MGMT_NO         varchar2(9)    not null,
        AUTH_CLSS           char(2)        not null,
        AUTH_DATE           varchar2(8)    not null, --> Partition Column
        AUTH_TIME           char(6)        not null,
        AUTH_AMT            number(13,2)   not null,
        EN_MODE             char(2)        not null,
        AUTH_REQ_MTHD       char(2)        not null,
        AUTH_REQ_CLSS       char(1)        not null,
        SALE_REF_PL         char(2)        ,
        REQ_MER_MGMT_NO     varchar2(9)    ,
        SALE_PL_NO          varchar2(12)   ,
        REQ_VALD_LIM        char(6)        ,

               --------
               --------
               --------
               --------
               --------
               --------

        USE_AMT             number(11)     ,
        RCP_AMT             number(11)     ,
        MER_RCP_FEE         number(11)     ,
        RCP_DATE            char(8)        ,
        POS_CODE            char(2)        ,
        CONSTRAINT  C_AUTH_DATE_199507
        CHECK (AUTH_DATE BETWEEN '19950701' AND '19950731'))
           ---> Partition Column Constraint
        TABLESPACE TS_TUNING01
        PCTFREE 10
        STORAGE(INITIAL 100M NEXT 100M PCTINCREASE 0
                MAXEXTENTS UNLIMITED FREELISTS 10);
```

* Index 정보

Index 1 : CARD_NO

```
CREATE INDEX TBCARDUSE_199507_IDX1
  ON TBCARDUSE_199507(CARD_NO)
  TABLESPACE TS_TUNING_I01
  STORAGE(INITIAL 50M NEXT 50M PCTINCREASE 0
          MAXEXTENTS UNLIMITED)
  PARALLEL (DEGREE 4);
```

Index 2 : USE_NO

```
CREATE INDEX TBCARDUSE_199507_IDX2
  ON TBCARDUSE_199507(USE_NO)
  TABLESPACE TS_TUNING_I01
  STORAGE(INITIAL 50M NEXT 50M PCTINCREASE 0
          MAXEXTENTS UNLIMITED)
  PARALLEL (DEGREE 4);
```

Index 3 : AUTH_DATE + MER_MGMT_NO

```
CREATE INDEX TBCARDUSE_199507_IDX3
  ON TBCARDUSE_199507(AUTH_DATE,MER_MGMT_NO)
  TABLESPACE TS_TUNING_I01
  STORAGE(INITIAL 50M NEXT 50M PCTINCREASE 0
          MAXEXTENTS UNLIMITED)
  PARALLEL (DEGREE 4);
```

Index 4 : MER_MGMT_NO + AUTH_DATE

```
CREATE INDEX TBCARDUSE_199507_IDX4
  ON TBCARDUSE_199507(MER_MGMT_NO, AUTH_DATE)
  TABLESPACE TS_TUNING_I01
  STORAGE(INITIAL 50M NEXT 50M PCTINCREASE 0
          MAXEXTENTS UNLIMITED)
  PARALLEL (DEGREE 4);
```

* Table Analyzing

```
analyze table TBCARDUSE_199507 compute statistics;
```

* Partition View Creation

```
CREATE OR REPLACE VIEW PV_TBCARDUSE
AS
  SELECT * FROM TBCARDUSE_199507
```

```
UNION ALL
SELECT * FROM TBCARDUSE_199508
UNION ALL
SELECT * FROM TBCARDUSE_199509
UNION ALL
SELECT * FROM TBCARDUSE_199510
UNION ALL
SELECT * FROM TBCARDUSE_199511
UNION ALL
SELECT * FROM TBCARDUSE_199512
UNION ALL
SELECT * FROM TBCARDUSE_199601
UNION ALL
SELECT * FROM TBCARDUSE_199602
UNION ALL
SELECT * FROM TBCARDUSE_199603
UNION ALL
SELECT * FROM TBCARDUSE_199604
UNION ALL
SELECT * FROM TBCARDUSE_199605
UNION ALL
SELECT * FROM TBCARDUSE_199606
UNION ALL
SELECT * FROM TBCARDUSE_199607
UNION ALL
SELECT * FROM TBCARDUSE_199608
UNION ALL
SELECT * FROM TBCARDUSE_199609
```

4. Test Procedure

1) Partition View의 동작 여부 확인

(1) Partition Column이 '='로 비교 되었을 때

```
SELECT CARD_NO,USE_NO,AUTH_DATE,AUTH_AMT
FROM   PV_TBCARDUSE
WHERE  USE_NO = '00002037'
AND    AUTH_DATE = '19960723';

Rows  Execution Plan
----  -------------------------------------------------
   0  SELECT STATEMENT   GOAL: CHOOSE
   1   VIEW OF 'PV_TBCARDUSE'
   1    UNION-ALL (PARTITION)
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199507'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199507_IDX2'
```

```
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199508'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199508_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199509'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199509_IDX2'
0     FILTER                    ---> Access 불 필요 Skip Operation
0      TABLE ACCESS OF 'TBCARDUSE_199510'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199510_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199511'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199511_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199512'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199512_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199601'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199601_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199602'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199602_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199603'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199603_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199604'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199604_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199605'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199605_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199606'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199606_IDX2'
1     TABLE ACCESS OF 'TBCARDUSE_199607'
2      INDEX (RANGE SCAN) OF 'TBCARDUSE_199607_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199608'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199608_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199609'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199609_IDX2'
```

(설명) Partition View의 가장 일반적인 사용 Case로 Partition Column이 *=*로 비교 되었을 때는 Access 불 필요 Table은 Skip하는 Operation (FILTER)이 동작 함.

(2) Partition Column이 " BETWEEN ~ AND ~ "로 비교 될 때

```
SELECT CARD_NO,USE_NO,AUTH_DATE,AUTH_AMT
FROM   PV_TBCARDUSE
WHERE  USE_NO = '00002037'
AND    AUTH_DATE BETWEEN '19960701' AND '19960731'

Rows  Execution Plan
----  --------------------------------------------------
   0  SELECT STATEMENT   GOAL: CHOOSE
   1   VIEW OF 'PV_TBCARDUSE'
   1    UNION-ALL (PARTITION)
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199507'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199507_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199508'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199508_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199509'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199509_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199510'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199510_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199511'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199511_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199512'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199512_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199601'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199601_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199602'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199602_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199603'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199603_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199604'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199604_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199605'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199605_IDX2'
   0     FILTER
   0      TABLE ACCESS OF 'TBCARDUSE_199606'
   0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199606_IDX2'
   1     TABLE ACCESS OF 'TBCARDUSE_199607'
   2      INDEX (RANGE SCAN) OF 'TBCARDUSE_199607_IDX2'
```

```
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199608'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199608_IDX2'
0     FILTER
0      TABLE ACCESS OF 'TBCARDUSE_199609'
0       INDEX (RANGE SCAN) OF 'TBCARDUSE_199609_IDX2'
```

(설명) Partition Column이 BETWEEN ~ AND ~ 로 비교 되었을 때는
Access 불 필요 Table은 Skip하는 Operation(FILTER)이 동작 함

(3) Partition Column이 " LIKE "로 비교 될 때

```
SELECT CARD_NO,USE_NO,AUTH_DATE,AUTH_AMT
FROM   PV_TBCARDUSE
WHERE  USE_NO = '00002037'
AND    AUTH_DATE LIKE '199607%'

Rows  Execution Plan
----  -------------------------------------------------
   0  SELECT STATEMENT   GOAL: CHOOSE
   1   VIEW OF 'PV_TBCARDUSE'
   1    UNION-ALL (PARTITION)
   0     TABLE ACCESS OF 'TBCARDUSE_199507'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199510_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199508'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199510_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199509'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199510_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199510'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199510_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199511'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199511_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199512'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199512_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199601'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199601_IDX2'
   1     TABLE ACCESS OF 'TBCARDUSE_199602'
   2      INDEX (RANGE SCAN) OF 'TBCARDUSE_199602_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199603'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199603_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199604'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199604_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199605'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199605_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199606'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199606_IDX2'
   1     TABLE ACCESS OF 'TBCARDUSE_199607'
   2      INDEX (RANGE SCAN) OF 'TBCARDUSE_199607_IDX2'
```

```
   0     TABLE ACCESS OF 'TBCARDUSE_199608'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199608_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199609'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199609_IDX2'
```

(설명) Partition Column이 *LIKE*로 비교 되었을 때는
FILTER가 동작하지 않음. (주의 요망)

(4) Partition Column이 " OR "로 비교 될 때

```
SELECT CARD_NO,USE_NO,AUTH_DATE,AUTH_AMT
FROM   PV_TBCARDUSE
WHERE  USE_NO = '00002037'
AND    (    AUTH_DATE BETWEEN '19960601' AND '19960630'
         OR AUTH_DATE BETWEEN '19960701' AND '19960730')

Rows  Execution Plan
----  -------------------------------------------------
   0  SELECT STATEMENT   GOAL: CHOOSE
   1   VIEW OF 'PV_TBCARDUSE'
   1    UNION-ALL (PARTITION)
   0     TABLE ACCESS OF 'TBCARDUSE_199507'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199510_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199508'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199510_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199509'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199510_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199510'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199510_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199511'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199511_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199512'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199512_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199601'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199601_IDX2'
   1     TABLE ACCESS OF 'TBCARDUSE_199602'
   2      INDEX (RANGE SCAN) OF 'TBCARDUSE_199602_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199603'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199603_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199604'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199604_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199605'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199605_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199606'
   1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199606_IDX2'
   1     TABLE ACCESS OF 'TBCARDUSE_199607'
   2      INDEX (RANGE SCAN) OF 'TBCARDUSE_199607_IDX2'
   0     TABLE ACCESS OF 'TBCARDUSE_199608'
```

```
1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199608_IDX2'
0     TABLE ACCESS OF 'TBCARDUSE_199609'
1      INDEX (RANGE SCAN) OF 'TBCARDUSE_199609_IDX2'
```

(설명) Partition Column이 *OR*로 비교 되었을 때는
FILTER가 동작하지 않음. (BUG로 등록되어 있음)

(5) Partition Column이 " BETWEEN "으로 비교되고 다른 비교 Column이 없을 때

```
SELECT CARD_NO,USE_NO,AUTH_DATE,AUTH_AMT
FROM   PV_TBCARDUSE
WHERE  AUTH_DATE BETWEEN '19960601' AND '19960630'

EXECUTION PLAN
----------------------------------------------------
SELECT STATEMENT
  VIEW PV_TBCARDUSE
    UNION-ALL (PARTITION)
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199507
          INDEX (RANGE SCAN) TBCARDUSE_199510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199508
          INDEX (RANGE SCAN) TBCARDUSE_199510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199509
          INDEX (RANGE SCAN) TBCARDUSE_199510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199510
          INDEX (RANGE SCAN) TBCARDUSE_199510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199511
          INDEX (RANGE SCAN) TBCARDUSE_199511_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199512
          INDEX (RANGE SCAN) TBCARDUSE_199512_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199601
          INDEX (RANGE SCAN) TBCARDUSE_199601_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199602
          INDEX (RANGE SCAN) TBCARDUSE_199602_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199603
          INDEX (RANGE SCAN) TBCARDUSE_199603_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199604
```

```
          INDEX (RANGE SCAN) TBCARDUSE_199604_IDX3
        FILTER
          TABLE ACCESS (BY ROWID) OF TBCARDUSE_199605
            INDEX (RANGE SCAN) TBCARDUSE_199605_IDX3
        TABLE ACCESS FULL TBCARDUSE_199606
        FILTER
          TABLE ACCESS (BY ROWID) OF TBCARDUSE_199607
            INDEX (RANGE SCAN) TBCARDUSE_199607_IDX3
        FILTER
          TABLE ACCESS (BY ROWID) OF TBCARDUSE_199608
            INDEX (RANGE SCAN) TBCARDUSE_199608_IDX3
        FILTER
          TABLE ACCESS (BY ROWID) OF TBCARDUSE_199609
            INDEX (RANGE SCAN) TBCARDUSE_199609_IDX3
```

(6) Partition Column이 " OR "로 비교되고 다른 비교 Column이 없을 때.

```
SELECT CARD_NO,USE_NO,AUTH_DATE,AUTH_AMT
FROM   PV_TBCARDUSE
WHERE  AUTH_DATE BETWEEN '19960501' AND '19960531'
OR     AUTH_DATE BETWEEN '19960601' AND '19960630'
OR     AUTH_DATE BETWEEN '19960701' AND '19960731'

EXECUTION PLAN
------------------------------------------------
SELECT STATEMENT
  VIEW PV_TBCARDUSE
    UNION-ALL (PARTITION)
      TABLE ACCESS FULL TBCARDUSE_199507
      TABLE ACCESS FULL TBCARDUSE_199508
      TABLE ACCESS FULL TBCARDUSE_199509
      TABLE ACCESS FULL TBCARDUSE_199510
      TABLE ACCESS FULL TBCARDUSE_199511
      TABLE ACCESS FULL TBCARDUSE_199512
      TABLE ACCESS FULL TBCARDUSE_199601
      TABLE ACCESS FULL TBCARDUSE_199602
      TABLE ACCESS FULL TBCARDUSE_199603
      TABLE ACCESS FULL TBCARDUSE_199604
      TABLE ACCESS FULL TBCARDUSE_199605
      TABLE ACCESS FULL TBCARDUSE_199606
      TABLE ACCESS FULL TBCARDUSE_199607
      TABLE ACCESS FULL TBCARDUSE_199608
      TABLE ACCESS FULL TBCARDUSE_199609
```

최악의 경우

(7) 위 (6)의 Query를 " BETWEEN "으로 변환 경우

```
SELECT /*+ FULL(PV_TBCARDUSE) */
       CARD_NO,USE_NO,AUTH_DATE,AUTH_AMT
FROM   PV_TBCARDUSE
WHERE  AUTH_DATE BETWEEN '19960501' AND '19960731'

EXECUTION PLAN
------------------------------------------------
SELECT STATEMENT
  VIEW PV_TBCARDUSE
    UNION-ALL (PARTITION)
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199507
          INDEX (RANGE SCAN) TBCARDUSE_199510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199508
          INDEX (RANGE SCAN) TBCARDUSE_199510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199509
          INDEX (RANGE SCAN) TBCARDUSE_199510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199510
          INDEX (RANGE SCAN) TBCARDUSE_199510_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199511
          INDEX (RANGE SCAN) TBCARDUSE_199511_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199512
          INDEX (RANGE SCAN) TBCARDUSE_199512_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199601
          INDEX (RANGE SCAN) TBCARDUSE_199601_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199602
          INDEX (RANGE SCAN) TBCARDUSE_199602_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199603
          INDEX (RANGE SCAN) TBCARDUSE_199603_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199604
          INDEX (RANGE SCAN) TBCARDUSE_199604_IDX3
      TABLE ACCESS FULL TBCARDUSE_199605
      TABLE ACCESS FULL TBCARDUSE_199606
      TABLE ACCESS FULL TBCARDUSE_199607
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199608
          INDEX (RANGE SCAN) TBCARDUSE_199608_IDX3
      FILTER
        TABLE ACCESS (BY ROWID) OF TBCARDUSE_199609
```

INDEX (RANGE SCAN) TBCARDUSE_199609_IDX3

2) Insert, Delete, Update의 사용

현재까지는 Partition View에 직접적으로 Insert/Delete/Update를 할 수 없다.
(이는 8.0 Version에 가능)
따라서 현재는 2 가지 방법으로 가능.

(1) DYNAMIC SQL을 이용하는 방법
(2) LOGIC적으로 IF-ELSE 방법을사용하여 가능한 일어 날 수 있는 모든 경우를 기술하는 방법

(비교) (1)의 방법은 Application 작성이 간단하기는 하나 LOOP안에서 수행되거나 빈번히 수행되는 경우에는 Parsing Overhead가 심각해질 정도로 증가 됨.
(2)의 방법은 (1)의 단점을 해소 할 수 있으나 Partition Table이 추가,변경 됨에 따라 주기적으로 APPLICATION이 같이 변경해 주어야 한다.

3) Partition View의 효율적인 사용 방법

(1) Parsing Time의 감소

Partition View는 많은 수의 Table을 UNION ALL View로 만든 것이므로 Query 수행 시 Parsing Time의 Overhead로 인해 성능 저하를 가져 올 수 있다.

이에 대해서 물리적인 방법(HOLD_CURSOR, CURSOR_SPACE_FOR_TIME=true 등)으로 Re-Parsing Time을 줄일 수 있으나 Partion View를 효율적으로 생성하므로써 Overhead를 줄일수 있다.

(가) 수직적 분할

(사례)에서는 약 100개 정도의 Column에 대해서 "Select * "의 형태로 되어 있어 Data Dictionary Call이 그만큼 많아 진다. 그러나 Table의 속성을 보면 필수 NOT NULL Column(10ro)을 제외한 90개의 Column이 실제적으로는 '신용판매','현금서비스','해외이용'의 3가지로 GROUPING 되는 것으로 PARTITION VIEW자체를 3개 정도로 분할(수직적인 분할)하고 사용하여 Overhead를 줄인다.

(나) 수평적 분할

가장 ACTIVE한 범위를 정하여 수평적으로 분할하여 Partition View를 생성한다. (사례)에서는 3개월,6개월,9개월 단위의 Partition View를 만들어 사용하고 있다.

(2) Partition Column이 비교절에 나오지 않을때

이 때는 어쩔 수 없이 Partition Column이 사용 될 수 없으므로 일반적인 UNION-ALL View형태로 Access 될 수 밖에 없다.
그러나 SELECT 결과가 1건일 경우에는 ROWNUM 제한을 시켜 될 수 있는 한

TABLE ACCESS를 제한시킨다.

(UNION ALL의 순서를 역순으로 하여 VIEW를 생성)

```
CREATE OR REPLACE VIEW PV_TBCARDUSE1
 AS
   SELECT * FROM TBCARDUSE_199609
   UNION ALL
   SELECT * FROM TBCARDUSE_199608
   UNION ALL
   SELECT * FROM TBCARDUSE_199607
   UNION ALL
   SELECT * FROM TBCARDUSE_199606
   UNION ALL
   SELECT * FROM TBCARDUSE_199605
   UNION ALL
   SELECT * FROM TBCARDUSE_199604
   UNION ALL
   SELECT * FROM TBCARDUSE_199603
   UNION ALL
   SELECT * FROM TBCARDUSE_199602
   UNION ALL
   SELECT * FROM TBCARDUSE_199601
   UNION ALL
   SELECT * FROM TBCARDUSE_199512
   UNION ALL
   SELECT * FROM TBCARDUSE_199511
   UNION ALL
   SELECT * FROM TBCARDUSE_199510
   UNION ALL
   SELECT * FROM TBCARDUSE_199509
   UNION ALL
   SELECT * FROM TBCARDUSE_199508
   UNION ALL
   SELECT * FROM TBCARDUSE_199507

 SELECT * FROM PV_TBCARDUSE1
 WHERE  USE_NO = '123456'              ---+
 AND    CARD_NO = '45991276182212'     ---+ (UNQUE)
 AND    ROWNUM = 1;
```

(사례)에서 시간적으로 최근의 TABLE을 먼저 UNION ALL하여 사용하였다.
이는 원하는 정보가 최 에 있을 확률이 높기때문에 먼저 찾게되면 더이상
TABLE의 ACCESS를 하지 않기때문이다.

(3) HINT의 사용

Partition View에 Hint 사용은 보통의 Query에 대해서와 같이 사용한다.

(1) PARALLEL HINT

```
SELECT /*+ PARALLEL(PV_TBCARDUSE,2) */
       SUM(AUTH_AMT)
FROM   PV_TBCARDUSE
WHERE  AUTH_DATE BETWEEN '19960501' AND '19960731'
```

TBCARDUSE_199605,TBCARDUSE_199606,TBCARDUSE_199607 Table을
Parallel 처리한다.

(2) INDEX_DESC HINT

이는 INDEX이름이 HINT내에 들어가야 하므로 원칙적으로는 불가능하지만
사용 될 Table이 명확하게 정의된다면 사용 할 수도 있다.

```
SELECT /*+ INDEX_DESC(a TBACRDUSE_199605_IDX1) */
         *
FROM   PV_TBCARDUSE a
WHERE  CARD_NO = '4599243141847231'
AND    AUTH_DATE BETWEEN '19960501' AND '19960531'
AND    ROWNUM = 1
```

(V7.3) PARTITION VIEWS IN RELEASE 7.3

Bulletin no : 10380

---

1. 소개

Partition view 는 Oracle Release 7.3 에서 대용량 테이블(예를 들면, data warehousing에서 주로 사용함)을 지원하기 위해서 추가된 새로운 방법이다. Partition View는 대용량 테이블에 대한 관리성 및 가용성을 향상시키고, 이들 테이블에 대한 질의 속도를 향상시킬 수 있다.
Partition view 의 예를 보면,

```
create view SALES as
        select * from jan_sales
    union all
        select * from feb_sales
    ...
    union all
        select * from dec_sales;
```

1월에서 12월까지의 sales table 들을 합한 뷰가 *SALES* 이다. 이들 월별 테이블들은 column name, data type, index 가 모두 같아야 한다. 또한 이들 월별 테이블들은 partition column 에 check constraint 가 반드시 존재해야 하는데, 위의 예를 통해서 보면 jan_sales 의 date column 의 값들은 1월 1일에서 31일 사이의 date 만이 가능하도록 check constraint 가 있어야 한다.
Base table, indexes, constraints 뿐 만 아니라 UNION-ALL view 의 정의는 DBA 에 의해 생성된다.

2. 관리성 및 가용성에 대한 장점

Partition view 는 대용량 테이블의 관리를 매우 쉽게 해 준다. 예를 들면, 위에서 정의한 *SALES* 라는 대용량 테이블을 살펴보자. 매월마다 DBA 는 새로운 달의 sales data 를 *SALES* 에 추가해야 한다. 따라서, DBA 는 *SALES* 대한 모든 index 들을 drop 하고, 새로운 데이타를 추가한 후에 index 를 재 생성해야 할 것이다. 더구나, *SALES* 가 큰 테이블이므로 이 작업 시간도 상당히 오래 걸릴 것이다. 또한, 이 작업 동안 *SALES* 테이블은 사용할 수 없게 된다.
그러나, Partition view 를 사용한다면, 새로운 달의 테이타를 새로운 Partition을 만들어 추가하고, 이 새로운 Partition 에 index 를 생성하는 동안 현재 사용 중인 Partition 을 계속 사용할 수 있다. 따라서, 새로운 Partition 과 index 가 완성되었을 때, DBA 는 이 새로운 Partition 을 포함하는 UNION-All view 를 재 생성 하면 된다. 이 UNION-ALL view 를 재생성하는 짧은 시간 동안만 *SALES* 가 사용 불가능 하게 된다.

3. Partition view 의 성능 향상

Release 7.3 부터 제공하는 UNION-ALL view 를 사용하면, 단일 테이블 접근 보다 적어도 같거나 대부분의 경우 더 좋은 성능 향상을 얻을 수 있다. 단, 모든 Partition 들은 반드시 같은 column 에 check constraint 가 있어야 하고, column 정의와 index 가 같아야 성능 향상의 효과를 얻을 수 있다. Partition view 에는 다음 두 가지의 성능이 향상되었다.

① Partition 제거

② UNION-ALL view 의 Parallel 수행

먼저, Partition 제거란 다음과 같다. 위에서 생성된 *SALES* partition view 에 대한 임의의 질의들은 일부의 Partition 들로부터 데이타를 가져온다. 예를 들어,

```
select sum(revenues) from SALES
  where sales_date between *15-JAN-96* and *15-FEB-96*;
```

와 같은 질의는 Partition view 로 처리 될 경우, JAN partition view와 FEB partition view 만으로 산출된다. 즉, 나머지 10개의 Partition 은 접근하지 않는다. 이것을 Partition 제거라고 한다. Partition 제거는 Partition column 에 대한 질의에서만 사용될 수 있다. 위의 예에서는 sales_date column 이 partition column 이다.이러한 제약 사항은 있지만, 속도 향상은 주목할 만 한다. 위의 예에서 보듯이 12개중 10개의 Partition 을 제외하여 6배의 성능 향상을 얻었다.

Partition view 를 사용 했을 때, 또 다른 장점은 UNION-ALL view 를 parallel 수행한다는 것이다. Parallel Query Option 이 설치된 경우, 모든 UNION-ALL view 에 대한 질의는 parallel 로 수행된다. 이점은 다른 경쟁 회사들의 parallel query 구조(이들 제품들은 데이타 partition 의 물리적 구조가 parallelism 을 결정한다)와는 다르게 parallelism degree 와는 독립적으로 수행된다는 점은 매우 특이하다. 오라클은 UNION-ALL view 의 데이타를 모든 parallel query processes 에게 동적으로 균등하게 배분하고, 위에서 언급한 par

* Partition view 의 제약 사항
Partition view 는 DML 연산을 제공하지 않는다. 따라서, READ-OLNY application (data warehousing 등등) 에 적합하다.

(V7.3) STANDBY DATABASE IN ORACLE 7.3

Bulletin no : 10322

Standby database 에 대하여

1. 오라클 7.3 버젼에 추가된 new feature 로써, archive mode 상태에서 online tablespace backup 이 필요없는 새로운 backup/recovery 방법이다. 즉, archive log mode 에서도 각 tablespace 의 주기적인 online backup 이 필요하였고, 이 작업에 따라 회복시간이 좌우 되었다. standby database 를 잘 유지하여 줄 수 있는 환경이라면, online tablespace backup 대신 유용하게 사용될수 있다.

2. 회복 시간은 작업량과 standby database 에 archive log file 가 어느 정도 반영 되었느냐에 따라 다르다.

3. OPS 와의 비교
instance 를 두개 이상 사용하는 Oracle Parallel Server와 비교하는 것 보다 위의 1번 에서 소개한 환경과 비교하는 것이 더 적절하다.

4. 생성 방법

4.1 create the standby database
; standby db 의 생성

1) back up the datafiles
2) startup mount;
alter database create standby controlfile as 'filename';
3) alter system archive log current;
4) transfer *.log, *.dbf to the safe place for standby database

4.2 maintain the standby database
; log file 을 standby db 에 적용시켜 최신의 데이타로 유지함.

1) startup nomount;
2) alter database mount standby database;
3) recover staneby database ;

4.3 activate the standby database
; primary database 가 정상적으로 startup 할 수 없을 때

1) alter database activate standby database;

!!! 일단 standby database 를 기동시켰으면, 이전의 primary database 상태로 되돌릴수 없으니 신중하게 사용하여야 함.

ARCHIVE LOG MODE 운영방법

Bulletin no : 10384

Oracle에서 Online Backup을 받거나 완벽한 Recovery 작업을 수행하기 위해서는 DB를 Archive log mode로 운영하여야 한다.

Archive log mode로 운영하기 위해서는 다음과 같이 변경하여야 한다.

1. initSID.ora file에서 다음의 parameter를 setting한다.

(1) LOG_ARCHIVE_START = TRUE
-> ARCH process 가 기동
-> log switch 발생시 automatic archive를 수행한다. 만약 이 parametrer가 false이면 manual archive를 실시하여야 한다.

(2) LOG_ARCHIVE_DEST = /home/oracle7/dbs/archive_file/arc
-> archive장소의 디렉토리와 확장자를 포함하지 않는 파일명을 지정
-> 여기에서 archive_file까지는 directory이며 마지막에 있는 arc는 archive log의 file initial명이다.

(3) LOG_ARCHIVE_FORMAT = %s.log
-> archive file의 확 자와 log sequence번호의 형식을 지정
-> 이는 (2)에서 정의된 archive log의 initial file명과 함께 나타난다.

[예] arc123.log, arc124.log (123과 124는 log sequence number 이다.)

initSID.ora file에 위의 정보를 setting한후 save한다.

2. 다음과 같이 작업하여 archive log mode로 변환한다.

```
$ sqldba lmode=y
   SQLDBA> connect internal
   SQLDBA> startup mount                                          -> (1)
   SQLDBA> alter database archivelog                              -> (2)
   SQLDBA> archive log list                                       -> (3)
         Database log mode              ARCHIVELOG                -> (4)
         Automatic archival               ENABLED                 -> (5)
         Archive destination              ?/dbs_ar/offline_log/offline     -> (6)
         Oldest online log sequence     123                       -> (7)
         Current log sequence           125                       -> (8)
         SQLDBA>alter database open;                              -> (9)
```

(1) DB를 startup mount까지만 한다.
(2) 이 Command를 이용하여 archivelog mode로 DB를 변경한다.
(3) Archivelog mode로 변경되었는지를 확인한다.

(4) DB가 Archivelog mode임을 나타낸다.
    만약 NOARCHIVELOG 로 되어있으면 변경되지 않은 것을 의미한다.
(5) initSID.ora file에서 LOG_ARCHIVE_LOG parameter를 TRUE로 정의하였음을 나타내며 false인 경우에는 DISABLED로 나타난다.
(6) initSID.ora file의 LOG_ARCHIVE_DEST parameter에서 정의한 archive 할 장소이다.
(7) 3개의 redo log중 가장 오래된 redo log의 sequence가 123임을 의미한다.
(8) 현재 사용중인 redo log의 sequence가 125임을 의미한다. 만약 이전부터 archivelog mode로 운영중이었다면 archivelog file은 log sequence 124까지 archiveing되어 있다는 것을 의미한다.
(9) Archive mode로 운영 가능하도록 변경 후 DB를 사용 가능하도록 open한다.

AUDITING 이란 무엇인가?

Bulletin no : 10196

---

1) 개 요

AUDITING 기능은 사용자의 행동을 감시하거나 데이타베이스에 관한 통계자료를 얻는 목적으로 사용된다. AUDITING 기능을 사용함으로써 누가 어떤 테
이블을 언제 사용하고, 언제 어떤 작업을 하는지를 기록할 수 있다. 하지만
레코드에 대한 AUDITING 은 불가능하며 AUDITING 을 걸게 되면 시스템에 부
하를 줄 수도 있기 때문에 자주 사용되는 기능은 아니다. AUDITING 에서 제
공되지 않는 기능은 사용자가 트리거를 만들어서 구현을 해야 한다.

2) 종 류

AUDITING 에는 3가지 종류가 있다.

STATEMENT AUDITING : 명령의 종류에 따라서 AUDITING 설정
EX)AUDIT TABLE BY SCOTT BY ACCESS WHENEVER SUCCESSFUL;
scott 유저가 내리는 명령중 테이블에 관련된 명령(create table,drop table 등)이 성공한 경우 기록된다.

PRIVILEGE AUDITING : 사용되는 PRIVILEGE 에 따른 AUDITING 설정
EX)AUDIT CREATE TABLE BY SCOTT BY SESSION;
scott 유저가 CREATE TABLE 권한이 필요한 명령(CREATE TABLE)을 내린 경우 접속된 세션에 대해서 한번만 기록된다.

OBJECT AUDITING : 적용되는 OBJECT 에 따른 AUDITING 설정
EX)AUDIT ALL ON SCOTT.EMP;
scott.emp 테이블에 대한 모든 명령(SELECT,DELETE,DROP 등)에 관한 사항이 기록된다.

설정된 AUDIT 기능은 NOAUDIT 명령으로 없앨 수 있다.
EX)NOAUDIT ALL ON SCOTT.EMP;

3)설 정

AUDIT 기능은 init.ora 화일에 AUDIT_TRAIL 파라미터를 설정하고 DB를 shutdown/startup 하면 작동한다.

AUDIT_TRAIL=TRUE, AUDIT_TRAIL=DB 로 설정이 되어 있으면 결과는SYS.AUD$ 테이블에 쌓이게 되고 AUDIT_TRAIL=OS 로 설정하면 결과가 화일로생기게 된다.

SYS.AUD$ 테이블에서 얻을 수 있는 정보는 매우 많은데 대표적인 것으로 다음을 들 수 있다.

USER NAME : 오라클 유저 이름
SESSION ID : 오라클 세션 ID

TERMINAL ID : 유닉스 시스템 상의 터미날 ID
OBJECT NAME : 테이블, 뷰 등의 이름
ACCTION ID : 명령 코드
DATE : 명령이 실행된 날짜,시간
COMPLETION CODE : 0 이면 성공, 실패하면 에러코드

ACCTION ID 는 사용된 명령을 나타내는데 이것은 숫자로 기록이 되어 있으므로 AUDIT_ACTIONS 딕셔너리를 통하여 실제의 명령으로 변환해 볼 수 있다.

4)특 징

*내려진 명령이 ROLLBACK 되더라도 AUDIT 에 의해 기록된 정보는 그대로 남아 있다.
*SYS 또는 INTERNAL 로 접속하여 행하는 명령은 AUDIT 에 의해서 기록되지 않는다.
*오라클은 AUDIT 기능의 세팅 여부와는 관계없이 STARTUP, SHUTDOWN,CONNECT INTERNAL 과 같은 정보는 OS 상의 화일로 기록을 남겨준다.
*REMOTE DB 에 대해서 행해지는 작업은 AUDIT 대상이 되지 않는다. 이것은 REMOTE DB 에서 처리해 주어야 한다.
*만약 두개의 AUDIT 가 동시에 적용되는 경우라면 하나의 레코드만 생성된다. 예를 들어 TABLE 과 CREATE TABLE 이 적용되는 경우이다.
*OBJECT AUDITING 은 모든 유저에 대해서 적용된다. 특정한 유저를 지정하지 못한다.
*BY SESSION 은 접속된 세션에 대해서 하나의 레코드만 생성하고 BY ACCESS 는 해당 명령이 내려질 때 마다 정보를 기록한다.
*DDL 을 AUDIT 하는 SYSTEM AUDIT, PRIVILEGE AUDIT 의 경우는 BY ACCESS 가 디폴트 이고 그외의 경우는 BY SESSION 이 디폴트이다.

BITMAP INDEX 생성과 SPACE 비교

Bulletin no : 10319

. Bitmap index 의 생성 방법은 다음과 같다.

```
SQL>  create bitmap index id_dept  on dept (dname);
```

. Explain plan 의 결과

```
select /*+ index( dept ID_DEPT ) */ *
from dept where dname = 'sales'

Rows     Execution Plan
-------  ---------------------------------------------
      0  SELECT STATEMENT   GOAL: CHOOSE
      1  TABLE ACCESS (BY ROWID) OF 'DEPT'
      1  BITMAP CONVERSION (TO ROWIDS)
      2  BITMAP INDEX (SINGLE VALUE) OF 'ID_DEPT'
```

. Index space 이용도 비교

. bitmap index 도 B-tree index 와 같이 where 절에 주로 나타나는 column 을 지정하여 생성한다.
. single bit 에대한 lock 방안이 없어 row level lock 대신 block level lock 이 적용되므로 OLTP 전용 업무에서는 불리 하다.
. bitmap index 는 많은 DML load 가 있는 table 은 불리하다

Indexed column	Cardinality of column indexed	Index Size (in MB) conventional B-tree index	bitmap index
KSEQ	1000000	14.33	23.90
K500K	500000	15.29	12.35
K250K	250000	15.27	8.46
K100K	100000	15.21	5.25
K40K	40000	15.06	3.75
K10K	10000	14.34	2.99
K1K	1000	14.26	2.25
K100	100	13.40	1.38
K25	25	13.40	1.31
K10	10	13.40	1.11
K5	5	13.40	0.78
K4	4	13.40	0.67
K2	2	13.40	0.37

BITMAPPED INDEX

Bulletin no : 10206

최근에 각광받고 있는 Datawarehousing환경 과 End-User-Computing환경등에서 필요한 RDBMS의 첨단 Indexing Access기법으로 경쟁사에서는 Bitwised Index를 발표하고 있습니다. 이 기법은 Oracle Server V7.3에서도 Bitmapped Index라는 이름으로 발표될 예정이므로 이에 대한 정확한 이해를 위해 다음사항을 기술해 보았습니다.

1. Bitmapped Index란 ?

RDBMS의 Table로부터 특정 자료의 검색을 위해 기존의 일반 RDBMS에서는검색효율의 향상을 위해 B-Tree Index를 구현하여 사용하여 왔습니다. 그외에도 B-tree Cluster Index, Hash Cluster Index등을 이용하여 대부분의 검색효율을 보장하여 왔지만 B-Tree Access를 위한 특성로 인해 몇가지 어려움을 안고 있었습니다.

1) B-tree Index Access 방식을 위해선 실지 조건비교되는 Column값에 대한Table의 원시값을 Index에도 보관하고 있어야 함으로 인해 Index를 위한 실지 data의 중복 저장으로 저장공간의 낭비를 감수하여야 으며, 특히 현실 상황에서 자주 사용케 되는 여러 컬럼의 Concatenated Index (결합 인덱스) 에서 자료량이 많은 Table에 대해서는 크나큰 부담이었습니다.

2) Index설정 Column값의 분포도(Unique성)가 넓은 Column에 대한 (예:성별)조건검색시에는 Index access를 했을때 보다 오히려 Full Table Scan방식이 오히려 빠른 성능을 보장하기 때문에 RDBMS의 Cost Based Optimizer는 Random access비용이 많이드는 Index access를 포기하고 빠른속도의 순차 Table Scan을 하여 높은 Hit Ratio를 보장하려 하는 경우를 우리는 흔히 보게됩니다.

3) End-User-Computing 환경에서 흔히 예상되는 복잡한 질의조건으로 인해 Optimizer가 최적 실행계획에서 Index를 포기하게되는 경우를 초래합니다. (예: 복잡한 OR 연산자등)

이러한 상황을 해결하기 위한 새로운 Index Access 방법 로 Bitmapped Index가 새롭게 등장하게 되었습니다.

다음 상황을 통하여 구체적 설명을 제시합니다.
예) 한국오라클(주) 에서 관리하고 있는 고객정보 Table이 다음과 같이 정의 되어있다고 가정합시다.

고객번호	혼인여부	성별	중요등급
1001	결혼	여	하
1002	미혼	남	상
1003	재혼	남	중

상기 Column들의 속성을 보면 고객번호는 유일한 값들이며 B-Tree Index로 탁월한 조회성능 보장.
혼인여부(3종류),성별(2종류),중요등급(3종류) 은 30 - 50%의 넓은 분포도를 갖는 값들로서 B-Tree Index사용시 효율성이 거의 없슴.
즉, B-Tree Index에서는 자료의 값에 따라 Sort하여 이분식(Binary) 가지치기(Leaf & Node) 구조를 유지하여, 조건에 들어오는 값에따라 검색자료 범위(Access Range)를 좁혀어 나가는데 반해, 저장되

어 있는 자료가 동종(같은값)이 많다면 해야할일(Access Range)이 줄지 않고 비용이 많이드는 Random Access(Index에서 Rowid를 가지고 실 Table Data를 찾을때)만 많아지고, Scan해야할 자료가지(Leaf) 수는 쉽게 줄지않게되어 B-Tree Index방식으로는 빠른 성능을 보장하기가 어렵습니다.

그렇다면 다음과 같이 분포도가 좋지않은 중요등급 Column을 몇 개의 Bit들만 가지고 정보를 저장키로 하여봅니다.

중요등급 = '상'	1	0	0	0	0	0
중요등급 = '중'	0	1	0	0	1	1
중요등급 = '하'	0	0	1	1	0	0

*혼인여부(3종류),성별(2종류)도 같이 적용하였다고 가정.

이런 상황에서 아래의 SQL을 실행한다고 가정하여 봅니다.

```
Select count(*) from 고객정보
 where 결혼여부 = '미혼'  and 중요도 in ('상','중');
("미혼자 인 고객중에 중요도가 '상' 이거나 '중'인 고객은 몇 ?")
```

이 SQL을 실행하기 위한 Optimizer가 이러한 Operation을 다음과 같이 결정한다면 훨씬 빠른속도의 성능을 낼 수 있을 것 입니다.

혼인여부 = '미혼' AND (중요도 = '상' OR 중요 = '중')
즉, '011001' AND ('100000' OR '010011')
다시 풀어보면 '011001' AND '110011' 이고 결과는 '010001'이다.
Optimizer는 두개 컬럼의 논리연산결과가 '010001'인 Row들을 찾아 건수를 세면 될 것 입니다.

상기와 같이 논리연산과 Bit처리 방식은 Computer에게 훨씬 쉽게 처리할 수 있는 길을 만들어주게 되면서, 몇종류 안되는 값들과의 별도의 조그만 연결표(Bitmap)와 몇Bit만의 저장공간만을 가지고 Index 구성을 가능케 함으로써 공간의 절약효과도 상당할 것이다.특히, Data warehousing, Decision Support System 에서와 같은 방대한 정보량에서 유용할 수 있습니다.

2. Bitmapped Index의 장단점 ?

Index를 사용하는데 있어서 우리는 몇가지의 고려사항을 검토해야 합니다. 조회성능측면, 저장공간측면, 유지관리측면 정도를 고려할 수 있습니다. 앞으로, 이러한 측면에서의 Bitmapped Index특성을 설명하기로하겠습니다. 우선, Bitmapped Index의 장,단점을 간추려보면 다음과 같습니다. 장점으로는 아주적은 Index저장공간을 사용하여 좋 않은 분포도의 값을 갖는 다량의 자료를 빠른속도로 Access 할 수 있다는 것과 복잡다양한 조건에 대해 Index Access Path를 적용할 수 있다는 점이 있는
반면, 조건 유형이 Pattern Match형태가 자주 사용될시 효용성이 극소화되며, B-Tree Index와 같이 모든 Query에 대해 Index Path로 사용될 수 없다는 단점이 있습니다. 한예로 Insert, Update, Delete 와 같은 Query에서는 무의미 합니다.
Bitmapped Index는 다음과 같은 측면에서 각각 유용한 이점이 있습니다.

조회성 측면
- 각각의 독립적인 Column들에 대하여 여러유형 (AND,OR...)의 조건절에 대해 Index가 사용 되어지게 하는 규칙이 불필요하다.(ad hoc query)

- 분포도가 나쁜 값에 대한 Index Access가 빠름.
- Index Column으로 추가 필요시 독립적으로 추가 및 적용 가능.
- 복잡하게 길어지는 조건절에서도 모두 유효하게 작동된다.(복잡한 질의 및 ad hoc query에서 유용)
- 특히 다량의 자료에 대한 계 질의(aggregate query)에서 탁월. (예:COUNT operator)
- 분포도가 좋은 (Unique성:값의종류가 많다) 값에대한 Index는 불리.

저장공간측면

- Index에 가지고 있어야 될 자료값 에대한 공간 절약.

. 하지만, Bitmapped Index로 적용하게될 Column의 특성상 실제 값의 Size도 크지않다 예를들어 '남' 과 '여'값의 겨우 2Byte 이하일 것이다. 즉, Bitmapped Index적용 Column의 후보는 대개 5가지 정도 이내의 값을 갖는 경우가 되므로 실지 값의 Size도 작게됨.고로, 이러한 측면에서의 저장공간 절약 측면 보다는 실제 현업상 요구되는 Index구성에서 예를 들면 더나은 이해가 될 것 입니다.

예) 기존에 고객정보 검색속도를 위해 Index1을 다음과 같이 구성 하였다고 가정 하겠습니다.

B-Tree Index1 = (혼인여부,성별,중요도)

이경우는 Where절에서 혼인여부가 조건에 오지않는 경우나 조건절에 성별만 혹은 중요도만을 가지고 검색할 경우에는 Index1을 사용할 수 없기에 Index2(성별,중요도,혼인여부),Index3(중요도,성별,혼인여부)등의 추가 Index가 있어야 만족한 성능을 낼 수 있었다면 거의 10 - 100배의 공간절약을 하며 Bitmapped Index1(혼인여부),Index2(성별),Index3 (중요도)등의 독립적인 3개의 적은공간으로 만족한 성능을 보장할 수 있다. 이러한 측면에서의 이점이 클 것 입니다.

유지관리측면

- Bitmapped Index는 Decision Support System과 같은 조회전용 업무나 OLTP업무 비중이 작은 업무에서 적합.
- 아직 Single Bit에 대한 Lock방안이 없고, Bitmapped Index에서 Row-Level Locking대신에 Block-Level Locking이 적용되기 때문에 OLTP전용 업무에서는 Lock Contention 및 Deadlock 가능성 많음.
- Bitmapped Index에서 Update등의 Transaction은 Block level Lock을 사용하여야 하므로 Oracle의 기본Locking 인 Row level Lock 사용할 수 없게 되며 결국 Oracle block (예:2KB)내의 한Row에 대한 변경이 필요시(Update등)에는 해당 Block전체가 Locking되므로 잦은 변경이 예상되는 OLTP업무에서는 큰부담이 되므로 Batch성 Bulk Operation 이 가미되어 운용할 수 있는 방안이 필요.

예) Data warehousing에서는 주업무가 조회이며, 자료변경 추가시는 대부분 Batch성으로 처리하는 사례에서 Batch작업시 Bitmapped Index들을 Disable시키고 처리후 Enable하 것이 바람직.

3.Oracle's Bitmapped Index의 특성 ?

Oracle에서의 Bitmapped Index는 다음의 이점을 제공합니다.

검증된 기술

- 이미 Oracle7의 Text Server에서 Bitmapped Technology를 적용하여 사용하여 왔으며 이러한 기술을 Oracle7 Release7.3에서 Production 으로 제공합니다.

통합기능으로 제공

- 별도의 기능옵션 추가없이 Oracle Server에 통합되어 제공됩니다.
- 기존 사용 모든 환경과 수정없이 통합 적용이 가능합니다.
  (예: DB Trigger, Distributed, Parallel, Integrity constraints...)
- 각종 SQL, Tool 및 Utility, Application등에서 수정이 필요없습니다.

병렬 Index 생성 지원

- 방대한 자료에 대한 Index생성시 병렬수행으로 Bitmapped Index생성

압축기능
- 저장공간을 현저히 줄일 수 있게 구현된 압축기법을 제공합니다.

입력/수정/삭제 지원
- Oracle의 Bitmapped Index는 Insert,Update,Delete 를 지원합니다.
  (은행 계정업무 와 같은 무리한 OLTP transaction이 아닌 하루업무 중에 약 10% 이내의 자료변경 업무는 그대로 사용 가능)
- 사용자로 하여금 가끔 유발되는 변경에 대해서 Bitmapped Index를 재생성이 불필요하게 투명하게 사용가능.

병렬 Index 검색 지원
- 대량의 Table에 대한 검색시 병렬로 Bitmapped Index 검색 가능.
여러컬럼에 대한 Bitmapped Index 지원
- 분포도가 나쁜 몇개의 컬럼이 항시 같이 조건에 오는 경우 B-Tree Index에서의 Concatenated Index 같이 Multicolumn Bitmapped Index 사용가능.

유연성 보장
- 하나의 Table에 B-tree Index와 Bitmapped Index를 동시 혼용가능.
  (Clustered Index도 혼용 생성 가능)

Oracle에서의 테스트 예제

Oracle에서의 Bitmapped Index의 특성을 확인할 수 있는 예시 입니다.

고객정보자료는 약100만건의 자료로 구성되어 있으며 현재 고객번호는 유일한 값으로 구성되며 Primary Key로 선언되어 있습니다. 그러나, 성별 과 중요도, 혼인여부 컬럼의 값은 2가지 내지는 3가지 종류이어서 B-Tree Index로는 검색수행 속도를 보장할 수 없는 상황임.

1) Select * from 고객정보 where 성별 = '남';
   본 SQL을 Optimizer는 최적의 Execution Plan으로 Parallel Full Table Scan을 선택하게 됩니다. (전체의 50%이므로)

2) Select * from 고객정보 where 고객번호 = 1001;
   본 SQL을 Optimizer는 최적의 Execution Plan으로 Unique B-tree Index를 Access 하여 Rowid를 가지고 Table 을 1번 Scan.

3) Select * from 고객정보
   where 성별 = '남' and 중요도 in ('상','중');
   본 SQL을 Optimizer는 최적의 Execution Plan으로 Bitmapped Index 를 선택하게 됩니다.

4) Select * from 고객정보
   where 성별 = '남' and 고객번호 = 1001;
   본 SQL을 Optimizer는 최적의 Execution Plan으로 B-tree Index를 Range Scan하는 방 을 선택하게 됩니다.
   (성별 Bitmapped Index 보다는 Unique한 고객번호 Index가 효율적)

상기와같이 Oracle 에서의 Bitmapped Index는 Cost Based Optimizer에 의해 투명하고 유연성있게 수

행속도를 보장하게 되며, 필요시 Oracle의 특징인 Optimizer 취사 선택기능으로 사용의도에 맞추어 사용이 가능합니다.

결 론

Data Warehousing 또는 Decision Support System에서의 필요한 요소인 Oracle의 Bitmapped Index는 Release7.3 Server에 탑재되게 되었으며 기존 RDBMS에서의 조회 수행속도의 몇가지 걸림돌을 해결케 하였고, 또한 방대한 자료량에 대한 Index의 저장공간 절약 및 End-User-computing에 꼭 필요한 Indexing 구현기법입니다.
단, Bitmapped Index만이 지상 최대의 해법이 아니란 것도 우리는 알아야 하며 사전에 OLTP에서의 부담감에 대한 분석이 필요 며, 과연 나쁜분포도에서 저장공간에 대한 낭비에는 많은 관심이 없고 단지 좋은 조회속도를 보장키 위한다면 이미 기존 Oracle7에서부터 사용해온 Clustered Index를 검토해 보는 것이 타당하리라고 사료됩니다.

CONNET INTERNAL에 대해서 PASSWORD 설정하는 방법

Bulletin no : 10240

---

connect internal 시에 password 를 입력하도록 하려면 다음과 같이 하면 됩니다. DB Extender와 같은 Client Tool을 이용하여 서버에 접속하는 경우에도 마찬가지로 적용됩니다.

1)orapwd를 이용해서 password file을 만들어 줍니다.

```
orapwd file=orapwORA7 password=manager entries=1
```

이 화일은 $ORACLE_HOME/dbs 디렉토리에 만들어 주며 ORACLE SID가 ORA7인경우 화일 이름은 orapwORA7 으로 해 줍니다

2)initORA7.ora 화일에서 다음을 추가합니다.

```
remote_login_passwordfile=exclusive
compatible=7.1 또는 compatible=7.2
```

이렇게만 해 주면 connect internal 시의 password가 manager로 세팅됩니다.

이것은 Sun Solaris 2.4, Oracle 7.2.2 에서 테스트 해 본 것인데 다른 기종에서는 방법이 다를 수도 있습니다. 자세한 방법은

```
Oracle7 Server Documentation Addendum(Release 7.1) Chap.5
```

을 참조하면 됩니다.

DB Extender에서 접속할 때 ORA-9910 에러가 발생하는 경우도 똑같이 이 방법대로 password file을 만들어 주면 됩니다. 단, 이 경우에는 initORA7.ora 화일에 remote-login_passwordfile 을 세팅할 필요까지는 없고 compatible 만 세팅하고 password file을 만들어 주면 됩니다.

CONTEXT SIZE & CURSORS (ORA-1046,1050,1051)

Bulletin no : 10208

---

1. Context size 에 관련한 error message

.ora-1046 :can't acquire space to extend context area.
.ora-1050 :can't acquire space to open context area.
.ora-1051 :maximum context area extents exceeded.

2. Context size란 무엇인가?

(1) 쉽게 말하면 Cursor의 initial size이다 . 즉, Cursor 에 allocate 되는 user memory 이다.
(2) 이는 init.ora 의 CONTEXT_SIZE 에 의해 결정된다.
(3) Cursor 에 할당되는 additional space 는 CONTEXT_INCR 에 의하며 50 extents 를 갖는다.
(4) Recommended context size increment 는 4096 bytes(4K) 이다.
(5) SQL statement가 수행시마다 cursor 가 open 되며,같은 cursor가 reuse 되도록 design 되어 SQL*PLUS session은 2-3 개 이상 open 되어지지 않는다 그러나 SQL*FORMS 는 여러 다른 task 를 수행하므로 많은 cursor를 open한다 .(100 or more)

(6)Cursor 가 hold 하는 item
* the SQL statement
* the parsed SQL statement
* one row of the result

3. ORA-1051 은 무엇이 문제인가?

(1) cursor 의 size 를 줄인다
(2) CONTEXT_SIZE,CONTEXT_INCR 를 늘린다.

4. OPEN_CURSORS 수를 줄이는 전략

(1) Commit을 자주한다.
(2) Synonym이나 view 를 사용하지 않음으로써 implicit cursor 수를 줄인다.
(3)SQL*FORMS 에서 select 문대신 #COPY 로 바꿔 사용한다.
(4)SQL*FORMS 에서 large forms를 여러개의 작은 forms 로 나눈다
(5)ASAP, EXEC SQL CLOSE C1; 을 수행한다.
(6)HOLD_CURSOR=NO & RELEASE_CURSOR=YES 를 사용한다.

CONTROLFILE 재생성하는 방법

Bulletin no : 10069

---

1. CONTROL FILE 은 다음 경우에 다시생성한다.

- CONTROL FILE 이 손상되었으나 이용가능한 BACKUP FILE 이 없을 경우
- 데이타베이스 이름을 변경 할 경우
- DATA FILE , LOG FILE 의 MAX 갯수를 확장할때

CONTROL FILE 을 다시 생성하는경우 ONLINE REDO LOGILE 및 DATA FILE 이 손상될 경우가 있으므로 CONTROL FILE 을 생성하기전에 모든 데이타베이스의 FULLBACKUP 을 실행한다.

CONTROL FILE 을 재 생성 하기위해서는 DATABASE 에 관계되는 모든 FILE NAME 을 알아야한다. 다음 SQL 문을 DB 구조가 변경될때 즉 DATA FILE을 추가하거나 DROP 할때 실행하여 DATA FILE 과 LOG FILE 이름을 항상 알아둔다.

```
SQLDBA > SELECT * FROM V$DBFILE;

FILE#  NAME
-----  ------------------------------
    1  /usr/oracle/dbs/systORA7.dbf
    2  /usr/oracle/dbs/rbsORA7.dbf
    3  /usr/oracle/dbs/tempORA7.dbf
    4  /usr/oracle/dbs/userORA7.dbf

SQLDBA > SELECT * FROM V$LOGFILE

GROUP#  STATUS  MEMBER
------  ------  ------
     1           /usr/oracle/dbs/log1ORA7.dbf
     2           /usr/oracle/dbs/log2ORA7.dbf
```

2. CONTROL FILE 생성 절차

1) DATABASE 을 SHUTDOWN 한다.

```
SQLDBA > SHUTDOWN;
```

2) 모든 DATABASE FILE 을 BACKUP 받는다.(DATA FILE, ONLINE REDO LOG FILE)

3) INSTANCE 를 START 한다.

```
SQLDBA > STARTUP NOMOUNT
```

4) CONTROL FILE 을 생성한다.

```
SQLDBA > CREATE CONTROLFILE  [REUSE ]
         [SET] DATABASE ORA7
         LOGFILE '/usr/oracle/dbs/log1ORA7.dbf',
                 '/usr/oracle/dbs/log2ORA7.dbf',
                 '/usr/oracle/dbs/log2ORA7.dbf'
         NORESETLOGS [ RESETLOGS ]
         DATAFILE
                 '/usr/oracle/dbs/systORA7.dbf',
                 '/usr/oracle/dbs/rbsORA7.dbf',
                 '/usr/oracle/dbs/tempORA7.dbf',
                 '/usr/oracle/dbs/userORA7.dbf',
                 '/usr/oracle/dbs/dataORA7.dbf'
         MAXLOGFILES 10
         MAXDATAFILES 60;
```

[REUSE] : 기존에 CONTROL FILE 이 존재하는 경우에 사용. 생략하면
?/dbs 디렉토리에 CONTROL FILE 이 없어야 한다.
[SET] : 데이타베이스 이름을 변경할때 용

5) ?/dbs/initORA7 혹은 ?/dbs/config.ora (ORACLE7 의 경우) FILE 내의 CONTROL_FILES PARAMETER 에 새로 생성된 CONTROL FILE 을 추가한다.

```
CONTROL_FILES = (/usr/oracle/dbs/ctrl1ORA7.ctl,
                 /usr/oracle/dbs/ctrl2ORA7.ctl)
```

6) 데이타베이스를 MOUNT, RECOVER, OPEN 한다.

```
SQLDBA > SHUTDOWN
SQLDBA > STARTUP MOUNT
```

[NORESETLOGS] OPTION 을 사용한경우

```
SQLDBA > RECOVER DATABASE
  Media recovery complete.
SQLDBA > ALTER DATABASE OPEN;
```

[RESETLOGS] OPTION 을 사용한경우

```
SQLDBA > RECOVER DATABASE
         USING BACKUP CONTROLFILE
         UNTIL CANCEL;
```

incomplete cancel-based recovery 를 실행한다.

```
SQLDBA > ALTER DATABASE OPEN RESETLOGS;
 Statement processed
```

CORE DUMP 에 대한 설명

Bulletin no : 10311

---

1. 서론

여기서는 오라클 product 가 발생하는 core dump 로 부터 stack trace 를 모으는방법을 설명 하고자 한다.

1) 'core dump' 는 무엇인가?
core dump 는 process 들의 image copy 이다.이것은 file 의 형태로 구성되며 현재 수행중인 directory 에 발생한다

2) 발생되는 원인

원인은 여러가지가 있을 수 있으나 대체적으로 임의의 process 가 OS가 원하지 않는 일을 수행하려 할때 발생한다.
공통적으로
. program 이 자신의 영역밖의 memory 를 access 하고자 시도할 때 .
. program 이 고갈되었거나 사용이 불가능한 resource 얻고자 할 때.
. Illegal instructions 에 의해 무언가가수행되려 할 때 .

Unix systems 에서는 process의 불법적인 행위를 금하고자 signal 을 보내는데 이것은 user가 이러한 signal 을 manual 하게 보냄으로써 발생시킬수도 있다.

2. 처리 방안 ?

1) 우선 각 product 의 full version 을 check 하여야 한다.
RDBMS,PL/SQL , 각 연관된 product 의 version 을 check 한다. - Eg: Forms and Menu 가 연결되었다면 SQL*Menu version 도 check 한다.

2) 수행하고자 한 정확한 command 를 check 한다.
Eg: 만약 SQL*Forms 문제이면 'mrunform30' 으로 note 한다.

3) 다음을 순서대로 수행한다.
(1) 해당 command 를 수행했던 directory 또 ORACLE_HOME/dbs 또는 $ORACLE_HOME/dbs/core_NNNN 에서 'core' file을 check 한다.

(2) ORACLE id 로 login 을 해서 $ORACLE_HOME/bin directory 로 간다. 그리고 다음 command를 수행한다.

```
$ file program
```

(여기에서 program 은 core dump 를 발생시킨 command이다)

만약 이 command의 결과가 'dynamic' 또는 'dynamically linked' 이면 해당 executable을 relink 후core를 다시 생성한다 .

오라클은 dynamic link 를 support 하지 않기때문이다.

(3) $ chmod +r program (read permission 부여)
(4) Logout 후 error 가 발생했던 user로 login 한 후 다음 command를 실행한다.
이 command는 UNIX version 에 dependent 하다

Command	NB	Exit command	Stack Trace command
dbx		quit	where
xdb	(HP ONLY)	quit	t
sdb		q	t
adb		$q (or Ctrl-D)	$c

우선 해당 directory 로 이동 후

DBX :
```
$ script /tmp/mystack
$ dbx $ORACLE_HOME/bin/program core
        (dbx) where
        ...                 << Stack should appear here
        (dbx) quit

$ exit
```

XDB:
```
$ script /tmp/mystack
$ xdb $ORACLE_HOME/bin/program core
        (xdb) t
        ...                 << Stack should appear here
        (xdb) quit
$ exit
```

SDB:
```
$ script /tmp/mystack
$ sdb $ORACLE_HOME/bin/program core
        (sdb) t
        ...                 << Stack should appear here
        (sdb) q
$ exit
```

ADB:
```
$ script /tmp/mystack
$ adb $ORACLE_HOME/bin/program core
$c      << NB: adb has no prompt so just enter $c
        ...
$q
$ exit
      ...
$q
$ exit
```

(5) 만일 debug command 가 실패하면 위 list 의 다른 debugger 를 사용해 다시 실행함.
만약 모든 debugger 가 실패하면 file type 의 permission 에 문제가 있는경우 이다.

system 에따라 만들어지는 core file의 maximum size 를 control 하는 kernel parameter 가 있는데 값이 너무 작으면 core file 이 쓸모가 없고 다시 재현하는 것이 불가능할 수 있다.

이는 다음 command 로 check 해볼수 있다.

```
        limit          in the C shell
   OR   ulimit -a      in the Bourne / Korn shells.
```

5) mystack 이라는 trace file 이 생성되면 적당한 exit command 를 실행한다

6)stack trace file 을 print 해서 본사에 reporting 한다.

CORRUPTED BLOCK을 가진 OBJECT 찾는법

Bulletin no : 10059

ORA-1578 : FILE ID # 6, BLOCK # 82 라는 ERROR 가 나올 경우 해당되는 OBJECT를 찾아야 한다.
다음과 같은 SQL문으로 해당 OBJECT를 찾아서 RECOVERY 를 실행한다.

```
$ sqldba lmode=y
  sqldba > connect internal
  slqdba > select * from dba_extents
           where file_id = 6 and
           82 between block_id and block_id + blocks -1;
```

COST BASED OPTIMIZER 와 RULE BASED OPTIMIZER

Bulletin no : 10239

Prior to 07.3 the CBO still behaves somewhat erratically by making poor execution plan choices and actually slowing down the query compared to the RBO implementation. The use of SQL HINTS is strongly encouraged to help the CBO make "more informed" decisions about the choice of execution plans. The general reaction of the audience was that CBO "has a long way to go" before becoming a mature, reliable product. I would recommend that you suggest to your customer that they approach the CBO carefully by setting up various tests to compare RBO and CBO performance. You should encourage the customer to begin experimenting with CBO and tell them that by the time they upgrade to 07.3 the product will have matured into a dependable tool that will produce quality execution plans. In my opinion, some of the negative feedback from customers using CBO occurred because they did not take the time to learn how it works and how best to use it.

1.) The RBO will be supported in Oracle8, but will not be enhanced and will eventually be desupported.

2.) PL/SQL V1 (which is the PL/SQL engine linked with most tools) is basically an Oracle 6 product and does not contain all the logic to support CBO. Only PL/SQL V2 understands the CBO concept of HINTs.

3.) The CBO does not understand the characteristics associated with an application, nor can it fully understand the impact of complex relationships between joined tables (the cost of the analysis would far out-weigh the benefit). Therefore, hints may be required to ensure a specific behavior ofthe CBO.

4.) The CBO is still relatively immature and (under some conditions) improperly estimates the cost of some query components.

5.) The CBO assume evenly distributed values and therefore does not always estimate selectivity correctly. In the case of bind variables, selectivity is assumed to be some fixed value (maybe 25%?). Oracle 7.3 will address some of this by utilizing column histograms.

6.) Use of the LIKE predicate sometimes results in unexpected execution plans. In particular, the CBO does not propagate LIKE-predicate transitivity.

7.) Application queries should be tuned and monitored for performance degradation, regardless of the optimizer type being used.

8.) All things being equal, the CBO will select a driving table working from left-to-right in the FROM list. This is just the opposite of the RBO behavior.

9.) By default, the CBO optimizes for throughput (overall resource consumption), NOT response time. Response time should not be confused with throughput.

10.) Generally speaking, interactive/Forms-based applications with canned queries should be tuned for response time (OPTIMIZER GOAL=FIRST_ROWS).

11.) Observation of CBO behavior suggests that SORT-MERGE Joins are often favored over NESTED-LOOP Joins. This has a negative impact on the response time for the first row(s).

12.) Accurate statistics for all tables & indexes are a MUST for proper behavior of the CBO. (The CBO will make hard-coded assumptions for statistics that are not provided).

13.) If OPTIMIZER_MODE is set to CHOOSE, the CBO will be used if ANY of the tables in the query has statistics. The CBO will optimize for throughput, NOT response time.

14.) If OPTIMIZER_MODE is set to RULE, the RBO will be used regardless of the presence of statistics.

15.) If OPTIMIZER_MODE is set to FIRST_ROWS or ALL_ROWS (or if the session OPTIMIZER_GOAL is set to FIRST_ROWS or ALL_ROWS), then the CBO will be used regardless of the presence of statistics. FIRST_ROWS is optimized for response time, and ALL_ROWS has a goal of optimizing throughput.

16.) Whenever possible statistics should be updated by using the ANALYZE...COMPUTE command (the ESTIMATE option had some problems in earlier releases of Oracle7).

17.) Performing an ANALYZE operation on a table automatically cascades to all related indexes.

18.) If tables are too large to allow a full ANALYZE... COMPUTE, statistics should be updated with ANALYZE... ESTIMATE using at least 20-25% of the table. A full ANALYZE... COMPUTE should be performed on the INDEXES following the ANALYZE... ESTIMATE operation on the table.

19.) The ANALYZE command should be performed regularly on dynamic tables. The frequency of ANALYZE operations should be commensurate with the dynamics of the table.

20.) When formatting trace files using TKPROF, it is important to understand that optionally selected EXCUTION PLANs are derived using the current optimizer environment (the plans are NOT stored in the trace file). Settings such as OPTIMIZER_MODE should be consistent with the settings at the time the trace file was created.

21.) For complicated multi-table joins, the number of execution plan permutations can be very large. Therefore, though Oracles documentation might suggest otherwise, the CBO uses heuristics to estimate certain costs and to prune execution plan options. (The order of the tables in the FROM clause can therefore have an impact on the final execution plan selected).

22.) PL/SQL blocks executed from SQLPLUS and PL/SQL stored procedures use an optimizer goal of CHOOSE when the session has been altered to a goal of FIRST_ROWS. I assume this is a bug. This prevents optimization for time unless you provide a specific HINT.

23.) There are some known bugs associated with the optimizer goal of FIRST_ROWS and queries that have ORDER BY -- in some cases the optimizer may not use an available index in lieu of a SORT operation. (see bugs 211500, 247561)

24.) Since Oracle Forms uses the PL/SQL v1 engine, HINTs are stripped from the SQL statement. There is a block property to supply a HINT. Additionally, the session may be altered to optimize for response time (FIRST_ROWS goal for all queries) by invoking the FORMS_DLL built-in: FORMS_DLL(alter session set optimizer_goal=FIRST_ROWS).

25.) Optimizer Goal can be set at an instance level in both Oracle 7.0.16 and 7.1.4. Though the feature is not documented in 7.0.16, the paramter OPTIMIZER_MODE=FIRST_ROWS can be used (but not officially supported).

26.) There is a bug associated with the ANALYZE ... ESTIMATE option that sometimes results in division by zero errors. It appears that this problem is most likely encountered when estimating statistics on smaller tables.

27.) There is a bug in pre-7.1 releases of Oracle7 that can cause the CBO to return no rows, but the RBO will correctly return n rows. The bug number is 187592 and is a duplicate of 184630. Basically, queries with outer joins involving more than 3 tables may return incorrect number of rows when using the cost based optimizer. Use the ORDERED hint as a possible work-around.

28.) Most of these issues relate to Oracle 7.0.16 and 7.1.4. My understanding is that some of the CBO problems and BUGs are being fixed in Oracle 7.2.

29.) Any suspected problems should be documented in an Oracle TAR. WWS should open a BUG for any newly discovered problems. Problems that don't get BUG'ed don't get fixed.

One other thing. If hints are used and are specified incorrectly (ie. improper syntax) or the hint makes no sense (improper semantics) then the hint is simply ignored without any error msgs or other feedback to the user.

DATA BLOCK CORRUPTION :
ORA-1578 해결방법

Bulletin no : 10062

모든 오라클 데이타 블럭은 sequence 번호(seq)와 incarnation 번호 (inc)를 갖고 있다. ORA-1578 에러는 seq=0 이고 inc<>0(새로운 블럭이 아님)일 때 발생한다. ORA-1578 에러는 ORA-600[3339] 에러와 함께 발생하곤 한다.

ORA-1578 에러가 발생하면 corruption 발생한 화일번호와 블럭번호를 알려준다. 여기서는 이 때의 화일번호를 f, 블럭번호를 b 라고 부르기로 한다.

해결방법

1)우선 해야 할 일은 어떠한 오브젝트가 corrup 되었는가를 알아내는 것이다.
다음의 스크립트를 이용하면 알 수 있다.

```
SQL>select segment_name, segment_type
    from dba_extents
    where file_id = f and
    b between block_id and block_id + blocks - 1;
```

2)만약 해당 세그먼트가 인덱스이면 drop 시키고 다시 생성하면 된다.

3)만약 해당 세그먼트가 테이블이면 corrup 블럭의 데이타는 손상된 것이다.

4)만약 해당 테이블이 들어있는 엑스포트 화일이 있다면 손상된 테이블을 drop 시키고 임포트 받는 것이 제일 간단한 방법이다. 하지만 만약 엑스포트 받은 화일이 없거나 백업해둔 화일도 없다면 해당 테이블에 인덱스가 생성되어 있는 경우에 한해서 다음의 방법을 사용해서 복구를 하도록 한다.

5)만약 해당 테이블에 대한 인덱스가 생성되어 있다면 이를 이용해서 corrup 된 블럭을 피해갈 수 있다. 이 방법은 다음과 같다.

- empno, ename, deptno 를 컬럼으로 가지는 emp 테이블이 corrup되었다고 가정하자. 그리고 empno컬럼에 인덱스가 생성되어 있다고 하자.클러스터화되지 않은 모든 테이블은 유니크한 rowid를 가진다. rowid를 varchar2/hexadecimal 형식으로 표현하려면 rowidtochar 함수를 이용한다.

```
SQL>select rowidtochar(rowid) from emp;
```

rowid는 총 18자로 블럭어드레스(8자), 점(1자), 로우 어드레스(4자), 점(1자), 화일 어드레스(4자)로 구성되어 있다.

```
SQL>select empno, rowid
    from emp
    where empno > 0
```

위의 스크립트를 실행시키면 다음과 같은 결과를 얻게 된다.

```
EMPNO          ROWID
-----          ------
100            00000003.0000.0006
101            00000003.0001.0006
102            00000003.0002.0006
103            00000003.0003.0006
.
.
500            00000004.0000.000A
501            00000004.0001.000A
.
.
755            0000001A.0005.000A
756            0000001A.000C.000A
```

만약 인덱스가 character 컬럼에 대한 것이었다면 위의 query 문장을 다음과 같이 바꿀 수 있다.

```
SQL>select empno, rowid
    from emp
    where empno > '';
```

예를 들어 다음과 같은 에러메시지가 떨어졌다고 하자.

01578, 00000, "ORACLE data block corrupted (file # 10, block # 4)

그러면 다음의 스크립트를 사용하여 손상된 블럭에 있는 employee 에 대한 empno를 구할 수 있다.

```
SQL>select empno from emp
    where empno > 0
    and rowidtochar(rowid) like '00000004.%.000A';
EMPNO          ROWID
-----          -----
500            00000004.0000.000A
501            00000004.0001.000A
```

이제 emp 테이블과 같은 구조를 갖는 새로운 테이블을 만든다.

```
SQL>create table temp
    as select * from emp
    where 1 = 2;
```

그리고는 손상된 부분을 피해서 새로운 테이블에 손상된 테이블의 데이타를 추가한다.

```
SQL>insert into temp select * from emp where empno < 500;
SQL>insert into temp select * from emp where empno > 501;
```

손상된 테이블을 drop 시키고 temp 테이블의 이름을 emp로 변경한다.
그리고 백업된 자료나 문서자료를 통하여 손상된 부분에 대한 정보를 추가한다.

6)손상된 블럭에 여러개의 로우가 존재하고 있다면 다음의 방법을 이용한다.

```
SQL>create table empnos as
    select empno from emp
    where empno>0
    and rowidtochar(rowid) not like '00000004.%.000A';
```

이 스크립트를 이용하면 손상된 블럭에 포함되지 않는 empno 들을 알수 있다.
다음의 스크립트를 계속 실행시켜 복구를 한다.

```
SQL>create table temp  as select * from emp  where 1 = 2;
SQL>insert into temp
    select emp.empno, emp.ename, emp.deptno
    from emp, empnos
    where emp.empno > 0
    and emp.empno = empnos.empno;
```

7)만약 데이타 딕셔너리의 테이블이나 인덱스에서 손상된 블럭이 발생했다면 지원을 요청해야 한다.

DATABASE INTERNAL RECOVERY

Bulletin no : 10149

< Internal parameters >

_allow_resetlogs_corruption = true
. avoid rolling forward
_offline_rollback_segments=r01,..
. take the rollback segments offline during transactions recovery.
. temporarily prevents SMON from cleaning up the uncommitted transaction in the rollback segments involved.
_corrupted_rollback_segments=r01,...
. uncommitted transactions are marked as committed.
. used to corrupt the rollback segments and the rollback segment can be dropped
_db_block_cache_protect
. forces every process to lock a block in memory before writing to it.
. if a process writes it in a location memory that it hasn't locked.
then access violation occurs, giving diagnostics trace information.
VMS,MVS,OS/2, NEC, DGAviion,not implemented on most UNIX
_db_block_compute_checksums
. used to debug from disk to cache, oracle will computes and write a checksum to the block. next time the same block is read, oracle computes the checksum again and compares it with the value that's in the block. if that differs, the blcok is corrupted on disk.

< Automatic recovery failure >

a. ora-600[3339]
t1 - . use _offline_rollback_segments=r01,r02... in init.ora
. try to open the db
t2 - . if rollback segment corrupted,use
_corrupted_rollback_segments= r01,r02.. in init.ora
. rebulid rollback segments
t3 - . use _corrupted_rollback_segments=r01,r02
. use _allow_resetlogs_corruption=true
. make a controlfile as resetlogs and open the db using resetlogs.
. rebuild rollback segments

b. ora-600[3398]
t1 - . put _db_block_cache_protect=true on init.ora and
try to startup 2-3 times
t2 - . set events in init.ora
event="10210 trace name context forever,level 10"
event="10211 trace name context forever,level 10"

c. ora-600[2662]
   t1 - . retry shutdown abort and startup 3 times
   t2 - . event="immediate trace name adjust_scn level 1" in init.ora
   t3 - . _corrupted_rollback_segments = r01,r02
        . use _allow_resetlogs_corruption=true

d. ora-600[16224],ora-472
   t1 - . event="10052 trace name context forever"
      -. startup db
         * event 10052 : don't clean up obj$

e. ora-600[3020] when start recovery
   before recovery, don't startup db

f. ora-600[4306]
   t1 - . event="10061 trace name context forever"
         * event 10061 : disable SMON from cleaning temp segment

< Database mount and hanging >

a. if archivelog mode, check whether archivelog is normal or not
   sqldba > archive log list
     ...
     automatic archive   enable
     current log seq   1000
     ..
     next archive log seq 998 --> abnoraml
  . check the disk space of archive log directory
  . automatic archive - enable

b. database not open and waiting
  . check alert.log file - if it is changing, then rolling back
  . bg processes are idle and no changing in alert.log - data block
    corrupted

    alter database open
    Beginning crash recovery of 1 threads
    ... waiting
     t1 - use _corrupted_rollback_segments=r01,r02 and try to startup
     t2 - make controlfile as resetlogs
         _allow_resetlogs_corruption=true
         _corrupted_rollback_segments = r01,r02

< Current online log deleted >

  . make controlfile as resetlogs
  . use _allow_resetlogs_corruption and _corrupted_rollback_segments=r01,
  . rebuild rollback segments

DATABASE LINK 사용 방법

Bulletin no : 10091

우선 고려되어야 사항은 ORACLE INSTANCE가 두 개 이상이고 각기 다른 PLATFORM에서 운용된다는 가정하에서 각각의 HOST NAME과 ORACLE_SID는 다르고 NLS_CHARACTER_SET은 동일하게 되어 있어야 합니다.

만약 같은 MACHINE에서 INSTANCE의 ORACLE_SID가 같다면 TNS ERROR가 발생 할 것이다.
또한 미래를 위해 다른 MACHINE이라 할지라도 ORACLE_SID는 규칙에 의해 다르게 가져가는 것이 좋습니다.
그리고 NLS_CHARACTER_SET이 동일하게 되어 있지 않으면 DATA 입 출력시 한글 데이터가 ?????로 나타날 것입니다.

그럼 환경 점검이 끝났으니 ORACLE7에서 ORACLE V6(ORACLE7의 경우도 비슷)에 있는 TABLE의 DATA를 DB LINK를 이용하여 SELECT하거나 VIEW를 작성하여 보겠습니다.

	V7	V6
HOST NAME	V7DB	V6DB
ORACLE_SID	V7SID	V6SID

이라 할때

1> V7 에서 V6로 DBLINK생성(이하 V7 시스템에서)
- Public Database Link 생성
  sqlplus system/manager로 ORACLE에 LOGIN

```
SQL> create public database link V7TOV6 using 't:V6DB:V6SID';
```

- Private Database Link 생성
  scott/tiger 로 Login

```
SQL> create public database link  V7TOV6 connect to scott
     identified by tiger using 't:V6DB:V6SID';
```

로 하면 된다.

2> V6 에 있는 TABLE의 select 및 view 작성

```
SQL> select * from emp@V7TOV6;
SQL> create view emp_view as select * from emp@V7TOV6  a
     where a.deptno = 10;
```

3> V7 에서 SYNONYM을 생성하여 사용하는 경우

```
SQL> create synonym emp for emp@V7TOV6;
SQL> select * from emp;
```

로 한다면 간단히 분산 DB의 환경에서 사용 할 수 있습니다..
(단 SERVER TO SERVER로 NETWORK 환경이 구축되어 있어야 합니다.)

DATAFILE을 RENAME 하는 방법

Bulletin no : 10071

▶ NON-SYSTEM TABLESPACE 의 DATAFILE이름을 변경하는 경우

SPACE 때문에 데이타 화일을 다른 디렉토리에 옮겨야 하는 경우가 있다.
데이타 화일을 다른 디렉토리에 옮기는 것은 데이타 화일의 이름이 변경되는 경우이므로 다음 SQL 문을 사용하면 된다.

```
ALTER TABLESPACE tablespace RENAME DATAFILE 'filename' TO 'filename';
```

1. 데이타베이스 BACKUP을 시행하고 데이타베이스의 모든 작업을 중단하고 다음 작업을 실시한다.

2. 해당 TABLESPACE를 다음 명령어로 DB 가 OPEN된 상태에서 OFFLINE 한다.

```
ALTER TABLESPACE tablespace OFFLINE;
```

3. OS COPY 명령어을 사용하여 새로운 DIRECTORY에 FILE을 COPY 한다.

4. ALTER TABLESPACE 명령어를 사용하여 DATABASE FILE을 RENAME 한다.

5. TABLESPACE 를 ONLINE으로 전환한다.

```
ALTER TABLESPACE tablespace ONLINE;
```

6. OS 상에서 OLD FILE을 삭제한다.

위의 작업을 하기위해서는 OLD와 NEW 데이타 화일이 존재해야한다.
또한 데이타 화일의 이름이 변경 되었으므로

```
ALTER DATABASE BACKUP CONTROLFILE TO 'filename'
```

▶ SYSTEM TABLESPACE DATAFILE 이름을 변경하는 경우

SYSTEM TABLESPACE에 있는 DATAFILE NAME을 변경하는 경우 SYSTEM TABLESPACE는 OFFLINE 할 수 없으므로 DATABASE를 OPEN 하지않고 MOUNT된 상태에서 ALTER DATABASE RENAME FILE 명령어를 사용한다.

1. EXPORT로 FULL BACKUP을 받는다.
```
$ exp system/manager file=system.dmp full=y
```

2. 데이타베이스를 정상적으로 DOWN 한다.
```
$ sqldba lmode=y
  SQLDBA> connect internal
  SQLDBA> shutdown
```

3. DATAFILE을 변경하고자 하는 화일 이름으로 COPY 한다.

```
$ cp /oracle/dbs/syst_old.dbf  /oracle/dbs/syst_new.dbf
```

4. DATAFILE의 이름을 RENAME 한다.

```
$ sqldba lmode=y
  SQLDBA > connect internal
  SQLDBA > startup mount
  SQLDBA > alter database rename file '/oracle/dbs/syst_old.dbf' to
               '/oracle/dbs/syst_new.dbf' ;
  SQLDBA > alter database open;
```

5. 데이타베이스가 정상적으로 OPEN 되는지 SHUTDOWN 후 다시 STARTUP 한다.

6. 데이타베이스가 정상적으로 STARTUP되면 OLD 화일을 삭제한다.

```
SQLDBA > !rm /oracle/dbs/syst_old.dbf
```

DBA 정보와 CONVERT SCRIPT

Bulletin no : 10065

DBA는 데이타블럭에 대한 file#와 block#의 조합으로 이루어져 있다. 길이는 32bit 이며 상위비트들은 file#를 나타내고 하위비트들은 block#를 나타낸다.

V6에서는 대부분의 유닉스 플랫폼에서 file#로 6bit를 사용했다. file#를 이용하여 가능한 최대데 타화일의 수를 계산하는 공식은 다음과 같다.

최대 데이타화일 수 = 2^n - 2

여기서 n은 file#를 표시하는 비트수를 나타낸다. 만약 n=6이라면

최대 데이타화일 수 = 62 개

가 된다.

V7에서는 플랫폼마다 file#, block# 를 구성하는 비트수가 달라진다.
일반적으로 file#에는 10비트가 사용되는데 그러면

최대 데이타화일 수 = 2^10 -2 = 1022개

가 된다.

그 동안 V7에서 DBA의 file# 를 어떻게 구성할까 하는가에 대해 많은 논란이 있어왔다. 어떠한 구조를 가지든 모든 플랫폼에서 최소한 1022개의 데이타 화일을 지원할 수 있고 V6의 DBA가 V7에서도 그대로 사용될 수 있도록 하여야만 했다.
이는 V6에서 V7으로의 migration을 용이하게 해주기 때문이다. 우선 1022개의 화일을 지원하려면 10bits가 필요하므로 DBA의 file#는 10bits로 결정되었다. 하지만 이렇게 함으로써 file#가 6bits로 구성된 V6와의 호환성이 문제가 되었다. 아래에 한가지 예를 들었다.

(ex)block#=1, file#=1 일때

2진수로
V6: 0000 0100 0000 0000 0000 0000 0000 0001
V7: 0000 0000 0100 0000 0000 0000 0000 0001

16진수로
V6: 04000001
V7: 00400001

V6와의 호환성 문제를 해결하기 위하여 간단한 방법이 이용된다. 즉 앞에서 결정한 V7의 file# 10bits(아직 실제로 V7에서 사용하는 값이 아님)에서 MSbs와 LSBs를 바꾼 값이 실제로 V7이 file#로 사용하는 값이 되는 것이다.
MSBs와 LSBs의 bit 수는 다음과 같이 결정된다.

MSBs의 bit 수 = 10 - (V6에서의 file# bit 수)
= 4(일반적으로)

LSBs의 bit 수 = 10 - (MSB의 bit 수)
= 6(일반적으로)

하나 예를 들어보기로 하자

(ex)file# = 4, block#=3 일때
file#를 10bit 이진수로 표시하면 00 0000 0100
MSBs = 0000
LSBs = 0001 00
따라서
file# = 0001 0000 00
그러므로
DBA = 0001 0000 0000 0000 0000 0000 0000 0003

이 값은 V6와 호환됨을 알 수 있다.

다시한번 V7에서 file#와 block#를 가지고 DBA를 계산하는 방법을 알아보기로 하자.

1) file #를 10bit 이진수로 변환한다.
2) 10에서 V6에서 file#로 사용되는 bit 수를 뺀다. 이 값을 X 라고 부르기로 한다.
3) 과정 1)에서의 10bit에서 X 만큼의 MSBs를 구하여 그 나머지와 순서를 바꾼다.
4) 과정 3)의 결과를 32bit 이진수의 MSB로 설정한다. 나머지 22bit는 0으로 설정한다.
5) block#를 32bit 이진수로 변환한다.
6) 과정 4)와 과정 5)의 결과를 OR 연산에 의하여 bit단위로 결합한다.
7) 이 값을 16진수로 변환한다.

하지만 V7 DBA의 file#가 플랫폼 다 어떻게 달라지는가는 아직 설명하지 않았다. V6의 file#가 6bit로 구성되지 않을 수도 있기 때문이며 과정 2)에서 X 라는 값을 설정하였음을 기억하기 바란다.

V6에서 file#를 구성하는 bit 수를 아는 방법은 다음과 같다.

```
SQL>select dump(chartorowid('00000000.0000.0001')) from dual;
```

이것은 rowid의 내부저장형태를 보여줌으로써 file#의 구성을 알 수 있다.

ex)AIX에서

```
SQL>select dump(chartorowid('00000000.0000.0001')) from dual;

DUMP(CHARTOROWID('00000000.0000.0001'))
----------------------------------------
Typ=69 Len=6: 1,0,0,0,0,0
1 row selected
```

앞의 4개의 숫자가 DBA(file#, block#)를 가리킨다. 뒤의 2개의 숫자는 row# 이다.

이 값을 2진수로 쓰면

00000001 00000000 00000000 00000000

AIX플랫폼의 V6에서 file# 가 위와 같이 표현되었으므로 file#로 8bit가 사용됨을 알 수 있다.
V7에서 사용되는 DBA의 file#는 보통 5~12bit로 구성된다. V7에서 file#를 구성하는 bit수를 아는 방법은 두가지가 있다.

하나는 해당 플랫폼의 가능한 최대 데이타화일 수를 알아서 다음의 공식으로 변환하면 된다. n은 file#의 bit 수 이다.

2^n -2 = 최대 데이타화일 수

다른 하나는 file#=ffff 인 경우에 대해 select 문을 사용하여 알아 보는 방법이다.

```
SQL>select dump(chartorowid('00000000.0000.ffff')) from dual;

DUMP(CHARTOROWID('00000000.0000.ffff'))
---------------------------------------
Type=69 Len=6: 255,192,0,0,0,0
1 row selected
```

2진수로는

11111111 11000000 00000000 00000000

여기서 AIX에서는 10bit가 file#에 사용됨을 알 수 있다. -END-

다음은 file# 및 block#와 DBA간의 상호 변환을 수행하는 스크립트이다.
여기에 나오는 함수들을 이용하여 변환을 하기 바란다.

```
REM ***************************************************************
REM  CATDUMP.SQL - (run as SYSTEM or other DBA user)
REM
REM  This utility script creates stored objects for dumping blocks:
REM   (1) dba(file,blk,port)   == returns decimal dba or level.
REM   (2) dbax(file,blk,port)  == returns hex dba.
REM   (3) dbarev(decdba,port)  == returns file, block.
REM   (4) dbarevx(hexdba,port) == returns file, block.
REM   (5) block(file,blk,port) == dumps the specified block
REM
REM  Needs ALTER SESSION, CREATE PROCEDURE/FUNCTION/TABLE to be
REM  explicitly granted. Privs to execute DBMS_SQL routines also.
REM
REM ***************************************************************

REM connect system/manager;
```

```
drop table portinfo;
drop function dba;
drop function dbarev;
drop function dbax;
drop function dbarevx;
drop function dectohex;
drop function hextodec;

drop public synonym dba;
drop public synonym dbarev;
drop public synonym block;
drop public synonym dbax;
drop public synonym dbarevx;

create table portinfo
( portid        number  primary key,
  sfdbaofbits   number,
  sfdbanfbits   number,
  port varchar2(25));
insert into portinfo values (1, 8, 10, 'DEC VMS');
insert into portinfo values (2, 6, 10, 'HP/UX');
insert into portinfo values (28, 5, 8, 'SCO Unix');
insert into portinfo values (30, 12, 12, 'IBM MVS');
insert into portinfo values (84, 5, 8, 'SCO MPX');
insert into portinfo values (87, 8, 10, 'DEC Alpha');
insert into portinfo values (58, 6, 10, 'DEC MIPS Ultrix');
insert into portinfo values (116, 6, 10,'Pyramid DC/OSx');
insert into portinfo values (162, 6, 10,'DG/UX Aviion');
insert into portinfo values (168, 6, 10,'SGI Unix');
insert into portinfo values (198, 6, 10,'Sequent DYNIX/ptx');
insert into portinfo values (202, 8, 10,'ATT 7000');
insert into portinfo values (263, 8, 8, 'OS/2');
insert into portinfo values (319, 8, 10,'IBM RS/6000');
insert into portinfo values (358, 8, 10,'ATT/NCR 3000');
insert into portinfo values (451, 6, 10,'Sun 4 Unix');
insert into portinfo values (453, 6, 10,'Sun Solaris');
insert into portinfo values (610, 8, 10,'IBM SP2');
insert into portinfo values (611, 6, 10,'Pyramid MESH');
commit;

create or replace FUNCTION hextodec(a IN VARCHAR) RETURN NUMBER IS
        x NUMBER := 0;
        y NUMBER;
        z VARCHAR2(1);
    BEGIN
        IF length(a) > 8 THEN   -- only allow 32-bit DBAs
            RAISE invalid_number;
        END IF;
```

```
        FOR i IN 1..length(a) LOOP
            z := upper(substr(a,i,1));
        IF z = 'A' THEN         -- decode() leads to bug 240820
            y := 10;
        ELSIF z = 'B' THEN
            y := 11;
        ELSIF z = 'C' THEN
            y := 12;
        ELSIF z = 'D' THEN
            y := 13;
        ELSIF z = 'E' THEN
            y := 14;
        ELSIF z = 'F' THEN
            y := 15;
        ELSE
            y := to_number(z,9);
        END IF;
            x := x * 16;
            x := x + y;
        END LOOP;
        RETURN x;
    END;
/

create or replace FUNCTION dectohex(a IN NUMBER) RETURN VARCHAR IS
        x VARCHAR2(8) := '';
        y VARCHAR2(1);
        z NUMBER;
        w NUMBER;
    BEGIN
        IF a > power(2,32) OR a < 0  THEN
            RAISE invalid_number;
        END IF;
        w := a;
        WHILE w > 0 LOOP
            z := w mod 16;
        IF z = 10 THEN
            y := 'A';
        ELSIF z = 11 THEN
            y := 'B';
        ELSIF z = 12 THEN
            y := 'C';
        ELSIF z = 13 THEN
            y := 'D';
        ELSIF z = 14 THEN
            y := 'E';
        ELSIF z = 15 THEN
            y := 'F';
```

```
        ELSE
            y := to_char(z);
        END IF;
            w := trunc(w / 16);
            x := concat(y,x);    -- build x string backwards
        END LOOP;
        RETURN x;
    END;
/

create or replace function
 dba(file_id number, block_id number, port_id number)
return number is
m number;
n number;
diff number;
obits number;
nbits number;

begin
  begin
    select sfdbaofbits, sfdbanfbits into obits, nbits
      from portinfo
      where portid = port_id;
    EXCEPTION
      WHEN no_data_found THEN
      RAISE invalid_number;
  end;
  IF file_id > 1022 THEN
    RAISE invalid_number;
  END IF;
  diff:= nbits-obits;
  m := floor(file_id/power(2, obits));   /* value of higher DIFF bits */
  n := mod (file_id, power(2, obits));   /* value of lower OBITS bits */
  return (n*power(2, diff) + m)*power(2, 32-nbits) + block_id;
                                  /* swap higher/lower, add block_id */
end;
/

create or replace function
 dbarev (decdba number, port_id number)
return varchar is
obits number;
nbits number;
fl number;
bk number;
diff number;
maxbk number;
```

```
begin
  begin
    select sfdbaofbits, sfdbanfbits into obits, nbits
      from portinfo
      where portid = port_id;
    EXCEPTION
      WHEN no_data_found THEN
      RAISE invalid_number;
  end;
  IF (decdba > power(2,32)) OR (decdba < 0)  THEN
    RAISE invalid_number;
  END IF;
  diff:= nbits-obits;
  maxbk:= power(2,32-nbits);            /* prepare block mask */
  bk:= mod(decdba, maxbk);              /* extract block_id value */
  fl:= floor(decdba/maxbk);             /* extract file_id value */
  fl:= mod(fl, power(2,diff))*power(2,obits) +
        floor(fl/power(2,diff));        /* swap "diff" bits */
  return (to_char(fl) || ' . ' || to_char(bk)); /* "FILE . BLOCK" */
end;
/

create or replace function
 dbax (file_id number, block_id number, port_id number)
return varchar is

begin
  return dectohex(dba(file_id,block_id,port_id));
end;
/

create or replace function
 dbarevx (hexdba in varchar2, port_id in number)
return varchar is

begin
  return dbarev(hextodec(hexdba), port_id);
end;
/

create or replace procedure
 block (file_id in number, block_id in number, port_id number) as
cur integer;
dummy integer;
lev number;

begin
  cur:= dbms_sql.open_cursor;
```

```
  lev:= dba(file_id, block_id, port_id);
  IF (lev > power(2,32)) OR (lev < 0)  THEN
    RAISE invalid_number;
  END IF;
  dbms_sql.parse(cur, 'alter session set events
    ''immediate trace name blockdump level '||lev|| ' ''', dbms_sql.v7);
  dummy:= dbms_sql.execute(cur);
  dbms_sql.close_cursor(cur);
end;
/

grant execute on dba to public;
grant execute on dbarev to public;
grant execute on dbax to public;
grant execute on dbarevx to public;
grant execute on block to public;

create public synonym block for block;
create public synonym dba for dba;
create public synonym dbarev for dbarev;
create public synonym dbax for dbax;
create public synonym dbarevx for dbarevx;
```

DBMS PACKAGE : DBMS_ALERT을 이용하여 SESSION간 통신하는 방법

Bulletin no : 10394

DBMS_ALERT이란?

DBMS_ALERT은 DATABASE에 일어나는 변화를 자동으로 알려줄 수 있는 FUNTION이다. APPLICATION은 관심있는 TABLE에 대해 일어나는 변화에 대한 확인을 언제든지 받을 수 있게 된다. 이렇게 되기 위해서 해당 TABLE에 대해 변화가 일어났을 때 ALERT을 줄 수 있는 몇개의 PROGRAM이 DATABASE에 등록되어 있어야 한다.

DBMS_ALERT은 어떤 TABLE에 대해 일어나는 INSERT, UPDATE, DELETE등의 변화를 MONITORING하는 PRO*C PROGRAM등에 이용될 수 있겠다.

ALERT을 사용하려면?

다른 DBMS PACKAGES와 마찬가지로 DBMS_ALERT PACKAGE는 SYS가 OWNER이며
CATPROC.SQL을 실행하여 생성된다. DBMS_ALERT PACKAGE를 사용하려면
이 PACKAGE에 대한 EXECUTE PRIVILEGE가 있어야 한다.

```
SQLDBA> GRANT EXECUTE ON DBMS_ALERT TO USER_NAME;
```

한번 EXECUTE PRIVILEGE를 받으면 다른 SQL*PLUS, PRO*C, PRO*COBOL, PL/SQL같은 다른 ORACLE TOOL에서 DBMS_ALERT을 사용할 수 있다.

DBMS_ALERT은 어떻게 실행되는가?

여기에서는 몇 개의 SESSION간에 ALERT이 어떻게 사용되는지 설명하기 위해 DBMS_ALERT PACKAGE중 4개의PROCEDURE를 사용하겠다.

REGISTER PROCEDURE:

이 PROCEDURE는 ALERT를 등록한다.
하나의 SESSION에서 여러개의 ALERT을 등록할 수 있다.

```
DBMS_ALERT.REGISTER( ALERT_NAME IN  VARCHAR2);
```

SIGNAL PROCEDURE:

이 PROCEDURE는 ALERT에 SIGNAL을 보내고,MESSAGE를 보낸다.
SIGNAL을 TRANSACTION에서 COMMIT을 만나야지만 영향이 있게 된다.
만약 TRANSACTION에서 ROLL BACK을 하면 SIGNAL을 아무런 영향을 받지 않는다.

```
DBMS_ALERT.SIGNAL(ALERT_NAME IN VARCHAR2,
                  MESSAGE IN VARCHAR2);
```

WAITONE PROCEDURE:

이 PROCEDURE는 SESSION이 특정한 ALERT이 발생되기를 기다리게 한다.

SIGNAL이 실행되면, MESSAGE는 WATING하고 있는 모든 SESSION에 전달되게 된다.
TIME-OUT SECONDS에 지정된 시간 내에 SIGNAL이 오지 않으면 WAITONE은 STATUS 1을
RETURN하게 된다. 그 전에 ALERT이 일어나면 STATUS 0를 RETURN한다.
TIME-OUT시간은 USER가 선택적으로 지정할 수 있다.

```
    DBMS_ALERT.WAITONE(ALERT_NAME IN VARCHAR2,
                               MESSAGE OUT VARCHAR2,
                               STATUS OUT INTEGER,
                               TIME-OUT IN NUMBER);
```

만약 한 SESSION에서 여러개의 ALERT을 사용할 경 에는 WAITANY PROCEDURE를
사용하여야 한다.

REMOVE PROCEDURE:

    이 PROCEDURE는 등록LIST에서 해당 ALERT명을 삭제하는 기능을 한다.

```
    DBMS_ALERT.REMOVE(ALERT_NAME IN VARCHAR2);
```

더 자세한 사항은 ORACLE7 SERVER APPLICATION DEVELOPER'S GUIDE 참조 바람

SAMPLE

DBMS_ALERT과 TRIGGER를 이용하여 SESSION간에 TABLE에 관련된 정보를 전달하는 과정을 본다.
세번째 SESSION에서 EMP TABLE에 UPDATE, DELETE, INSERT문장을 실행하면 SIGNAL을 보내는 DB
TRIGGER가 FIRE되게 된다.
그러면 기다리고 있는 2개의 PRO*C,PL/SQL SESSION에서 SIGNAL을 받아 MESSAGE를 내게 되는 것이다.

첫번째 SESSION은 PRO*C PROGRAM이다.
이 SESSION에서는 I_GOT_IT이라는 이름의 ALERT를 지정하여 SIGNAL을 기다린다.

```
=====================================================
#include
#include
#define     UNAME_LEN       20
#define     PWD_LEN         40
VARCHAR     username[UNAME_LEN];
VARCHAR     password[PWD_LEN];


EXEC SQL BEGIN DECLARE SECTION;
  int             status;
  int             timeout=20;
  varchar         name[40];
  char            message[100];
  short           name_ind;
  short           message_ind;
  short           status_ind;
EXEC SQL END DECLARE SECTION;

#include
/* Declare error handling function. */
void sql_error();
```

```
main()
{
    strncpy((char *) username.arr, "SCOTT", UNAME_LEN);
    username.len = strlen((char *) username.arr);
    strncpy((char *) password.arr, "TIGER", PWD_LEN);
    password.len = strlen((char *) password.arr);

    EXEC SQL WHENEVER SQLERROR DO sql_error("ORACLE error--\n");

    EXEC SQL CONNECT :username IDENTIFIED BY :password;
     printf("\nConnected to ORACLE as user: %s\n", username.arr);


    strcpy((char *)name.arr, "I_GOT_IT");
    name.len = strlen((char *) name.arr);

    memset(message,'\0',100);
    EXEC SQL EXECUTE
      BEGIN
        DBMS_ALERT.REGISTER(:name);
      END;
    END-EXEC;
    printf("\n WE ARE WAITING FOR WAITONE SIGNAL\t%s\n\n",name.arr);
    EXEC SQL EXECUTE
      BEGIN
        DBMS_ALERT.WAITANY(:name:name_ind, :message:message_ind,
        :status:status_ind,  :timeout);
      END;
    END-EXEC;
    if (!status)
    {
        printf("\n WE HAVE RECEIVED THE SIGNAL \t\n\n");
        printf("\n THE MESSAGE THAT HAS BEEN RECEIVED IS\t%s\n\n", message);
        printf("\n THE STATUS\t\t  THE TIME\n");
        printf("\n\t %d \t\t %d\n\n", status, timeout);
        printf("\nG'day.\n\n\n");
    }
    else
       printf("NO MESSAGE HAS BEEN RECEIVED. EXIT SESSION\n\n");
    EXEC SQL EXECUTE
      BEGIN
        DBMS_ALERT.REMOVE(:name);
      END;
    END-EXEC;
    EXEC SQL COMMIT WORK RELEASE;
    exit(0);
   }
=====================================================
```

두번째 SESSION은 PL/SQL이다.
이 SESSION에서도 I_GOT_IT이라는 이름의 ALERT을 지정하여 SIGNAL을 기다린다.

```
=====================================================
DECLARE
  status  NUMBER;
  timeout NUMBER;
  message CHAR(30);
  name    VARCHAR2(20);
BEGIN
  name:='I_GOT_IT';
  timeout:=20;
  DBMS_ALERT.REGISTER(name);
  DBMS_ALERT.WAITANY(name, message, status, timeout);
  IF status = 0 THEN
    DBMS_OUTPUT.PUT_LINE(' RECEIVED SIGNAL   ' || name);
    DBMS_OUTPUT.PUT_LINE('THE MESSAGE THAT HAS BEEN RECEIVED IS ' || message);
  ELSE
    DBMS_OUTPUT.PUT_LINE('NO MESSAGE HAS BEEN RECEIVED.  EXIT SESSION');
  END IF;
  DBMS_ALERT.REMOVE(name);
END;
/
=====================================================
```

세번째 SESSION을 SQL*PLUS에서 SQL을 실행하는 SESSION이다.
이 SESSION은 먼저 다음과 같이 단순 게 I_GOT_IT이라는 ALERT을 다음과 같이 등록한 후 INSERT, UPDATE, DELETE등의 SQL을 실행한다.

```
SQL> EXECUTE DBMS_ALERT.REGISTER ('I_GOT_IT');
```

위의 3개의 SESSION을 생성하기 전에 사용자는 먼저 다음과 같은 TRIGGER를 등록해 놓아야 한다.

```
=====================================================
CREATE OR REPLACE TRIGGER my_trig
  AFTER INSERT OR UPDATE OR DELETE on EMP
  FOR EACH ROW
BEGIN
  IF INSERTING THEN
    DBMS_ALERT.SIGNAL('I_GOT_IT', 'INSET INTO EMP  ' || :new.ename );
  END IF;
  IF UPDATING THEN
    DBMS_ALERT.SIGNAL('I_GOT_IT', 'UPDATE ENAME TO  ' || :new.ename );
  END IF;
  IF DELETING THEN
    DBMS_ALERT.SIGNAL('I_GOT_IT', 'DELETE FROM EMP  ' || :old.ename );
  END IF;
END;
/
=====================================================
```

위의 PRO*C,PL/SQL SESSION을 실행한 후 3번째의 SQL*PLUS SESSION을 실행한다.
SQL*PLUS SESSION에서 다음과 같은 SQL문장을 실행한 후 COMMIT한다.

SQL> UPDATE EMP SET ENAME= 'HADI' WHERE EMPNO = 7844;

COMMIT되고나면 TRIGGER가 FIRE되어 SIGNAL이 실행되고,PRO*C, PL/SQL SESSION에서 ALERT을 받아 MESSAGE를 내게 된다.

< 위의 PROGRAM들을 RDBMS 7.1.6 , PRO*C 2.0.6에서 작성된 것이다. >

DBMS PACKAGE :
DBMS_PIPE NOT DECLARED ERROR발생시

Bulletin no : 10342

발생원인 1)

DBMS_PIPE는 oracle install시에 $ORACLE_HOME/rdbms/admin/catproc.sql을 실행함으로써 생성되도록 되어 있는 stored package이다. 다 stored package(DBMS_OUTPUT....)등은 생성직후 sys를 owner로 한
stored package에 대해 public synonym을 생성한 후 바로 이 synonym에 대해 public에게 execute previlege가
grant되지만 DBMS_PIPE는 생성만 되도록 되어 있기 때문에 사용하고자 하는 user에게 별도의 grant작업이 필요하다.

-> 확인사항 및 해결 방법

1) sqldba에서
   select * from dba_objects where object_name = 'DBMS_PIPE';
   를 수행하여 DBMS_PIPE에 관한 자료가 sys가 owner인 package spec,package body,synonym만 나타나야 한다.
2) sqldba에서
   grant execute on DBMS_PIPE to user_name;
   을 실행하여 해당 user에게 grant한다.

발생원인 2)

DBMS_PIPE는 sys가 owner이어야 한다.
user의 실수로 system이나 다른 별도의 user로 connect하여 catproc.sql을 실행한 경우에 발생한다.

-> 확인사항 및 해결방법

1) sqldba에서
   select * from dba_objects where object_name = 'DBMS_PIPE';
   를 수행하여 DBMS_PIPE에 관한 자료가 sys나 public이 아닌 다른 user가 owner로 되어 있는 DBMS_PIPE object를 drop시킨다.
2) sqldba에서
   grant execute on DBMS_PIPE to user_name;
   을 실행하여 해당 user에게 grant한다.

DBMS PACKAGE : DBMS_PIPE사용 (SESSION간의 MESSAGE SENDING)

Bulletin no : 10300

DBMS_PIPE Package는 동일한 Instance내의 둘 혹은 그 이상의 Session간에통신을 가능하게 한다. ORACLE PIPE는 UNIX에서 사용하는 PIPE의 개념과 흡사하나, ORACLE PIPE는 OS PIPE의 MECHANISM을 사용하지 않는다.

ORACLE PIPE를 통해 전해지는 정보는 SGA에 저장되며 LRU Algorithm을 사용한다.(DB CREATE 시에 CATPROC.SQL이 실행되면서 자동 생성된다.)

* 이 Package를 사용하려면 PL/SQL Version 2.0이상이 되어야만 지원이 가능하며, 따라서 Forms3.0이나 Forms4.5등에서는 사용할 수 없으며 경우에 따라Stored Procedure를 작성하여 Call함으로써 사용할 수 있다.

또한 SYS에서 해당 User에게로 Execute할 수 있는 권한을 Grant 시켜주어야만 비로소 실행할 수 있다.(예: SQL> grant execute on dbms_pipe to scott;)

* 한 User에 의하여 정보가 읽혀지면, Buffer는 비 되고, 같은 PIPE라 하더 라도 다른 User가 읽을 수 없게 된다.

* 예제

```
create or replace procedure SendMessage(info VARCHAR2)
is
   s NUMBER;
begin
  DBMS_PIPE.PACK_MESSAGE(info);
  s := DBMS_PIPE.SEND_MESSAGE('pipe_name');
end;
/
 (여기서 pipe_name은 임의로 줄 수 있다.)

create or replace function ReceiveMessage
return VARCHAR2 is
  n NUMBER;
  chr VARCHAR2(200);
begin
  n := DBMS_PIPE.RECEIVE_MESSAGE('pipe_name',5);
   /* 5 는 timeout기간(초단위)이며 default maxwait는
       86400000(1000일) 이다. */

  DBMS_PIPE.UNPACK_MESSAGE(chr);
  return(chr);
end;
/
```

* SQLPLUS에서 DATA PASSING

```
SQL> execute SendMessage('Hello');

PL/SQL procedure successfully completed.

SQL> variable a varchar2(200);
SQL> execute :a := ReceiveMessage;

PL/SQL procedure successfully completed.

SQL> print a;

A
----------------------
Hello
```

* FORMS에서 DATA PASSING

[A FORM]

1. DEPT Table상의 Default Block으로 Form을 작성한다.
2. Button을 만들고 그 Button에 WHEN-BUTTON-PRESSED Trigger를 아래와 같이 준다.

```
DECLARE
   x  VARCHAR2(200);
BEGIN
   x := :DEPT.DNAME;
   DBMS_PIPE.PURGE('pipe_name');
   SendMessage(x);
END;
```

[B FORM]

1. PRE-QUERY TRIGGER

```
BEGIN
  :DEPT.DNAME := ReceiveMessage;
END;
```

2. WHEN-BUTTON-PRESSED TRIGGER

```
execute_query;
```

* DBMS_PIPE Package의 Function과 Procedure

보안적인 면에 따라, PUBLIC PIPE와 PRIVATE PIPE중 선택하여 사용한다.

FUNCTION	DESCRIPTION
CREATE_PIPE	명시적 PIPE를 생성한다. (PRIVATE PIPE를 사용하는데 필요)
SEND_MESSAGE	NAMED PIPE에 MESSAGE를 보낸다.(NAMED PIPE가 없는 경우 묵시적인 PUBLIC PIPE를 생성한다)
RECEIVE_MESSAGE	NAMED PIPE에서 LOCAL BUFFER로 MESSAGE를 COPY한다
NEXT_ITEM_TYPE	BUFFER내의 다음ITEM의 DATA TYPE을 RETURN한다
REMOVE_PIPE	NAMED PIPE를 앤다
UNIQUE_SESSION_NAME	UNIQUE SESSION명을 RETURN한다

PROCEDURE	DESCRIPTION
PACK_MESSAGE	LOCAL BUFFER안에 MESSAGE를 만든다
UNPACK_MESSAGE	BUFFER내의 다음 ITEM을 ACCESS한다
PURGE	NAMED PIPE내의 내용을 없앤다
RESET_BUFFER	LOCAL BUFFER의 내용을 없앤다

* 더 자세한 사항은 Oracle7 Application Developer's Guide 를 참조.

DB_FILE_MULTIBLOCK_READ_COUNT와 퍼포먼스의 관계

Bulletin no : 10101

---

* db_file_multiblock_read_count 파라미터는 한번의 I/O 작업으로 읽어들이는 최대 블럭 수를 가리키며 Multi Block I/O 에 관련된 파라미터이다. 디폴트 값은 O/S에 따라서 다른데 보통 db_block_buffers와 Processes에 의해서 결정되며 4~32 정도가 많이 사용된다.
이 값을 키우고 SORT/MERGE 를 이용하는 것이 Nested Loop를 이용하는 경우 보다 퍼포먼스가 증가하는가 하는 문제는 Query의 종류와 읽어들이는 데이타의 양(전체에 대한 Percentage)에 따라서 달라지게 된다.

즉, 이 값이 크면 한번의 I/O로 여러개의 블럭을 읽 오므로 Full Table Scan시 에는 퍼포먼스가 증가한다. 하지만 특정 블럭만이 필요한 경우에 불필요한 블럭까지 함께 읽어들이므로 디스크를 읽는 시간이 증가할 뿐 아니라 크기가 한정된 Buffer Pool을 많이 차지해 버림으로써 자주 쓰이는 데이타를 밀어내어 오히려 Cache의 효율을 떨어뜨리는 결과를 가져올 수도 있으므로 작업특성에 따라서 적절한 값을 세팅하여야 한다.

오라클의 Optimizer는 이 값이 크면 작업의 효율성을 위해서 Full Table Scan 을 선택할 가능성이 커지게 된다.

DB_NAME과 SID 변경하는 방법

Bulletin no : 10162

Modifying a database to run under a new ORACLE_SID

---

1. Shutdown instance
2. Backup all control, redo and data files.
3. Go thru the .profile, .cshrc, .login, oratab, tnsnames.ora(for net v2)
   , and redefine the environment variable ORACLE_SID to a new value.
   ie search thru disks and do a grep ORACLE_SID *
4. cd $ORACLE_HOME/dbs and rename the following files:
   - o init.ora (or use pfile to point to the init file.)
   - o control file(s) This is optional if you don't rename any of the
     controlfiles, and the control_files parameter is
     used. control_files would be set in the initSID.ora
     file or in a file it references with the ifile
     parameter. Make sure control_files doesn't point to
     any old file names, if you renamed them.
   - o crdb.sql & crdb2.sql This is optional. These are
     only used at database creation.
5. cd $ORACLE_HOME/rdbms/admin and rename the file:
   - o startup.sql This is optional.

   (On some platforms, this file may be in $ORACLE_HOME/rdbms/install.)
   Make sure the contents of this file do not reference old initSID.ora
   files that have been renamed. This file simplifies the process to
   "startup exclusive" your database.
6. To rename the database files and redo log files, you would follow the
   instructions in the bulletin: 98863.723.
7. Change the ORACLE_SID environment variable to the new value.
8. start up database and verify it works. Once you have done this,
   shutdown the database and take a final backup of all control, redo and
   data files.
9. When the instance is started, the control file gets updated with the
   current ORACLE_SID.

Changing the dbname for a database

---

1. sqldba
2. connect internal
3. alter database backup controlfile to trace;
   This will write in a trace file, the CREATE CONTROLFILE command that
   would recreate the controlfile as it currently exists.
4. Exit and go to the directory where your trace files are located.

They are usually in the $ORACLE_HOME/rdbms/log directory.
If user_dump_dest is set in the initSID.ora, then go to the directory listed in the user_dump_dest variable.
The trace file will have the form "ora_NNNN.trc with NNNN being a number.

5. Get the CREATE CONTROLFILE command from the trace file and put it in a new file called something like ccf.sql.
6. Edit the ccf.sql file and modify the CREATE CONTROLFILE command.
   Just change the word "REUSE" to "SET",and "NORESETLOGS" to "RESETLOGS", and modify the dbname.
   Old line:
   CREATE CONTROLFILE REUSE DATABASE "olddbname" NORESETLOGS ...
   New line:
   CREATE CONTROLFILE set DATABASE "newdbname" RESETLOGS ...
   Then save the ccf.sql file.
7. Rename the old control files for backup purposes and so they are not in the way of creating the new ones.
8. Edit initSID.ora so that db_name="newdbname".
9. sqldba
10. connect internal
11. startup nomount
12. @ccf
13. alter database open;
14. Make sure the database is working. Shutdown and backup the database.

DB를 COLD BACKUP(FULL BACKUP) 받는 방법

Bulletin no : 10167

콜드 백업은 데이타베이스가 정상적으로 SHUTDOWN 된 상태에서 데이터화일, 로그 화일, 콘트롤 화일을 모두 백업 받는 것을 말한다. SHUTDOWN 하지 않고 OPEN 된 상태에서 백업을 받으면 백업받은 내용을 나중에 사용할 수가 없으므로 유의해야 한다.

콜드 백업을 위해서 데이타베이스를 SHUTDOWN 할 때에는 NORMAL,IMMEDIATE 옵션을 사용해야 하며 ABORT 를 사용해서는 안된다. IMMEDIATE 를 사용한 경우에는 SHUTDOWN 후에 다시 STARTUP 하고 NORMAL 로 SHUTDOWN 하도록 한다.

콘트롤 화일과 데이타 화일 및 로그 화일의 위치를 확인하여 이들을 tar, cpio, dd 등의 명령을 이용하여 백업 받도록 한다.

콘트롤 파일 확인 방법

```
SQLDBA>show parameter control_files
```

데이타 화일 확인 방법

```
SQLDBA>select name from v$datafile
```

로그화일 확인 방법

```
SQLDBA>select member from v$logfile;
```

필요에 따라서는 파라미터 화일까지 받아두는 것이 좋다.
파라미터 화일은 $ORACLE_HOME/dbs/init.ora 이다. 여기서 는 오라클 인스턴스 이름으로 유닉스 환경변수 ORACLE_SID 로 지정이 되어 있다.

DDB 기본기능 및 분산 트랜잭션 문제의 해결

Bulletin no : 10295

여기서는 DDB(분산 데이타베이스)의 기능과 역할 및 사용 시에 발생하는 문제점들에 대해서 알아보기로 한다. 분산 DB 환경과 관련된 init.ora화일의 파라미터는 다음과 같은 것들이 있다.

- DB_NAME
- DB_DOMAIN
- GLOBAL_NAME
- DISTRIBUTED_RECOVER_CONNECTION_HOLD_TIME
- DISTRIBUTED_TRANSACTIONS
- DISTRIBUTED_LOCK_TIMEOUT
- COMMIT_POINT_STRENGTH
- OPEN_LINKS

분산 DB와 관련된 VIEW와 TABLE은 다음과 같다.

- GLOBAL_NAME
- DBA_DB_LINKS, ALL_DB_LINKS, USER_DB_LINKS
- DBA_2PC_PENDING
- DBA_2PC_NEIGHBORS

* RECO:Recoverer Process

분산 트랜잭션의 Failure를 해결하는 Process로 RECO는 In-doubt 분산 트랜잭션을 가지고 있는 다른 DB에 자동적으로 Connect 하여 모든 In-doubt 트랜잭션을 해결한다. 만일 RECO가 Remote Server에 Connection을 시 했는데 Remote Server가 사용 가능하지 않거나 Network가 복구 되어 있지 않을 경우 RECO는 일정한 시간 후에 다시 Connect를 시도한다.

* DATABASE LINK

1. Database Link에는 Private Database Link와 Public Database Link의 두 종류가 있는데 일반적으로 Public Database Link로 정의하여 사용한다.
2. Public Database Link
   Local 데이타베이스의 모든 사용자가 Remote 데이타베이스의 데이타를 사용할 수 있다. 이 경우 Remote Database에 대해서 필요한 권한을 가지고 있어야한다.
3. Public Database Link Creation
   EX) CREATE PUBLIC DATABASE LINK sales.acme.com USING 'dbstring';
4. Dropping Databsae Link
   EX) DROP PUBLIC DATABASE LINK sales.acme.com;

분산처리 문제와 해결 방법

1. Two-Phase Commit을 인터럽트하는 Failures

[현상]
ORA-02050: Transaction ID rolled back, some remote dbs may be in-doubt
ORA-02051: Transaction ID committed, some remote dbs may be in-doubt
ORA-02054: Transaction ID in-doubt

만일 프로그램이 사용 중 위의 에러 가운데 하나가 발생한다면 그 트랜잭션에 대한 정보 자동적으로 저장되며 이 정보는 후에 분산 트랜잭션에 대한 Manual Recovery 시에 사용될 수 있다.

[조치 방법]
DBA는 특별한 Action을 취할 필요가 없다. RECO Process가 Session Tree의 모든 노드에서 같은 결과가 발생되도록 (즉 모두 Commit 혹 모두 Rollback) In-doubt 트랜잭션을 자동적으로 Recovery 한다. 그러나 장기적인 Power Failure인 경우 Lock된 리소스를 Release 하기 위해 강제적으로 Commit나 Rollback을 시켜준다.

2. 데이타의 Access를 금지시키는 Failure

[현상1.]
Transaction Time-out
ORA-02049: time-out : distributed transaction waiting for lock

Local의 DML 문장이 Remote의 다른 Transaction에 의해 Lock 되어 있는 데이타를 수정 및 삭제 하려고 할 때 Time-out이 발생하고 그 명령은 Rollback된다.

(Scenario)
[in REMOTE]
SQL>update dept set deptno=10;
[in LOCAL]
SQL>update dept set deptno=10;
SQL>update dept@ORA7 set deptno=10;

ORA-02049 : time-out

[조치 방법]
Local DB에는 Update가 발생하므로 항상 Rollback을 시켜야 한다. RemoteDB는 변경되지 않으므로 Time-out의 결과로 다른 조치는 필요 없다. 위 상황에서 Time-Out Interval은 DISTRIBUTED_LOCK_TIMEOUT 파라미터로 Time Out시간을 조정할 수 있다.

[현상2.]
Lock from In-doubt Transaction
ORA-01591 : Lock held by in-doubt distributed transaction ID

Local DB에 Lock을 발생시키는 DML 문장을 In-doubt 분산 트랜잭션에 의해 Lock된 Resource에 계속해서 실행할 경우 발생하는 에러

(Scenario)
(1) update dept@ORA7 set dname='AA';
(2) Remote DB down
(3) Commit
ORA-02054 : transaction 2.1.207 is in-doubt transaction
(4) select * from dept;
ORA-01591 : Lock held by in-doubt distributed transaction id 2.1.207
Local에서 dba_2pc_pending으로부터 In-doubt 트랜잭션 정보를 확인할 수 있다.

[조치 방법]
이 경우 SQL 문장은 즉시 Rollback 되 만 Lock이 계속해서 걸려있으면 DBA에게 알리도록 한다. 이런 현상은 드물게 발생하며 Network나 System Failure가 빠르게 복구되면 문제는 자동적으로 해결된다.

3. 두개 NODE 사이에 Connection이 만들어진 이후에 Network Failure가 발생한 경우

(Scenario)
(1) Forms 화면에서 Data Update
(2) Commit - DB Trigger에 의해서 Server Machine에 Server process가 생기며 Network와 Remote DB가 정상적인 경우 Commit 된다.
(3) Network Fail(Line 을 Disconnect)
(4) Forms 화면에서 Update
(5) Commit
(6) Forms Screen이 Holding 상태로 됨

[조치 방법]
Network이 복구되면 자동적으로 Commit 되지만 Network Failure가 장기화될 경우 강제로 Abort 시키는 조치가 필요하다. 강제로 Abort 이후 자동적으로 Rollback 되므로 더이상 조치가 필요하지 않다.

4. Network이 Down된 상태에서 Connection을 시도하려는 경우

(Scenario)
(1) Network Down
(2) Forms 화면에서 Data 수정
(3) Commit

[조치 방법]
모든 Transaction이 Rollback 되므로 더 이상 조치가 필요하지 않으나 다만 Network Down된 상태에서 Connect를 시도하려고 하는 시간을 줄여서 곧바로 Error가 발생되도록 해야 한다.

5. Server Machine의 orasrv가 Down된 상태에서 Connect를 하려고 하는 경우
: ORA-06108 error 발생

6. In-doubt 트랜잭션을 Manually Overriding
ORA-01591 Message가 발생하여 User가 조치를 요구하는 경우
In-doubt 트랜잭션이 Rollback Segment의 Extension를 막는 경우
Network 및 System Failure가 장기화 될 경우

t In-doubt 트랜잭션에 대한 정보를 얻는 방법

a.User Feedback을 기록

b.Local dba_2pc_pending View를 조회

c.Local dba_2pc_neighbors View를 조회

d.정상적으로 통신이 복구된 후에 Mixed Outcome Flag를 Check

* Record User Feedback

ORA-01591: Lock held by in-doubt distributed transaction 1.21.17
1.21.17 : local transaction id

- Query dba_2pc_pending

< global_db_name format >

global_database_name.hhhhhhhh.local_transaction_id

cf) global_database_name=global coordinator db name

hhhhhhhh=internal db id

- Query dba_2pc_neighbors

In-doubt 트랜잭션이 관 되어 있는 Connection에 대한 정보를 조회한다.
각 Connection에 대한 정보는 Connection이 Inbound, Outbound 이냐에 따라 다르다.

만일 Connection이 Inbound이면 그 Node는 다른 Node 의 Server이다. 만일 Connection이 Outbound이면 그 Node는 다른 Server의 Client이 . Interface Column 은 Local Node 혹은 Subordinate Node가 Commit Point Site 인지를 알려준다. 만일 Local Transaction id와 Global Transaction id가 일치하면 그 Node 는 Global Coordinator이다.

- Check for Mixed Outcome

트랜잭션이 Manual로 Commit 혹은 Rollback된 후에 해당 Row가 Pending 트랜잭션 테이블에 남아있다. 그 트랜잭션의 상태는 "forced commit" or "forced abort" 로 변경되어지고 Instance 사이에 Connection이 복구되면 RECO는 트랜잭션의 Global Outcome을 Check하여 정상적인 경우 Row를 삭제하지만 틀린 방법으로 트랜잭션을 Force 시키면 삭제되지 않고 Mixed Column이 "Yes" 로 변경된다.

7. Manual Commiting In-Doubt Transaction

```
COMMIT FORCE 'SALES.ACME.COM.55dic563.193.29';
COMMIT FORCE 'global_id', 829381993
                          (system commit number)
```

8. Manually Rolling Back In-Doubt Transaction

```
ROLLBACK FORCE '2.9.4';
```

9. Disabling and Enabling RECO

```
ALTER SYSTEM DISABLE DISTRIBUTED RECOVERY;
ALTER SYSTEM ENABLE DISTRIBUTED RECOVERY;
```

10. Commit Point Site 결정

. 번째 Commit하는 Node

. In-Doubt Data를 가질 수 없다.

. Prepared 상태로 될 수 없다.

. 분산 데이타베이스에 참여하는 시스템 중 가장 안정된 시스템에 가장 높은 값을 준다.

DBMS PACKAGE : DEBUGGING PL/SQL PROGRAMS USING DBMS_OUTPUT

Bulletin no : 10291

Introduction

To aid in the development of PL/SQL programs, stored procedures, and triggers,Oracle provides a public package named DBMS_OUTPUT for displaying debugging information. The procedures in this package provide much the same functionality as the printf() function in C.
To create the DBMS_OUTPUT package, the script DBMSOTPT.SQL must be run by the user SYS. This script is run automatically by the CATPROC.SQL script.
Both of these scripts can be found in $ORACLE_HOME/rdbms/admin on unix systems, or in the ORA_RDBMS directory on VMS. These scripts create the package - to run it you will also need execute privilege on it.

DBMS_OUTPUT and how it is used

The package is provided by Oracle to help programmers in debugging PL/SQL programs, procedures, and triggers. It is not designed to be used for report displaying, or formatting output to the standard output. The package is designed to display debugging info in stream-like processing. That is, the variables, string literals, and other info are written to a buffer. The buffer is divided into lines, each of which can have up to 255 characters. There is no other structure on the buffer. DBMS_OUTPUT is most helpful when used with SQL*Plus and SQL*DBA. In order to use the package with these two tools, you must set the 'serveroutput' before any info can be displayed to the screen.Use the following command in SQL*Plus or SQL*DBA:

```
SQL> set serveroutput on
```

Note that 'serveroutput' is one word. If serveroutput is not turned on, nothing will be displayed even if you call the DBMS_OUTPUT.ENABLE packaged procedure.

The way DBMS_OUTPUT works is as follows

In a PL/SQL program, calls to DBMS_OUTPUT.PUT_LINE will add lines to the buffer.
Nothing is actually displayed until the program executes completely and control is transfered back to SQL*Plus or SQL*DBA. At this point, if serveroutput is on, SQL*Plus or SQL*DBA will make a call to DBMS_OUTPUT.GET_LINES and print the results to the screen.

If your PL/SQL program runs for a long time, you actually have to wait until the program executes and control goes back to SQL*Plus in order to see any information displayed by DBMS_OUTPUT.PUT_LINE. Consider the following example - I had to wait about 13 minutes before the output was displayed:

```
-- PL/SQL script out1.sql displays time in hh-mi-ss format at the beginning
-- of the script, loops to eat cpu time, and displays time at the end of the
-- script.
-- Calls to the DBMS_OUTPUT package are capitalized for emphasis - as with
-- all database objects they are really case-insenitive.
DECLARE
  y1      varchar2(80);
  y2      number;
BEGIN
  SELECT to_char(sysdate, 'hh-mi-ss')
    into y1
    from dual;

  DBMS_OUTPUT.PUT_LINE(y1);

  y2 := 0;
  LOOP
    IF y2 > 100000 THEN
      exit;
    END IF;

    y2 := y2 + 1;
  END LOOP;

  SELECT to_char(sysdate, 'hh-mi-ss')
    into y1
    from dual;

  DBMS_OUTPUT.PUT_LINE(y1);
END;
/

SQL> @out1
12-43-26
12-56-19

PL/SQL procedure successfully completed.
```

Notice how long the program took to execute. Both prints were done at the same time - they are not printed as the block is executing.

Calls to DBMS_OUTPUT.PUT_LINE are dispalyed even if a rollback was issued at the end of the PL/SQL script, the following example demonstrates this:

```
SQL> BEGIN
  2    dbms_output.put_line('test rollback');
  3    ROLLBACK;
  4  END;
  5  /
test rollback
```

```
PL/SQL procedure successfully completed.
```

Even if the script has an unhandled exception, all the calls to DBMS_OUTPUT.PUT_LINE before the unhandled exception are displayed. Example:

```
SQL> DECLARE
  2    x number;
  3  BEGIN
  4    dbms_output.put_line('test errors');
  5    SELECT 'Junk' into x from dual;
  6  END;
  7  /
test errors
declare
 *
ERROR at line 1:
ORA-06502: PL/SQL: numeric or value error
ORA-06512: at line 5
```

Notice how the line was displayed regardless of the unhandled exception. It should be noted that in versions of SQL*Plus prior to 3.1.2.2, SQL*Plus did not call dbms_output.get_lines to display the information in the buffer if an unhandled exception occurred.

Procedures in DBMS_OUTPUT package

procedure enable(bufsize IN default 2000);

This procedure enables calls to put, put_line, new_line, get_line, and get_lines. If the dbms_output package has not been enabled, calls to these procedures will be ignored. If you use the dbms_output.disable procedure in your PL/SQL block, you will have to call dbms_output.enable if you later want to re-enable calls to procedures put, put_line, new_line, get_line, and get_lines.

This procedure is used to change the default size of the buffer. The default and minimum size is 2000. Maximum is 1,000,000. When you set serveroutput on,
dbms_output.enable is implicitly called, with the default size of 2000.

Example:

```
SQL> set serveroutput on
    SQL> BEGIN
      2    dbms_output.put('Example, show that enable is not needed ');
      3    dbms_output.put_line('when "set serveroutput on" is used.');
      4  END;
      5  /
    Example, show that enable is not needed when "set serveroutput on" is used.

PL/SQL procedure successfully completed.
```

procedure disable;

This procedure will disable all calls to put, put_line, new_line, get_line, and get_lines. It will also clear the buffer of all information. If you call the procedure after calling

dbms_output.put_line, nothing will be displayed. The SQL*Plus and SQL*DBA command "set serveroutput off" does the same thing.
Example:

```
SQL> set serveroutput on
SQL> BEGIN
  2    dbms_output.put_line('Example, show that enable is not needed');
  3    dbms_output.disable;
  4  END;
  5  /

PL/SQL procedure successfully completed.
```

Notice that the buffer was cleared before the block ended, so nothing was displayed.

```
procedure put(a varchar2);
procedure put(a number);
procedure put(a date);
Input parameters:
  -- a
  --   Item to buffer
```

This procedure puts the specified piece of information in the buffer. The procedure dbms_output.new_line or dbms_output.put_lines must be called after one or more calls to dbms_output.put to end the line. A line that has not been terminated with a newline character will not be displayed by a subsequent call to dbms_output.get_lines.

```
procedure put_line(a varchar2);
procedure put_line(a number);
procedure put_line(a date);
Input parameters:
  -- a
  --   Item to buffer
```

This procedure puts the specified piece of information in the buffer followed by an end-of-line marker.

```
procedure new_line;
```
This procedure places an end-of-line marker in the buffer.

```
procedure get_line(line OUT varchar2, status OUT integer);
procedure get_lines(lines OUT chararr, numlines IN OUT integer);
Output parameters:
  -- line
  -- Holds the line to be displayed
  -- status
  -- This will be 0 upon successful completion of the call. A 1 means there are no more lines.
```

```
  -- lines
  --  Type chararr is a PL/SQL table defined in the DBMS_OUTPUT package.
  --  type chararr is table of varchar2(255) index by binary_integer;
  --  When using this type in your Pl/SQL script, you should declare a
  --  variable as
  --    declare
  --      var1 dbms_output.chararr;
  --   If you redefine the type again, an Oracle error will be issued.
Input parameters:
  -- numlines
  --   The maximum number of lines to be returned.
```

These two procedures are used to get lines back from the buffer into varchar2 variables declared in the PL/SQL program. If you call put_line after calling get_line or get_lines, the buffer will be cleared and the only thing displayed is the information provided with calls to put_line afterward.

The different between get_line and get_lines is :

get_lines can return more than one line from buffer.

Example:

```
SQL> set serveroutput on
SQL> declare
  2    y1 varchar2(80);
  3    y2 number;
  4  begin
  5    dbms_output.put_line('First line before dbms_out.get_line');
  6    dbms_output.put_line('Second line before dbms_out.get_line');
  7    dbms_output.get_line(y1,y2);
  8    dbms_output.put_line('Line after dbms_out.get_line');
  9  end;
 10  /
Line after dbms_out.get_line

PL/SQL procedure successfully completed.
```

Notice how the buffer was cleared by the call to get_line, so the only line displayed is the argument to put_line after the get_line call.

Notes

1) As the package is implemented currently, there is no way to flush the buffer, print the contents, and then add new lines to it. This is because the buffer is not actually displayed to the screen until the block ends, and SQL*Plus or SQL*DBA calls get_lines and prints the buffer.

2) If you run into the error:

ORA-20000: ORU-10027: buffer overflow, limit of 2000 bytes just call enable with a larger buffer size - it can go up to 1,000,000. It is possible also to modify buffer size in SQL*PLUS and SQL*DBA by using the following command:
SET SERVEROUTPUT ON SIZE n
where n is an integer between 2,000 and 1,000,000.

3) There is little reason to use the procedures in DBMS_OUPUT with the precompilers. You can make calls to put_line, but the buffer will not be printed to the screen automatically. You could call get_line to get data from the buffer back into host variables and display them using regular host language output statements, but in this case, it would make more sense to assign to the host variables directly and bypass the buffer entirely.

4) DBMS_OUTPUT is not designed to be a report writer. It is meant to be used for debugging. It is not really powerful enough for much more than this.

5) For more information, look at Appendix A of the Oracle7 Server Application Developer's Guide. The DBMSOTPT.SQL file also has descriptions of the package and the procedures.

DISK STRIPING

Bulletin no : 10153

---

1. 서로 다른 DISK의 DATAFILE 로 TABLESPACE 를 만든다.

```
$ sqlplus system/manager
  SQL> CREATE TABLESPACE disk_stp
       DATAFILE '/disk1/file1.dbf' SIZE 100M,
                '/disk2/file2.dbf' SIZE 100M,
                '/disk3/file3.dbf' SIZE 100M;
```

2. 'CREATE TABLE' 명령을 사용하여 disk_stp TABLESPACE 에 TABLE을 생성한다.

```
$ sqlplus scott/tiger
  SQL> CREATE TABLE striptab (
       col1  NUMBER(2),
       col2  VARCHAR2(10))
       TABLESPACE disk_stp
       STORAGE ( INITIAL 95M   NEXT 95M
                 MINEXTENTS 3  PCTINCREASE 0);
```

DOWN TIME WITH OPS

Bulletin no : 10043

I can only answer in a generic format. First, to understand how long fail over will take, you must first understand the different operations that takes place on fail over. My short answer would be, no, a 5 second average down time is not feasible. My long answer is, it depends:

1) How long does it take for the other node(s) to realize that a node in the cluster is down?
2) How long it takes to "remaster" the locks?
3) How long does it take for Oracle to perform instance recovery?

All of these play a part on how long fail over will take. My testing on fail over has been focused on Pyramid's, but I believe it should be the same.

#1 is a tunable parameter - how often do you want each node in the cluster to send a heart beat across to the other nodes? On our machines, it's set to 20 seconds before cluster rebuild and remastering of locks occur if a node goes down.

#2 Only takes minutes at most. I've been testing remastering of locks any where between 1000 locks -> 200,000 locks and worst case scenario(remastering all 200,000 locks) has only taken less than 4 minutes. Setting gc_db_locks = 1000 only took seconds to remaster the locks.

#3 is probably the most time consuming. It's really dependent upon how much redo has been generated and I don't have any timing statistics on this.

A few things that we need to realize is that not all the locks may need to be remastered - only the locks which were found "dubious" so the timing statistics I gave on that are worst case scenario's for remastering locks.

Also, there's a possibility of absolutely no wait time if the data block you're trying to access is mastered by a node which is up and running.

DYNAMIC SQL(DBMS_SQL PACKAGE)

Bulletin no : 10172

DYNAMIC SQL 이란?

PL/SQL은 binding이 compile시에 일어나므로 database object의 name이 compile시에 고정되어야 하는 등의 제한이 있다. PL/SQL 2.1(RDBMS 7.1)이후 version에서는 DBMS_SQL package로 Dynamic SQL statement의 사용을 가능하게 한다. 이는 database object의 name을 runtime에 줄 수 있을 뿐더러 DDL문장을 기술 할 수도 있는 장점이 있다.

Function open_cursor
  : SQL문의 실행에 필요한 새로운 cursor를 열고 cursor ID number를 return 한다.

Function is_open
  : 주어진 cursor가 현재 open되어 있으면 TRUE를, 아니면 FALSE를 return한다.

Procedure parse
  : statement를 check하고 cursor와 결합시킨다.

Procedure bind_variable
  : program내에서 data를 저장한 placeholder의 값을 제공하는 역할을 한다.

Procedure define_column
  : cursor로부터 select된 column의 값을 받는 변수를 지정한다.

Function execute
  : SQL문을 실행하고 처리된 Row의 수를 return한다.
    (insert, update, delete인 경우에만 해당)

Function fetch_rows
  : cursor로부터 row를 fetch하고 실제로 fetch된 row의 수를 return 다. 이 row들은 buffer에 들어가며, column_value를 호출하여 읽어들여야 한다.

Function execute_and_fetch
  : execute와 fetch row를 동시에 수행하고 실제로 fetch된 row의 수를 return한다.

Procedure variable_value
  : 주어진 변수의 값을 return한다.

Procedure column_value
  : fetch_rows에 의해 fetch된 data의 값을 return한다.

Procedure close_cursor
  : cursor를 닫는다.

Using the DBMS_SQL Package to Execute DDL statements:

< Example 1: >

procedure는 table을 create하는 것으로 table name과 column,type을 parameter로 받는다.

```
CREATE OR REPLACE PROCEDURE ddlproc (tablename varchar2,
  cols varchar2) AS
  cursor1 INTEGER;
BEGIN
  cursor1 := dbms_sql.open_cursor;
  dbms_sql.parse(cursor1, 'CREATE TABLE ' || tablename ||
    ' ( ' || cols || ' )', dbms_sql.v7);
  dbms_sql.close_cursor(cursor1);
end;
/

SQL> execute ddlproc ('MYTABLE','COL1 NUMBER, COL2
                       VARCHAR2(10)');

PL/SQL procedure successfully completed.

SQL> desc mytable;
 Name                           Null?    Type
 ------------------------------ -------- ----
 COL1                                    NUMBER
 COL2                                    VARCHAR2(10)
```

DDL statement는 parse command에 의해 수행된다.
그러므로 DDL statement에서는 bind variable을 사용할 수가 없다.

다음은 DDL statement내에 bind variable을 사용한 잘못된 예이다.

```
CREATE OR REPLACE PROCEDURE ddlproc (tablename
  VARCHAR2, colname   VARCHAR2,  coltype   VARCHAR2)
AS
  cursor1 INTEGER;
  ignore  INTEGER;
BEGIN
  cursor1 := dbms_sql.open_cursor;
  dbms_sql.parse(cursor1, 'CREATE TABLE :x1 (:y1 :z1)',dbms_sql.v7);
  dbms_sql.bind_variable(cursor1, ':x1', tablename);
  dbms_sql.bind_variable(cursor1, ':y1', colname);
  dbms_sql.bind_variable(cursor1, ':z1', coltype);
  ignore := dbms_sql.execute(cursor1);
  dbms_sql.close_cursor(cursor1);
end;
/
```

Procedure를 create했을때에는 error를 만나지 않았지만, runtime에는 "ORA-00903: invalid table name" 이라는 error가 난다.

```
SQL> execute ddlproc ('MYTABLE', 'COL1', 'NUMBER');
begin ddlproc ('MYTABLE', 'COL1', 'NUMBER'); end;

*
ERROR at line 1:
ORA-00903: invalid table name
ORA-06512: at "SYS.DBMS_SYS_SQL", line 239
ORA-06512: at "SYS.DBMS_SQL", line 25
ORA-06512: at "SCOTT.DDLPROC", line 8
ORA-06512: at line 1
```

< Example 2: >

table을 drop하는 procedure로 table name을 parameter로 받는다.

```
create or replace procedure droptable (table_name varchar2) as
  cursor1 integer;
begin
  cursor1 := dbms_sql.open_cursor;
  dbms_sql.parse(cursor1, 'DROP TABLE ' || table_name,
                          dbms_sql.v7);
  dbms_sql.close_cursor(cursor1);
end;
/

SQL> begin
  2    droptable('MYTABLE');
  3  end;
  4  /

PL/SQL procedure successfully completed.
```

< Example 3: >

DDL statemenet를 수행하는 procedure로 DDL statement자체를 parameter로 받는다.

```
create procedure anyddl (s1 varchar2) as
  cursor1 integer;
begin
  cursor1 := dbms_sql.open_cursor;
  dbms_sql.parse(cursor1, s1, dbms_sql.v7);
  dbms_sql.close_cursor(cursor1);
end;
/
```

```
SQL> execute anyddl('CREATE TABLE MYTABLE (COL1 NUMBER)');

PL/SQL procedure successfully completed.

SQL> desc mytable;
 Name                           Null?    Type
 ------------------------------ -------- ----
 COL1                                    NUMBER

SQL> execute anyddl('drop table mytable');

PL/SQL procedure successfully completed.
```

Using the DBMS_SQL Package to Execute Dynamic SQL Statements:

DBMS_SQL package dynamic SQL statement를 수행하는데 이용되어 질 수 있는데 이는 rumtime전에 statement의 일부분 혹은 전체를 알 수 없는 경우에 쓰여진다.

< Example 4: >

이 예제는 runtime시에 주어진 number보다 더 높은 employee number를 가진 모든 employee 의 이름과 employee number를 return한다.

```
CREATE or REPLACE PROCEDURE rows_greater_than (low_value  number)
  AS
  cursor1  integer;
  rows_processed  integer;
  myempno number;
  myename varchar2(20);
BEGIN
  cursor1 := dbms_sql.open_cursor;
  dbms_sql.parse (cursor1, 'select empno, ename from emp
        where empno > :x', dbms_sql.v7);
  dbms_sql.bind_variable(cursor1, 'x', low_value);
  dbms_sql.define_column (cursor1, 1, myempno);
  dbms_sql.define_column (cursor1, 2, myename, 20);
  rows_processed := dbms_sql.execute (cursor1);
  loop
    if dbms_sql.fetch_rows (cursor1) > 0 then
      dbms_sql.column_value (cursor1, 1, myempno);
      dbms_sql.column_value (cursor1, 2, myename);
      dbms_output.put_line(to_char(myempno) || '   ' || myename);
    else
      exit;
    end if;
  end loop;
  dbms_sql.close_cursor (cursor1);
EXCEPTION
```

```
  WHEN OTHERS THEN
    dbms_output.put_line(sqlerrm);
    if dbms_sql.is_open (cursor1) then
      dbms_sql.close_cursor (cursor1);
    end if;
END;
/
```

DBMS_OUTPUT package을 사용하기 전에 먼저 SET SERVEROUTPUT ON command를 사용하여야 한다.

```
SQL> set serveroutput on
SQL> execute rows_greater_than(7500);
7521   WARD
7566   JONES
7654   MARTIN
7698   BLAKE
7782   CLARK
7788   SCOTT
7839   KING
7844   TURNER
7876   ADAMS
7900   JAMES
7902   FORD
8100   MILLER

PL/SQL procedure successfully completed.

SQL> execute rows_greater_than(8000);
8100   MILLER

PL/SQL procedure successfully completed.
```

< Example 5: >

이 예제는 where clause의 parameter를 column_name과 operator(<,<=,=,>=,>), new_value로 받아서 employee 이름과 number를 return한다.

```
CREATE or REPLACE PROCEDURE get_rows (column_name
varchar2,  comparison_type varchar2, new_value number)
 AS
  cursor1  integer;
  rows_processed  integer;
  myempno number;
  myename varchar2(20);
BEGIN
  cursor1 := dbms_sql.open_cursor;
```

```
  dbms_sql.parse (cursor1, 'select empno, ename from emp
                            where ' || column_name ||
                            ' ' || comparison_type || ' :x',
                    dbms_sql.v7);
  dbms_sql.bind_variable(cursor1, 'x', new_value);
  dbms_sql.define_column (cursor1, 1, myempno);
  dbms_sql.define_column (cursor1, 2, myename, 20);
    rows_processed := dbms_sql.execute (cursor1);
  loop
    if dbms_sql.fetch_rows (cursor1) > 0 then
      dbms_sql.column_value (cursor1, 1, myempno);
      dbms_sql.column_value (cursor1, 2, myename);
      dbms_output.put_line(to_char(myempno) || '   ' || myename);
    else
      exit;
    end if;
  end loop;
  dbms_sql.close_cursor (cursor1);
EXCEPTION
  WHEN OTHERS THEN
    dbms_output.put_line(sqlerrm);
    if dbms_sql.is_open (cursor1) then
      dbms_sql.close_cursor (cursor1);
    end if;
END;
/

SQL> begin
  2    get_rows('EMPNO', '<', 2000);
  3  end;
  4  /
1111

PL/SQL procedure successfully completed.

SQL> execute get_rows('SAL', '>', 3000);
7566   JONES
7788   SCOTT
7839   KING
7902   FORD

PL/SQL procedure successfully completed.

SQL> begin
  2    get_rows('DEPTNO', '>=', 20);
  3  end;
  4  /
7369   SMITH
```

```
7499   ALLEN
7521   WARD
7566   JONES
7654   MARTIN
7698   BLAKE
7788   SCOTT
7844   TURNER
7876   ADAMS
7900   JAMES
7902   FORD

PL/SQL procedure successfully completed.
```

< Example 6: >

이 예제는 where clase 전체를 runtime시에 parameter로 받는다.

```
CREATE or REPLACE PROCEDURE get_rows (where_clause varchar2) AS
  cursor1  integer;
  rows_processed  integer;
  myempno number;
  myename varchar2(20);
BEGIN
  cursor1 := dbms_sql.open_cursor;
  dbms_sql.parse (cursor1, 'select empno, ename from emp where '
              || where_clause, dbms_sql.v7);
  dbms_sql.define_column (cursor1, 1, myempno);
  dbms_sql.define_column (cursor1, 2, myename, 20);
    rows_processed := dbms_sql.execute (cursor1);
  loop
    if dbms_sql.fetch_rows (cursor1) > 0 then
      dbms_sql.column_value (cursor1, 1, myempno);
      dbms_sql.column_value (cursor1, 2, myename);
      dbms_output.put_line(to_char(myempno) || '   ' || myename);
    else
      exit;
    end if;
  end loop;
  dbms_sql.close_cursor (cursor1);
EXCEPTION
  WHEN OTHERS THEN
    dbms_output.put_line(sqlerrm);
    if dbms_sql.is_open (cursor1) then
      dbms_sql.close_cursor (cursor1);
    end if;
END;
/
```

```
SQL> execute get_rows('ENAME = ''KING'' ');
7839   KING

PL/SQL procedure successfully completed.

SQL> execute get_rows('SAL > 1000 AND DEPTNO = 10');
7782   CLARK
7839   KING
8100   MILLER

PL/SQL procedure successfully completed.
```

< Example 7: >

이 예제는 non-query SQL statement를 실행한다.

```
create procedure anysql (s1 varchar2) as
  cursor1 integer;
  return_value integer;
begin
  cursor1 := dbms_sql.open_cursor;
  dbms_sql.parse(cursor1, s1, dbms_sql.v7);
  return_value := dbms_sql.execute(cursor1);
  dbms_sql.close_cursor(cursor1);
end;
/


SQL> execute anysql('CREATE TABLE MYTABLE (COL1 number, col2 varchar2(3))');

PL/SQL procedure successfully completed.

SQL
SQL> desc mytable;
 Name                           Null?    Type
 ------------------------------ -------- ----
 COL1                                    NUMBER
 COL2                                    VARCHAR2(3)

SQL> execute anysql('INSERT INTO MYTABLE VALUES(1, ''ABC'')');

PL/SQL procedure successfully completed.

SQL> begin
  2    anysql(
  3      'declare
  4         var1 varchar2(3);
  5       begin
```

```
  6         select col2
  7           into var1
  8           from mytable
  9           where col1 = 1;
 10         dbms_output.put_line(''var1 = '' || var1);
 11       end;');
 12  end;
 13  /
var1 = ABC

PL/SQL procedure successfully completed.
```

****

더욱 자세한 사항은 Oracle7 Server Documentation Addendum을 참조.

****

Forms 3.0은 pl/sql 1.1만을 지원하므로 dynamic SQL을 쓸 수 없으며 stored procedure를 만들어 호출해서 사용해야 한다. 이때 Forms가 부른 stored procedure나 function은 그 내에서 commit을 사용할 수 없게 되어 있기 때문에 COMMIT이나 ROLLBACK을 만나면 ORA-00034: Commit and Rollback from PL/SQL disabled for this session 이란 error가 난다.

그러므로 DDL statement도 사용을 할 수 없다.

ENTERPRIZE BACKUP UTILITY

Bulletin no : 10395

1. The Problem

과거 메인 프레임 환경에서 사용하던 고객들이 저 비용, C/S환경으로 Migration시에는 주로 Open system과 Oracle RDBMS를 채택하고 있다. 이런 관계로 해당 제품 시장은 고 성장을 구가하고 있기도 하다.

이러한 고 성장을 가능케 하는 요소는 물론 H/W,S/W에서의 혁신적인 진보이나, 메인 프레임급에 걸맞는 대용량 DB를 관리하고 Backup하는 데 필수적인 System management tool은 과거와 크게 달라진 바가 없다.

tar, cpio, dd같은 UNIX backup tool들은 실제 backup media의 관리 측면에서 별 기여 를 하지 못하고 있다. 대용량 DB를 위한 Backup script는 source만 해도 수백line에 달하며 test와 troubleshoot에서도 많은 문제점을 갖고 있다. 뿐만 아니라 대용량 Database(이하 VLDB)의 Restore는 Backup보다도 훨씬 더 복잡한 문제를 갖고 있다.

최근의 Backup file이 어느 곳에 저장 되었는지를 알기 위해 Backup tape을 분류하고, damaged file들을 restore 해야 하는 등의 일련의 작업이 필요하다.

2. The Solution

최근에 몇몇 media management vendor들은 media management기능, scheduling기능, 강력한 security 기능, UNIX system에서 제공하는 backup tool이상의 신뢰성을 갖는 강력한backup product를 공급하기 시작했다. 이러한 product들은 system level에서의 관리를 용이하게 해주는 장점은 있으나 이미 위에서 지적된 VLDB의 backup시 제기되는 문제점들을 완전히 해결해 주지는 못한다.

Oracle7 Enterprise Backup Utility(이하 EBU)는 이런 문제점을 해결하는 solution이다. EBU는 backup/restore시 media management product와 강력한 인터페이스를 제공하며 다른 어느것과 비교 할 수 없는 강점을 갖고 있다.

3. Introducing Oracle7 Enterprise Backup Utility

EBU는 open system상에서 메인 프레임급 성능을 제공하여 한 기업 전반에 걸쳐 전략적으로 중요한 역할을 감당하게 된다. 이 utility는 DB의 신속한backup과 restore를 위해disk와 backup될 media device에서H/W parallelism을 이용한다.open system의 full capacity를 이용하기 위해 device가 add 될 때 performance가 증가된다.parallel H/W, backup configuration check ,error detection and cleaning ,restore시의 database의block verification과 같은 특징들이EBU가 high reliability를 갖게 하는 특징들이다.

Oracle7 데이타베이스는 data files, control files, redo log files로 구성되는데 EBU는data file, control file을 backup받고 archived redo log file을 받는다. (release2.0.8) Media failure의 경우 DB를 full restore/partial restore가 가능하며 data file도 선택적으로 최근의 last backup까지, 어느 일정 시점 까지 의 선택이 가능하다. 또한 recovery도 일정 시점으로 완전recovery가 가능하다.

4. Architecture

EBU는 크게 두 부분으로 구성되는데 backup하려 하는 DB에 관한 모든 current정보와history를 관리하는 catalog부분, 그리고 실행 파일 부분으로 구성된다. 실행 파일들과 catalog는 반드시 같은

host내에 존재할 필요는 없다. 실행 파일들은 다음으로 구성된다.

*obackup : 모든 다른 프로세스들을 monitor하며 backup하려하는 DB, catalog와 의 communication을 담당한다.또한 Instance Manager를 running시킨다.

*brio : obackup, brdk, brtp사이 조정 역할을 수행한다.parallel I/O stream당 하나의 brio process가 발생한다.

*brdk : Disk file의 read/write를 조절한다. File당 하나의 brdk를 발생시키므로 만일 여러 개의 file들을 하나의 single data stream으로 결합시키는 경우 하나의 I/O stream내에 여러 개의 brdk process를 create한다.

*brtp : Tape file들의 read/write를 조절한다. I/O stream당 하나의 brtp create.

*brd : Instance Manager로써 backup catalog, obackup을 monitor하는 daemon process이다. backup을 위한 cleanup과 비정상 종료된 작업을 복구한다.

Backup동안 database file들은 disk process인 brdk에 의해 읽혀져 backup buffer area로 넘겨지고 그 후 third vendor의 media management s/w를 경유하여 backup device에 brtp process에 해 write 된다. Restore시는 반대의 과정이 진행된다. 위의backup buffer는 hard disk와 tape device간에 발생하는 I/O상의 speed mismatch를 조절하는 역할을 한다.

5. EBU의 장점(Benefits)

Third vendor의 media management product와 함께 사용되는 EBU는 기존의 UNIX 에서 제공하는 backup tool보다 다음과 같은 강력한 장점을 지닌다.

(1) 강력한 Reliability.

*. Consistent backup procedures

기존의 hot backup script의 경우 만일 datafile이 추가될 경우 script에 새로 지정해 주어야 하며 VLDB의 경우 script만 해도 백line에 달하여 관리상 번거로운 게 사실 이였다. 그러나 EBU의 경우는 어느 datafile이 어느directory에 위치하는지를 작업자가 알 필요가 없고 script는 짧고 단순하여 누구든 쉽게 알아볼 수 있다. 또한 DB 구성이 자동으로 확인되며 backup media를 자동으로 인식 및 관리한다.

예).full online backup script

```
backup online database
db_name = "PRODB"
oracle_sid = "PROD"
```

.partial online backup script

```
backup online
db_name = "PRODB"
oracle_sid = "PROD"
   tablespace = "SALES"
```

.full restore script

```
restore database
db_name = "PRODB"
oracle_sid = "PROD"
```

이와 같이 간단한 script에 의해 DBA는 VLDB를 쉽게 backup과 restore를 할 수 있으며 작업에 필요한 세부정보는 backup catalog내에 있게 된다.

*. End-to-End checksum
Backup시 EBU는 각 backup data의 byte를 계산하고, restore시도 계산하여 정상 수행 여부를 판단 시 checksum 값을 상호 체크 한다.

*. Instance Manager
Instance Manager는 backup/restore시에 진행 상황을 monitor하고 작업 중에 에러가 발생시 복구 및 각종resource를 release시키는 기능을 담당하게 한다.

(2) High Performance
다음과 같은 EBU의 특징과 고속media device의 지원 아래 VLDB의 backup을 적은시간 내에 완전하게 수행 할 수 있게 되었다.

*. Parallel Hardware Capability
Backup/restore시 multiple device를 사용하여 동시에 여러 tablespace에 대한 작업을 수행 할 수 있다. VLDB와 같이 다수의 tablespace, datafile이 여러 disk에 분산되어 있는 경우 이와 같이 물리적으로 다수의 device를 parallel하게 사용함으로서 performance를 극대화 시킬 수 있다. Backup시 동시 사용되는 device의 수와 performance는 정비례 하게 나타난다.

*. Multiplexing
EBU는 backup device가 최고의 속도로 backup을 수행 할 수 있게 script에 지정 할 수가 있다.이것은 backup media access속도보다 disk access 속도가 훨씬 작은 경우에 여러 disk에 분산된 datafile들을 한device에 multiplexing되게 지정함으로 가능하다.

*. Null Block Compression
EBU는 backup시 null data block을 skip시키고 restore시 재구성함으로써 공간을 절약하며 performance를 증가시킨다.

*. Buffered I/O
EBU는 disk I/O와 device I/O사이의 access speed의 mismatch를 이기 위하여 모든 I/O stream을 bufferring한다. 양쪽의 I/O는 shared memory buffer를 거치게 함으로써 backup/restore수행 중 항시 일정 속도를 유지하게 한다. shared memory는 user가 parameter로 구성할 수 있으며 기본적으로는 parallel data stream의 개수와 buffer size에 dependent하다.

(3) Availability
한 기업에서 전략적으로 중요한 DB들은 대부분 24시간*7일 full로 가동되며, 사용 중에 backup을 수행할 수 밖에 없다. Offline full backup은 당연히 지원되며, EBU는 DB를 online상태로 full backup이 가능 며, 또한partial backup도 가능하다. 기존의 online backup이 backup으로 인해 performance를 저하시키는 단점이 있었으나 EBU는DB 운영에 최소의 영향만을 미친다.
Recovery시에도 full로 restore 시킬 수 있으며 나머지는 사용 중인 상태에서 필요 부분만 partial restore가 가능하다. 일단 EBU가 data를 restore시킨 후에는 기존의 Oracle7의 표준 recovery process가 적용되어 진다.

(4) Usability
*. Auto-configuration
Online backup을 수행 시 EBU는 current DB configuration과 backup catalog

내의 가장 최근의 configuration정보를 비교하여 backup catalog정보가 더 오래 전의 것이면 EBU는 DB의 backup직전으로 update시킨다. Auto-configuration은 default이며 명시적으로 지정하여 disable시킬 수 있다.

Offline backup시는 backup script내에 backup command기술 전에 register라는 명령어를 기술 해야만 configuration정보를 update시킬 수 있다.

*. Light-Out backups

Third party의 media management s/w를 이용하여 EBU를 operator의 개입 없이 자동으로 수행이 가능하다. UNIX의 cron에서도 자동으로 수행되게 할 수 있다.

*. Raw device support

EBU는 raw device로 잡힌 DB도 operator의 특별한 조치 없이 일반 file system의 backup과 똑같이 수행할 수 있게 한다.

*. Aggregated restore

기업의 mission-critical system에서의 backup 전략들은 흔히 partial backup에 의존하게 된다. 이와 같은 환경에서는 backup data내의 모든file에 대한 current version을 검색하는 일이 중요한데 EBU는 이것을 자동으로 수행한다. partial backup data는 restore시 media failure이전에 취해진 가장 최근의 full backup data와 aggregated 되어져 이용된다.

*. Point-in-Time restore

EBU는 backup catalog내에 backup하려하는 DB에 대한 모든 configuration history를 갖고 있어서, 만일 하나의 tablespace가 drop된 후 다시 필요하게 되면 비록 그 후 일정 시간이 경과되고 많은 변경이 일어났더라도 drop직전으로 일부분만 restore하여 사용이 가능하게 할 수 있다. 이 같은 tablespace는 다른 machine에 restore 될 수도 있어 현재의 DB작동에 아무 영향을 미치지 않고 수행할 수가 있다.

*. Backup catalog

EBU에 의해 제공되는 backup catalog는 모든 backup대상 DB의 configuration과 history의 저장소로써 제공된다. backup catalog에 저장된 정보는 backup된 file, tablespace 그리고 multiplexing정보, 마지막backup이나 restore의 시간기록, backup type(full or subset),backup들이 저장 되어 있는 file set등을 포함하고 있다. 하나의 backup catalog는 기업 내 모든 분산된 dB들을 관리할 수 있다.

*. Dry Runs

VLDB를 backup시 겪는 문제 중 하나는 과연 script가 정상적으로 수행이 완료되는지를 검증하는 것이다. 특별한 방법이 없는 한 긴 시간 동안 수행시켜 끝난 후 확인 할 수 밖에 없는데 EBU는 I/O없이 간단히 test로 수행을 시킬 수가 있다. 이로 인해 신뢰성 있는 backup전략을 DBA가 수립 가능하게 한다.

*. Enterprise-wide backup

이것은 backup이 단순히 single database에 국한되지 않고 한 기업 전체에 걸쳐서 global하게 수행 가능함을 말한다. 예로 전국적으로 다수의 site에 database가 구축되어진 상황에서 각database들이 two phase commit,snapshot 같은 분산처리 작업을 통해 다른 database들을 synchronous하게 또는 asynchronous하게 update할 경우 기업 전체를 하나의 logical database로 보고 이런 logical database를 중앙에서 EBU를 통해 backup시킬 가 있다.EBU는 중앙의 catalog내에 모든database

들에 대한 backup정보를 간직하고 있다.

이와 같은 중앙의 한 지점을 통하여 기업 전체 모든 database들에 대한 backup수행 기능은 EBU의 가장 큰 장점 중 하나이며 이런 경우 N/W상에 대량의 data가 전송되어 network의 bandwidth가 backup의 중요한 요소로 부각된다.

6. Flexibility

*. EBU API

EBU는 앞에서도 소개되었듯이 단독으로 사용되는 것이 아니고 third-part에서 제공하는 media management product와 같이 사용된다. 이때 두 product사이의 interface는 backup/restore Application Programming Interface(API Glue)에 의해서 이루어 진다. 이것은 third part에 의해 제공되며 EBU install시 installer에 의해 자동적으로 access된다.

*. Media management products

Epoch*s EpochBackup, IBM Adstar*s Distributed Storage Manager, Legato Systems*NetWorker, HP*s Omniback*, SprectraLogic*s Alexandria, StorageTek*s REEL, etc.

*. Media device

4mm and 8mm DATS, 3480s, 3490s, WORM, Writable Optical Devices and automated systems such as stackers and silos.

7. Pricing

EBU는 무상으로 공급되며 사용을 위해서는 third vendor의 media management product를 구입하여 설치하여야 한다.

E7XP/IMP 를 이용하여 SCOTT의 데이타를 LARRY로 옮기는 방법

Bulletin no : 10166

---

scott의 데이타를 larry 로 옮기려면 export/import 를 이용해야 한다.
larry가 만들어져 있지 않다면 다음과 같이 만든다.

```
$sqlplus system/manager

SQL>create user larry
    identified by lion
    default tablespace users
    temporary tablespace temp
    quota unlimited on users;
```

default tablespace, temporary tablespace 는 시스템에 따라 알맞게 설정한다.

다음에 권한을 부여한다.

```
sql>grant connect,resource to larry
sql>revoke unlimited tablespace from larry
```

물론 users 테이블스페이스는 이미 존재한다고 가정한다.

scott로 익스포트를 받고 larry로 임포트 한다.

```
$exp scott/tiger owner=scott file=scott.dmp
$imp larry/lion fromuser=scott touser=larry file=scott.dmp
```

만약 scott 가 dba 권한을 갖고 있었다면 다음과 같이 임포트를 해야 한다.

```
$imp system/manager fromuser=scott touser=larry file=scott.dmp
```

만약 임포트 도중 데이타 영역이 부족해서 에러가 발생한 다면 익스포트를 받을 때 compress=n 옵션을 사용하며, 임포트시 롤백 세그먼트 문제가 발생한다면 임포트시 commit=y 옵션을 사용하면 쉽게 해결이 가능하다.

EXPLAIN PLAN 사용법

Bulletin no : 10312

Explain plan 이란 SQl 이 요구한 데이타를 추출하기 위해 오라클이 차례로 수행하는 작업 방법을 말한다.

1.Plan_table 생성

explain plan 을 sql 에 포함해서 수행하면 옵티마이저가 실행 계획까지만 수립하여 plan_table 에 저장해 둔다. 이 table 을 생성하기 위한 script는 ?/rdbms/admin/utlxplan.sql 이다.

2.index 생성

테이블 생성후 수행 속도 향상과 동일한 statement_id 가 생성되는 것을 방지 하기 위해 index 를 생성한다.

```
create unique index plan_index on plan_table(statement_id,id);
```

3. SQL 문 사용

```
EXPLAIN PLAN SET STATEMENT_ID=a1' FOR
              SELECT .....
```

4. Plan_table 을 select 하는 sql 문

```
Select ipad(operation,length(operation)+ 2*(level-1) ||
          decode(id,0,'cost estimate:' ||decode(position,'0',
          'N/A',position),null) || ' ' ||options ||
          decode(object_name,null,null,':') ||rpad(object_owner,
          length(object_name)+1,',') || object_name ||
          decode (object_type,'UNIQUE' ,'(U) ','NON_UNIQUE',
          '(NU)',null) || decode(object_instance,null,null,'('||
          object_instance||')')
FROM PLAN_TABLE
START WITH ID= 0 and STATEMENT_ID = '&&id'
CONNECT by prior ID=PARENT_ID and STATEMENT_ID='&&id'
```

Explain plan 문의 'statement_id =' 에서 부여한 제목을 'id'에 지정하고 다음처럼 실행한다.

```
sql> def   id = a1
sql> @plan
```

EXPLAIN PLAN 을 자동으로 수행하는 방법

Bulletin no : 10353

다음은 sql*plus 에서 자동으로 explain plan 을 creation 하는 방법이다. 이 방법은 explain 을 실행하는 구문을 기억할 필요없이 현재 수행하는 sql 문장을 쉽게 체크해 볼 수 있습니다.

다음은 script 의 내용입니다.

```
save toto repl              -- Save current script
truncate table plan_table;  -- Clean Plan Table
get toto nolist             -- Restore script

1                           -- setup explain statement
ch //explain plan set statement_id='MyTest' for /
/
                            -- Go!
select  lpad(' ',2*(level-1))|| operation||' '||
        options||' '||object_name||
        ' '||object_type||' '||object_instance||' '||
        decode(id,0,'Cost = '||position) "Query Plan"
from    plan_table
        start   with id = 0  --  start with statement_id = 'MyTest'
        connect by prior id = parent_id;

get toto nolist              -- Restore script
```

EXAMPLE:

위의 script 를 'plan.sql' 라는 file 로 저장한후 sql*plus 를 login 한다.

```
SQL) select * from dept;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        10 ACCOUNTING     NEW YORK
        20 RESEARCH       DALLAS
        30 SALES          CHICAGO
        40 OPERATIONS     BOSTON

SQL) @plan
Wrote file toto

Table truncated.
  1* select * from dept
  1* explain plan set statement_id='MyTest' for select * from dept
```

```
Explained.

Query Plan
----------------------------------------------------------------------------
SELECT STATEMENT     Cost = 1
  TABLE ACCESS FULL DEPT  1
```

Script 는 plan.sql 로 저장 되어 있으므로 user 는 buffer 를 query 해서 어느 정도의 수정을 가할 수 있다.

```
 SQL)  1
  1* select * from dept
SQL)  i
  2  where deptno =20;

    DEPTNO DNAME          LOC
---------- -------------- -------------
        20 RESEARCH       DALLAS

SQL)  @plan
Wrote file toto

Table truncated.

  1* select * from dept
  1* explain plan set statement_id='MyTest' for select * from dept

Explained.

Query Plan
----------------------------------------------------------------------------
SELECT STATEMENT     Cost = 1
  TABLE ACCESS BY ROWID DEPT  1
    INDEX UNIQUE SCAN DEPT_PRIMARY_KEY UNIQUE
----------------------------------------------------------------------------
```

EXPORT 받은 화일의 CHARACTER SET 변경 프로그램

Bulletin no : 10159

```
/*********************************************************************
 *                                                                   *
 *      convdat : Data Conversion Program                            *
 *                                                                   *
 *       . converts exported files in different db-charset           *
 *         to specific db-charset files.                             *
 *                                                                   *
 *********************************************************************/
#include
#include
#include
#include

int us7_to_ksc, ksc_to_us7; /* conversion flag */

main(argc,argv)
int argc;
char *argv[];
{
 us7_to_ksc = ksc_to_us7 = 0;
 if(argc != 3) {
   usage(argv[0]);
   exit(1);
 }

 if(!strcmp(argv[2],"KSC5601"))
   us7_to_ksc = 1;
 else if(!strcmp(argv[2],"US7ASCII"))
   ksc_to_us7 = 1;
 else
   usage(argv[0]);
 data_conversion(argv[1]);
 printf("conversion completed successfully !!!\n");
}

usage(prog_name)
char *prog_name;
{
 printf("Usage: %s exported-file-name charset\n",prog_name);
 printf("     - charset : KSC5601¦US7ASCII \n");
```

```
 printf("       . KSC5601 - convert us7ascii format to ksc5601  format\n");
 printf("       . US7ASCII - convert ksc5601 format to us7ascii format\n");
}

data_conversion(usfile)
char *usfile;
{
 int fd;
 char conv_buffer[2];
 if((fd = open(usfile, 2)) < 0) {
  perror("open");
  exit(1);
 }
 if(lseek(fd, 1, 0) < 0) {
  perror("lseek");
  exit(1);
 }
 if(us7_to_ksc){
  conv_buffer[0] = 3;
  conv_buffer[1] = 'H';
 }
 else if(ksc_to_us7){
   conv_buffer[0] = '\0';
   conv_buffer[1] = 1;
 }
 if(write(fd,conv_buffer,2) != 2) {
   perror("write");
   exit(1);
 }
 close(fd);
}
```

EXPORT받은 DUMP파일의 CHARACTERSET 확인방법

Bulletin no : 10329

다른 server에서 export받아온 dump file을 import시킬경우 자주 겪게 되는 문제가 characterset에 관련된 것이다. 양쪽의 characterset이 같고 .profile의 환경변수(NLS_LANG)도 맞다고 생각되는 데도 import시 segment fault가 발생되거나,imp-16,36,37,38,42,43,45,49등의 error가 발생되면 dump받아온 file을 의심해볼 필요가 있다.

이때 dump file을 8진수로 변환해 보면 현재 export받은 file의 characterset을 쉽게 알수 있다.

오라클export file은 특별한 character set을 사용해서 쓰여진다.
version5,6에서는 export시 단지 두개의 character set-ASCII와 EBCIDC-만이 지원 되었으나, version7에서는 지원하는 모든 character set이 export시에 사용될수 있다.export시 사용된 character set은 dump file내에 code로써 기술되는데 file의 첫byte가 이것을 나타낸다.

1st byte	의미
1	version5,6의 ASCII character set
2	version5,6의 EBCDIC character set
3	version7의 character set.

"3"일 경우 두,세번째 byte를 보고서 character set을 판단한다.이때 2nd byte 보통 000이며 아래에는 우리가 흔히 접하는 character set에 대한 character set ID이다.

ID	value
1	US7ASCII
2	WE8DEC
31	WE8ISO885P1
37	KO16KSC5601
:	:
:	:

export file을 체크하기 위해서는 다음의 UNIX command를 사용한다.

```
$ od -b expdat.dmp¦more

0000000 003 000 001 105 130 120 117 122 124 072 126 060 067 056 060 062
0000020 056 060 062 012 104 112 115 113 111 115 012 122 124 101 102 114
     :
     :
```

위의 경우 1st byte가 003이므로 version7이며 3rd byte가 001이므로 ID가 1번인 US7ASCII가 character set이 된다.

EXPORT실행시 ORA-1403 조치방법

Bulletin no : 10104

EXPORT 실행시 ORA-1403이 발생되는 경우가 있는 데, 이 에러는 테이블 혹은 인덱스에 문제가 있는 경우 발생할 수 있다. 여기서는 ROWID를 사용하여 테이블을 복구시키는 방법을 소개한다.

1. 기존 테이블과 같은 구조를 갖는 테이블을 만든다.

```
SQL> CREATE TABLE TEMP
                AS  SELECT *
                    FROM EMP
                    WHERE 1=2;
```

2. 기존 테이블에서 RECORD를 FETCH하여 새로운 테이블에 입력.
   이 때, INDEX가 설정되어 있는 COLUMN을 WHERE 조건에 부여함.

```
declare
 row_id char(18);
 cnt    number;
 cursor c1 is select rowid
              from emp
              where empno>0;
 begin
  cnt := 0;
  open c1;
  loop
   fetch c1 into row_id;
    insert into temp select * from emp where rowid=row_id;
    cnt:=cnt+1;
    if cnt = 100 then      /* commit per 100 records */
       commit;
       cnt:=0;
    end if;
   exit when c1%notfound;
  end loop;
 commit;
end;
/
```

3. create.sql file실행

```
        sql> @create
```

EXTENT된 영역의 FREESPACE

Bulletin no : 10241

---

1. TABLE 에 할당된 SEGMENT 의 크기

```
SQL> select bytes SEGMENT_SIZE
     from user_segments where segment_name='EMP';
```

2. TABLE 이 사용하고 있는 영역(ORACLE BLOCK 단위)

```
SQL> select count(distinct(substr(rowid,1,8)||substr(rowid,15,4)))
           * 2048  TABLE_SIZE from emp;
```

3. TABLE SEGMENT 의 FREESPACE

```
FREE_SPACE = SEGMENT_SIZE - TABLE_SIZE
```

FOREIGN KEY CONSTRAINT의 MASTER TABLE NAME의 확인

Bulletin no : 10081

FOREIGN KEY CONSTRAINT 이름으로 REFERENCE 하는 TABLE(MASTER TABLE)을 찾는 SQL SCRIPT

```
col Primary_key_table format a20
col Constraint_name   format a20
select a.object_name Primary_Key_table,
       c.name        Constraint_name
from dba_objects a,
     cdef$       b,
     con$        c
where c.name       = 'EMP_FOREIGN_KEY'   -- CONSTRAINT NAME
       and b.con#  = c.con#
       and b.robj# = a.object_id
/
```

FREE LIST와 FREE SPACE관리

Bulletin no : 10103

---

* Free List 는 해당 세그먼트가 다시 사용할 수 있는 block에 관한 list 이다.
Free Block이 추가될 때는 Free List의 Head에 덧붙여진다. 예를 들면 Block B5가 Free List FL 에 있고 다른 B6 block이 FL에 덧붙여질 때

FL : B6 -> B5

즉 B6 가 free block으로 먼저 쓰여진다.

앞으로 사용할 약어 : V1(V6.0 ~ V6.0.34)
V2(V.6.0.35 ~ V7)

* Free List 에는 다음과 같은 것들이 있다.
. Txfl(Transaction Free List) : 모든 트랜잭션에 할당되는 Free List이다.
한 세그먼트에 최소 16 Txfl 이 있고 이는 블럭이 꽉 찰 때 까지 필요한 만큼 증가한다. 모든 프로세스는 해당 인스턴스에 주어진 Txfl에 접근 할 수 있다.
. Prfl(Process Free List) : 하나의 프로세스에 할당되는 Free List 이다.
하나의 프로세스는 다른 프로세스에 할당 되어진 Prfl 을 사용하지 않는다.
. Sgfl(Segment Free List) : Prfl과 같다.
. Msfl(Master Free List) : 모든 프로세스가 사용 할 수 있도록 미리 할당 되어진 Prfl이다.

* 세그먼트는 연속된 블럭의 모음인 익스텐트로 구성 되어 있다. 세그먼트가 생성될때 첫번째 익스텐트의 첫번째 블럭을 Segment Header라고 한다.
Segment Header는 Extent Control, Extent Map Table, Segment Header,Segment Free Space, Transaction Free Space로 이루어져 있다. Extent Control과 Extent Map Table은 세그먼트에 할당된 익스텐트에 관한 정보를 준다. Segment Header Control은 Transaction Free List 정보와 세그먼트가 생성 될 때 명시된 Instance List(nfb)와 Process Free List(nfl)의 갯수를 가지고 있다.
* Instance Free List(infl) 갯수는 디폴트가 1이며 1보다 클 경우 Prfl의 갯수는 Prfl/Sgfl = nfb * nfl 이다. nfb는 init.ora 의 free_list_inst, nfl은 free_list_proc 에서 지정된다.

* V6.0.35 에서는 기존의 Block Type 5에 Block Type 12가 추가됨으로써 Segment Header는 Type 5 또는 12가 가능하다. 이의 구별은 명시된 Free Group의 갯수에 의해 결정된다. 즉, Free Group의 갯수가 1보다 크면 Segment Header의 block Type은 12로 설정된다.
* V6.0.35 이상에서는 Free Lists 의 관련 정보를 Storage 항에서 표시할 수 다.

....storage (freelists n freelist groups m)

장점은 데이타 딕셔너리에 Free List의 정보가 저장 되어진다는 것이다. 즉 V1 에서 처럼 Free Lists 를 바꾸기 위해 데이타베이스를 Restartup 할 필요가 없다.
* V1 에서는 Freelists를 위해 Segment Header인 한 블럭이 할당된다. 그러나 V2 에서는 각 Free List Group 에 한개의 Free List Block 이 존재한다. 예를 들어, M=2 이면 Segment Header와 다음 2 block 이 할당 되어지는데 각 블럭은 Free List Group을 위한 것이다. (여기서 M 은 Free Group 이다).

< Mapping Function 과 Free Space 검색 알고리즘>

(Mapping Function)

* Mapping Function은 단순히 Thread Number % Freelist Groups 와 Process Number % Process Freelists이다. 예를 들어 M=3,N=5는 3개의 Free List Groups 와 각각은 5개의 Process Free List 를 가짐을 의미한다.

1(1 2 3 4 5) 2(1 2 3 4 5) 3(1 2 3 4 5)

. 임의의 프로세스가 인스턴스 2 에 접속되어 있고 Thread Number는 10 이고 Process Number는 26 이라고 가정하면 어떤 Freelist 가 할당되어질까?

이런 경우에는

10 % 3 = 1

26 % 5 = 1

즉, 첫번째 그룹의 첫번째 Prf1 을 선택하게 된다.

* 만약 인스턴스를 특정한 Freelist Block에 할당하고자 할 때는 init.ora Parameter 인 Instance_number를 이용한다.

(검색 알고리즘)

* V6.04 이하에서는 다음 순서를 따른다.

1. 자신의 Transaction Free List에서 Free List를 찾는다. 찾으면 #7, 아니면 #2로 간다.
2. Process Freelist 의 Free Space 를 찾는다. 찾으면 #7, 아니면 #3 으로간다.
3. Commit된 Txfls를 찾는다. 만일 Commit되지 않은 Txf1 Space를 발견하면 #4로 가고 아니면 각 Commit된 Txf1 Space 를 한 Prf1에 할당한다. 만약 Prf1 > Trf1 이면 어떤 Prf1는 Space를 얻지 못하고 Prf1 < Trf1 이면 어떤 Prf1은 보다 많은 Space를 얻으며 #2 로 간다.
4. High Water Mark를 Bump Up할 수 없으면 #5로 가고 가능하면 원래대로 Space를 Prf1에 복사한 후 #2로 간다.
5. 데이타 딕셔너리에서 fet$를 검색하고 익스텐트를 세그먼트에 할당하고 #4로 간다. 만일 fet$ 에 Space 가 없으면 #6로 간다.
6. 에러발생. 테이블스페이스에 더이상 Space 가 없다.
7. Space 를 사용한다.

* V6.0.35 이상에서는 다음 순서에 의한다.

1. 자신의 Transaction Free List에서 Free List를 찾는다. 찾으면 #8, 아니면 #2 로 간다.
2. Process Freelist 의 Free Space 를 찾는다. 찾으면 #8, 아니면 #3 으로 간다.
3. Master Freelist를 검색한다. 만일 Space를 찾으면 Prf1에 Space의 조각을 복사하고 #2, 아니면 4 로 간다.
4. Commit된 Txfls를 찾는다. 만일 Commit되지 않은 Txf1 Space를 발견하면 #5, 아니면 각 Commit된 Txf1 Space 를 Master Freelist 에 복사하고 #3으로간다.
5. High Water Mark 를 Bump Up할 수 없으면 #6으로 가고 가능하면 Bump Up하고 Space를 Prf1 에 복사한 후 #2로 간다.
6. 데이타 딕셔너리에서 fet$를 검색하고 익스텐트를 세그먼트에 할당하고 #4로 간다. 만일 fet$ 에 Space 가 없으면 #7로 간다.
7. 에러 발생 테이블스페이스에 더 상 Space 가 없다.
8. Space 를 사용한다.

FULL TABLE SCAN을 보다 빠르게 하는 방법

Bulletin no : 10102

---

* db_file_multiblock_read_count 는 Full Table Scan 시에만 영향을 미치는 파라미터이다. 오라클의 최대 I/O 크기는 64KB 이므로

db_blocksize * db_file_multiblock_read_count <= 64KB

이어야 한다.

Index를 이용한 Query 인 경우에는 Full Table Scan 과는 상황이 달라진다. 이 때에는 db_block_size의 설정이 중요하다. 오라클은 원하는 블럭 크기를 선택할 수 있으며 대부분의 유닉스 환경에서 최대 8KB(시퀀트는 16KB)까지 가능하다. 같은 크기의 B-tree Index 에서 블럭크기가 8KB 라면 2KB 인 경우보다 거의 4배 빨리 데이타를 읽어들이게 된다.

한편 Index Block 의 Pctfree도 중요한 의미를 갖는데 이 값이 너무 크고 인덱스의 Entry Size 가 늘어나지 않으며 기존의 인덱스 값에 새로운 레코드가 더 이상 추가되지 않는다면 공간의 낭비가 심하게 되므로 적절한 값으로 설정해야 한다.
db_file_simultaneous_writes 파라미터는 인덱스의 재구성시 퍼포먼스에 별로 영향을 끼치지 않는다.

HIERARCHICAL QUERIES

Bulletin no : 10325

START WITH root rows of the hierarchy
CONNECT BY parent row 와 child row 의 관계를 맺어준다.
(parent row의 children 을 찾기위해 PRIOR operator를 사용)

예제 1) 다은은 전 사원의 구성도 hierarchical order 에 따라 select 하는 방법이다. 각 사원번호는 manager 번호와 연결되어 있다.

```
select ename,empno,mgr,job
from emp
start with  job= 'president'
connect by prior empno=mgr ;
```

예제 2) 테이블이 아래처럼 구성되어 있고 데이타가 들어있다고 가정할때 Tree가 Dynamic하게 변하더라도 값을 가져오기 위한 SQL문은 다음과 같다.

Tree Table :

```
       create table Tree (name varchar2(2), parent varchar2(2));
       insert into Tree(name , parent)
            values('a', NULL);
       insert into Tree(name , parent)
            values('b', 'a');
       insert into Tree(name , parent)
            values('c', 'a');
           .....
```

```
select name
from tree x
where not exists (select name
                  from tree y
                  where x.name = y.parent)
start with name = 'b'
connect by prior name = parent
```

만약 Parents만을 구하고자 한다면 위의 SQL을not exists 를 exists로 바꾸어주면 된다.

HP-UX: ASYNCH I/O USING FS_ASYNC KERNEL PARAMETER

Bulletin no : 10037

This type of I/O caching is also found in numerous third-party disk cache products, which allow disabling "write-through". Since Oracle cannot tell if an I/O actually made it TO DISK, this is not supported on any O/S, including OpenVMS. This is the same reason why using a RAM disk is not supported....

****

Use of FS_ASYNC is very dangerous and should not be used on any database that is not a toy. This kind of async, means that the Oracle writes to disk are not guaranteed to have gotten to disk. This can corrupt your database if it crashes before the data is written to disk, when Oracle thinks it already has.

This is not a new HP-UX feature. I believe it may have existed all the way back to 4.2BSD to support extremely fast but unreliable writes on old 1 Mip Vaxen.

The Async I/O which requires raw devices makes use of a special HP device driver which allows writes and reads to be "batched" so we can do a lot of writes with one system call. The device driver kicks off multiple I/O operations at a lower level where disk scheduling can be optimized for maximum parallelization, and minimum OS overhead. This asynch device driver only works with raw devices, i.e. raw disks or raw logical volumes. This is an HP restriction. We wish we could use it on file systems as well. Oh, yes, and this type of asynch is safe. Oracle is reliably notified if the data made it all the way to disk or not.

****

INDEX 생성시 필요한 TEMP SEGMENT 크기 추정

Bulletin no : 10215

새로운 인덱스를 저장하기위해 필요한 TEMPORARY SEGMENT SPACE의 량은 다음 공식을 사용하여 추정가능 하다.

```
  1.3 * key_storage
  -----------------
   where :  key_storage = (number_of_rows)*
                          ( 10 + sum_of_column_sizes + number_of_columns )
```

- COLUMNS 은 INDEX 에 있는 COLUMN 들이다.
- NUMBER_OF_COLUMNS 는 COLUMN 당 1 LENGTH Byte.
- 10 은 ROWID 와 추가적인 OVERHEAD

* 1.3 은 SORTING 을 위해 필요한 추가 영역의 평균치. 이 값은 임의 순서로 된 데이타에 대해서 가장 적당한 값이고 만일 데이타를 완전히 정반대로 SORTING 하면 두배의 영역이 필요하다.
만일 데이타가 완벽하게 SORTING 되어 있으면 오직 INDEX 를 저장하기위한 SPACE 만 필요하고 이때 CONSTANT 값은 1 이 된다.

INDEX_DESC HINT 사용법

Bulletin no : 10309

- INDEX_DESC hint 사용법

```
select /*+ index_desc(table_alias_name index_name) */
       seq
from   table_name table_alias_name
where  key1 = :b1
and    rownum = 1

(index_name : key1+seq)
```

* SINGLE TABLE QUERY에서는 HINT절에 TABLE명을 주는것이 원칙이다.

* HINT내의 table_name은 alias를 사용하였을 경우에는 반드시 table alise name으로 지정하여야 한다.(이는 Version 7.2 / 7.3공히 적용 된다.)

* 그 이유는 다음과 같이 SQL이 사용 될 경우도 있기 때문이다.

```
(SELF-JOIN)

select /*+ index_desc(a a_pk) */
       *
from   t1 a, t1 b
where  a.key1 = 'a'
and    a.key1 = b.prev_c1(+)
and    a.seq = b.pseq_no(+)
and    rownum = 1
```

위와 같이 동일한 TABLE이 FROM절 뒤에 복수로 나오는 경우(RECURSIVE 관계)가 있을 수가 있는데 이때 HINT절의 table명을 유일하게 줄수가 없으므로 반드시 table alias name을 주어야 원하는 답이 나온다.

INSTR 함수의 의미와 특정 CHARACTER 를 지우는 방법

Bulletin no : 10324

---

1) INSTR function 의 의미

임의의 character 의 position 을 찾아내는 function 이다.

```
INSTR(char1,char2[,n[,m]])
```

char1 의 n 번째 position 부터 m 번째 char2 의 position 을 찾는 function 이다. 만약 n이 음수이면 char1 의 뒤 터 찾으라는 의미이다.

eg) select INSTR('corporate floor','or',3,2) "instring" from dual;

```
 instring
--------
      14
```

2) 특정 character 를 지우는 방법(position 을 모를때)

같은 위치에 있지 않은 특정 character 를 지우고 select 하기를 원할때는 다음과 같은 sql문을 이용한다.

```
SELECT DECODE(SUBSTR(col,INSTR(col,','),1),',',
SUBSTR(col,1,INSTR(col,',')-1)||SUBSTR(col,INSTR(col,',')+1),
col) FROM test;
```

이 문장은 col 에서 comma 의 위치를 알아내어 그 comma를 제외한 data 값만을 select 하는 문장이다.

3) 특정 character 의 position 을 알때

특정 character 인 $ 의 position 이 col 의 1번째 위치임을 알고 이 $를 제 하고 select 하고자 할때는 다음과 같다.

```
SELECT DECODE(SUBSTR(col,1,1),'$',SUBSTR(col,2,col) FROM
table;
```

LAST ORA-1578 RECOVERY METHOD

Bulletin no : 10056

Subject: 최후의 ORA-1578의 처리방법

일반적으로 ORA-1578 : ORACLE data block corrupted (file # num,block #num)은 해당 objects를 drop하고 recreate하여 처리 할수 있읍니다. 하지만, backup이 안되어 있거나 되어 있더라도 시점에 문제가 생기면,당황할 수 밖에 없습니다. 따라서,손상된 db block만을 제외한 다른 block들을 recover하는 방법을 알려 드립니다.

1. 손상된 table과 똑같은 spec의 new table생성

2. 아래의 pl/sql 실행

```
/*
** This program uses a cursor to select the ROWID from INDEX
** for the corrupted table. And insert the all of the normal rows
** into the new table.
**
*/

DECLARE
    CURSOR c1 is
       SELECT ROWID FROM corrupted_table
       WHERE  indexed_column > smallest_value_of_the_indexes
       AND    SUBSTR(ROWID,1,8) != '000005DA';

/* Here,the value of '000005DA' is displayed in decimal
   when ORA-1578 error happens. */

    rowid_value    CHAR(18);
    count_value    NUMBER;
BEGIN
    OPEN c1;

    FOR i IN 1..total_number_of_the_rows_from_corrupted_table LOOP
     FETCH c1 INTO rowid_value;
     EXIT WHEN c1%NOTFOUND;
       INSERT INTO new_table
       SELECT * FROM corrupted_table WHERE ROWID = rowid_value;
        count_value := count_value + 1;
    IF count_value = 10000 THEN /* Let's think commit per 10000 rows */
           COMMIT;
       count_value := 0;
        END-IF;
```

```
    END LOOP;

    CLOSE c1;
END;
/
```

3. new table의 data건수등 확인후에 기존 table을 다른 이름으로 rename한 후
   new table을 original table name으로 rename

```
  SQL> rename corrupted_table to table_save;
  SQL> rename new_table to corrupted_table;
```

4. 필요한 index생성이나 constraint enable

5. 손상된 row에 대한 내용은, 최소한 primary key나 indexed columns에 대한 내용을

```
   SQL> spool chk.log
   SQL> select rowid,indexed_column1,indexed_column2,indexed_column3...
        from corrupted_table
        where indexed_column1 > smallest_value_of_the_index;
   SQL> spool off
```

를 통하여 chk.log에 저장후,

```
   SQL> select count(*) from corrupted_table;
```

혹은

  table export등을 통하여 확인된 rowid의 block number를 비교하여 보면 됩니다.
  주의사항은 여기에서 ora-1578과 동반되는 block number는 decimal이니, hexa로 환산해야 합니다.

LOCK 문제 확인과 해결 방법

Bulletin no : 10221

- 응용프로그램들의 기능이 복잡해져가면서 Lock 문제가 빈번하게 발생함에 따라 이를 해결하는 것이 매우 중요한 일이 되었다. 도대체 어떠한 프로그램이 Lock을 발생시켜 데이타베이스를 Hangup 상태에 이르게 하는가를 알아내는 것은 쉬운 일이 아니지만 지금부터 이 문제를 해결하는 방법에 대하여 알아보도록 한다.

```
SQLDBA MONITOR SCREENS
----------------------
$ TERM=vt100;export TERM
$ sqldba
  [ connect internal ] -> [ key-0(ins key) ]

[ MONITOR SESSION ]
```

Session ID	Serial Number	Process ID	Status	Username	Lock Waited	Current Statement
6	35	28	ACTIVE	LTO2	C2D2B3B4	UPDATE
8	70	19	INACTIVE	SYSTEM		SELECT
12	15	25	INACTIVE	LTO		INSERT
14	17	27	ACTIVE	LTO3	C2D2B438	DELETE
15	30	26	ACTIVE	SYS		UNKNOWN

```
 -  Session Id :  SID - Session Identifier
 -  Serial Nr  :  Session Serial Nmber. Session 의 Object 별로 지정된다.
 -  Process Id :  V$PROCESS에서 PID(Oracle Process Identifier)
 -  Status     :  Session의 상태
 -  Username   :  Oracle Username
 -  Lock waited:  대기중인 Lock Address
```

여기서 가장 중요한 컬럼은 Lock Waited이다. 여기에 무엇인가 기록이 되어 있다면 이 세션은 특정 리소스를 얻기 위하여 대기 중인 상태임을 알 수 있다. 위의 예에서 LTO2,LTO3이 그 경우에 해당 된다.

```
[ MONITOR LOCK ]
```

Username	Session ID	Serial Number	Lock Type	Resource ID 1	Resource ID 2	Mode Held	Mode Requested
LTO2	6	35	TM	2294	0	RX	NONE
LTO2	6	35	TM	2295	0	RX	NONE
LTO2	6	35	TX	262167	87	NONE	X
LTO2	6	35	TX	327682	90	X	NONE

```
 LTO2            12      15  TM         2294          0  RX    NONE
 LTO             12      15  TM         2295          0  RX    NONE
 LTO             12      15  TX       262167         87  X     NONE
 LTO3            14      17  TM         2294          0  RX    NONE
 LTO3            14      17  TM         2295          0  RX    NONE
 LTO3            14      17  TX       262167         87  NONE  X
 LTO3            14      17  TX       196636         87  X     NONE
=============================================================================
 - Username   :  Oracle Username.
 - Session Id :  SID - Session Identifier.
 - Serial Nr  :  앞의 설명 참조

===============+=========================+==============================
  Type of Lock ¦ Resource ID1             ¦ Resource ID 2
---------------+-------------------------+------------------------------
TX(Transaction)¦ 롤백 세그먼트 번호와 슬롯¦ Wrap Number를 10진수로 변환한
               ¦ 번호를 10진수로 변환한 값¦ 값(롤백슬롯이 재사용된 횟수)
---------------+-------------------------+------------------------------
TM(Table Locks)¦ 수정된 테이블의 Object ID¦ 항상 0
---------------+-------------------------+------------------------------
RW(Row Wait)   ¦ File #, Block #를 10진수 ¦ 블럭내의 Row를 10  수로 변환
               ¦ 로 변환한 값 (V6)       ¦ 한 값
---------------+-------------------------+------------------------------
UL(User Defined¦ Oracle7 Concepts Manual Ch10, Oracle7 Admin Guide
    Locks)     ¦ Appendix B 81 의 리스트 참조
---------------+--------------------------------------------------------
Mode Held      ¦ 걸고 있는 Lock Mode
---------------+--------------------------------------------------------
Mode Requested ¦ 요구한 Lock Mode
=============================================================================

* 다음 사용자들은 Lock을 기다리고 있는 상태이다.
 LTO2             6      35  TX       262167         87  NONE  X
 LTO3             4      17  TX       262167         87  NONE  X

* 이들이 기다리고 있는 Lock은 다음과 같다.
 LTO             12      15  TM         2294          0  RX    NONE
 LTO             12      15  TM         2295          0  RX    NONE
 LTO             12      15  TX       262167         87  X     NONE
```

TX Lock의 경우, 동일한 리소스 대한 Lock을 요구하는 경우 각각의 ID1,ID2는 동일한 값을 갖는다. 하나의 데이타 블럭 내에는 그 블럭을 사용하고 있는 트랜잭션에 대한정보가 들어있는데 만약 이 트랜잭션이 Commit, Rollback 되지 않으면 다른 트랜잭션들은 그 트랜잭션이 잡고 있는 리소스의 Lock이 해제 되기를 기다리게 된다.

하나의 트랜잭션이 여러개의 테이블에 대해서 작업을 하고 있는 경우에는 Lock과 연관된 리소스가 어떤 것인지를 쉽게 파악하기가 힘들다. 이 때에는 Monitor Session과 Monitor Table을 모두 보아야 지만 확실하게 알 수 있다.

[MONITOR SESSION]

Session ID	Serial Number	Process ID	Status	Username	Lock Waited	Current Statement
5	31	19	INACTIVE	LTO		INSERT
6	43	25	ACTIVE	LTO2	C3D320F4	UPDATE
9	1	26	ACTIVE	LTO3	C3D320C8	DELETE

[MONITOR TABLE]

Session ID	Schema Name	Table Name
5	LTO	DEPT
6	LTO	EMP
9	LTO	EMP

* 위의 경우를 보면 Session 6와 Session 9가 LTO.EMP을 사용하려고 기다리고 있음을 알 수 있다.

■ 해결방법

대부분의 Lock 문제는 응용프로그램의 잘못된 설계 때문에 발생하게 된다.
이를 해결하기 위해서 다음과 같은 방법으로 Lock을 Release 시켜야 한다.

1. Lock Holder에게 Commit, Rollback을 요청한다.
2. Lock Holder의 세션을 Kill 시킨다.

```
SQLDBA>ALTER SYSTEM KILL SESSION 'sid,serial#';
```

3. 유닉스 Shadow Process(Server Process)를 Kill 시키는 것은 바람직하지 않다. 특히 MTS 환경하에서는 각별한 주의가 필요하다.
4. 2PC Pending 트랜잭션인 경우는 Rollback Force, Commit Force 로 해결한다.

■ LOCK 문제를 일으키는 SQL 명령 찾기

* 다음 Query는 Lock과 관련된 트랜잭션을 출력해준다.

```
column username format a10
column sid format 999
column lock_type format a15
column MODE_HELD format a11
column MODE_REQUESTED format a10
column LOCK_ID1 format a8
column LOCK_ID2 format a8
select a.sid,
```

```
 decode(a.type,
'MR', 'Media Recovery',
'RT', 'Redo Thread',
'UN', 'User Name',
'TX', 'Transaction',
'TM', 'DML',
'UL', 'PL/SQL User Lock',
'DX', 'Distributed Xaction',
'CF', 'Control File',
'IS', 'Instance State',
'FS', 'File Set',
'IR', 'Instance Recovery',
'ST', 'Disk Space Transaction',
'IR', 'Instance Recovery',
'ST', 'Disk Space Transaction',
'TS', 'Temp Segment',
'IV', 'Library Cache Invalidation',
'LS', 'Log Start or Switch',
'RW', 'Row Wait',
'SQ', 'Sequence Number',
'TE', 'Extend Table',
'TT', 'Temp Table',
a.type) lock_type,
decode(a.lmode,
0, 'None',            /* Mon Lock equivalent */
1, 'Null',            /* N */
2, 'Row-S (SS)',      /* L */
3, 'Row-X (SX)',      /* R */
3, 'Row-X (SX)',      /* R */
4, 'Share',           /* S */
5, 'S/Row-X (SSX)', /* C */
6, 'Exclusive',       /* X */
to_char(a.lmode)) mode_held,
decode(a.request,
0, 'None',            /* Mon Lock equivalent */
1, 'Null',            /* N */
2, 'Row-S (SS)',      /* L */
3, 'Row-X (SX)',      /* R */
4, 'Share',           /* S */
5, 'S/Row-X (SSX)', /* C */
6, 'Exclusive',       /* X */
to_char(a.request)) mode_requested,
to_char(a.id1) lock_id1, to_char(a.id2) lock_id2
from v$lock a
where (id1,id2) in
    (select b.id1, b.id2 from v$lock b where b.id1=a.id1 and
           b.id2=a.id2 and b.request>0)
/
```

(위의 Query를 실행시키면 다음과 같은 내용이 출력된다.

```
SID LOCK_TYPE         MODE_HELD   MODE_REQUE   LOCK_ID1   LOCK_ID2
--- ---------------   ----------  ----------   --------   --------
  5 Transaction       Exclusive    None        262172     90
  6 Transaction       None         Exclusive   262172     90
  9 Transaction       None         Exclusive   262172     90
```

SID 6과 9는 SID 5가 걸고 있는 Lock이 풀리기를 기다리고 있음을 알 수 있다.

* 다음 Query는 Lock과 관련된 테이블을 출력해 준다.

```
column username format a10
column lockwait format a10
column sql_text format a80
column object_owner format a14
column object format a15
select b.username username, c.sid sid, c.owner object_owner,
   c.object object, b.lockwait, a.sql_text SQL
from v$sqltext a, v$session b, v$access c
where a.address=b.sql_address and
   a.hash_value=b.sql_hash_value and
   b.sid = c.sid and c.owner != 'SYS'
/
```

(위의 Query를 실행하면 다음과 같은 결과가 출력된다.

```
USERNAME          SID OBJECT_OWNER   OBJECT          LOCKWAIT
--------------- --- ------------ -------------- ----------
SQL
------------------------------------------------------------------
LTO2                6  LTO               EMP             C3D320F4
update lto.emp set empno =25 where empno=7788
LTO3                9  LTO               EMP             C3D320C8
delete from lto.emp where empno=7788
LTO                 5  LTO               DEPT
insert into lto.dept values (60,'PROGRAMMER','LOS ANGELOS')
```

여기서는 USERNAME에 나와있는 유저가 OBJECT에 나와있는 테이블을 수정하려고 함을 나타낸다. LTO2,LTO3는 LTO가 Commit,Rollback 하기를 기다리고 있음을 알 수 있다. 하지만 여기에는 가장 최근의 DML 명령 하나만 나와있기 때문에 여기 나온 명령이 반드시 Lock을 걸고 있는 명령이라고 단정지을 수는 없다.

■ 관련된 프로세스

* 다음 Query를 행해 보면 프로세스와 관련된 정보를 얻을 수 있다.

```
column "ORACLE USER" format a11
column SERIAL# format 9999999
column "OS USER" format a8
select substr(s.username,1,11) "ORACLE USER", p.pid "PROCESS ID",
  s.sid "SESSION ID", s.serial#, osuser "OS USER",
  p.spid "PROC SPID",s.process "SESS SPID", s.lockwait "LOCK WAIT"
from v$process p, v$session s, v$access a
where a.sid=s.sid and
   p.addr=s.paddr and
   s.username != 'SYS'
/
```

* 위의 Query를 실행하면 다음과 같은 결과가 출력된다.

```
ORACLE    PROCESS  SESSION  SERIAL#  OS USER   PROC   SESS   LOCKWT
 USER          ID       ID                    SPID   SPID
------    -------  -------  -------  -------   ----   ----   ------
LTO            19        5       31  usupport  17312  17309
LTO2           25        6       43  usupport  17313  17310  C3D320F4
LTO3           26        9        1  usupport  17314  17311  C3D320D8
```

이 때는 다음과 같이 조치한다.

1. LTO에게 Commit,Rollback 할 것을 요구한다.
2. SQLDBA>ALTER SYSTEM KILL SESSION '5,31';
3. %kill -9 17309 (유닉스상의 Shadown Process)
   stop/id= (PROC SPID=SESS SPID on vms running single task)

여기서 SYS 유저는 제외시켰는데 필요하다면 Query의 Where 조건에서
s.username != 'SYS' 부분을 삭제하면 된다.

■ CATBLOCK.SQL & UTLLOCKT.SQL

$ORACLE_HOME/rdbms/admin 디렉토리에 있는 스크립트 가운데 catblock.sql과 utlockt.sql을 사용하여서 Lock 상황을 쉽게 파악할 수 있다. 이들은 다음과 같이 실행한다.

```
%cd $ORACLE_HOME/rdbms/admin
%sqldba lmode=y
SQLDBA>connect internal
SQLDBA>@catblock
SQLDBA>@catproc
```

결과는 다음 Query 문으로 알아 본다.

```
column waiting_session format a8
select lpad(' ',3*(level-1)) || waiting_session,
  lock_type,
  mode_requested,
  mode_held,
  lock_id1,
  lock_id1,
  lock_id2
from lock_holders
connect by  prior waiting_session = holding_session
start with holding_session is null;
```

위의 Query에 의한 출력은 다음과 같다.

```
WAITING_ LOCK_TYPE           MODE_REQUE MODE_HELD   LOCK_ID1   LOCK_ID2
-------- ----------------- ---------- ---------- ---------- ----------
  5       None
  6       Transaction        Exclusive   Exclusive   262172       90
  9       Transaction        Exclusive   Exclusive   262172       90
```

여기서 Session 6, Session 9가 Session 5를 기다리고 있음을 알 수 있다.

■ Lock & Hanging 문제를 추정하는 방법

프로그램 상에서 어느 부분이 Lock, Hanging 문제를 일으키는지 알아내기가 여의치 않을때 다음과 같은 방법을 사용해 보기 바란다.

1. init.ora의 sql_trace=ture로 세팅하면 연관된 SQL 명령이 출력될 것이다.
2. OS 상에서도 Process 상태를 점검하여 본다.
3. OS 상의 Debugging 기능을 사용하거나 만약 가능하다면 oradbx를 사용한다. Platform 에 따라서 없을 수도 있다.
4. 여러가지 Monitoring 방법과 Locking/Blocking 스크립트를 이용한다.

■ PROCESS STACKS 확인

때로는 Hanging Process나 리소스를 점유하고 있는 Process의 Process Stack을 점검해 보는 것이 문제 해결에 도움이 되는 경우가 있다.

1. OS Debugging 프로그램을 이용하여 Hangup 되기 전의 마지막 Call을 확인 한다.

   ex)%truss -p

2. oradbx(오라클 debugging 프로그램)은 Support,Development 시에만 사용 된다.

```
select substr(s.username,1,11) "ORACLE USER" ,
   p.pid "PROCESS ID", s.sid "SESSION ID", s.serial#,
   osuser "OS USER", p.spid "PROC SPID"
from v$session s, v$access a
where a.sid=s.sid and
   p.addr=s.paddr and
   s.username != 'SYS'
/
```

위의 Query를 실행하면 다음과 같은 결과가 출력된다.

ORACLE USER	PROCESS ID	SESSION ID	SERIAL#	OS USER	PROC SPID	SESS SPID	LOCKWT
LTO	19	5	31	usupport	17312	17309	
LTO2	25	6	43	usupport	17313	17310	C3D320F4
LTO3	26	9	1	usupport	17314	17311	C3D320D8

만약 oradbx가 없다면 다음과 같이 해서 만들어 준다.

```
%cd $ORACLE_HOME/rdbms/lib
%make -f oracle.mk oradbx
```

LTO Process가 무엇을 하고 있는지 알고 싶으면 Process Stack을 보면 알수 있다.

```
ps -ef ¦ grep 17312
  usupport 17312 17309 0 Sep 15 ?  0:00 oracleV713(DESCRIPTION=(LOCAL=YE
```

```
type
debug 17312 (이 유저의 oracle shadow process)
dump stack
dump procstat
```

위에서 생성된 트레이스 화일(user_dump_dest 에 생성됨)을 이용하면 Hanging 문제를 해결하는데 큰 도움이 된다.

■ 자주 발생하는 LOCK 문제들

1. Parent-Child 관계로 묶인 테이블에서 Child 테이블에 Index가 없는 상황서 Child 테이블을 수정하게 되면 Parent테이블에 TABLE LEVEL SHARE LOCK이 걸리게 되어서 Parent 테이블에 대한 모든 Update가 금지된다.
2. 블럭의 PCTFREE가 매우 작다면 한 블럭에 여러개의 레코드가 들어 있기 때문에 한 블럭에 과도한 트랜잭션이 들어와서 블럭의 Transaction Layer가 Release 되기를 기다리게 되는 경우도 있다.

Ex)

```
create table test (a number) initrans 1 maxtrans 1;

SYSTEM:  insert into test values (5);            /* completed   */
SCOTT:   insert into SYSTEM.test values (10);   /* Scott waits */
```

SID	OWNER	LOCK_TYPE	MODE_HELD	MODE_REQUE	LOCK_ID1	LOCK_ID2
7	System	Transaction	Exclusive	None	196639	54
10	Scott	Transaction	None	Share	196639	54

LOCKING CONFLICTS를 DETECTING/RESOLVING

Bulletin no : 10148

Problem Description:

With the growing complexity of applications, it has become a challenge to resolve and properly diagnose locking (hanging) issues. Although it may seem like quite a challenge to find out what is causing your application or your database to hang, we are providing you with a methodology of things to check when faced with a difficult "hanging" scenario.

Problem Explanation:

Bulletin 106450.503 by LTO explains how to use a combination of utilities:
SQL, MONITORING FACILITY and LOCKING scripts provided by Oracle.
The screens used for monitor are session, lock, and table. The locking scripts used are catblock.sql and utllockt.sql

USE OF MONITOR SCREENS:

MONITOR SESSION:

Session ID	Serial Number	Process ID	Status	Username	Lock Waited	Current Statement
6	35	28	ACTIVE	LTO2	C2D2B3B4	UPDATE
8	70	19	INACTIVE	SYSTEM		SELECT
12	15	25	INACTIVE	LTO		INSERT
14	17	27	ACTIVE	LTO3	C2D2B438	DELETE
15	30	26	ACTIVE	SYS		UNKNOWN

Name	Description
Session Id	SID - Session Identifier
Serial Nr	Session serial number. Used to uniquely identify a session's objects. Guarantees that session-level commands are applied to the correct session objects in the event that the session ends and another session begins with the same session ID
Process Id	PID in v$process. Oracle Process Identifier.
Status	Status of the session.
Username	User name.
Lock waited	Address of lock waiting for ; null if none.

The most important column would be LOCK WAITED. If it is NOT NULL, then

this user is waiting for a resource. In this example, LTO2 and LTO3 are waiting on locks.

MONITOR LOCK:

Username	Session ID	Serial Number	Lock Type	Resource ID 1	Resource ID 2	Mode Held	Mode Requested
LTO2	6	35	TM	2294	0	RX	NONE
LTO2	6	35	TM	2295	0	RX	NONE
LTO2	6	35	TX	262167	87	NONE	X
LTO2	6	35	TX	327682	90	X	NONE
LTO	12	15	TM	2294	0	RX	NONE
LTO	12	15	TM	2295	0	RX	NONE
LTO	12	15	TX	262167	87	X	NONE
LTO3	14	17	TM	2294	0	RX	NONE
LTO3	14	17	TM	2295	0	RX	NONE
LTO3	14	17	TX	262167	87	NONE	X
LTO3	14	17	TX	196636	87	X	NONE

Username	User name.	
Session Id	SID - Session Identifier.	
Serial Nr	Same as above.	
Type of Lock	LOCK ID1	LOCK ID2
TX(Transaction)	Decimal representation of rollback segment number and slot number	Decimal representation of "wrap" number (number times the rollback slot has been reused)
TM(Table Locks)	Object id of table being modified.	Always 0.
RW(Row Wait)	Decimal representation of file no. and block no. (Version 6)	Decimal representation of row within block
UL(User Defined Locks)	Complete list is found in chpt 10 of the Oracle 7 Concepts Manual or in Appendix B-81 in the Oracle 7 Admin Guide. Please note that the RW enqueue was taken out after version 6.0.34 and may be added back in a future release.	
Mode Held	Lock mode held.	
Mode Requested	Lock mode requested.	

The following users are waiting:

LTO2	6	35	TX	262167	87	NONE	X
LTO3	14	17	TX	262167	87	NONE	X

They are waiting on:

```
LTO                  12     15   TM       2294        0    RX   NONE
LTO                  12     15   TM       2295        0    RX   NONE
LTO                  12     15   TX     262167       87     X   NONE
```

For the TX lock, lock id1 and lock id2 are the same when they are contending for the same resouces. Within the block, we have a record of the each transactions pertaining to the data in that particular block. If the transaction is not committed or rolled back, then other subsequent
transactions may have to wait for that resource.

Often, the user(s) may be modifying many tables within the same transaction.
At times, this will make it difficult to find out which resource the WAITER is contending from the HOLDER. This is easily resolved by looking at a
combination of two monitors. MONITOR SESSION will tell you which user is
waiting on a lock and MONITOR TABLE will tell you the table that the user is currently trying to modify.

```
MONITOR SESSION:
---------------

Session Serial Process                          Lock

  ID    Number   ID    Status   Username        Waited   Current Statement

================================================================================
      5     31      19 INACTIVE LTO                      INSERT
      6     43      25 ACTIVE   LTO2            C3D320F4 UPDATE
      9      1      26 ACTIVE   LTO3            C3D320C8 DELETE

MONITOR TABLE:
-------------
Session
   ID   Schema Name                  Table Name
================================================================================
      5 LTO                          DEPT
      6 LTO                          EMP
      9 LTO                          EMP
```

Session 6 and 9 are waiting for locks. They are both attempting to modify
lto.emp TABLE.

RESOLUTION:

Most locking issues are application specifics. To resolve locking contention, one needs to free the resource by:

1. Asking the HOLDER to commit or rollback,
2. Killing the session which holds the lock,
   ALTER SESSION KILL SESSION sid, serial#; or
   use KILL USER SESSION menu found in the sqldba form.
3. Killing the unix shadow process which is not recommended.
   When killing the unix shadow process, please be careful of shared servers in a multi-threaded environment.
4. ROLLBACK FORCE or COMMIT FORCE if 2pc pending transaction.

SQL STATEMENTS TO DECIPHER LOCKING ISSUES:

TRANSACTIONS INVOLVED:

This query defines the transactions involved in a locking situation:

```
column username format a10
column sid format 999
column lock_type format a15
column MODE_HELD format a11
column MODE_REQUESTED format a10
column LOCK_ID1 format a8
column LOCK_ID2 format a8

select a.sid,
   decode(a.type,
   'MR', 'Media Recovery',
   'RT', 'Redo Thread',
   'UN', 'User Name',
   'TX', 'Transaction',
   'TM', 'DML',
   'UL', 'PL/SQL User Lock',
   'DX', 'Distributed Xaction',
   'CF', 'Control File',
   'IS', 'Instance State',
   'FS', 'File Set',
   'IR', 'Instance Recovery',
   'ST', 'Disk Space Transaction',
   'IR', 'Instance Recovery',
   'ST', 'Disk Space Transaction',
   'TS', 'Temp Segment',
```

```
   'IV', 'Library Cache Invalidation',
   'LS', 'Log Start or Switch',
   'RW', 'Row Wait',
   'SQ', 'Sequence Number',
   'TE', 'Extend Table',
   'TT', 'Temp Table',
   a.type) lock_type,
   decode(a.lmode,
   0, 'None',           /* Mon Lock equivalent */
   1, 'Null',           /* N */
   2, 'Row-S (SS)',     /* L */
   3, 'Row-X (SX)',     /* R */
   4, 'Share',          /* S */
   5, 'S/Row-X (SSX)',  /* C */
   6, 'Exclusive',      /* X */
   to_char(a.lmode)) mode_held,
   decode(a.request,
   0, 'None',           /* Mon Lock equivalent */
   1, 'Null',           /* N */
   2, 'Row-S (SS)',     /* L */
   3, 'Row-X (SX)',     /* R */
   4, 'Share',          /* S */
   5, 'S/Row-X (SSX)',  /* C */
   6, 'Exclusive',      /* X */
   to_char(a.request)) mode_requested,
   to_char(a.id1) lock_id1, to_char(a.id2) lock_id2
from v$lock a
   where (id1,id2) in
     (select b.id1, b.id2 from v$lock b where b.id1=a.id1 and
     b.id2=a.id2 and b.request>0)
/
```

Sample output:

```
 SID LOCK_TYPE       MODE_HELD   MODE_REQUE LOCK_ID1 LOCK_ID2
---- --------------- ----------- ---------- -------- --------
   5 Transaction     Exclusive   None       262172   90
   6 Transaction     None        Exclusive  262172   90
   9 Transaction     None        Exclusive  262172   90
```

This is very similar to the monitor lock output. Again, SID 6 and 9 are waiting on SID 5.

OBJECTS INVOLVED:

This query will defined the objects that are involved in the locking conflict:

```
column username format a10
column lockwait format a10
column sql_text format a80
column object_owner format a14
column object format a15

select b.username username, c.sid sid, c.owner object_owner,
   c.object object, b.lockwait, a.sql_text SQL
from v$sqltext a, v$session b, v$access c
where a.address=b.sql_address and
   a.hash_value=b.sql_hash_value and
   b.sid = c.sid and c.owner != 'SYS'
/
```

Sample output:

```
USERNAME          SID OBJECT_OWNER   OBJECT          LOCKWAIT
---------- ---------- -------------- --------------- ----------
SQL
------------------------------------------------------------------
LTO2                6 LTO            EMP             C3D320F4
update lto.emp set empno =25 where empno=7788

LTO3                9 LTO            EMP             C3D320C8
delete from lto.emp where empno=7788

LTO                 5 LTO            DEPT
insert into lto.dept values (60,'PROGRAMMER','LOS ANGELOS')
```

One can see the DMLs resulting from LTO2 and LTO3 who are the WAITERS.
They are waiting for LTO to commit or rollback; unfortunately, the DML
statement listed under LTO is only the most recent DML. This may not indicate the statement that is holding the resource. In this example, it is NOT the DML. The only DML was an update statement. OBJECT_OWNER.OBJECT is the object USERNAME is attempting to modify.

PROCESSES INVOLVED:

To ascertain process information, one can execute this query:

```
column "ORACLE USER" format a11
column SERIAL# format 9999999
column "OS USER" format a8

select substr(s.username,1,11) "ORACLE USER", p.pid "PROCESS ID",
   s.sid "SESSION ID", s.serial#, osuser "OS USER", p.spid "PROC SPID",
```

```
   s.process "SESS SPID", s.lockwait "LOCK WAIT"
from v$process p, v$session s, v$access a
where a.sid=s.sid and
   p.addr=s.paddr and
   s.username != 'SYS'
/
```

Sample output:

```
ORACLE USER PROCESS ID SESSION ID  SERIAL# OS USER  PROC SPID SESS SPID LOCKWT
----------- ---------- ---------- -------- -------- ---------------------
LTO                 19          5       31 usupport 17312     17309
LTO2                25          6       43 usupport 17313     17310   C3D320F4
LTO3                26          9        1 usupport 17314     17311   C3D320C8
```

What can you do??

1) Can ask LTO to commit or rollback or
2) alter system kill session '5,31'; or
3) kill -9 17309   (shadow process on unix)
   stop/id= (PROC SPID=SESS SPID on vms running single task)

Notice that the user SYS has been excluded. If your application was created under SYS, then you may want to include it in the query.

USING CATBLOCK.SQL and UTLLOCKT.SQL:

These are very helpful scripts that oracle provides with the other RDBMS installation scripts (CATALOG.SQL, CATPROC.SQL, etc.) For example, ?/rdbms/admin directory on unix and ora_rdbms directory on vms.

First run catblock.sql as sys and then run utllock.sql as sys.

Print out the result in a tree structured fashion:

```
column waiting_session format a8

select lpad(' ',3*(level-1)) || waiting_session waiting_session,
   lock_type,
   mode_requested,
   mode_held,
   lock_id1,
   lock_id2
from lock_holders
connect by  prior waiting_session = holding_session
start with holding_session is null;
```

Sample Output:

```
WAITING_ LOCK_TYPE         MODE_REQUE MODE_HELD  LOCK_ID1   LOCK_ID2
-------- ----------------- ---------- ---------- ---------- ----------
   5        None
   6     Transaction       Exclusive  Exclusive  262172     90
   9     Transaction       Exclusive  Exclusive  262172     90
```

Sessions 6 and 9 are waiting on session 5.

DRASTIC METHODS IN DECIPHERING A LOCKING OR "HANGING" SITUATION:

At times, it may be difficult to find out the root cause of the "hanging" problem within your application. One may need to resort to drastic measures which include both tracing on the database and operating system layer. Here are some recommendations:

1) Turn on sql_trace. This will reveal the sql statements involved.
2) Set timing on and monitor processes on the operating system side as well. This will reveal if the process is getting any cpu and the status of the process.
3) Utilize the operating system debugging facilities or utilize "oradbx" if available. (oradbx is only for support and development use and may not be available on your platform.)
4) Utilize monitor screens and the locking (blocking) scripts.
5) One can build your own self monitoring script.

For example:

Create a file called monitor.sql:

```
select b.username username, c.sid sid, c.owner object_owner,
   c.object object, b.lockwait, a.sql_text SQL
from v$sqltext a, v$session b, v$access c
where a.address=b.sql_address and
   a.hash_value=b.sql_hash_value and
   b.sid = c.sid and
   c.owner != 'SYS'
/
@?/rdbms/admin/utllockt

!ps -ef

!sleep
```

Create a file name runtest with executable permissions:

```
sqlplus name/password << test
spool output.txt
set time on
set echo on
@monitor
```

ASCERTAINING PROCESS STACKS:

In some rare cases, one may encounter a "hanging" problem that can not be fathom. One may find that a process stack of the hanging process or the process that holds the resource may be helpful in resolving the problem.

1) Operating system debugging tools can be used to find out the last call before it hangs. ie. truss -p
2) ORACLE'S debugging utility, oradbx, is for SUPPORT and DEVELOPMENT's use ONLY.

```
select substr(s.username,1,11) "ORACLE USER" ,
   p.pid "PROCESS ID", s.sid "SESSION ID", s.serial#,
   osuser "OS USER", p.spid "PROC SPID"
from v$session s, v$access a
where a.sid=s.sid and
   p.addr=s.paddr and
   s.username != 'SYS'
/
```

Sample Output:

```
ORACLE USER PROC ID SESSION ID  SERIAL# OS USER  PROC SPID SESS SPID LOCK WAIT
---------------------------------- --------------------------------------
LTO            19          5       31   usupport  17312     17309
LTO2           25          6       43   usupport  17313     17310    C3D320F4
LTO3           26          9        1   usupport  17314     17311    C3D320C8
```

```
cd $ORACLE_HOME/rdbms/lib
make -f oracle.mk oradbx
```

To find out what LTO process is actually doing, one can dump the process stack.

```
ps -ef ¦ grep 17312
usupport 17312 17309  0  Sep 15  ?     0:00 oracleV713  (DESCRIPTION=(LOCAL=YE
```

```
type
debug 17312 (which is the oracle shadow process for this user)
dump stack
dump procstat
```

These trace files may be instrumental in resolving a hanging problem. Please remember that Oracle's debugging utility, oradbx, is for Support and Development's use ONLY. The trace files should go to your USER_DUMP_DEST and should be sent to Worldwide Support for debugging.

UNUSUAL LOCKING PROBLEMS:

1) When your application has referential integrity and attempts to modify the child table, Oracle will get a TABLE LEVEL SHARE LOCK on the parent table when there is NO index on the foreign key.

2) When a table's PCTFREE is set too low , the block is full with data, and there are many concurrent DML's occurring on rows within the block, one may see a Share Lock being requested when doing a DML. To my knowledge, this is only time we grab the SHARED lock. Instead of waiting for a lock, this process is waiting for some extra space or a release of an INITRANS within the transaction layer of the block.

Here's a simple example:

```
create table test (a number) initrans 1 maxtrans 1;

SYSTEM:  insert into test values (5);   /* completed */
SCOTT:   insert into SYSTEM.test values (10);    /* Scott waits */
```

In normal situations, SCOTT should not have to wait but SCOTT has to wait until SYSTEM releases the one and only INITRANS in the block. If the MAXTRANS was not set to 1, then oracle will try to allocate another INITRANS in the block if space permits. I believe each INITRANS is about 24 bytes.

```
 SID OWNER   LOCK_TYPE       MODE_HELD   MODE_REQUE LOCK_ID1 LOCK_ID2
---- -----   --------------- ----------- ---------- -------- --------
   7 System  Transaction     Exclusive   None       196639   54
  10 Scott   Transaction     None        Share      196639   54
```

LOCKING CHART:

The following table describes which table accesses are gotten for which table operations and what equivalent DML modes are actually gotten in various conditions.

		DML Table Lock Mode				
		Yes	Yes	No	No	Row-Locking
Operation	Table Access	No	Yes	No	Yes	Serializable
Select	Read	NULL	S	NULL	S	
Select For Update	Row-Read	SS	S	SS	S	
Insert	Row-Write	SX	SX	SSX	SSX	
Update	Read-Row-Write	SX	SSX	SSX	SSX	
Delete	Read-Row-Write	SX	SSX	SSX	SSX	
Lock For Update	Row-Read	SS	S	SS	S	

Lock Share		S	S	S	S
Lock Exclusive		X	X	X	X
Lock Row Share		SS	SS	SS	SS
Lock Row Exclusive		SX	SX	SX	SX
Lock Share Row Exclusive		SSX	SSX	SSX	SSX
Alter	Write	X	X	X	X
Drop	Write	X	X	X	X
Create Index		S	S	S	S
Drop Index	Write	X	X	X	X

Lock Mode Compatibility:

	NULL	SS	SX	S	SSX	X
NULL	YES	YES	YES	YES	YES	YES
SS	YES	YES	YES	YES	YES	no
SX	YES	YES	YES	no	no	no
S	YES	YES	no	YES	no	no
SSX	YES	YES	no	no	no	no
X	YES	no	no	no	no	no

MESSAGE CONVERSION PROGRAM

Bulletin no : 10029

많은 site에서 영문 DB에서 한글 메시지를 사용하고 싶다고 문의가들어옵니다. 현재는, 지금 가지고 있는 *ko.msb를 가지고 영문 DB에서 사용하면 ???로 display가됩니다. 이것을 해결하기 위해 2가지 방법이 있습니다.

1. lmsgen를 다시만드는 것입니다.
   Ex> lmsgen iacko.msg forms30 iac korean_korea.us7ascii
  로 하면 iacko.msb로 만드는 동시에 이 화일에 영문이라는 정보가 들어가기 때문에 영문 DB에서 아무런 문제없이 사용하실 수 있습니다.

2. lmsgen는 각 system에마다 가지고 있어야 합니다. Sun에서 만든 *ko.msb를 hp나 다른 system에서 사용할 수 없습니다. 현재 각 system에 해당하는 lmsgen를 보관하고 있는것은 아닙니다.
 그래서 현재 각 tool에 있는 message에 korean 정보를 us7ascii로 바꾸는 program를 만들었습니다.

   사용하는 방법
   $ cc conlm.c -o conlm
   $ sh -x k (conlm 화일이름 KO16KSC5601 --> 화일에 영문 DB를 KO DB로 바꾸는것) 또는 sh -x u
     (conlm 화일이름 US7ASCII --> 화일에 KO DB를 영문 DB로 바꾸는것) 를 하시면 됩니 .

   - conlm.c 프로그램.

```
#include
#include
#include

int  us_to_ko, ko_to_us; /*message conversion flag */
main(argc,argv)
int       argc;
char      *argv[];
{
   us_to_ko = ko_to_us = 0;
   if ( argc != 3) {
           usage(argv[0]);
           exit(1);
}

if (!strcmp(argv[2], "KO16KSC5601"))
       us_to_ko = 1;
else if (!strcmp(argv[2], "US7ASCII"))
       ko_to_us = 1;
else
       usage(argv[0]);
data_conversion(argv[1]);
```

```
printf("conversion completed sucessfully !!!\n");
}

usage(prog_name)
char *prog_name;
{
        printf("Usage: %s exported_file_name charset \n",prog_name);
        printf("       charset :KO16KSC5601¦US7ASCII \n");
        printf("       . KO16KSC5601 - convert us7ascii format to
                ko16ksc5601 format\n");
        printf("       . US7ASCII -convert ko16ksc5601 format to us7ascii
                format\n");
}

data_conversion(usfile)
char *usfile;
{
  int     fd;
  char    conv_buffer[11];
  if((fd = open(usfile, 2)) < 0) {
           perror("open");
           exit(1);
}
if(lseek(fd, 0, 0) < 0) {
           perror("lseek");
           exit(1);
}
if(us_to_ko){
          conv_buffer[0] = 11;
          conv_buffer[1] = 15;
          conv_buffer[2] = 28;
          conv_buffer[3] = '!';
          conv_buffer[4] = 11;
          conv_buffer[5] = 19;
          conv_buffer[6] = 3;
          conv_buffer[7] = ' ';
          conv_buffer[8] = '!';
          conv_buffer[9] = 27;
          conv_buffer[10] = 28;
}
else if(ko_to_us){
          conv_buffer[0] = 21;
          conv_buffer[1] = 19;
          conv_buffer[2] = '"';
          conv_buffer[3] = 1;
          conv_buffer[4] = 19;
          conv_buffer[5] = 3;
          conv_buffer[6] = '\t';
```

```
            conv_buffer[7] = '\t';
            conv_buffer[8] = '\0';
            conv_buffer[9] = '\0';
            conv_buffer[10] = '\0';
}
if(write(fd,conv_buffer,11) !=11) {
            perror("write");
            exit(1);
}
close(fd);
}
```

METHOD 4를 이용한 SAM FILE생성 PRORAM

Bulletin no : 10302

아래의 Program은 Pro*C의 Dynamic Method 4를 응용하였으며, SQL*Loader에서 사용할 수 있는 Data file및 Control file을 자동으로 생성해 준다.
(Source File은 천리안 "go oracle" 로 가서 구할 수 있다)

(1) 사용 방법 :

```
$ db2file  id/passwd  Table_name
```

(2) 출력 File :

Table_name.dat, Table_name.ctl

(3) SQL*Loader로 Loading방법:

```
$ sqlload id/pwd control=Table_name.ctl
```

아래의 Program을 $ORACLE_HOME/proc/demo 디렉토리로 db2file.pc라는 이름으로 생성한 후 make file을 수행한다.

```
$ make -f proc.mk EXE=db2file OBJS=db2file.o
```

(현재 Max Item의 갯수는 40으로 설정되어있으며, 필요에따라 수정하여 사용 한다. 또한 자세한 내용은 Pro*C 메뉴얼을 참조.)

```
#include
#include
#include

EXEC SQL BEGIN DECLARE SECTION;
    VARCHAR     username [40];
    VARCHAR     statement[40];
EXEC SQL END DECLARE SECTION;

EXEC SQL INCLUDE sqlca.h;
EXEC SQL INCLUDE sqlda.h;

#define MAX_ITEMS         40
#define MAX_VNAME_LEN     30
#define MAX_INAME_LEN     30

SQLDA *bind_dp;
SQLDA *select_dp;

extern SQLDA *sqlald();
extern void sqlprc();
```

```
int precision, scale;
short null_ok;
extern void sqlnul();
void print_rows();

FILE *fp, *fp_ctl;
main(argc, argv)
int argc;
char **argv;
{
    int i, j, num_ret = 0;
    char filename[12], ctl_file[12];

    if(argc != 3) {
        printf("\n\n      Input data type error !!!\n");
        printf("\n      Usage: db2file scott/tiger emp \n\n\n");
        exit(0);
    }

    EXEC SQL WHENEVER SQLERROR GOTO sqlerror;

    strcpy (username.arr, argv[1]);
    username.len = strlen(username.arr);
    EXEC SQL CONNECT :username;

      if(alloc_descriptors(MAX_ITEMS, MAX_VNAME_LEN, MAX_INAME_LEN)
                                        != 0)
        exit(1);

    sprintf(statement.arr, "select * from %s", argv[2]);
    statement.len = strlen(statement.arr);

    /* Prepare the statement and Declare a cursor */
    EXEC SQL PREPARE S FROM :statement;
    EXEC SQL DECLARE C CURSOR FOR S;

    bind_dp->N = MAX_ITEMS;
    EXEC SQL DESCRIBE BIND VARIABLES FOR S INTO bind_dp;

    /* If "F" is negative, there were more bind variables
       than was originally allocated using "sqlald" */

    if(bind_dp->F < 0) {
        printf("\nToo many bind variables for descriptor.");
        goto sqlerror;
    }

    /* Set the max number of array elements to the number found */
```

```
  bind_dp->N = bind_dp->F;

  EXEC SQL OPEN C USING DESCRIPTOR bind_dp;
  EXEC SQL DESCRIBE SELECT LIST FOR S INTO select_dp;

  /* If "F" is negative, there were more select list variables
     than was originally allocated using "sqlald" */

    if(select_dp->F < 0) {
       printf("\nToo many select list variables for descriptor:
                              %d",-(select_dp->F));
      goto sqlerror;
  }

  sprintf(filename, "%s.dat", argv[2]);
  sprintf(ctl_file, "%s.ctl", argv[2]);
  fp     = fopen(filename, "w");
  fp_ctl = fopen(ctl_file, "w");

  printf ("\n Table %s date unloading ......\n", argv[2]);

/* Set the max number of elements in descriptor to that found */

   select_dp->N = select_dp->F;

/* Now allocate storage for each of the SELECT list variables.*/

  /* Control File을 생성하는 부분(Header) */
  printf("Control file creating .....\n");

  fprintf(fp_ctl, "load data\n");
  fprintf(fp_ctl, "infile '%s'\n", filename);
  fprintf(fp_ctl, "insert\n");
  fprintf(fp_ctl, "into table %s\n", argv[2]);
  fprintf(fp_ctl, "fields enclosed by X'22' \n(\n");

  for(i=0; iF; i++) {

      sqlnul (&(select_dp->T[i]), &(select_dp->T[i]), &null_ok);

      switch (select_dp->T[i])
      {
          case  1 : /* CHAR datatype   */
                    break;

          case  2 : /* NUMBER datatype */
                    sqlprc(&(select_dp->L[i]), &precision, &scale);
                    /* Max size of NUMBER */
```

```
                if(precision == 0) precision = 40;
          /* Also allow for decimal point and possible sign */
                select_dp->L[i] = precision;
                /* allow for negative scale */
                if(scale < 0)  select_dp->L[i] += -scale;
                break;

           case  8 : /* LONG datatype */
                select_dp->L[i] = 240;
                break;

           case 12 : /* DATE datatype */
                select_dp->L[i] = 9;
                break;

           case 23 : /* RAW datatype */
                break;

           case 24 : /* LONG RAW datatype */
                select_dp->L[i] = 240;
                break;
      }

      if(select_dp->T[i] != 2)
          select_dp->V[i] = (char *) realloc(select_dp->V[i],
                            select_dp->L[i] + 1);
      else
          select_dp->V[i] = (char *) realloc(select_dp->V[i],
                            select_dp->L[i]);

  /* Coerce ALL datatypes except for LONG RAW's to character */
      if(select_dp->T[i] != 24)  select_dp->T[i]  = 1;

if(select_dp->T[i] == 2)
   if(scale > 0) select_dp->T[i] = 4;
   else          select_dp->T[i] = 3;

      /* Control File을 생성하는 부분 */

      if(select_dp->T[i] == 2) {
         if(select_dp->F != i+1) {
       fprintf(fp_ctl,"\"%.*s\"   CHAR(%d) NULLIF  \"%.*s\"=BLANKS,
                       \n",select_dp->C[i],select_dp->S[i],
                      (int)select_dp->L[i],select_dp->C[i],
                      select_dp->S[i]);
         }
         else  {
                fprintf(fp_ctl,"\"%.*s\"   CHAR(%d) NULLIF
```

```
                    \"%.*s\"=BLANKS \n", select_dp->C[i],
                    select_dp->S[i],(int)select_dp->L[i],
                    select_dp->C[i],select_dp->S[i]);
        }
      }
      else {
        if(select_dp->F != i+1) {
                    fprintf(fp_ctl,"\"%-.*s\"  CHAR(%d) NULLIF
                      \"%-.*s\"=BLANKS,\n", select_dp->C[i],
                      select_dp->S[i],(int)select_dp->L[i],
                      select_dp->C[i],select_dp->S[i]);
        }
        else {
         fprintf(fp_ctl,"\"%-.*s\"  CHAR(%d) NULLIF
             \"%-.*s\"=BLANKS \n",select_dp->C[i],select_dp->S[i],
          (int)select_dp->L[i], select_dp->C[i],select_dp->S[i]);
        }

      }
  } /* end of for */

fprintf(fp_ctl, ")\n");
fclose(fp_ctl);

  EXEC SQL WHENEVER NOT FOUND GOTO end_select_loop;

  for (;;)
  {
      EXEC SQL FETCH C USING DESCRIPTOR select_dp;

      for (i = 0; i < select_dp->F; i++)
      {
          if(*select_dp->I[i] < 0)
            if (select_dp->T[i] == 4)
              fprintf(fp,"\"%-*c\" ",(int)select_dp->L[i]+3, ' ');
            else
              fprintf(fp,"\"%-*c\" ",(int)select_dp->L[i], ' ');
          else
              if(select_dp->T[i] == 3)     /* int datatype */
                fprintf(fp,"\"%*d\" ", (int)select_dp->L[i],
                              *(int *)select_dp->V[i]);
              else if(select_dp->T[i] == 4)   /* float datatype */
                fprintf(fp,"\"%*.2f\" ", (int)select_dp->L[i],
                              *(float *)select_dp->V[i]);
              else                          /* character string */
                fprintf(fp,"\"%-*.*s\" ",(int)select_dp->L[i],
                         (int)select_dp->L[i], select_dp->V[i]);
```

```
        }
        fprintf (fp,"\n");
    }
end_select_loop:
   printf("\n\n Unloaded to %s (rows= %d) \n",
            filename, sqlca.sqlerrd[2]);

      /* Free up space used by descriptor (gracefull exit)   */
  /*1. Free the "malloc"'ed space for the select list variables*/
    for (i=0; iF; i++) {
        free (select_dp->V[i]);
        free (select_dp->I[i]);
    }

    /* 2. Free up space used by the descriptors themselves */
    sqlclu (bind_dp);
    sqlclu (select_dp);

    fclose(fp);

    EXEC SQL CLOSE C;
    EXEC SQL COMMIT WORK RELEASE;
    return;

/* Oracle Error Handler */
sqlerror:
    printf ("\n%.70s\n",sqlca.sqlerrm.sqlerrmc);
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    EXEC SQL ROLLBACK WORK RELEASE;
    return;
}

alloc_descriptors(size, max_vname_len, max_iname_len)
int size;
int max_vname_len;
int max_iname_len;
{
    int i;

    if ((bind_dp =
        sqlald(size, max_vname_len, max_iname_len)) == (SQLDA *) 0)
    {
        fprintf(stderr,
            "Cannot allocate memory for bind descriptor.");
        return -1;  /* Have to exit in this case. */
    }

    if ((select_dp =
```

```
    sqlald (size, max_vname_len, max_iname_len)) == (SQLDA *) 0)
{
    fprintf(stderr,
        "Cannot allocate memory for select descriptor.");
    return -1;
}
select_dp->N = MAX_ITEMS;

/* Allocate the pointers to the indicator variables, and the
   actual data. */
for (i = 0; i < MAX_ITEMS; i++) {
    bind_dp->I[i] = (short *) malloc(sizeof (short));
    select_dp->I[i] = (short *) malloc(sizeof(short));
    bind_dp->V[i] = (char *) malloc(1);
    select_dp->V[i] = (char *) malloc(1);
}

return 0;
}
```

NETWARE LISTENER 활용(MAX USERS)

Bulletin no : 10034

현재 netware listener(spxsrv)의 최대 연결가능 사용자수가 기본으로 30명이 설정되어 있습니다.
이를 변경하려면 다음 parameter를 추가하세요.

어디에 : netware server의 SYS:SYSTEM\config.ora
무엇을 : SPX_MAX_CLIENTS = 사용자수 (DEFAULT:30)

NETWARE SUPPORT MATRIX

Bulletin no : 10316

NetWare Version	3.11	3.12	3.11 SFTIII	4.01	4.02	4.1	4.1 SMP	4.1 SFTIII
6.0.36/6.0.37	Yes	Yes	Yes	No	No	No	No	No
7.0.15.4.x	Yes	Yes	Yes	Yes	Yes	No	No	No
7.0.15.5.0 ^	No	Yes	No	No	No	No	No	No
7.0.15.5.x	Yes	Yes	Yes	Yes	Yes	No	No	No
7.0.16.6.x	Yes	Yes	Yes	Yes	Yes	No	No	No
7.1.3.3.x	Yes	Yes	Yes	Yes	Yes	Yes	No	No
7.1.4.3.x	Yes	Yes	Yes	Yes	Yes	Yes	No	No
7.2.2.4.x	Yes	Yes	No	No	No	Yes	Yes	Yes
7.3.2.1.x *	No	Yes	No	No	No	Yes	Yes	Yes

ORACLE의 RELEASE NOTE에 보면 NETWARE에 대한 PATCH를 요구하는 경우가 있으므로 해당 PATCH를 NOVELL로부터 받아서 INTALL을 한 다음에 ORACLE을 설치해야 한다.

7.0.15.5.0 버젼은 OracleWare를 위한 특별버젼으로 기존에 이 버젼을 사용하던 고객은 모두 UPGRADE를 하였으므로 이 버젼은 현재 사용되지 않는다.

NLS CHARACTERSET 변경방법 (DB REBUILD 없이)

Bulletin no : 10016

데이타베이스의 CHARACTER SET은 데이타 딕셔너리 테이블인 sys.props$에 들어 있다

```
SQL>desc sys.props$
Name                            Null?                Type
------------------------------  -----------------    ---------------
NAME                             NOT NULL            VARCHAR2(30)
VALUE$                                               VARCHAR2(2000)
COMMENT$                                             VARCHAR2(2000)

SQL>column c1 format a30
SQL>select name c1, value$ c1 from sys.props$;

C1                            C1
----------------------------  ------------------------------
DICT.BASE                      2
NLS_LANGUAGE                   AMERICAN
NLS_TERRITORY                  AMERICA
NLS_CURRENCY                   $
NLS_ISO_CURRENCY               AMERICA
NLS_NUMERIC_CHARACTERS         .,
NLS_DATE_FORMAT                DD-MON-YY
NLS_DATE_LANGUAGE              AMERICAN
NLS_CHARACTERSET               US7ASCII
NLS_SORT                       BINARY
GLOBAL_DB_NAME                 NLSV7.WORLD
```

여기서 NLS_CHARACTERSET에 현재 DB의 CHARACTER SET이 들어 있는데 이 값을 변경하여 DB의 CHARACTER SET을 변경할 수 있다. 여기서는 US7ASCII에서 KO16KSC5601 로 옮기는 경우를 알아보자.

우선 바꾸고자 하는 CHRACTER SET이 지원되는 지를 다음 명령으로 확인한다.

```
select convert('a','KO16KSC5601','US7ASCII') from dual;
```

만약 이 Select 문에서 ORA-01482 에러가 발생하면 지정한 CHARACTER SET이 지원되지 않는 경우이며 에러가 발생하지 않으면 CHARACTER SET을 변경할 수 있다.

작업을 하기전에는 만약을 위해서 DB 전체를 백업 받아두도록 한다.
CHARACTER SET 을 잘못 변경하면 DB 를 OPEN 할수가 없기 때문이다.
---

1. 다음의 Update문을 실행하여 CHARACTER SET을 변경한다.

```
UPDATE sys.props$
SET value$ = 'KO16KSC5601'
WHERE name = 'NLS_CHARACTERSET';
```

Update 시에 NLS_CHARACTERSET을 지원되지 않는 값으로 잘못 설정하거나 실수로 콘트롤 문자 같은 것이 들어가게 되면 DB가 Shutdown 된 다음에는 Startup 이 안 되므로 Update 후에 다음 명령으로 확인을 한 다음에 Commit을 하도록 한다.

```
select name, value$
from sys.props$
where value$ = 'KO16KSC5601';
```

Select 제대로 출력되면 Commit 하고 Shutdown 했다가 Startup 하게 되면 새로운 CHARACTER SET 값을 갖게 된다. SELECT가 안 되면 ROLLBACK하고 UPDATE 부터 다시 하도록 한다.

2. 환경 변수 NLS_LANG 을 변경한다.

.profile (or .cshrc) 에서

```
NLS_LANG=American_America.KO16KSC5601; export NLS_LANG
```

or

```
setenv NLS_LANG American_America.KO16KSC5601
```

OPEN VMS 시스템에서의 ORACLE AUTO STARTUP

Bulletin no : 10132

---

* OpenVMS 의 SYSTEM Account는 DBA 관련 작업을 할 수 없게 되어 있기 때문에 오라클의 Startup도 할 수 없다. 이 문제를 해결하기 위해서는 두가지 방법을 생각해 볼 수 있다.

[방법1] 시스템의 기동 시에 ORACLE7 에 의해서 작업이 수행되도록 설정한다.

1. 다음과 같은 내용을 갖는 프로시져(STARTUP_ORACLE7.COM)를 만든다.

```
$@ddcn:[oracle_root_directory.UTIL]ORAUSER.COM
$@ORA_RDBMS:INSORACLE.COM
$@ddcn :
    [oracle_root_directory.DB_dbname]STARTUP_EXCLUSIVE_dbname.COM
```

여기서

ddcn:	: 오라클이 인스톨된 디스크
oracle_root_directory	: 오라클의 최상위 디렉토리
dbname	: 오라클 데이타베이스 이름

만약 병렬 서버를 사용중이라면 프로시져 이름을 STARTUP_EXCLUSIVE_dbname.COM에서 STARTUP_PARALLEL_dbname.COM 으로 바꾼다. 그리고 여러 개의 데이타베이스를 Startup 시키려고 한다면 세번째 라인과 같은 항목을 각각의 데이타베이스에 대해서 모두 기술해 주도록 한다.

2. 위에서 만든 프로시져를 ORACLE7 에 의해서 수행되도록 한다.

```
$SUBMIT/USER=ORACLE7 STARTUP_ORACLE7.COM
```

여기서 ORACLE7은 OpenVMS 의 ORACLE7 의 Account 이다. 필요한 내용이 있으면 SUBMIT 명령에 추가해도 된다. 예를 들어 ORACLE을 위의 프로시져로 Startup 시킬 때 에러가 발생하는 경우를 고려하여 배치 로그 화일을 저장하려고 한다면 /LOG/NOPRINT 스위치를 사용하면 된다.

[방법2] SET UIC 명령으로 ORACLE 을 Startup 시키기

1. 다음의 명령들을 시스템의 startup procedure 에 추가한다.

```
OpenVMS V5.x: SYS$STARTUP:SYSTARTUP_V5.COM
OpenVMS V6.x: SYS$STARTUP:SYSTARTUP_VMS.COM
```

```
$ SET UIC[ORACLE7]
$ SET RIGHTS_LIST/ENABLE ORA_DBA
$ @ddcn:[oracle_root_directory.UTIL]ORAUSER.COM
$ @ORA_RDBMS:INSORACLE.COM
```

```
$ @ddcn:[oracle_root_directory.DB_dbname]STARTUP_EXCLUSIVE_dbname.COM
$ SET UIC[SYSTEM]
```

여기서

ddcn:	: 오라클이 인스톨된 디스크
oracle_root_directory	: 오라클의 최상위 디렉토리
dbname	: 오라클 데이타베이스 이름
ORACLE7	: OpenVMS ORACLE7 account
SYSTEM	: OpenVMS SYSTEM account

만약 병렬 서버를 사용중이라면 프로시져 이름을 STARTUP_EXCLUSIVE_dbname.COM에서 STARTUP_PARALLEL_dbname.COM 으로 바꾼다. 그리고 여러 개의 데이타베이스를 Startup 시키려고 한다면 여섯번째 라인과 같은 항목을 각각의 데이타베이스에 대해서 모두 기술해 주도록 한다.

SID 별로 ORA__DBA 와 같은 ORACLE 권한 Identifier를 사용한다면 이와 같은 것들도 Enable 시켜야 한다. ORA_DBA 권한 Identifier 를 사용하지 않는다면 이것은 Enable 시키지 말도록 하고 만약 ORA_DBA 와 ORA__DBA 를 모두 사용한다면 두 지 모두 Enable 시킨다.

2. OpenVMS SYSTEM Account 가 ORA_DBA and/or ORA__DBA 권한 Identifier를 부여하였음을 확인하고 그렇지 않다면 다음과 같이 한다.

```
$ SET DEFAULT SYS$SYSTEM
$ RUN AUTHORIZE
   UAF>GRANT/IDENTIFIER ORA_DBA SYSTEM
      and/or
   UAF>GRANT/IDENTIFIER ORA__DBA SYSTEM
   UAF>EXIT
```

OPS 에서 새로운 NODE 추가 방법

Bulletin no : 10192

---

1. hardware와 o/s setting(dlm)이 완벽한 상태

2. oracle home directory의 software를 모두 rcp .

3. 새로운 node의 init.ora file 수정

   o instance_number = 3 (ex: 기존에 두개의 node에서 추가시)
   o thread = 3
   o rollback_segment = () --> 원하는 만큼 생성.

4. primary node에서 작업
   o alter database add logfile thread 3
   group 7 '/dev/rmt/c0t1d0s1' size 9M,
   group 8 '/dev/rmt/c0t1d0s2' size 9M,
   group 9 '/dev/rmt/c0t1d0s3' size 9M;
   alter database enable public thread 3;

   o 3번에서 지정한 rollback segment를 생성하고 online 시킨다.

5. 추가된 node에서 database를 기동한다. (parallel mode)

ORA-54 WHEN DROP A TABLE(SESSION KILL)

Bulletin no : 10060

TABLE에 LOCK이 걸려 DML 및 DDL 명령 사용시 ORA-54 ERROR 가 나오는 경우가 있다. 이때 DB를 RESTART하면 되지만 더 효율적인 해결 방법은 다음과 같이할 수 있다. (serial number 가 negative 인 경우 그 값에 65536 을 더해야 함)

1. 예를 들어 LOCK 이 걸려 있는 테이블이 EMP 라고 가정하면 다음 SELECT문으로 EMP 테이블에 LOCK 을 걸고있는 SESSION을 찾는다.

```
$ sqlplus system/manager
  SQL> select  a.sid,a.serial#
       from    v$session a, v$lock b, dba_objects c
       where   a.sid=b.sid and
               b.id1=c.object_id and
               b.type='TM' and
               c.object_name=upper('EMP');

     SID       SERIAL#
     ---       -------
      5             1
      6             1
```

2. 다음 명령으로 SESSION들을 KILL 한다.

```
SQL> alter system  kill session '5,1';
SQL> alter system  kill session '6,1';
```

데이타베이스 STARTUP 시 ORA-312,313 에러
(ONLINE LOG CRASH)

Bulletin no : 10214

[ONLINE LOG 가 손상되었을때 DB에 OPERATION 이 없었던 경우는 다음과 같은 절차로 DB 을 OPEN 할수있다 - 확률 70%]

---

1. CONTROLFILE 생성

-. 손상된 online log 는 포함시키지 않는다.
-. resetlogs option 으로 생성한다.
-. reuse option 은 생략하고 기존 controlfile 은 다른이름으로 move 시킴.

```
sqldba> startup mount
sqldba> alter database backup controlfile to trace;
```

위와 같이 명령을 입력하면 ORACLE_HOME/rdbms/log 디렉토리에 트레이스 화일이 생긴다. 그 트레이스 화일에서 create controlfile 명령부분을 남기고 삭제 한다.

콘트롤화일 생성 문장 예 - : GROUP 1 이 ONLINE LOG 라고 가정

```
CREATE CONTROLFILE DATABASE "RC722" RESETLOGS NOARCHIVELOG
   MAXLOGFILES 32                    ********
   MAXLOGMEMBERS 2
   MAXDATAFILES 30
   MAXINSTANCES 8
   MAXLOGHISTORY 800
LOGFILE
 GROUP 2 '/oracle/oracle/dbs/log2RC722.dbf'  SIZE 5M,
 GROUP 3 '/oracle/oracle/dbs/log3RC722.dbf'  SIZE 5M
DATAFILE
 '/oracle/oracle/dbs/systRC722.dbf',
 '/oracle/oracle/dbs/rbsRC722.dbf',
 '/oracle/oracle/dbs/toolRC722.dbf',
 '/oracle/oracle/dbs/usrRC722.dbf',
 '/oracle/oracle/dbs/tempRC722.dbf',
 '/oracle/oracle/rcdata.dbf'
;
```

2.절차

```
$ sqldba lmode=y
  SQLDBA> connect internal
  SQLDBA> shutdown abort
  SQLDBA> startup nomount
     statement processed
  SQLDBA> @cnt
  SQLDBA> recover database using backup controlfile until cancel;
      ....
      ...
    CANCEL (Return)
    Recovery canceled
  SQLDBA> alter database open resetlogs;

: 만일 정상적으로 open 되면 log file 추가
  SQLDBA> alter database add logfile '?/dbs/log1ORA722.dbf' size 1M;
: 정상적으로 open 안되면 RC에 다시 연락
```

ORA-600[3339]
DATA BLOCK CORRUPTION DETECTION

Bulletin no : 10061

개 요 : ORA-00600 [3339] [arg1] [arg2] [] [] [] []
ORA-1578 : Data block corrupted in file # block #

ORA-600 에러는 ORACLE이 직접 버퍼로 데이타를 읽어들일 때 읽은 블럭의 DBA(Data Block Address)가 잘못되었음(INVALID)을 의미한다. arg1은 실제 읽어들인 블럭의 DBA를 나타내며 arg2는 ORACLE이 읽고자 하였던 블럭의 DBA를 나타낸다. arg1과 arg2이 다르면 위에서와 같은 에러가 발생하며 주로 OS나 HW의 문제가 그 원인이 되는 경우가 많다.데이타의 손상 원인은 여러가지가 있으며 대개 데이타가 손상될 당 에는 그 사실이 드러나지 않다가 손상된 부분의 데이타가 사용될 때 비로소 손상된 사실이 알려지게 된다.

ORACLE은 모든 정보를 블럭 형태로 관리한다. ORACLE 데이타 블럭은 하나 이상의 OS 블럭으로 이루어져 있다. 그 크기는 init.ora 화일의 DB_BLOCK_SIZE에 나타나 있다.

ORACLE 데이타 블럭은 블럭의 정보를 담은 fixed header 를 갖고 있으며 이 정보를 이용하여 각각의 블럭과 데이타베이스 전체의 INTEGRITY를 유지한다. DBA는 fixed header의 한 부분으로 32bit 길이의 정수이며 데이타베이스의 화일 번호와 화일에서 블럭위치를 나타낸다.

DBA에 문제가 있으면 ORACLE은 OERI(3339)와 함께 위에서 말한 2개의 인수, 즉 arg1과 arg2를 표시하게 된다. 이 두 값이 다르면 OERI(3339)가 발생하게 되는 것이다.

ORACLE은 화일에 블럭을 읽고 쓸때 시스템 펑션 콜을 이용한다. 여기 는 lseek(), read(), readv(), write(), writev() 등이 포함된다. 블럭이 OS에 의해서 읽혀지면 SGA로 들어가며 곧이어서 DBA에 대한 검사가 이루어진다.

원인과 증상:

경우1)
ORA-00600 [3339] [0] [large number] [] [] [] []
ORA-1578: Data block corrupted file # block #

이 경우에는 arg1은 0이고 arg2는 ORACLE이 찾고 있는 DBA이다. 이 에러의 원인은 ORACLE 블럭의 일부가 모두 0이기 때문인 경우가 많다.
ORACLE 블럭내의 첫번째 OS 블럭은 디스크상의 문제로 해서 OS 또는 디스크 복구 프로그램이 해당 블럭을 복구하려는 과정에서 모두 0으로 되어버리곤 한다.
유닉스 플랫폼의 경우에는 여러개의 DBWR을 동시에 작동시키면 이와같은 문제가 발생하는 것으로 알려져 있으나 V6.0.33.2 이후 버전에서부터는 수정되었다.
ORA-1578 메시지는 반드시 OERI(3339)를 유발시키지는 않는다.

원인)
arg1이 0이되는 또다른 원인은 ORACLE의 버그 때문인데 롤백세그먼트가 드롭되기 전 그 롤백 세그먼트를 포함한 테이블스페이스가 먼저 드롭되면 발생한다. 이 문제는 V6.0.32 이후의 버전에서 수정되었다.

경우2)
ORA-00600 [3339] [large number] [large number] [] [] [] []
ORA-1578: Data block corrupted file # block #

원인1)
이번에는 디스크상의 물리적 블럭의 DBA가 틀린 경우이다. 이것은 블럭이 메모리 상에서 손상되었으나 그대로 디스크에 기록된 경우에 발생한다. 이 문제는 매우 드물게 나타나며 이 때의 DBA는 통 아무의미없는 값이다.

만약 메모리에 문제가 있다고 생각된다면 다음과 같은 event parameter를 init.ora에 추가함으로써 블럭 검사를 할 수 있다.

event = "10210 trace name context forever, level 10"
event = "10211 trace name context forever, level 10"

DBWR가 디스크에 데이타를 쓰기 전에 캐쉬에서 손상된 블럭을 발견하면 OERI(3398) 메시지를 출력하고 인스턴스를 정지시킬 것이다. 따라서 문제의 블럭은 디스크에 저장되지 않는다. 이 때 DBA를 포함한 많은 인수들이 OERI(3398) 내부 에러 처리기에 전달된다. 이와 은 경우에는 인스턴스를 다시 기동시키고 리턴된 인수와 tarce 화일을 가지고 Oracle World Wide Customer Support에 연락하여야 한다.

원인2)
블럭이 데이타화일상에 기록될 위치를 잘못 찾는 경우이다. 이것은 "write blocks out of sequence" 라고 불리운다. 때 OERI(3339)에서 표시되는 DBA는 모두 유효한 값이다. 이것은 ORACLE이 lseek()을 호출했을 때 OS가 블럭을 잘못된 곳에 기록해서 생기는 경우가 많다.

어떤 HW/OS 공급업체들은 4.2G 를 넘는 큰 화일을 다룰 수 있는 기능을 제공한다. 이것은 32bit unsigned number로 다룰 수 있는 범위를 벗어나기 때문에 OS는 offset 값을 ORACLE이 사용할 수 있도록 적절히 변환시켜야 한다. ORACLE은 OS의 지원 여부와 관계없이 2G 이상의 화일을 지원하지 않으며 위와같은 큰 화일을 다룰 수 있는 환경에서는 lseek() 시스템 콜이 정확한 위 로의 변환을 시켜주지 못함에따라 보다 작은 크기의 화일에서도 문제가 발생하는 경우도 있다.

원인3)
세번째 원인은 I/O기능이 전혀 작동하지 않는 경우이다. 이 경우에 표시되는 DBA 값은 모두 유효한 값들이지만 arg1은 이전에 SGA로 읽혀진 블럭의 DBA 값이 된다. ORACLE은 lseek(), read() 시스템 콜이 리턴하는 에러 코드를 검사하며 read()가 읽어들인 바이트 수가 BLOCK SIZE의 정수배인지를 검사한다. 이 검사를 통과하면 ORACLE은 성공적으로 READ가 수행되었다고 가정한다. 만약 DBA가 정확하지 않다고 체크되면 DATABASE 대한 읽기 요구는 실패하기 때문에 실제의 블럭 읽기는 일어나지 않는다. 이 경우에 리턴된 DBA는 실제로는 다른 화일의 다른 블럭을 지시하게 될 수도 있다.

원인4)
또 다른 원인은 동일한 디바이스에서 다른 블럭을 읽어온 경우이다. 이것은 작업이 메우 바쁜 디스크에서 발생하곤 한다. 어떤 경우에는 수백개 이상 떨어진 곳의 블럭을 읽어오는 경우도 있다. 이때에도 DBA 값은 모두 유효한 값이다.

세번째와 네번째 경우에는 다시 한번 동작을 반복함으로써 문제는 해결될 수 있으며 이것은 일반적으로 ORACLE의 문제가 아니라 OS나 HW의 문제인 경우가 많다.
OERI(3339)에서 표시되는 인수들의 값은 플랫폼에 따라 다르다.

ORA-604 조치 방법

Bulletin no : 10082

이 에러는 내부적으로 SQL명령이 실행될 때 발생한다. 예를 들어 현재 할당된 익스텐트가 가득 차서 다음 익스텐트를 할당 받으려고 할 때 오라클이 다음 익스텐트의 크기와 위치를 결정하기 위하여 SELECT명령을 내리게 되는 것과 같은 경우이다.

이 문제가 발생하 우선 alert.log 화일을 검사하여 ORA-600 과 같은 에러가 발생했는가를 확인한다. ORA-600 에러가 발생했다면 오라클측에 지원을 요청하도록 하고 그렇지 않다면 다른 원인을 검사해 봐야 한다.

가장 먼저 고려할 사항은 ?/dbs/init.ora 화일에 지정된 open_cursors 의 크기를 알아보는 것이다. 이 값이 설정이 안되어 있으면 Default가 50이므로

open_cursors=255

와 같이 설정하도록 한다. 이 값은 단지 커서의 최대 값을 지정하는 것이므로 커서를 적게 쓰는 프로그램에 아무런 영향을 끼치지 않는다. open_cursors를 변경하고 DB를 Shutdown 하고 Startup 시키면 된다.

만약 이 방법으로 해결이 안되면 다음의 방법을 따른다.
정확한 에러의 원인을 찾기 위해서 init.ora 화일에 다음과 같은 라인을 추가한다.

events = "604 trace name errorstack"

이렇게 init.ora를 변경하고 DB를 Shutdown 하고 Startup 하면 ORA-604 에러가 발생하는 경우에 자세한 정보를 Trace 화일에 기록해 주므로 이 화일을 검사하여 에러의 원인을 찾을 수 있다.

에러의 다른 원인으로는 init.ora 화일의 파라미터 가운데 DC_FREE_EXTENTS 나 ROW_CACHE_ENQUEUES 의 값이 너무 작게 설정된 경우를 생각해 볼 수 있다.
이와같은 경우는 이들 값을 크게 설정해 주도록 한다.

테이블 스페이스가 가득 차거나 Extent 갯수의 최대 허용값을 초과해서 에러가 발생하는 경우 ORA-604 에러가 함께 발생할 수가 있는데 이와같은 경우에는 이들 문제를 먼저 해결하면 ORA-604 에러는 함께 해결 된다.

ORA-1118
조치방법(MAXDATAFILES 와 DB__FILES)

Bulletin no : 10096

테이블 스페이스를 만들거나 데이타 화일을 추가하다 보면

ORA-01118: cannot add any more database files: limit of XXX exceeded

와 같은 에러가 발생하는 경우가 있다. 오라클의 데이타 화일의 최대 갯수는 MAXDATAFILES와 DB_FILES 에 의해서 제한을 받는데 여기서는 이 에러를 해결하는 방법을 알아보기로 한다.

데이타베이스를 처음 만들 때 사용되는 CREATE DATABASE 명령에서 MAXDATAFILES 라는 파라미터를 찾아볼 수 있다. 여기서 지정된 값(명시를 안하면 디폴트로 설정)은 콘트롤 화일에 기록된다. 이 값은 데이타베이스에 대해서 설정된 최대 데이타화일 갯수이다. 이 값을 변경하려면 콘트롤 화일을 다시 만들어야 한다.

한편, init.ora(UNIX 에서는 init.ora, VMS 에서는 __init.ora)에는 DB_FILES 라는 파라미터가 있는데 이 값은 해당 인스턴스에 대해서 지정된 최대 데이타화일 갯수이다. DB_FILES는 단순히 에디터상에서 init.ora 화일을 수정한 다음에 DB를 Restartup 하면 새로운 값이 적용된다.

1. 왜 MAXDATAFILES 와 같은 한계값을 설정하는가?
   -> O/S는 오라클 화일의 갯수를 지정하기 위하여 특정한 갯수 bit를 사용하며 이 값이 Platform 에 따라 다르다. MAXDATAFILES는 이 값의 영향 하에 있게 된다.
   일반적인 최대 값은 다음과 같다.

	V6	V7
UNIX	62	1022
VMS	254	1022
DOS	254	NA

[참조]일부 유닉스 Platform상의 V7.0.16 이전 버전의 경우 1022보다 작은 경우도 있다.

2. 왜 MAXDATAFILES 값을 가능한 최대로 설정해 두지 않는가?
   -> MAXDATAFILES 를 크게 지정하면 그만큼 콘트롤 화일의 크기도 늘어나기 때문이다.

3. 왜 DB_FILES를 MAXDATAFILES 의 크기만큼 크게 설정해 두지 않는가?
   -> DB_FILES를 늘리면 각 User Process 에 할당되는 PGA(Program Global Area) 의 크기가 커지기 때문이다.

4. 사용하는 시스템 MAXDATAFILES를 알 수 있는 방법은?
   -> 시스템에 따라서 다르므로 Installation Guide 를 참조해야만 한다.

5. 콘트롤 화일의 위치를 아는 방법은?
  -> SQLDBA>show parameter control_files;
      또는
    SQLDBA>select * from v$controlfile; (7.0.16 이상)
  위의 Query 를 이용하거나 $ORACLE_HOME/dbs/config.ora 화일을 보면 알 수 있다.

[ORA-1118 해결방법]
 -> ORA-1118 에러는 데이타 화일의 갯수가 MAXDATAFILES 값에 도달한 경우 발생한다. DB_FILES 값에 도달한 경우라면 ORA-59 에러가 발생한다. ORA-59 에러는 init.ora 의 DB_FILES 를 늘려주고 DB를 Restartup 하면 해결 되지만 ORA-1118 에러는 이와같이 간단하게 해결되지는 않는다. 다음과 같은 방법이 있다.

1. 여러개의 데이타화일로 구성된 테이블 스페이스가 있으면 이를 Export 받고 테이블스페이스를 Drop한 다음 하 의 큰 데이타화일을 갖도록 테이블 스페이스를 만들고 Import를 한다.

2. V6.0.33 이전 버젼을 사용중이라면 MAXDATAFILES를 늘리기 위해서는 DB를 새로 만들어야 하며 그 이후 버젼을 사용중이라면 콘트롤 화일을 새로 만들어서 MAXDATAFILES를 늘릴 수 있다.

1. DB가 mount 또는 open 된 상태에서
      SQLDBA>alter database backup controlfile to trace;

와 같은 명령을 내리면 에 지정된 디렉토리 -
ORACLE_HOME/rdbms/log - 에 트레이스 화일이 하나 생긴다. 이 화일 찾으려면 해당 디렉토리에서 가장 최근에 생긴 트레이스 화일을 찾으면 된다.

 이 화일을 다른 이름으로 복사한 다음 에디터로 열어서 CREATE CONTROLFILE 명령 부분 외의 불필요한 부분과 'REUSE' OPTION 을 지우고 MAXDATAFILES를 늘려 준다. 이 화일의 이름을 newctl.sql 로 하기로 한다.

2. ORCACLE_HOME/dbs 에 있는 Control file 을 다른 이름으로 MOVE 한다.

3. DB를 NORMAL 또는 IMMEDIATE로 Shutdown 하고, 콘트롤 화일 생성시 DB
영향을 줄 수 있기 때문에 만약을 위해서 DB 전체를 백업 받도록 한다.

4. 현재 사용중인 콘 롤 화일을 다른 이름으로 옮기고 다음을 실행한다.

```
SQLDBA>startup nomount
SQLDBA>@newctl
SQLDBA>alter database open;
```

5. 이제 필요한 모든 작업은 끝났지만 여기서 다시 한번 Full Bakcup 을 받는 것이 좋다.

[참고] 다음은 콘트롤 화일을 생성하는 명령의 예이다.

```
SQLDBA> STARTUP NOMOUNT
SQLDBA> CREATE CONTROLFILE REUSE
      DATABASE ORACLE
```

```
        LOGFILE '/users/oracle/dbs/log1ORACLE.dbf',
                '/users/oracle/dbs/log2ORACLE.dbf',
                '/users/oracle/dbs/log3ORACLE.dbf'
        NORESETLOGS
        DATAFILE '/users/oracle/dbs/systORACLE.dbf',
                 '/users/oracle/dbs/rbsORACLE.dbf',
                 '/users/oracle/dbs/tempORACLE.dbf',
                 '/users/oracle/dbs/tool1ORACLE.dbf',
                 '/users/oracle/dbs/usrORACLE.dbf'
        MAXDATAFILES 121;
SQLDBA> ALTER DATABASE OPEN NORESETLOGS;
```

ORA-1547 조치 방법

Bulletin no : 10083

ORA-1547 에러 발생의 원인으로는 TABLESPACE가 에러에 명시된 ORACLE block 수 만큼의 요청된 EXTENT를 할당할 충분한 FREE SPACE를 갖고있지 못할 경우에 발생된다. 현재의 EXTENT가 실제 데이타 화일에 존재하는 FREE SPACE 중에서 할당 가능한 연속적인 ORACLE block을 요구하게 된다.

< ORA-1547 에러가 발생하는 경우 >

1. 데이타 INSERT나 UPDATE시 DATA SEGMENT가 차지하게될 연속적인 ORACLE 블럭을 할당받지 못할 경우에 발생한다.

2. 인덱스를 생성할 경우에 발생한다.

- ROLLBACK SEGMENT가 사용할 RBS 또는 USER TABLESPACE의 영역이 부족하여 발생할 수 있다.
- 인덱스 생성시 SORT 영역으로 사용되는 TEMPORARY TABLESPACE내의 SPACE의 부족으로 발생할 수 있다.

3. SQL*FORMS30, SQL*REPORTWRITER등의 프로그램을 데이타베이스에 [SAVE]시 관련 테이블들을 포함고 있는 SYSTEM 또는 TOOLS TABLESPACE등의 영역이 부족한 경우에 발생된다.

< ORA-1547 에러 확인 >

이러한 경우 EXTENT에 관련된 DATA DICTIONARY VIEW인 USER_TABLES, USER_EXTENTS, USER_SEGMENTS와 DBA_FREE_SPACE등을 조회해서 관련 내용을 확인한다.
예를 어, 데이타 INSERT시 "ORA-1547 : Failed to allocate extent of size 'num' in tablespace 'TOOLS'" 에러가 발생될 경우를 고려해 보자.

1. [USER_TABLES]에서 INSERT에 관련된 테이블의 NEXT_EXTENT 크기를 확인 한다.

```
SQL> SELECT * FROM USER_TABLES
       WHERE TABLE_NAME = 'EMP';

  INITIAL_EXTENT  NEXT_EXTENT  PCT_INCREASE  MIN_EXTENTS  MAX_EXTENTS
  --------------  -----------  ------------  -----------  -----------
           10240       190464            50            1          121
                         (A)
```

(A) : 다음에 할당되는 EXTENT의 크기를 나타내며 BYTES 단위이다.

2. [DBA_FREE_SPACE]에서 현재 TABLESPACE에 존재하는 FREE SPACE 중 가장 큰 연속된 영역을 확인한다. [DBA_FREE_SPACE]는 SQLDBA에서 확인한다.

```
SQLDBA> SELECT MAX(BYTES) MAX_CONTIGUOUS_SPACE
          FROM DBA_FREE_SPACE
          WHERE TABLESPACE_NAME = 'TOOLS';
```

```
MAX_CONTIGUOUS_BYTES
--------------------
          19730432
             (B)
```

(B) : 현재 TABLESPACE에 남아있는 FREE SPACE 중 가장 큰 연속된 영역으로 BYTES 단위로 나타난다.

3. 위에서 살펴본바와 같이 2)-(B)의 MAX(BYTES) 크기가 1)-(A)의 NEXT_EXTENT 크기보다 커야하므로 최소 1)-(A)의 NEXT_EXTENT 크기 이상의 데이타 화일을 "TOOLS" TABLESPACE에 "ALTER TABLESPACE tablespace_name ADD DATAFILE.... " 명령을 이용하여 추가한다.

< ORA-1547의 처리 방법 >

1. 해당 TABLESPACE내에서 연속된 영역의 ORACLE block 할당할 수 있도록 데이타 화일을 추가한다. 3) 에서 "TOOLS" TABLESPACE 에 영역을 추가하는 방법 다음과 같다.

```
$ sqlplus system/manager
  SQL> ALTER TABLESPACE tools ADD DATAFILE
       '/usr/../tools10RA7.dbf' SIZE 50M;
```

<참고>DATAFILE 의 이름 즉 '/usr/../tools10RA7.dbf' 는 이미 존재하는 DATAFILE 이름과 다르게 함.

2. TABLE의 STORAGE PARAMETER에서 INITIAL EXTENT, NEXT EXTENT의 크기를 조정하여 TABLE을 재구축 할 수 있다. 즉, TABLE의 STORAGE PARAMETER 중에서 NEXT를 현재 TABLESPACE에 남아 있는 FREE SPACE 중 가장 큰 연속된 영역(DBA_FREE_SPACE의 MAX(BYTES)) 다 작게 변경할 수 있다.
예를 들면, "ALTER TABLE..." 명령을 이용하여 다음과 같이 재 조정한다.

```
SQL> ALTER TABLE EMP STORAGE ( NEXT 100K );
```

3. 다음의 방법으로는 관련 TABLESPACE를 재구성하는 것이다. 여기서는 TABLESPACE를 재 구축한 후 TABLE을 시 생성함으로써 DISK FRAGMENTATION을 없앨 수 있다.

ORA-1552 조치 방법

Bulletin no : 10098

ORA-1552 (CANNOT USE SYSTEM ROLLBACK SEGMENT FOR NON-SYSTEM TABLESPACE '%S') 에러는 SYSTEM TABLESPACE 이외의 TABLESPACE를 포함한 OPERATION을 위하여 SYSTEM TABLESPACE의 ROLLBACK SEGMENT를 사용할 경우에 발생되는 현상이다. SYSTEM TABLESPACE가 추가적 로 하나의 ROLLBACK SEGMENT를 포함하기 전에는 새로운 TABLESPACE에 임의의 데이타베이스 오브젝트를 생성할 수 없다.

ORA-1552 발생 원인을 해결 하기위해 SYSTEM TABLESPACE에 하나 이상의 ROLLBACK SEGMENT를 추가한 다음, 데이타베이스 오브젝트를 생성한다. CREATE ROLLBACK SEGMENT 문에서 PRIVATE/PUBLIC으로 ROLLBACK SEGMENT를 생성한 후, ORACLE 데이타베이스를 SHUTDOWN, STARTUP 한다. PRIVATE으로 생성된 ROLLBACK SEGMENT는 INITSID.ORA 화일의 "ROLLBACK_SEGMENTS" PARAMETER에 등록한다.

< ORACLE V6>

```
$ sqldba
SQLDBA> CONNECT INTERNAL;
SQLDBA> CREATE ROLLBACK SEGMENT R0;
  $ORACLE_HOME/dbs/initSID.ora 화일의 "ROLLBACK_SEGMENTS" PARAMETER에  R0를  등록한다.
SQLDBA> SHUTDOWN
SQLDBA> STARTUP
```

< ORACLE7>

```
$ sqldba lmode=y
SQLDBA> CONNECT INTERNAL
SQLDBA> CREATE ROLLBACK SEGMENT R0;
SQLDBA> ALTER ROLLBACK SEGMENT R0 ONLINE;
```

ORA-1555 조치방법

Bulletin no : 10105

ORA-1555 Snapshot Too Old 에러가 발생하는 원인은 여러가지가 있을 수 있다.
이를 이해하기 위해서 오라클의 내부 메카니즘에 대해서 알아볼 필요가 있다. 오라클은 항상 Statement-Level Read Consistency를 유지한다. 즉, 하나의 Query 에서 읽어들이는 값은 그 Query 가 발생한 시점의 값, 즉 Snapshot 을 반영한다. 따라서 Query 가 진행되는 동안 데이타가 변하더라도 Query의 결과값에는 아무런 영향을 끼치지 않는다.

오라클은 특정한 시점을 SCN(System Change Number)로 관리한다. 달리 말하면 SCN은 특정한 시점에서 데이타베이스의 상태라고도 할 수 있다. Read Consistency를 유지하기 위하여 Query가 실행되는 순간 해당 시점의 SCN이 세팅된다. 어떠한 Query든 조회 시점 이후의 SCN 값을 갖는 데이타를 읽어들여 서는 안된다.

Read Consistent Snapshot을 구현하기 위한 방법으로 롤백 세그먼트가 사용된다. 데이타가 변경되면 변경되기 이전의 값은 롤백 세그먼트에 기록되고 데이타 블럭 헤더에는 변경되기 이전 값이 기록된 롤백 세그먼트 블럭의 위치가 기록된다. 모든 데이타 블럭에는 가장 최근의 커밋 시점의 SCN이 기록되어 있다. Query 가 진행되면 데이타 블럭에서는 Query의 SCN 보다 이전의 SCN값을 갖는 블럭만이 읽혀지고 그 이후의 SCN 값을 갖는 블럭과 커밋되지 않은 값을 갖는 블럭에 대한 Query는 롤백 세그먼트에서 읽혀지게 된다. 이 때 만약 롤백 세그먼트에 원하는 블럭이 없어서 읽어들이 못하게 되면 ORA-1555 에러가 발생하게 된다.

롤백 세그먼트는 트랜잭션이 끝나기 전까지 변경된 데이타의 원래 상태를 기록하고 있다. 그런데 트랜잭션이 종료되면 롤백 세그먼트에 의해 점유된 영역은 Free 상태로 되면서 다른 트랜잭션에 의해서 덮어씌워질 도 있게 된다. 따라서 Query가 필요로하는 값을 가진 롤백 세그먼트 블럭이 더 이상 존재하지 않는 상황에서 그 블럭을 요구하게 되면 ORA-1555 에러가 발생하게 되는 것이다.

ORA-1555 에러가 발생하는 경우

1. 데이타의 변경이 심한 데이타베이스에서 롤백 세그먼트의 갯수와 크기가 작을 경우에 발생한다. 많은 트랜잭션이 데이타를 자주 변경하고 커밋하게 되면 커밋된 트랜잭션이 이용하던 롤백 세그먼트 공간을 다른 트랜잭션이 이용하게 될 가능성이 많아진다.
   따라서 긴 Query의 경우 원하는 값을 롤백 세그먼트에서 얻고자 할 때 이미 다른 트랜잭션이 그 값이 저장된 공간을 이용해 버리는 결과가 발생할 수 있다. 이와같은 경우에는 크기가 큰 롤백 세그먼트를 이용하면 어느정도 예방이 가능하다.

2. 롤백 세그먼트가 손상되어 읽을 수 없게 된 경우

3. Fetch Across Commit

한 테이블에 대하여 Query가 커서를 열고 루프 내에서 데이타를 Fetch하고 변경하고 커밋하는 과정에서 발생한다. 이 경우에는 ORA-1555 에러가 자주 발생할 소지가 있는데 예를 들어 커서가 SCN=10 에 Open되었다고 하자. 따라서 이 커서에 관련된 Fetch는 SCN<=10 인 블럭만을 읽어들여야 한다. 이 프로그램은 데이타를 Fetch 한 후에 변경하고 다시 커밋하는 과정을 계속 반복하는데 SCN=20에서 커밋했다고 하자. 만약 이후의 Query가 이전의 커밋된 블럭의 데이타를 요구할 경우 그 값이 이미

변경되었으므로 롤백 세그먼트를 검색 하지만 많은 변경이 있어왔기 때문에 SCN=10인 블럭을 찾지 못하고 ORA-1555 에러를 유발할 수 있다. 이를 방지하기 위해서는 커서가 Open된 상태에서는 커밋을 자주하지 않고 롤백 세그먼트 크기를 키워 나가도록 한다.

4. Fetch Across Commit 와 Delayed Block Clean Out

Delayed Block Clean Out 도 이 에러를 유발할 수 있다. 데이타 블럭이 변경되고 커밋되면 오라클은 롤백 세그먼트 헤더에 그 트랜잭션이 커밋되었다고 기록하지만 데이타 블럭을 바로 변경하지는 않는다 (Fast Commit). 그리고 다음 트랜잭션이 변경된 블럭을 요구할 때야 비로소 변경 시키는데 이를 Delayed Block Clean Out 이라고 한다.

예를 들어 3 과 같은 경우에서 두개의 테이블에 대한 Query를 알아보자.

즉, 하나의 테이블로부터 데이타를 Fetch 하고 다른 테이블의 데이타를 변경한다고 하자. 데이타가 한쪽 테이블에서 커밋되고 있지만 데이타를 가져오는 테이블에 대해서는 Clean Out이 이루어지지 않았기 때문에 ORA-1555 에러가 발생할 수 있다. 이 문제는 커서를 사용하기 전에 Full Table Scan을 해주면 예방이 가능하다. 커서가 오픈된 상 에서의 커밋은 ANSI 표준에는 들어있지 않지만 오라클에서는 지원이 된다. 하지만 ORA-1555 에러를 일으킬 수도 있다는 점에 유의하여야 한다.

ORA-1562 처리 방법

Bulletin no : 10086

IMPORT등의 대량 데이타를 일괄적으로 처리시 발생되는 ORA-1562(Failed to extend rollback segment(id = %s)) 에러에 대하여 살펴보자

ORA-1562 에러 발생의 원인으로는 현재 사용중인 ACTIVE 상태의 ROLLBACK SEGMENT가 다음 EXTENT를 할당하고자 할 경우, 해당 ROLLBACK SEGMENT에 대하여 발생 가능한 최대 EXTENT 수를 초과할때 발생된다. DATA DICTIONARY VIEW중 DBA_ROLLBACK_SEGS를 살펴보면 MAX_EXTENT에 지정된 수 만큼의 EXTENT가 발생될 수 있음을 알수 있다.

```
SQLDBA> SELECT * FROM DBA_ROLLBACK_SEGS;

SEGMENT_NAME  SEGMENT_ID  INITIAL_EX  NEXT_EX  MIN_EXTENT  MAX_EXTENT
------------  ----------  ----------  -------  ----------  ----------
SYSTEM                 0       51200    51200           2         121
R01                    1      131072   131072           2         121
R02                    2      131072   131072           2         121
R03                    3      131072   131072           2         121
R04                    4      131072   131072           2         121
```

예를 들어, 현재 사용중인 ROLLBACK SEGMENT인 R01(SEGMENT_ID=1)이 NEXT_EXTENT(131 KB) 크기로 EXTENT를 MAX_EXTENT인 121개까지 확장시킨 다음, 122번째를 발생시킬 경우에 ORA-1562 에러가 발생된다. 이것은 상대적으로 적은 NEXT EXTENT의 크기(131 KB)로 인하여 EXTENT가 빈번하게 발생되어 MAX EXTENT 수에 도달하는 경우이다.

대부분의 시스템에서는 한 ROLLBACK SEGMENT에서 확장 가능한 최대 EXTENT 수를 121개로 제한한다. 이러한 경우 ALTER ROLLBACK SEGMENT 문장에 의해서 EXTENT의 최대 수를 증가 시키더라도 시스템 제한으 인해 아무런 영향을 미치지 못하므로 ROLLBACK SEGMENT의 재생성이 요구된다.

해당 ROLLBACK SEGMENT를 재생성하기 위해서는,
첫번째 문제의 ROLLBACK SEGMENT를 OFFLINE 시킨후 ROLLBACK SEGMENT를 DROP 한다.

```
SQLDBA > ALTER ROLLBACK SEGMENT R01 OFFLINE;
SQLDBA > DROP ROLLBACK SEGMENT R01;
```

두번째 ROLLBACK SEGMENT 생성시 STORAGE 절을 이용하여 재생성한다.
ROLLBACK SEGMENT를 재생성시 INITIAL EXTENT의 크기와 NEXT EXTENT의 크기를 증가시킴으로서 EXTENT 발생 횟수를 줄일 수 있다.

```
SQLDBA> CREATE ROLLBACK SEGMENT R01 STORAGE( INITIAL 1M  NEXT 1M )
        TABLESPACE RBS;
SQLDBA> ALTER ROLLBACK SEGMENT R01 ONLINE;
```

만약, ORA-1562에서 명시된 ROLLBACK SEGMENT의 SEGMENT ID가 "SYSTEM" TABLESPACE에 포함된 "SYSTEM" ROLLBACK SEGMENT인 경우에는 DROP ROLLBACK SEGMENT문을 수행한 후, 재성성할 수 없음을 명심하자. 이러한 경우에는 현재의 트랜잭션이 "SYSTEM" ROLLBACK SEGMENT가 아닌 다른 TABLESPACE에 포함된 ROLLBACK SEGMENT를 이용할 수 있도록 지정한후, 트랜잭션을 수행할 수 있다.

ORA-1628,1630,1631,1630 조치 방법

Bulletin no : 10094

다음 ORA 에러들은 오라클의 오브젝트들이 MAXEXTENTS에 도달 했을때 발생하는 것들이다.

```
01628, 00000, "max # extents (%s) reached for rollback segment %s"
01630, 00000, "max # extents (%s) reached in temp segment  in tablespace
               %s"
01631, 00000, "max # extents (%s) reached in table %s.%s"
01632, 00000, "max # extents (%s) reached in index %s.%s"
```

또한 ORA-1628 다음에는 ORA-01562 도 함께 발생한다.
이 에러들은 다음 모든 LEVEL 에서 발생 될수 있다.

- 에플리케이 LEVEL (GL, AOL, Financials, Etc)
- TOOLS LEVEL (Reports, Forms, Etc)
- 커널 LEVEL (Insert, Update, Delete)

이 에러의 이유는 오브젝트의 익스텐트가 MAX # 에 도달 했기 때문에 발생되며 오브젝트의 MAXEXTENTS는 STORAGE 의 MAXEXTENTS 파라미터에 의해 결정된다.

다음 예를 보기로 하자.

```
SQL> INSERT INTO TAB1 SELECT * FROM TAB1;
        ORA-01631: max # extents (2) reached in table JANE.TAB1

SQL> SELECT INITIAL_EXTENT, NEXT_EXTENT, MAX_EXTENTS
           FROM USER_TABLES
           WHERE  TABLE_NAME = 'TAB1';

    INITIAL_EX TENT      NEXT_EXTENT        MAX_EXTENT
    ---------------      -----------        ----------
     6144                10240              2
```

위 예에서 오브젝트의 MAXEXTENTS 는 2 인데 이 값은 HARDCORD된 MAX 값이 아니다.HARDCORD 된 MAXEXTENTS의 최대 값은 데이타베이스가 생성될 당시 지정된 DB_BLOCK_SIZE 의 값에 따라 다르다.

DB_BLOCK_SIZE	최대 MAXEXTENTS 값
512	25
1K	57
2K	121
4K	249
8K	505

만일 최대 MAXEXTENTS 값 보다 더 큰 MAXEXTENTS를 Storage 에서 지정하면 다음 에러가 발생한다.

```
ORA-02226, 00000, "invalid MAXEXTENTS value (max allowed: %s)"
```

다음은 ORA-0163x 에대한 해결 방법이다. 만일 플레폼의 최대 MAXEXTENTS 에 도달이 안되었으면

ALTER TABLE .. STORAGE (MAXEXTENTS n)

를 사용하여 최대 MAXEXTENTS 값보다 작은 수로 MAXEXTENTS를 늘려준다.

만일 최대 MAXEXTENTS 값에 도달 했으면 해결 할수있는 방법은 MAX 제한에 도달되지 않도록 EXTENT SIZE 를 더 크게하여 오브젝트를 다시 생성하는 것이다.

만일 이 에러가 ROLLBACK SEGMENT 에서 발생되면 DROP하고 다시 생성 한다.

만일 에러가 TEMPORARY TABLESPACE에서 발생되면 TEMPORARY TABLESPACE의 STROAGE를 변경한다. TEMP SEGMENT는 그것이 생성된 TABLESPACE의 Default Storage Parameter를 사용하기 때문에 다음과 같은 방법으로 해결 할수 있다.

ALTER TABLESPACE 'tempname' DEFAULT STORAGE (INITIAL n NEXT n);
. n 은 기존 지정된 값보다 큰값을 지정 한다.
ALTER TABLESPACE 'tempname' DEFAULT STORAGE (PCTINCREASE m);
. m 은 기존 지정된 값보다 큰값을 지정한다.

만일 에러가 TABLE에서 발생되면 import/export utility를 사용하여 그 TABLE을 다시 생성한다.

[단계]

1. COMPRESS=Y OPTION을 사용하여 엑스포트를 실행함.
   예) $ exp scott/tiger file=emp.dmp tables=emp compress=y
2. 기존 TABLE을 RENAME 함.
   예) SQL> RENAME EMP TO EMP_OLD;
3. 사용자의 SCHEMA 에 임포트를 실행하여 TABLE을 생성함.
   예) $ imp scott/tiger file=emp.dmp commit=y
4. ALTER TABLE 명령을 사용하여 임포트된 TABLE의 STORAGE를 변경함.
   예) SQL> ALTER TABLE emp STORAGE(NEXT 1M)
5. 기존 테이블을 DROP 함.
   예) SQL> DROP TABLE EMP_OLD;

위 작업의 결과로 한개의 큰 EXTENT를 가진 TABLE이 생성 되고 모든 GRANT,INDEX,CONSTRAINT 들이 그대로 보존 된다.

ORA-1652 조치 방법

Bulletin no : 10100

테이블이나 인덱스 등을 만들 때 자신의 TEMP TABLESPACE 가 아닌 곳에서 ORA-1652(temp tablespace가 부족함) 에러가 발생하는 경우가 있다. 이와같은 문제는 V7.1 에서만 발생하는 데 이것은 사용자를 상당히 혼란스럽게 만든다.

V7.1 에서는 테이블, 인덱스 등을 병렬로 생성할 수 있다. 이를 위하여 실제로 테이블 등이 생성될 공간에 Temporary Segment 를 만들게 되는데 이 과정에서 Temporary Segment 를 만들 공간이 부족하게 되면 실제의 테이블이 생성되는 테이블 스페이스에 대하여 ORA-1652 에러가 발생하게 되는 것이다.

ORA-1652 에러를 해결하는 방법은 에러메시지에서 보여주는 대로 해당 테이블스페이스에 Temporary Segment 가 생성될 만한 연속된 공간을 마련하여 주는 것이다. 즉,

1. 데이타 화일을 추가하여 테이블스페이스의 크기를 확장한다.

```
SQLDBA > ALTER TABLESPACE data ADD DATAFILE
         '/usr/../oracle/data2.dbf' SIZE 100M;
```

2. 테이블의 storage parameter 를 조정하여 현재 남아있는 영역에 들어갈 수 있도록 한다.

```
SQLDBA > ALTER TABLE emp STORAGE(NEXT 1M);
```

3. 테이블스페이스가 fragmentration 이 심한 상태이면 exp/imp 를 이용하여 테이블스페이스를 재구성 한다.

[예제] 다음의 테이블 생성 문장을 보자

```
SQL>  CREATE TABLE FEATURE
      (feature_code varchar2(4)  primary key,
       feature_desc varchar2(3)  );
ORA-01652, 00000, "unable to extend temp segment by 6144 in tablespace
                  "VESSEL"
```

테이블 스페이스 VESSEL 에 남아있는 가장 큰 연속된 공간을 확인해 보면

```
SQL>SELECT MAX(blocks), MAX(bytes)
    FROM DBA_FREE_SPACE
    WHERE TABLESPACE_NAME = 'VESSEL';
     blocks      bytes
     ------      ---------
      6143       12,580,864
```

위의 결과를 보면 현재 VESSEL 에 남아있는 가장 큰 연속된 공간은 6143 블럭인데 오라클은 6144 블럭이 사용하려다 이를 할당받지 못하여 에러가 발생하게 된 것이다.

ORA-4031 조치 방법

Bulletin no : 10095

우리는 다음과 같은 작업 수행시 Oracle 이 Shared pool 에서 연속적인 메모리 부분을 찾지못해 ORA-4031 Error 를 발생시키는 것을 볼수 있다.

. PL/SQL Routine . Procedure 수행시 . Compile 시

. Form Generate 또는 Running 시 . Object 생성하기 위해 Installer 사용시

ORA-1547과 유사한 Error들은 서로 다른 Application, 서로 다른 환경에서 발생하지만 이 문제들은 여전히 같이 취급되고 같은 방법으로 해결 되고 있다. (eg. ORA-4031)

[Problem 설명]

Error 발생의 주된 원인은 Shared Pool 사용가능한 Memory 가 시간이 흐름에 따라 작은 조각으로 분할 되어 진다는 것이다. 그래서 큰 부분의 Memory 를 할당하려 한다면 Shared Memory 가 부족하다는 ORA-4031 Error 가 발생한다. 즉 전체적으로는 많은양의 사용가능 한 Space 가 있다하더라도 충분한 양의 연속적인 Space 가 없으면 이 Error가 발생한다.

[Problem 해결 방안]

이 Error 해결방안을 살펴 보면 다음과 같다.

1. Object를 Shared Pool에 맞추어 Fragmentation을 줄인다
   (Dbms_Shared_Pool Procedure 이용)
2. Shared Pool 을 효율적으로 사용하도록 Application Program을 조절한다
3. Shared Pool 의 Size 를 적절히 조절한다. 이 Size는 Default 값이 3.5M이며 이 Size를 수정시는 DB를 Shutdown 후 다시 Start 시켜야 하므로 항상 가능한 해결 방법이 될수 없다.

```
< ORACLE_HOME/dbs/initSID.ora >

  shared_pool_size  = 9000000     <- 단위 : Byte
==============================
```

위와 같이 변경하고 DB 를 RESTARTUP 한다.

DBMS_SHARED_POOL STORED PROCEDURE

이 stored pakage는 dbmspool.sql을 포함하며 7.0.13 이상 version에서 사용 가능하다. 이는 다음과 같이 3가지 부분으로 나누어 진다.

Procedure sizes(minsize number);
-> Shared_Pool 안에서 정해진 Size 보다 큰 Object 를 보여준다

Procedure keep(name varchar2, flag char Default 'P')
-> Object (Only Package)를 Shared Pool 에 유지한다.또한 일단 Keep한 Object는 LRU Algorithm 에 영향을 받지 않으며 Alter System Flush Shared_Pool Command 에 의해 Package 의 Compiled Version 이 Shared Pool 에서 Clear 되지 않는다.

Procedure unkeep(name varchar2); -> keep() 의 반대기능이다

이 Procedure들과 사용법에 대해 보다 더 자세한 정보를 위해서는
$ORACLE_HOME/rdbms/admin/dbmspool.sql script 를 참조.

ORACLE DATABASE HOME DIRECTORY 변경

Bulletin no : 10090

ORACLE HOME DIRECTORY를 옮기는 과정은 다음의 세 단계로 나누어 진다.

1. 오라클 소프트웨어의 이동
2. 데이타 화일, 로그 화일, 콘트롤 화일 등의 데이타베이스 부분의 이동
3. 각종 환경 설정의 변경

만약 오라클을 다른 MACHINE으로 옮기는 경우라면 두 OS 가 Binary Compatible 해야 한다. 일반적으로 OS 가 같더라도 버전이 다르면 Binary Compatible 하지 않은 경우가 있으니 유의해야 한다.
모든 작업은 오라클을 셧다운 한 상태에서 진행한다.

1. 오라클 소프트웨어의 이동

OS 상의 명령어(tar, cpio, cp 등)을 사용하 오라클 소프트웨어를 새로운 디렉토리로 옮긴다. ORACLE HOME DIRECTORY 이하 부분을 모두 옮기면 된다. 작업이 끝난 후에 화일의 크기나 checksum(sum 명령)을 이용하여 제대로 옮겨졌는지를 확인하도록 한다.

2. 데이타베이스의 이동

데이타베이스를 구성하는 화일들을 다음 명령을 이용하여 확인한 다음 앞의1.과정에서 복사되지 않은 화일이 있으면 새로운 디렉토리로 복사한다.

```
SQLDBA>select * from dba_data_files;
SQLDBA>select * from v$logfile;
SQLDBA>show parameter control_files
```

그 다음에 다음과 같은 과정을 거쳐 데이타 화일과 로그 화일의 이동 상태를 콘트롤 화일에 반영한다. 이 명령은 위에서 출력된 데이타 화일과 로그 화일 모두에 대해서 수행되어야 하며 한번의 명령에 여러개의 화일을 지정할 수 있다. 여기서는 2개의 화일에 대한 예를 들었다.

```
SQLDBA>  startup mount
SQLDBA>  alter database rename file
                    '/file1', '/file2' to
                    '/file1', '/file2';
```

만약의 경우를 위하여 콘트롤 화일을 생성해 주는 스크립트를 다음 명령으로 만들 두도록 한다.

```
SQLDBA>alter database backup controlfile to trace
```

이 결과는 디렉토리에서 가장 나중에 생긴 트레이스 화일에 약간의 편집을 통하여 사용할 수 있는 형태로 저장된다.

3. 환경 설정의 변경

/etc/oratab 또는 /var/opt/oracle/oratab 화일을 새로운 값으로 수정한다.
그리고 $ORACLE_HOME 을 설정할 때에는 소프트 링크를 포함해서는 안된다.
init.ora의 내용과 여기에 ifile 로 기술된 config.ora 화일의
내용을 수정한다. 기본적인 것은 다음과 같은데 이외의 내용도 있을 수도 있다.

ifile
control_files
background_dump_dest
log_archive_dest
db_name(변경된 경우)

만약 다른 MACHINE으로 옮기는 경우라면 SHARED MEMORY 와 SEMAPHORE의
설정도 적절히 세팅되어야 하며 Swap Space 도 고려하여야 한다.

다음과 같은 환경 수를 새로운 값으로 변경한다.

$ORACLE_HOME
$ORACLE_SID(변경된 경우)
$FORMS30PATH
$ORAKITPATH
$TNS_ADMIN

그리고 $ORACLE_HOME/rdbms/lib/sysliblist 에 절대패스로 지정된 것이 있으면
이 값도 새로운 값으로 변경한다.
makefile 의 내용중 절대패스로 지정된 것이 있으면 이 값도 변경한다.

새로운 시스템으로 옮긴 경우에는 NO-SUID flag이 설정되어 있지 않아야
한다. 대부분의 시스템은 /etc/fstab 화일에 이 flag을 갖고 있다.

ORACLE INSTALLER에서 사용되는 User Default File인 usrdfl.log 화일은
더이상 의미하므로 삭제한다. 이 화일은 일반적으로 $ORACLE_HOME/orainst
디렉토리에 있다.

시스템을 변경하는 경우라면 /etc/services 화일에 SQL*Net listener 항목을
추가하고 ORACLE OWNER ID를 만들고 DBA 그룹에 등록한다.

기존에 만든 스크립트나 cron job 가운 디렉토리를 참조하는 부분의 변경이
필요하면 수정한다.

ORACLE DATABASE의 AUTOMATIC STARTUP/SHUTDOWN(한글판)

Bulletin no : 10168

시스템이 startup 하면서 오라클도 함께 startup 하거나 시스템을 shutdown 할 때 오라클도 같이 shutdown 하는 것이 가능하다. 그 방법은 OS마다 차이가 있는데 여기서는 UNIX 에 기반한 여러 플랫폼에 대해서 알아보기로 한다.

본 내용에서 가장 많이 사용되는 화일은 다음과 같다.

$ORACLE_HOME/bin/dbstart
$ORACLE_HOME/bin/dbshut
/etc/oratab 또는 /var/opt/oracle/oratab

dbstart 와 dbshut 은 오라클을 startup/shutdown 시키는 스크립트인데 이것은 automatic startup/shutdown 시에만 사용하도록 하고 평상시의 경우에는 사용하지 않도록 한다.
oratab 화일은 일반 텍스트 화일로서 시스템에 설치된 오라클 인스턴스에 대한 정보를 가지고 있는데 보통 3개의 필드로 이루어 있으며 첫번재 필드는 ORACLE_SID, 두번째 필드는 ORACLE_HOME, 세번째 필드는 Y 또는 N 으로 구성되어 있다. 해당 인스턴스를 autostartup 시키려면 꼭 Y로 세팅되어 있어야 한다.

SYSTEM V

System V 에 기반한 시스템은 /etc/rc.d 디렉토리에 초기화 스크립트를 가지고 있다. 여기서 은 해당 스크립트의 run-level 을 의미한다. 일반적으로 run-level 0 은 power shutdown mode 이고 run-level 2는 multi-user mode 이다. 각각의 디렉토리에 들어있는 초기화 스크립트는 다음과 같은 형태의 이름을 갖고 있다.

[K¦S]{two-digit number][descriptive filename]

예를 들면 S75cron, K30tcp 등이 있을 수 있다.

S로 시작되는 스크립트는 startup시에 실행되는 것이고 K 로 시작하는 것은 shutdown 시에 실행된다.

two-digit number가 의미하는 것은 스크립트가 실행되는 순서인데 숫자가 작은 것부터 먼저 실행된다. 따라서 오라클 스크립트는 보통 다음과 같은 이름을 갖게 된다

S99oracle, K01oracle

이와같이 설정하면 시스템이 startup 될 때 모든 초기화가 끝난 후 마지막으로 오라클이 startup 되며 시스템을 shutdown 할 때에는 오라클을 가장 먼저 shutdown 하게 된다.

BSD UNIX

BSD 에 기반한 시스템은 시스템이 startup 될 때 초기화 화일로 /etc/rc*(예를 들면, /etc/rc, /etc/rc.local) 를 사용한다.
시스템을 shutdown 할 때에는 보통은 /etc/shutdown 만 실행하게 되므로 특별히 실행되는 스크립트가 없지만 /etc/rc.shutdown 이 실행되는 플랫폼도 있다.

이제 각 플랫폼별로 자세히 알아보기로 한다.

주의 : 는 oracle owner id 로 대체 <$ORACLE_HOME> 은 oracle home directory 명으로 대체

(1)SunOS 4.1.3

SunOS 4.1.3 에서는 /etc/rc.local 을 startup 스크립트로 사용한다. 즉 시스템이 startup 되면서 /etc/rc.local 이 실행된다. 따라서 이 화일의 끝에 다음과 같은 라인을 추가하면 된다.

```
su  -c <$ORACLE_HOME>/bin/dbstart
```

예를 들어 oracle owner oracle7 이고 $ORACLE_HOME 이 /home/oracle 이라면 다음과 같이 해주면 된다.

```
su oracle7 -c /home/oracle/bin/dbstart
```

여기서 /home/oracle 대신에 $ORACLE_HOME 과 같은 환경변수를 이용하여 세팅하면 안된다.

SunOS 에서는 시스템이 shutdown 될 대 실행되는 스크립트는 따로 없다.
따라서 dbshut 을 실행시키는 방법은 다음과 같은 스크립트를 만들어서 시스템을 shutdown 시킬 때 /etc/shutdown 을 실행시키지 말고 대신 이것을 실행시켜야 한다.

```
#!/bin/csh
su  -c <$ORACLE_HOME>/bin/dbshut
/etc/shutdown $*
```

만약 오라클을 shutdown 하지 않고 시스템을 shutdown 하면 다음에 오라클을 startup 하려고 할 때 startup이 되지 않는다. 이 경우에는 $ORACLE_HOME/dbs/sgadef.dbf 화일을 지우고 startup 시키면 된다.

(2)OSF/1(DIGITAL UNIX)

```
/etc/inittab          : 초기화 과정을 콘트롤
/sbin/rc3             : run-level 3 에서 사용되는 script
/sbin/rc0             : run-level 0 에서 사용되는 script
/sbin/rc3.d/S99oracle : /sbin/init.d/oracle 로 link
/sbin/rc0.d/K01oracle : /sbin/init.d/oracle 로 link
```

/etc/initab 화일에는 초기화 과정에 대한 정보가 들어 있다. 예를 들면 다음과 같다.

```
s3:3:wait:/sbin/rc3 < /dev/console > /dev/console 2>&1
```

이렇게 설정하면 run-level 3 일 경우 /sbin/rc3 스크립트가 실행된다.
/sbin/rc3 은 /sbin/rc3.d 디렉토리에 들어있는 스크립트를 실행한다. 만약 스크립트 이름이 K 로 시작하면 이것은 stop 파라미터를 갖고 실행되어서 이 스크립트에 지정된 프로세스를 정지시킨다. 한편, 스크립트가 S 로 시작하면 /sbin/rc3 는 이 스크립트에 startup 파라미터를 부여하고 여기서 지정된 프로세스를 실행시킨다. 따라서 /sbin/rc3.d/S99oracle 을 사용하여 dbstart를 실행한다.

shutdown 과정도 거의 비슷하다. /etc/inittab 에 다음과 같은 라인을 추가 한다.

```
s0:0:wait:/sbin/rc0 off < /dev/console > /dev/console 2>&1
```

이것은 /sbin/rc0 를 실행 키는데 /sbin/rc0 는 /sbin/rc0.d 디렉토리에 들어있는 스크립트를 실행시킨다. 스크립트의 이름이 S 로 시작하면 run time parameter 로 start 를 사용하게 되고 K 로 시작하면 stop 을 사용한다. 따라서 /sbin/rc0.d/K01oracle 스크립트는 stop 파라미터를 갖고 실행되며 이 스크립트는 $ORACLE_HOME/bin/dbshut 스크립트를 실행시켜서 오라클을 shutdown 시킨다.

/sbin/rc3.d/S99oracle 과 /sbin/rc0.d/K01oracle 은 모두 /sbin/init.d/oracle에 대한 symbolic link 이다. /sbin/init.d/oracle 에 대한 symbolic link 는 이들 외에 더 있을 수 있다.

(3)Solaris 2.3 & SCO UNIX

```
/etc/initab           : 초기화 과정을 콘트롤
/etc/rc2              : run-level 2 에서 사용되는 스크립트
/etc/rc0              : run-level 0 에서 사용되는 스크립트
/etc/rc2.d/S99dbstart : /etc/init.d/dbstart 에 대한 link
/etc/rc0.d/K01dbshut  : /etc/init.d/dbshut 에 대한 link
/var/opt/oracle/oratab
```

Solaris 의 경우는 OSF/1 과 매우 유사하다. 차이점이라고 한다면 /sbin 디렉토리 대신에 /etc 디렉토리에 필요한 스크립트가 들어있다는 것이다.
rc0.d, rc2.d 같은 디렉토리는 실제로 /etc 디렉토리에 위치하고 있으며 rc0, rc2 와 같은 스크립트는 /sbin 에 존재하는 같은 이름의 스크립트에 대한 symbolic link 이다.

/etc/init.d/dbstart 와 /etc/init.d/dbshut 스크립트의 내용은 각각 다음과 같이 한줄로 구성되어 있다.

```
su  -c <$ORACLE_HOME>/bin/dbstart
```

```
su  -c <$ORACLE_HOME>/bin/dbshut
```

Solaris 에서는 start,stop 같은 파라미터는 /etc/init.d/dbstart,/etc/init.d/dbshut 에서는 무시된다.

/etc/init.d/dbstart, /etc/init.d/dbshut 화일은 owner 가 oracle owner id

이어야 한다. /etc/init.d/README 를 읽어보면 좀 더 자세한 내용이 설명되어 있으니 참조하기 바란다.

(4)AIX

/etc/inittab : 시스템 초기화 화일
/etc/rc : 시스템 초기화 스크립트
/etc/mkitab : /etc/inittab 을 수정하는 유틸리티
/etc/shutdown : 시스템 shutdown 스크립트

AIX 에서는 오라클 autostartup 방법은 여러가지가 가능하지만 shutdown 은 한가지만 가능하다. 우선 autostartup 은 다음과 같은 방법 가운데 하나를 사용하면 된다.

a)/etc/mkitab 를 사용하여 /etc/inittab 에 entry 를 추가한다. 즉,

```
/etc/mkitab ":2:wait:/bin/su  -c
              <$ORACLE_HOME>/bin/dbstart > /dev/console 2>&1"
```

이렇게 하면 /etc/inittab 화일에 dbstart 기능이 추가된다. 이렇게 하면 시스템이 뜨면서 run-level 2일 경우 오라클이 자동으로 startup 된다.

b)다음과 같은 방법도 가능하다.

```
/etc/mkitab "rcoracle:2:wait:/etc/rc.oracle > /dev/console 2>&1"
```

그리고 root 소유의 /etc/rc.oracle 스크립트를 다음과 같이 작성한다.

```
su  </bin/dbstart
EOF
```

c)/etc/rc 스크립트의 마지막에 다음을 추가한다.

```
/bin/su  - c <$ORACLE_HOME>/bin/dbstart
```

AIX 에서 시스템 shutdown 은 /etc/shutdown 에 의한다. 시스템을 내릴 때 오라클을 shutdown 하려면 별도의 시스템 shutdown 스크립트(root 소유로)를 다음과 같이 만들어서 /etc/shutdown 을 사용하지 말고 이것을 사용해야 한다.

```
#!/bin/sh
/bin/su  -c <$ORACLE_HOME>/bin/dbshut
/etc/shutdown $*
```

(5)HP/UX

/etc/rc : 시스템 startup 화일
/etc/shutdown : 시스템 shutdown 실행화일

/etc/shutdown.d : 시스템 shutdown 디렉토리

HP/UX 에서는 시스템 startup 시에 /etc/rc 가 실행된다. 따라서 다음 라인을 /etc/rc 스크립트의 마지막에 추가하면 autostartup 이 가능해진다.

```
su  -c <$ORACLE_HOME>/bin/dbstart
```

시스템의 shutdown 은 /etc/shutdown 화일을 실행하여 수행된다. 이 프로그램은 /etc/shutdown.d 디렉토리에 들어있는 스크립트를 실행시키는데 다음과 같은 세가지 방법 가운데 하나로 autoshutdown 을 세팅할 수 있다.

a)$ORACLE_HOME/bin/dbshut 화일을 /etc/shutdown.d 디렉토리 아래에 copy
b)/etc/shutdown.d/dbshut 을 $ORACLE_HOME/bin/dbshut 으로 symbolic link 지정
c)다음과 같은 스크립트 작성

```
#!/bin/sh
su  -c <$ORACLE_HOME>/bin/dbshut
```

단,이 스크립트는 root 의 소유이고 root 에 의해서만 실행가능해야 한다.

만약 V6 와 V7 을 동시에 사용하는 경우라면 V7 에서 제공하는 dbshut을 사용하도록 한다.

Oracle Enterprise Manager Install Procedure

Bulletin no : 10403

System requirement
HW (32M RAM, 30 M DISK)
SW (MS Windows NT ver 3.51 or Windows 95)

1. 먼저, Oracle server 7.3 sqlplus 에서OEM 유저를 생성한다. (grant resource, connect, dba to user) 이때, 반드시 default tablespace 를 지정하고, 가급적 다른 tablespace 와 분리하는 것이 좋다. (이 유저가 각자의 repository 유저로써 OEM 작업에 대한 정보가 저장된다)

2. Oracle installer(Orainst 또는 Setup) 를 수행 시킨 후에 Oracle Enterprise Manager와 Performance pack 을 설치한다.

3. Oracle Enterprise Manager 폴더에서 Network Topology Generator 를 선택 하거나 /Orawin95/bin/voot.exe 를 수행시킨다.
(이때, /Orawin95/network/admin/topology.ora 가 없기 때문에 ALERT 메세지가 뜨는데, OK를 클릭한다.) topology.ora 화일을 만들 때는 listener, database, agent 순으로 만든다.

4. Oracle for windows 95 폴더에서 SQL*plus 32 를 수행하여 Oracle Server 7.3 으로 tnsname을 이용하여 접속하여 Network 접속 여부를 확인한다. (잘 안된다면, tnsnames.ora 를 다시 설정) Oracle Enterprise Manager 폴더에서 SQL worksheet를 통하여 /Orawin95/sysman/admin/smpcre.sql 와 perfcre.sql를 차례로 수행 시킨다.

ORACLE MEDIA SERVER

Bulletin no : 10189

ORACLE MEDIA SERVER OVERVIEW

Oracle Media Server는 정보가 어떻게 저장되고, 검색되며, 목적지까지 도착하는 지를 정의하여주는 멀티미디어용 "자료 저장고(Data Warehouse)" 소프트웨어 입니다. 초병렬컴퓨터(Massively Parallel Computer)를 지원하는 강력한 데이타베이스 및 메시징 기술을 근간으로, 멀티미디어 서버는 초당 수천개의 트랜잭션을 처리할 뿐 아니라 대화형 서비스(Interactive Service)에 필요한 여러가지 데이타-오디오, 비디오 및 텍스트-를 관 하여 줍니다.

Oracle Media Net은 여러가지 복잡한 전송기술(전화선, 위성시스템 및 케이블 연결 등)을 감추어 투명하게 해주고, 정보를 가정으로 전달하기 위해 "TV"와 통신합니다.
Oracle Media Objects는 요구즉응 비디오(Video-on-demand), 요구 즉 뉴스 (News-on-demand), 그리고 홈 쇼핑(Home Shopping)과 같은 차세대의 애플리케이션을 창출해 낼 수 있는 최초의 애플리케이션 제작 도구(Authoring Tool) 입니다.

최근에 정보 고속도로와 관련하여 발표된 내용은 소비자가 어떻게 500여개의 TV 채널을 받고 조종할 수 있을 것인가에 그 촛점이 맞추어지고 있지만 Oracle의 새로운 소프트웨어는 사용자들이 각자의 구미에 맞도록 원하는 채널만을 짜맞출 수 있도록 하여줍니다.
예를 들면, 소비자들은 미디어 서버의 서비스를 이용하여 저녁 7시에 퇴근하고도 즐겨보는 방송의 6시 뉴스를 시청할 수 있고, 또는 템플리트(Template)를 만들어 자신이 원하는 토픽의 뉴스만을-스포츠뉴스 같은 -스캔(scan) 하여 볼 수도 있는 것입니다.
쇼핑도 현재의 TV쇼핑 프로그램이 상품선전 화면을 끝까지 지켜앉아 보아야하는 것과는 달리 소 자가 원하는 상품을 원하는 때 구매할 수 있습니다. 예를 들어 소비자들은 홍콩으로 부터 실크블라우스를 구입할 수 있고, 여러 상점으로부터 테니스 신발 가격을 비교해 볼 수도 있으며 크리스마스 선물도 집에서 구입할 수있게 됩니다. 또한 엔터테이먼트-영화, TV, 음악 기타 등등-도 요구즉응 서비스로 가능해 집니다.

다수의 전신전화 회사와 컴퓨터 제조회사들이 Oracle과 협력관계를 모색하고 있으며 이미 대화형 서비스 분야의 선두를 달리는 양대 전화회사-Bell Atlantic과 US West-가 그들의 대화형 서비스에 Oracle Media Server를 사용하겠다고 발표한 바있습니다. 대화형TV 제공의 주역자인 Philips, Compression Labs, Kaleida, 3DO,Echelon, General Magic, Apple Computer등도 Oracle의 미디어 서버를 지원하겠다고 발표하였으며, nCUBE와 Silicon Graphics 도 그들의 강력한 컴퓨터로 Oracle Media Server를 지원하겠다고 약속하였습니다

OMS Release

지난 수년동안 Oracle은 DBMS기술을 On-line 서비스의 세계에 의욕적으로 적용해 왔습니다. 이러한 노력의 일환으로 Oracle의 소프트웨어는 비디오, 오디오, 그리고 문자 데이타등을 수용할 수 있게 되었습니다. 또한 Oracle은 초병렬컴퓨터(Massively Parallel Computer)라고하는 새로운 형태의 컴퓨터에서 작동하는 최초의 상용 소프트웨어를 공급하는 회사가 되었습니다.
초병렬컴퓨터는 값이 비싸지 않은 마이크로프로세서 수천개를 배열하고 각각의 프로세서에 독자적인 메모리를 부여하여 외부와의 통신로를 제공하여 줍니다. 따라서 전체 시스템은 기존의 컴퓨터에서 성가신 문제로 여겨졌던 밀집된 데이타 경로를 피할 수 있고 수천명의 사용자들이 동시에 요청하는 요

구사항 즉, 요구즉응 영화(Movie-on-demand) 와 같은 서비스를제공할 수 있게되는 것입니다. nCUBE 2와 같은 전형적인 초병렬시스템에서 작동하는 미디어 서버의 경우, 25,000개의 개별적인 비디오 채널-또는 스트림(stream)-을 채널당 500달러의 가격으로 제공하며 이때 사용자들은 기존의 VCR과 똑같이 pause,rewind, fast forward 같은 기능을 이용 할 수 있게 됩니다.

Oracle 미디어 서버는 4가지의 주요 구성품으로 나누어 집니다.
* 시스템 관리 도구(System management tools)는 시스템관리자로 하여금 서버를 관리하도록 하여주는데 이에는 멀티미디어 자료의 로딩과 목록만들기 (cataloging), 사용자 프로파일(profile) 갱신, 시스템 파라미터(parameter)의 세팅 등이 포함됩니다.
* 서비스(Services)는 사용자의 요청에 의해 작동이 되어 요청된 정보를 라이브러리(libraries)에서 찾을 수 있도록 메시지를 전송하는 것과 관련된 일련의 기능을 총칭하는 것입니다.
* 액세스 라이브러리(Access libraries)는 서버의 데이타베이스내의 데이타를 액세스 하기 위한 데이타 조작루틴(manipulation routine)을 총칭합니다.
이로 인해 데이타의 위치지정, 갱신 및 검색등이 가능해 지며, 정지,빨리감기,되감기 등 기존의 VCR에서와 같은 조정/조작도 가능해 집니다.
* 데이타베이스(Databases)는 대화형의 서비스를 지원하기 위해 여러종류의 데이타를 저장합니다. 미디어 서버에는 6개의 데이타베이스가 있습니다.

- 실시간 스트림 데이타베이스(Real-time stream database)는 실시간으로 처리될 오디오 및 비디오 정보를 저장합니다. 사용자가 요구 즉응 오디오나 비디오 서비스를 요청하면 이 데이타베이스가 1초내에 요청된 자료를 찾아 소비자에게 보내게 됩니다.
- 관계형데이타베이스 서버(Relational database server)는 Oracle 7 데이타베이스이며 청구서를 위한 정보, 영화 리스트, 사용자 프로파일 등 표의 형식으로 분류될 정보를 관리합니다.
- 화일서버(File server)는 OS의 화일에 대한 액세스를 제공합니다.
- BLOB(Binary Large Object)데이타베이스는 이미지 및 실행화일을 저장합니다.
- 오브젝트 데이타베이스(Object database)는 오브젝트-오리엔티드 프로그래밍을 위한 오브젝트를 관리합니다.
- 텍스트 데이타베이스(Text database)는 텍스트와 도큐먼트 신속하게 추적, 검색하기 위한 것으로 인덱싱과 요약기능 등이 포함됩니다.

Oracle의 미디어 서버는 1994년 3월에 시중에 공급될 예정입니다. 오브젝트 서버와 텍스트 서버는 이때 제외될 것입니다. 다른 모든 Oracle 제품과 마찬가지로 미디어 서버도 역시 플랫폼에 제한 받지 않도록 설계되어 있습니다. nCUBE에 포팅하는 것을 시작으로 내년 까지는 다른 초병렬시스템과 다중처리 시스템에도 포팅될 것입니다.

OMO Release

Oracle Media Objects Software는 맞춤형식의 홈 서비스(Customized home services)를 제공하기 위해 가칭 "스마트(smart) TV" 시스템에서 OOP(Object-oriented programming)를 통하여 요구즉응 비디오나 홈 쇼핑 등을 가능하게 해주는 소프트웨어 입니다. 미디어 오브젝트는 대화형 애플리케이션의 프로토타이핑, 테스팅등을 신속하게 처리해 주는 영 와 흡사한 언어인 Oracle Media Talk 와 그래피컬 툴,그리고 재사용이 가능한 오브젝트들로 이루어집니다. 미디어 오브젝트의 기본 구조는 여러개의 "스택(stacks)"이며 이들 각각의 스택은 여러 개의 "카드(cards)"로 이루어집니다. 각각의 카드는 스크린 상에서 사용자가 직접 원격제어를 할 수 있도록 텍스트와 그래픽 등으로 이루어진 하나의 스크린 정보를 의미합니다. 사용자들은 카드 즉, 스크린을 마음대로 바꿀 수 있게 됩니다. 개발자들은 미디어 오브젝트의 애플리케이션용 오브젝트 세트를 이용하여 사용하기 쉬운 고도의 그래피컬 사용자

인터페이스를 만들어 낼 수 있습니다. 오브젝트란 독특한 모습과 작동원리가 부여된 하나의 구조물로서 스크린 상에 나타나는 것을 말합니다. 오브젝트의 작동에 관한 통제는 미디어 토크(Media Talk)로 하게 됩니다. 미디어 오브젝트의 개발 환경은 윈도우 맥킨토시에서 모두 가능하며, 오브젝트를 생성하고 편집하는 그래피컬 인터페이스와 미디어 토크 스크립트(scripts)는 물론 테스팅과 디버깅을 위한 런타임 (run-time)환경도 모두 지원됩니다.

OMN Release

Oracle Media Net는 가정으로 대화형의 서비스를 제공하는데 사용되는 여러가지 전송기법을 사용자로 부터 감추어주는 즉, 투명하게 해주는 고도의 프로토콜 입니다. 미디어 네트를 통하여 요구즉응 서비스에 대한 요청이 미디어 서버와 연결되는 것입니다.

Oracle 미디어 네트는 비대칭형(asymmetric)과 대칭형(symmetric)등 두가지의 네트워크 형태를 지원합니다. 비대칭형 네트워크는 서로 다른 밴드 폭을 갖는 두개의 방향이 다른 채널로 이루어지며, 이중 하나는 아주 빠른 속도로 가정으로 향하고, 또 하나는 그 보다는 느린 속도로 가정으로부터 빠져나오게 됩니다. 가정의 전화선이 좋은 예 입니다. 1994년 3월중에 계획되어 있는 Bell Atlantic 과 Oracle의 최초의 대화형서비스에는 기존의 "Twisted Pair" 네트워크에 ADSL 이라고 하는 프로토콜을 적용시켜 요구즉응 비디오 서비스 즉, 비디오 온 디맨드 서비스를 공급하게 됩니다.

대칭형 네트워크는 소위 말하는 "브로드밴드(broadband)"서비스를 말하며, 가정으로 들어오고 나가는 양쪽 채널 모두 고속의 밴드 폭을 제공합니다. 케이블과 화이버 (fiber)로 구성이 되며 ATM(Asynchronous Transmission Mode) 방식이 주로 쓰이는 프로토콜 입니다.

미디어 트는 또한 가정으로부터의 사용자 요청을 서버로 보내주는 소프트웨어를 제공합니다. 미디어 네트는 서비스 단위의 메시징을 기본으로 하기 때문에 특정한 비디오 또는 홈쇼핑 트랜잭션등은 한 번의 문의(조회)로 처리 됩니다.

또한 비디오 컨퍼런스(conference)나 자우편, 홈 뱅킹(home banking)등과 같이 지정된 상대와 메세지를 주고 받는 담화형식의 메시징도 지원합니다. Oracle 미디어네트는 미디어 서버와 어떠한 셋-탑박스(set-top box:TV위에 장착되는 부수장비)나"스마트 TV"와도 연결시켜 줌으로 사용자는 마음에 드는 시스템을 선택하면 됩니다.

ORACLE SERVER AND UNIX TRANSACTION PROCESSING(TP) MONITOR-1

Bulletin no : 10021

This file contains commonly asked questions about Oracle Server and UNIXT ransaction Processing Monitors (TPMs). The topics covered in this article are

- o What is a Transaction Processing Monitor (TPM)?
- o What is the X/Open Distributed Transaction Processing Model?
- o How does the Oracle Server works with TPMs?
- o How should I position TPMs with my customer?
- o What Oracle products must a customer purchase?
- o Where can my customer purchase a TPM?
- o Availability and packaging

What is a Transaction Processing Monitor?

Under UNIX, a Transaction Processing Monitor (TPM) is a tool that coordinates the flow of transaction requests between front-end client processes that issue requests and back-end servers that process them. A TPM is used as the "glue" to coordinate transactions that require the services of several different types of back-end processes, such as application servers and resource managers, possibly distributed over a network.

In a typical TPM environment, front-end client processes perform screen handling and ask for services from back-end server processes via calls to the TPM. The TPM then routes the requests to the appropriate back-end server process or server processes, wherever they are located on the network. Through configuration information, the TPM knows what services are available and where they are located. Generally, the back-end server processes are specialized so that each one handles one type of requested service. The TPM provides location transparency as well and can send messages through the network utilizing lower-level transport services such as TCP/IP or OSF DCE.

The back-end servers process the requests as necessary and return the results back to the TP monitor. The TP monitor then routes these results back to the original front-end client process.

A TPM is instrumental in the implementation of truly distributed processing.
Front-end clients and back-end processes have no knowledge of each other. They operate as separate entities, and it is this concept that provides flexibility in application development. Front-end and back-end processes are developed in the UNIX client-server style, with each side optimized for its particular task. Server functionality can be deployed in stages, which makes it easy to add functionality as needed later in the product cycle. It also makes it easy to distribute both the front-end and back-end processes throughout the network on the most appropriate hardware for the job. In addition, multiple back-end server processes of the same type might be activated to handle increasing numbers of users.

What is the X/Open Distributed Transaction Processing Model?

The X/Open Transaction Processing working group has been working for several years to establish a standard architecture to implement distributed transaction processing on open systems. In late 1991, X/Open published the initial Distributed Transaction Processing (DTP) model specification and defined the first of several interfaces that exist between the components of the model. Subsequently, other publications and a revised model specification have been published.

An important function of the TPM in the X/Open DTP model is the synchronization of any commits and rollbacks that are required to complete a distributed transaction request. The Transaction Manager (TM) portion of the TPM is the entity responsible for ordering when distributed commits and rollbacks will take place. Thus, if a distributed application program is written to take advantage of the TM portion of the TPM, then it, and not the DBMS, becomes responsible for enabling the two-phase commit process. Article 2 has more detail on this model.

How does the Oracle Server work with TPMs?

When a TPM is used without invoking an X/Open TM component to manage the transactions, Oracle Server needs no special functionality. The transaction will be managed by Oracle itself. However, when the TPM X/Open TM component is used to manage the transaction, the Oracle Server, that is the Oracle DBMS, acts as a Resource Manager--a type of back-end process. In the case of TPM-managed transactions, the TM needs a way to tell the RMs about the stages of the transaction. This is done by a standard, X/Open defined interface called XA. Article 2 of of this document gives more information about both the X/Open model and Oracle7's use of XA.

Because the XA interface provides a standard interface between the TM and the resource manager, it follows that the TM can communicate with any XA-compliant resource manager (e.g., RDBMS), and, conversely, that a resource manager can communicate with any XA-compliant TM. Thus, the Oracle Server, beginning with Oracle7, works with any XA-compliant TM.

How should I position TPMs with my customer?

There's been a great deal of confusion about the need for TPM technology. Some software suppliers, most notably IBM, will assert that a TPM like CICS is a necessary requirement for high volume OLTP. Other vendors will assert that there is seldom a need for such technology. And yet others promote TPMs as providers of higher transaction throughput.

From Oracle's standpoint, customers might choose TPM technology under any of the following conditions:

1. **For heterogeneous database access, especially for 2PC capability**

   This means that a TPM can be used to coordinate 2PC between Oracle DBMS and any other XA-compliant database, such as Informix. This does NOT provide SQL heterogeneity - SQL calls to Oracle DBMS may be different than SQL calls to Informix. The TPM handles the routing,
   communication, and two-phase commit portion of the transaction, but does not translate one type of SQL call into another.

2. For transaction monitoring and workload control

The leading TPMs supply tools to actively manage the flow of transactions between clients and servers and to load balance the work load across all available processors on a network, not just on a single multi-processor system. Some TPMs also have the ability to dynamically bring up additional back-end services during peak work hours.

3. For more flexible application development and installation

One of the key features of the DTP model is application modularity. Modularity, that is, the decomposition of a large program into small, easily defined, coded and maintained "mini-programs" makes it easy to add new functionality as needed. Modularity also makes it much easier to distribute the front-end and back-end processes and the resource managers across hardware throughout a network.

4. For isolating the client from details of the data model

By using the service oriented programming model, the client program is unaware of the data model. The service can be recoded to use a different one with no change to the client. To get this advantage, the application developer must explicitly code the server and client to fit the service model.

5. For connection of thousands of users

TP Monitors, because of their three-tier architecture, can be used to connect users to an intermediate machine or machines, removing the overhead of handling terminal connections from the machine arunning the database. See Article 4 for more information.

There are also several cases where TPM technology is not the right answer.
These include:

1. If the customer is simply looking for a performance improvement The customer may have heard a theory that "higher performance is possible for large scale applications only if they use a TP monitor". First, no performance gain can be achieved for existing applications; in fact, they won't even run under a TP Monitor without recoding. Second, performance improvements have only been documented for large numbers of users, and "large" means many hundreds or thousands. Without a TP Monitor, Oracle Server can handle several hundred users with its normal two-task architecture and several times that using the Multi Threaded Server. For more on performance, see Article 4.

2. If the customer has made large investment in his existing Oracle applications
TP monitor applications must be designed from the ground up to take advantage of TP monitor technology. Current Oracle customers will find it difficult to "retrofit" a TP monitor to their existing applications. The Multi Threaded Server, on the other hand, allows the use of existing Oracle applications without change.

3. If the customer is committed to the Oracle tool set
Currently, none of Oracle's front-end tools (Oracle Forms, etc.) is designed to work with TP monitors. It is possible to invoke a TP Monitor by using user exits. However, the fact that the TP Monitor model hides the data model from the client means that only the screen display parts of Forms can be used, not the automatic mapping from screen blocks to tables.

4. If the customer does not have a staff of experienced software engineers
This is still very young technology for UNIX. There is not a lot of knowledge in the industry on how to build TP monitor applications or what techniques are most useful and which are not. Furthermore, integrating products from different vendors, even with the support of standard interfaces, is more complex than deploying an integrated all-Oracle solution. Because TP monitor technology is fairly complex, we recommend that you let the TP monitor supplier promote the virtues of their technology and differentiate themselves from their competitors.

What Oracle products must a customer purchase?

If your customer is only interested in building Oracle-managed TP Monitor transactions, the only Oracle products required are the Oracle Server and the appropriate Oracle precompiler for whatever language the application is being written in--most likely C or Cobol. If TPM-managed transactions are required, the Oracle7 Server with the distributed option is also required. SQL*Net is optional because the TPM takes care of the network services. Article 2 describes when you would choose to have the TP Monitor manage the transactions.

Where can my customer purchase a TPM?

There are many vendors offering the UNIX TPM products. (Oracle does not relicense TPMs.) Information on the most well known products is provided below:

The following support XA:

Product & Vendor	FCS	Known OS/Platform Ports
"TUXEDO System/T" UNIX System Laboratories 190 River Road Summit, NJ 07901	1986	UNIX SVR4 & SVR3: Amdahl, AT&T, Bull, Compaq, Dell, Fujitsu, ICL, Motorola, Olivetti, Pyramid,Sequent, Sun, Toshiba, Unisys, NCR, Stratus Other: IBM AIX, HP/UX, DEC Ultrix
"TOP END" NCR Corporation 1334 S. Patterson Blvd. Dayton, OH 45479	1992	UNIX SVR4: NCR
"ENCINA" Transarc Corporation 707 Grant Street Pittsburgh, PA 15219	1992	IBM AIX, HP, Sun (SunOS and Solaris) Other: OS/2, DOS, HP-UX, STRATUS (Depends on DCE)
"CICS/6000" IBM Corporation	1993	AIX: IBM (Depends on DCE)

"CICS 9000" HP	1994	HP-UX

The following do not currently support XA:

Product & Vendor	FCS	Known OS/Platform Ports
"VIS/TP" VISystems, Inc. 11910 Greenville Avenue Dallas, TX 75243	unknown	unknown
"UniKix" UniKix	1990	UNIX: ARIX, AT&T, NCR, Pyramid, Sequent, Sun, Unisys
"MicroFocus Transaction System" Micro Focus 26 West Street Newbury RG13 1JT UK	1993	SCO Unix, AIX

There are also several third parties who are reselling the products listed above.

In addition, Groupe Bull, Digital, Siemens-Nixdorf, and several other hardware vendors are planning to redesign their proprietary TPMs to be XA-compliant and suitable for use on UNIX systems.

Availability and Packaging

On what platforms is the XA Library available?

Oracle provides the XA interface with Oracle7 Server on all platforms that support an XA-compliant TPM. Support for XA is included as part of the Oracle7 Server distributed option and has no extra charge in and of itself.

Which version of XA does Oracle Server support?

Oracle7 Server supports the Common Application Environment (CAE) version of XA, based on the specification published by X/Open in late 1991. It will require that the TM also be at that level. This means Tuxedo /T version 4.2, for example.

Oracle Server supports all required XA functions. There are some optional features Oracle Server does not support, such as asynchronous operation. None of those options affect application programming.

This file contains commonly asked questions about Oracle Server and UNIX Transaction Processing Monitors (TPMs). The topics covered in this article are

- o Oracle Server Working with UNIX TPMs
- o TPM Application Architecture

The questions answered in part 2 provide additional detail to the information provided in part 1.

Oracle Server Working with UNIX TP Monitors

Do I need XA to use Oracle Server with TPMs? If I don't use it, what are the consequences?

There are a number of real applications running today with Oracle Server and TPMs but not using XA. To use a TPM with Oracle without using XA, the user would write an "application server" program which could handle one or more "services". For example, a server program might handle a service called "debit_credit". The key requirement is that the entire transaction, including the "commit work", must be executed within a single service. This is the restriction which XA will remove, as we'll see later. Each server process can serially handle requests on behalf of different clients.

Because a server process can handle many client processes, this can reduce the total number of active processes on the server system, thereby reducing resource requirements and possibly increasing overall throughput.

When Oracle is used with a TPM in this mode, we call it an Oracle-managed transaction since the transaction commit or rollback is done with a SQL statement.

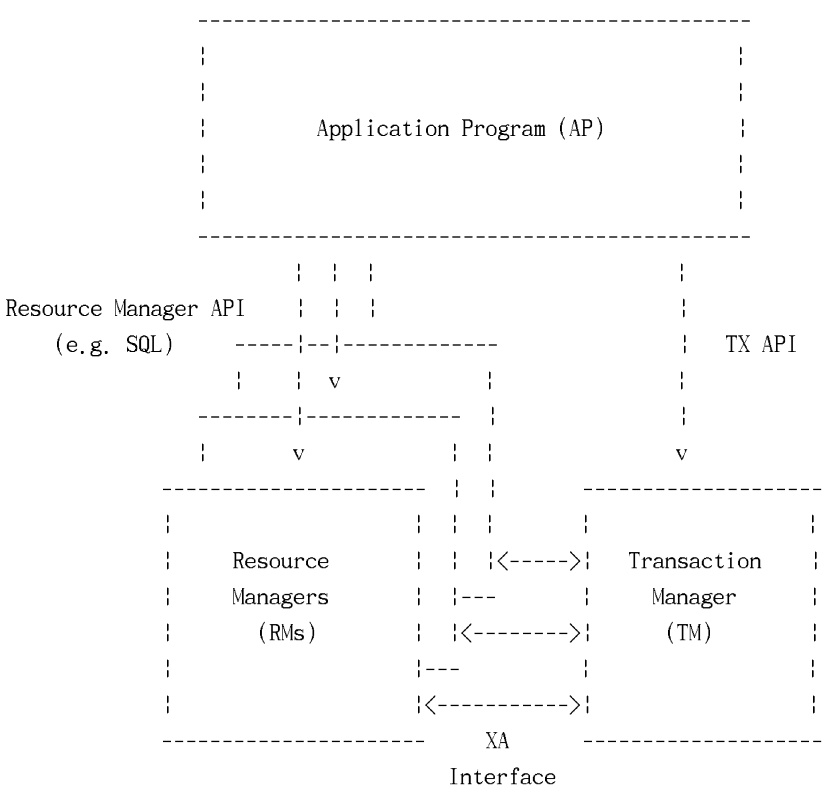
What is XA? How does XA help Oracle7 work with UNIX TPMs?

XA is an industry standard interface between a Transaction Manager and a Resource Manager. A Resource Manager (RM) is an agent which controls a shared, recoverable resource; such a resource can be returned to a consistent state after a failure. For example, Oracle7 Server is an RM and uses its redo log and undo segments to be able to do this. A Transaction Manager (TM) manages a transaction including the commitment protocol and, when necessary, the recovery after a failure. Normally, Oracle Server acts as its own TM and manages its own commitment and recovery. However, using a standards-based TM allows Oracle7 to cooperate with other heterogeneous RMs in a single transaction.

The commonly used TPMs include a TM component for this purpose. In order to use the TM capability of the TPM rather than Oracle7's own transaction management, the application uses a transaction demarcation API (called TX) provided by the TPM rather than the SQL transaction control statements (e.g. "commit work"). For each TX call, the TM then instructs all RMs, by the appropriate XA commands, to follow the two-phase commit protocol. We call this a TPM-managed transaction.

The following picture shows these interfaces within a monolithic application program model. This is the model most commonly described in the DTP literature. We'll see later what the picture looks like when we add Oracle7 and when we switch to a modularized client-server

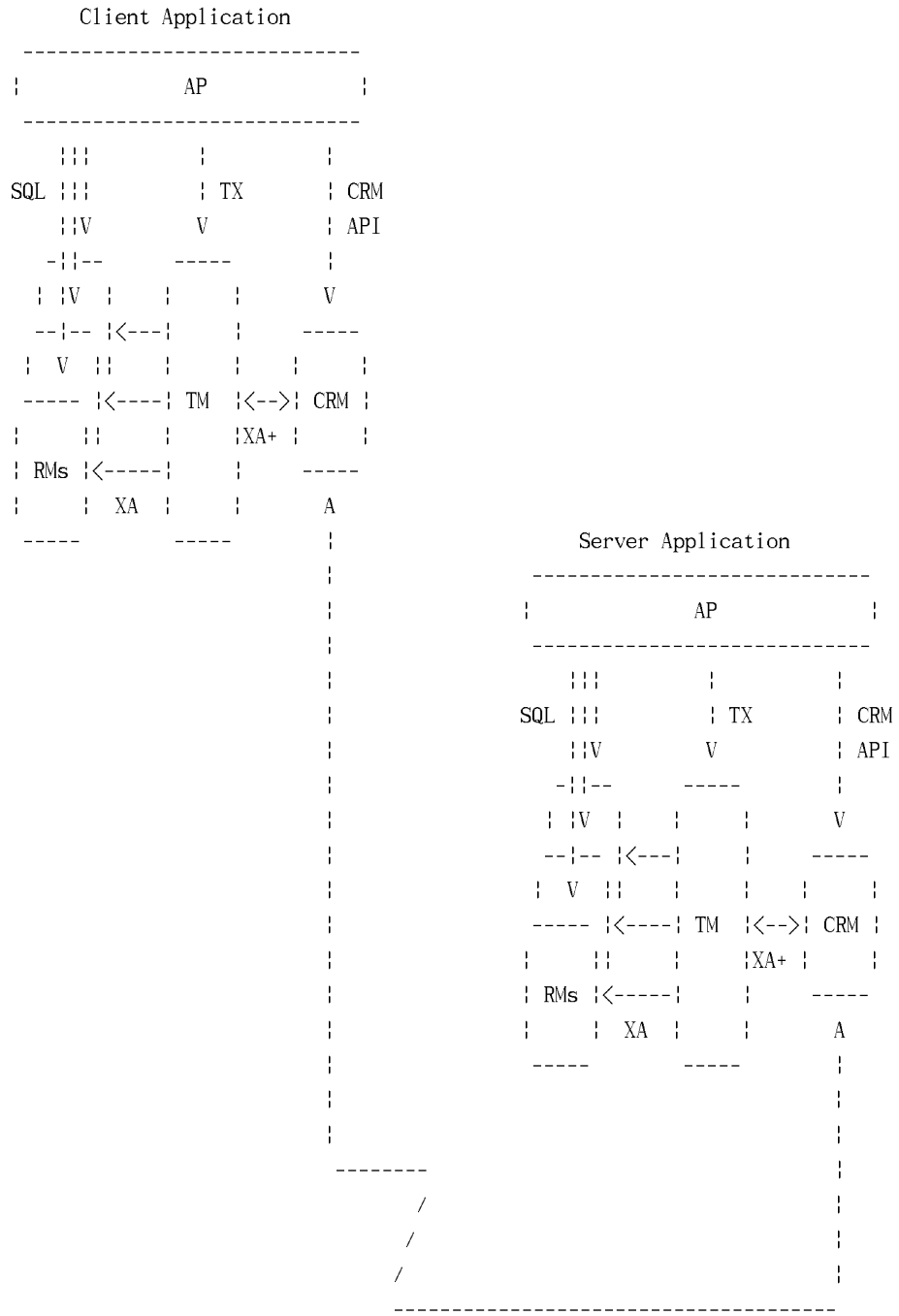
application program model.



The XA interface is an interface between two system components, not an application program interface: the application program does not write XA calls nor need to know the details of this interface. The TM cannot do transaction coordination without the assistance of the RM; the XA interface is used to get that assistance.

How does the DTP Model support client-server?

The above picture was actually simplified to make it easier to explain the role of XA. In a true distributed transaction architecture, there are multiple applications, each with an Application Program, a Resource Manager, and a Transaction Manager. The applications communicate by using a Communication Resource Manager. The CRM is generally provided as a component of the TPM. It includes the transaction information when it sends messages between applications, so that both applications can act of behalf of the same transaction. The following picture illustrates this:



Most TP Monitor products include both a TM and a CRM, and also provide additional functions such as task scheduling and workload monitoring.

What is XA+? What does Oracle need to do to comply with it?

XA+ is an interface that lets the X/Open model actually be distributed because it allows a communication resource manager to tell a TM on the server that a message from a client just came in for a particular transaction. Oracle is not currently planning to provide an X/Open communication resource manager, so we don't have any plans right now to do XA+. Version 2

of the DTP model paper from X/Open describes it. The status of the current XA+ specification is "snapshot".

When would I choose an Oracle-managed transaction vs a TPM-managed transaction?

Oracle Server is very efficient at managing its own transactions. If the TPM manages the transaction, in general some additional overhead will be incurred.

The two main reasons a customer might prefer to use a TPM-managed transaction are as follows:

(1) He may need to update RMs from different vendors. Experience so far has been that the most common case is wanting to update both Oracle and a TP Monitor managed resource such as a transactional queuing service in the same transaction (see Article 3).

(2) He may want to use the model of having several different services in a transaction, even to the same database. For example, the "debit_credit" service could be split into a "debit" service and a "credit" service. This is a very attractive model, but this type of modularity does exact a performance penalty (see Article 4).

Can I get a version of XA to run on Oracle Server version 6?

No, the XA functionality uses two underlying mechanisms in the Oracle Server which are not available in version 6: two-phase commit and session switching. The upi calls for these functions do not not exist in version 6.

When would I use XA vs Oracle7 to coordinate all-Oracle distributed transactions?

Generally speaking, Oracle Server should be used to coordinate all-Oracle distributed transactions. The main reason for using XA to coordinate transactions would be that you want to use the TP Monitor service-oriented architecture. That is, you would like to construct an application built of services and service requests in order to benefit from the modularity and workload control such an environment provides.

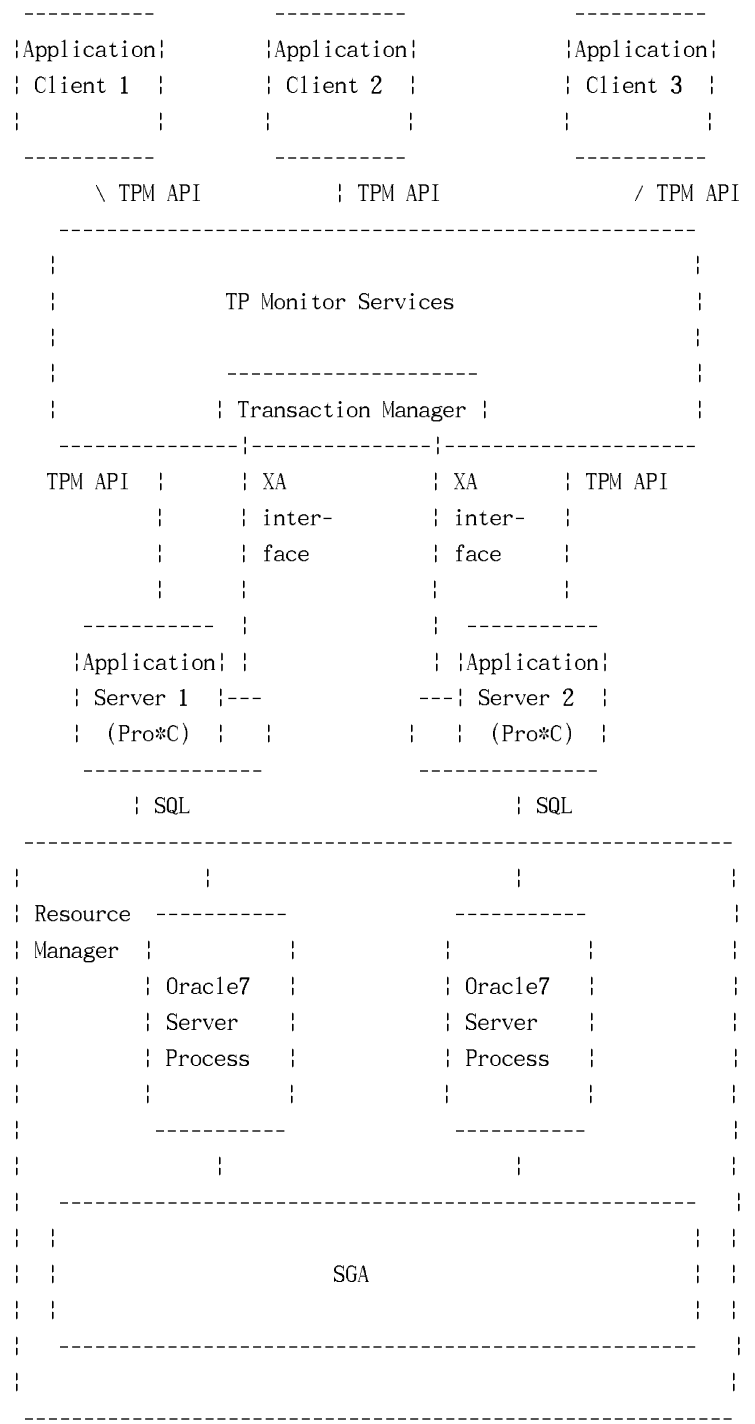
TP Monitor Application Architecture

What might a TP Monitor application look like?

Most TPM applications will consist of two more more programs, where there are front-end client programs which request services and back-end server programs which provide services. In this case, the TPM supplies an additional capability which is transactional communication. The client describes the boundaries of the transaction, through the use of the TX API, and the TPM relays that transaction information to each requested service.

The overall application structure generally looks like the following in the client-server model. The "TP Monitor Services" box is not necessarily a process. It could be one or more processes, or just libraries coordinating through shared memory. Each client process and server process could be on a different machine. Normally, the application server processes

would be connected to their Oracle Server processes using the IPC driver; the TPM would be used to deliver messages between application client processes on one machine and application server processes on another. However, the application server processes could also be connected with the standard Oracle SQL*Net to shadow processes on different machines. This might be useful if one of the databases was on a machine which did not support TPMs.



Application client programs might be written in C and be linked with TPM libraries.

Alternatively, they could use a screen painter product. Application server programs would be written in Pro*C or Pro*COBOL and be linked with TPM libraries, the normal Oracle7 user-side libraries and libxa.a. The Oracle7 Server process is the regular Oracle7 executable.

More complicated application architectures can also be constructed. Most of the TPMs allow a server to become a client of another service, so you can involve additional servers.

Could I use Oracle7's Multi Threaded Server as the SQL*Net connection in the previous picture?

Yes, but that will not be needed in many cases. For example, both application server processes in the previous picture could talk to a single Oracle7 Server process through the Multi Threaded Server in the previous picture. However, since the TPM architecture typically reduces the number of server processes, the reduction in processes using Multi Threaded Server may be less significant than in an architecture without TPMs. If the application will use database links, however, then MTS will be required.

How do I write an Oracle TP Monitor application?

The actual API used to talk to the TPM varies between vendors, so you need to get the documentation from the vendor. However, all have a way to indicate where a transaction begins and ends and a way to send a request and receive a response from a client to a server. Some use an RPC model, some use a pseudo-RPC model, and some use a send/receive model. The TX API
described earlier is a subset of the TPM API as defined by each of the TPM providers.

The client program and server program might look something like the following examples. We h
(such as Tuxedo's "tpacall

ORACLE7 BACKUP 및 RECOVERY 방법

Bulletin no : 10080

DATABASE 의 BACKUP 은 DB 관리자 및 DB 를 사용하는 모든 사용자에게 가장중요한 사항이다. ORACLE 은 DB BACKUP 을위해 몇 가지방법을 제시 하고 있으나 선택은 데이타의중요도,DB 이용도 등에 따라 가장적절한 방법을 택하고 가능하면 여러가지 방법으로 BACKUP 을 받 것이 안전하다.

< BACKUP 및 RECOVERY 방법 >
. EXPORT & IMPORT
. INCREMENTAL EXPORT & IMPORT
. IMAGE BACKUP (COLD,HOT BACKUP)
. ARCHIVING

1. EXPORT & IMPORT

EXPORT 는 DATA 를 ORACLE DATABASE 로부터 ORACLE Binary 형태의 OS FILE로 만들고 IMPORT 를 이용하여 DATABASE 로 LOADING 한다.

< EXPORT 의 형태 >
Entire Database (FULL) - 모든 DATABASE 을 EXPORT 한다.
User(s) - 특정 USER 의 OBJECT 를 EXPORT 한다.
Table(s) - 특정 TABLE 을 EXPORT 한다.

< 예 1 > 전체 데이타베이스 EXPORT (Interactive Method)

```
$ exp system/manager
Connected to: ORACLE7 Server Release 7.0.16.4.0 - Production
With the procedural and distributed options
PL/SQL Release 2.0.18.1.0 - Production
Enter array fetch buffer size : 4096 > (RETURN)
Export file : expdat.dmp >
(1) E(ntire database), (2) U(sers), (3) T(ables) : (2) U > e
Export grants (Y/N) : Y > y
Export table data (Y/N) : Y > y
Compress extents (Y/N) : Y > y
About to export the entire database....
. exporting tablespace definitions
. exporting profiles
. exporting user definitions
. exporting role
. exporting rollback segment definitions
. exporting database links
. exporting sequence numbers
. exporting sequence numbers
. exporting cluster definitions
```

```
. exporting stored procedures
. about to export SYSTEM's tables ...
. about to export SCOTT's tables ...
. exporting synonyms
. exporting views
. exporting referential integrity constraints
. exporting triggers
Export terminated successfully without warnings.
```

< 예 2 > 전체 데이타베이스 EXPORT (Command Line Method)

```
$ exp userid=system/manager full=y compress=y grants=y rows=y
```

< 예 3 > 전체 데이타베이스 EXPORT (Dynamic Method)

EXPORT PARAMETERS 을 다음과 같은 FILE(tusc.par) 형태로 만든다.

```
system/manager
full=y
compress=y
grants=y
rows=y
```

```
$ exp parfile=tusc.par
```

< 예 4 > USER only EXPORT

```
$ exp system/manager
Connected to: ORACLE7 Server Release 7.0.16.4.0 - Production
With the procedural and distributed options
PL/SQL Release 2.0.18.1.0 - Production
Enter array fetch buffer size : 4096 > (RETURN)
Export file : expdat.dmp >
(1) E(ntire database), (2) U(sers), (3) T(ables) : (2) U > u
Export grants (Y/N) : Y > y
Export table data (Y/N) : Y > y
Compress extents (Y/N) : Y > y

About to export specified users
User to be exported: (RETURN to quit) > scott
. exporting snapshots
. exporting snapshot log
. exporting database links
. exporting sequence numbers
. exporting sequence numbers
. exporting cluster definitions
. exporting stored procedures
. about to export SCOTT's tables ...
```

```
. exporting synonyms
. exporting views
. exporting referential integrity constraints
. exporting triggers
Export terminated successfully without warnings.
```

< 예 5 > USER only EXPORT (Command Line Method)

```
$ exp userid=system/manager owner=scott compress=y grants=y rows=y
```

< 예 6 > USER ONLY EXPORT (Dynamic Method)

EXPORT PARAMETERS 을 다음과 같은 FILE(tusc.par) 형태로 만든다.

```
system/manager
owner=scott
compress=y
grants=y
rows=y
```

```
$ exp parfile=tusc.par
```

< 예 7 > TABLES EXPORT

```
$ exp scott/tiger tables=emp,dept compress=y grants=y rows=y
```

< DATA IMPORTING >

DATA IMPORT 는 EXPORT 와 비슷하고 항상 EXPORT 된 FILE 로만 IMPORTING이 된다.

< 예 8 > 전체 데이타베이스 EXPORT FILE 로부터 SINGLE TABLE 을 IMPORT (INTERACTIVE MODE)

```
$ imp system/manager
Connected to: ORACLE7 Server Release 7.0.16.4.0 - Production
With the procedural and distributed options
PL/SQL Release 2.0.18.1.0 - Production

Import file: expdat.dmp > (RETURN)
Enter insert buffer size (minimum is 4096) 30720 > (RETURN)
Export file created by EXPORT:V07.00.16
List contents of import file only (yes/no) : no >
 ( "yes" 이면 data는 import 되지않고 Display 만됨 )
Ignore create errors due to object existence (yes/no) : yes >
 ("yes" : 만일 table 이존재하면 record 가 추가된다.
  "no"  : 이미 존재한 Object 에대해서 import 를 하지않는다.)
Import grants (yes/no) : yes >
```

```
Import table data (yes/no) : yes >
Import entire export file (yes/no) : yes >  no
User name : SCOTT
Enter table name. Null list name all tables for user
Enter table name or . if done : TEST
 importing SCOTT's objects into SCOTT
 importing table TEST       900 rows imported
Import terminated successfully.
```

주의 : IMPORT 작업은 보통 생성되는 INDEX 수에따라 1.5 - 4 배의 시간이걸리는데 EXPORT 는 "CREATE INDEX ...." 문만을 FILE 에 쓰고 IMPORT 시에는 INDEX 를 생성하기때문이다.

(COMMAND LINE MODE)

```
$ imp system/manager full=n grants=y rows=y tables=test commit=y
```

주의 : IMPORT 시는 데이타를 DB 에 WRITE 하기때문에 ROLLBACK SEGMENT가 사용된다. commit=y 는 row 만큼 데이타를 LOAD 하고나서 COMMIT 을 하기 때문에 ROLLBACK SEGMENT 가 커지는것을 피할수있다. commit=n 이면 TABLE 단위로 IMPORT 하고나서 COMMIT 이 되기 때문에 ROLLBACK SEGMNET 확장 ERROR 가 발생 할수있다.

2. INCREMENTAL EXPORT & IMPORT

< INCREMENTAL EXPORT >

INCREMENTAL EXPORT 는 SYS 혹은 SYSTEM USER 로서 EXPORT 한다.만일 FILE 이름을 "expdat.dmp" 에서 다른 이름으로 변경하고자 할때는 COMMAND LINE 에서 FILENAME 만 지정하면된다.

INCREMENTAL EXPORT 의 3 가지 형태

. COMPLETE - <예 1> 에서와같이 전체 데이타베이스를 EXPORT 한다.
. CUMULATIVE - 마지막 "COMPLETE" 혹은 "CUMULATIVE" EXPORT 이후에
변경된 정보를 EXPORT 한다.
. INCREMENTAL - 마지막 "COMPLETE","CUMULATIVE" 혹은 "INCREMENTAL"
EXPORT 이후에 변경된 정보를 EXPORT 한다.

BACKUP 방법은 매월 1 일에 "COMPLETE", 주말에 "CUMULATIVE", 주중에는
"INCREMENTAL" 로 하면 가장 적절하다.

< 예 9 > INCREMENTAL EXPORT

```
$ exp system/manager inctype=complete file=com0304.dmp
$ exp system/manager inctype=incremental file=inc0305.dmp
```

주의: INCREMENTAL EXPORT 데이타량은 "NEW 혹은 UPDATE" 된 TABLE의 갯수에따라 크게 다를수있다. 즉 하나의 큰 TABLE 이 매일 UPDATE 되면 항상 EXPORT 되고 UPDATE 가 안되면 FULL EXPORT 의 1% 정도 될 수있다. 따라서 이방법은 모든 TABLE 이 매 UPDATE 가 발생한면 비효율적인 방법이 된다.

< INCREMENTAL IMPORTING >

< 예 10 >
EXPORT 된 FILE 이 다음과 같다고 하면
F : full C : cumulative I : incremental
F1 I1 I2 C1 I3 I4 C2 I5 I6

a. 데이타베이스를 다시 생성한다.(CREATE DATABASE 문장을 사용)
b. IMPORT 하기위해 충분한 ROLLBACK SEGMENT 를 추가한다.
c. 가장최근 KEY 정보를 IMPORT 한다.
$ imp system/manager inctype=SYSTEM full=y file=I6
d. 가장최근 COMPLETE 로 EXPORT 한 FILE 을 IMPORT 한다.
$ imp system/manager inctype=RESTORE full=y file=F1
e. 모든 CUMULATIVE IMPORT 를 순서적으로 실행한다.
$ imp systme/manager inctype=restore full=y file=C1
$ imp systme/manager inctype=restore full=y file=C2
f. INCREMENTAL IMPORT 를 순서적으로 실행한다.
$ imp systme/manager inctype=restore full=y file=I5
$ imp systme/manager inctype=restore full=y file=I6

3. IMAGE BACKUP

IMAGE BACKUP 은 모든 데이타베이스 FILE 들을 다른 장소로 COPY 하는 것이다. 이 방법은 가장빠르고 안전한 방법이나 BACKUP 시점까지 데이타를 저장 할 수 밖에 없고, BACKUP 을 하는동안 데이타베이스를 DOWN 해야하는 문제가 있다. 그러나 IMAGE BACKUP 관련된 문제는 대부분 ARCHIVING 과 같이 사용하면 해결 될 수가 있다. 또한 IMAGE BACKUP 은 PERFORMANCE 를 고려하여 DISK 로 하는 것이 좋다. DATABASE 를 STARTUP 하고나서 TAPE 로 COPY 하면 DB 가 DOWN 되는 시간을 줄일수 있기때문이다.

a. 데이타베이스를 정상적으로(NORMAL) DOWN 한다.
b. 모든 DATA FILES, CONTROL FILES, ONLINE REDO LOG FILE 들을 OS 의
BACKUP UTILITY(UNIX 경우 cp, tar, cpio) 를 사용하 COPY 한다.

< 복구방법 >

데이타복구는 BACKUP 했던 시점까지 복구되고 이후변경된 데이타는
데이타베이스가 NOARCHIVELOG MODE 이면 복구 불가능 하다.
a. DISK 에있는 모든 데이타베이스 FILE (CONTROL,ONLINE REDO LOG, DATA)
을 삭제한다.
b. COLD BACKCUP 을 RESTORE 한다.
c. 데이타베이스를 STARTUP 한다.

4. ARCHIVING

ARCHIVING 은 주어진 기간동안에 데이타베이스에 발생한 변경사항을 모두 저장하는 방법이다. 따라서 IMAGE BACKUP 을 하고난 이후에 모든 ARCHIVE FILE을 가지고있다면 어떠한 데이타베이스 장애에도 최근에 변경된 내용까지 복구가 가능하다.

< 예 11 > 월요일 11pm : 전체 데이타베이스 IMAGE BACKUP
화요일 : ARCHIVE FILE 에 모든 변경된 내용이 저장
수요일 : 다음 IMAGE BACKUP 전에 DB 장애

<복구방법>

월요일 11pm IMAGE BACKUP 을 RESTORE 한다.
모든 ARCHIVE FILE 을 DB 에 적용한다.
ONLINE REDO LOG (아직 ARCHIVE 되지 않는내용)을 적용한다.

< 데이타베이스를 ARCHIVELOG MODE 로 전환하는 방법 >

a. ARCHIVELOG MODE 로 전환하기위해서 ONLINE REDO LOG FILE 이 3개이상 되어야한다.(ORACLE7 은 기본적으로 3 개)
ONLINE REDO LOG FILE 을 추가 하는 방법은 다음과 같다.(ORACLE 6경우)

```
$sqldba
SQLDBA > CONNECT INTERNAL
SQLDBA > STARTUP MOUNT
SQLDBA > ALTER DATABASE ADD LOGFILE '?/dbs/log3SID.dbf' SIZE 5M;
SQLDBA > ALTER DATABASE OPEN;
```

b. 자동으로 ARCHIVING 하기위해서 ?/dbs/initSID.ora FILE 에 다음내용을 추가한다.

```
LOG_ARCHIVE_START=TRUE
LOG_ARCHIVE_DEST=/arch/log   (/arch DIRECTORY 가 생성 되어야한다)
LOG_ARCHIVE_FORMAT=%s.arc
```

c. 데이타베이스를 MOUNT 한다

```
$ sqldba  lmode=y
SQLDBA > CONNECT INTERNAL
SQLDBA > STARTUP MOUNT
SQLDBA > ALTER DATABASE ARCHIVELOG
SQLDBA > ALTER DATABASE OPEN
```

(ARCHIVING 상황을 DISPLAY 하기위하여)

```
SQLDBA > ARCHIVE LOG LIST

   Database log mode              ARCHIVELOG
   Automatic archival             ENABLED
   Archive destination            /arch/log
   Oldest online log seq.         155
   Next log sequence to archive   156
   Current log Sequence           156
```

데이타베이스를 위와같이 ARCHIVELOG MODE 로 사용하면 ONLINE REDO LOG FILE이 LOG SWITCH 가 발생 할때 ARCHIVER(arch) PROCESS 는 CURRENT ONLNE REDO LOG FILE 을 /arch DIRECTORY 에 COPY 한

다. ARCHIVE FILE NAME은 log156.arc, log157.arc 등과 같이 LOG SEQUENCE NO 가 FILE NAME에 붙여진다. ARCHIVER PROCESS는 ORACLE BACKGROUND PROCESS 와 비동기적으로 작동 하기때문에 COPY 하는 동안에 DISK I/O 가 증가되는 현상은 발생하나 데이타베이스의 동작이 중단되는 현상은 발생하지 않는다.

< 복구 >
ARCHIVE FILE 을 사용하여 데이타베이스를 복구하는 경우는
. MEDIA FAILURE
. 데이타 FILE 이 DROP 되기전에 REMOVE 했을경우
. ORACLE BLOCK 이 CORRUPT 된경우
. ONLINE LOG FILE 이 CORRUPT 된경우 등이다.

복구방법으로는 COMPLETE MEDIA RECOVERY 와 INCOMPLETE MEDIA RECOVERY 등이있다.

방법 1 non-SYSTEM TABLESPACE 데이타 FILE 이 CORRUPT 된경우

CORRUT 된 DATA FILE 을 복구하거나, TABLESPACE 를 복구한다. TABLESPACE를 복구하는경우 TABLESPACE 에있는 모든 DATA FILE 을 복구한다.

a. CORRUPT 된 DATAFILE 을 삭제하고 BACKUP 된 FILE 을 RESTORE 한다.
$ tar xvf /dev/rmt0 user1.dbf

b. CORRUPT 된 DATAFILE 을 복구 한다.(TABLESPACE 복구는 생략)

데이타베이스를 STARTUP 하면 MOUNT 까지는 실행되지만 OPEN 할때
DATA FILE 의 LOG SEQUENCE 가 일치하지않기 때문에 OPEN 되지 않는다.

```
SQLDBA > STARTUP MOUNT
SQLDBA > RECOVER DATAFILE '/usr/oracle7/dbs/user1.dbf';

ORA-00279: Change 5035 generated at 05/05/94 16:10:00 needed for thread1
ORA-00289: Suggestion : /arch/log156.arc
ORA-00280: Change 5035 for thread 1 is in sequence # 156
Specify log : {=suggested ¦ filename ¦ AUTO ¦ FROM logsource ¦
              CANCEL ¦
```

(RETURN 을 하면 제시된 ARCHIVE LOG FILE 이 APPLY 한다.)

```
Applying suggested logfile...
```

(계속해서 제시된 ARCHIVE LOG FILE 로 APPLY 하면 최종적으로
"Media recovery complete" MESSAGE 가 나타나면서 복구가 완료된다.)

```
SQLDBA > ALTER DATABASE OPEN;
```

방법 2. SYSTEM TABLESPACE 데이타 FILE이 CORRUPT 된경우

a. CORRUPT 된 DATAFILE 을 삭제하고 BACKUP 된 FILE 을 RESTORE 한다.
   $ tar xvf /dev/rmt0 systORA7.dbf

b. 데이타베이스 전체를 복구 한다.

```
SQLDBA > STARTUP MOUNT
SQLDBA > RECOVER DATABASE;

ORA-00279: Change 5035 generated at 05/05/94 16:10:00 needed for thread1
ORA-00289: Suggestion : /arch/log156.arc
ORA-00280: Change 5035 for thread 1 is in sequence # 156
Specify log : {=suggested ¦ filename ¦ AUTO ¦ FROM logsource ¦
              CANCEL ¦
```

(RETURN 을 하면 제시된 ARCHIVE LOG FILE 이 APPLY 한다.)

```
Applying suggested logfile...
```

(계속해서 제시된 ARCHIVE LOG FILE 로 APPLY 하면 최종적으로
  "Media recovery complete" MESSAGE 가 나타나면서 복구가 완료된다.)

```
SQLDBA > ALTER DATABASE OPEN;
```

방법 3. CONTROL FILE 과 ONLINE REDOLOG FILE 복구방법

a. CONTROL FILE 은 서로다른 DISK 에 2 개이상 COPY 하여 운영하면
   하나의 FILE 이 CORRUPT 된경우 다른 CONTROL FILE 을 COPY 하여 사용한다.
b. ONLINE REDO LOG FILE 은 ORACLE7 에서는 MIRRORED ONLINE REDO LOG
   방법을 사용하여 운영하면 하나의 GROUP 에있는 모든 REDO LOG FILE
   이 CORRUPT 되는경우를 제외하고 데이타베이스 운영에 장애를 받지 않는다.

5. HOT BACKUP

데이타베이스를 DOWN 하지않고 IMAGE BACKUP 하는방법이다. 이방법은 COLD BACKUP 보다 더 어렵지만 데이타베이스를 OPEN 중에 할수 있고 또한 TABLESPACE 별로 BACKUP 을 할수있는 장점이있다.
HOT BACKUP 은 항상 데이타베이스를 ARCHIVELOG MODE상태에서 실시한다.

< 방법 > TABLESPACE 단위로 BACKUP 을 실시한다.

```
SQLDBA > CONNECT INTERNAL
SQLDBA > ALTER TABLESPACE SYSTEM BEGIN BACKUP;
```

(SYSTEM TABLESPACE 의 모든 DATAFILE 에대해서 OS BACKUP 한다)

```
$ tar cvf /dev/rmt0 /usr/oracle7/dbs/syst1ORA7.dbf
$ tar cvf /dev/rmt0 /usr/oracle7/dbs/syst2ORA7.dbf
```

```
SQLDBA > ALTER TABLESPACE SYSTEM END BACKUP;

SQLDBA > ALTER TABLESPACE USERS BEGIN BACKUP;

$ tar cvf /dev/rmt0 /usr/oracle7/dbs/user1ORA7.dbf
$ tar cvf /dev/rmt0 /usr/oracle7/dbs/user2ORA7.dbf

SQLDBA > ALTER TABLESPACE USERS END BACKUP;
```

(다 TABLESPACE 에 대해서도 같은 방법으로 한다)

주의: BEGIN 과 END 사이에는 해당 TABLESPACE 의 DATAFILE HEADER가 변경되지 않는다. 따라서 OS BACKUP 이 종료됨과 동시에 "ALTER TABLESPACE .. END BACKUP" COMMAND 를 실행하여 DATAFILE의 HEADER 가 변경되도록 한다.

복구방법은 ARCHIVING 경우의 RECOVERY 방법과 동일하다.

ORACLE7 INSTALL 후
데이타베이스 FILE 의 구성

Bulletin no : 10070

ORACLE7 INSTALL 이 끝나면 ORACLE7 이 작동되기위해서 필요한 기본적인 DATABASE FILE 즉 CONTROL FILE, LOG FILE, DATA FILE 들이생성 된다.

이러한 FILE 들은 UNIX 인경우 ORACLE_HOME/dbs 밑에 생기며 INSTALL 과정에서 변경도 가능하다. 이 FILE 들은 항상 정상적인 상태로 존재해야하고 TABLESPACE 및 DATA FILE 을 DROP,OFFLINE 하기전에 OS 상에서 REMOVE 해서는 안된다. 각 FILE 들의 NAME 과 사용 내역을 알기위해 ORACLE HOME DIRECTORY 와 ORACLE SID 를 확인해야 하는데ORACLE 환경 변수는 $ env 를 해보면 알수있다.

ORACLE_SID = ORA7
ORACLE_HOME = /user/oracle7

1. PARAMETER FILE
내용 : 각종 PARAMETER

FILE NAME : initORA7.ora

2. CONTROL FILE
내용 : DATABASE NAME
DB CREATTION TIMESTAMP
DATABASE 및 ONLINE LOG FILE의 이름, 위치
현재 LOG SEQUENCE NUMBER
CHECK POINT 정보

FILE NAME : ctrl1ORA7.ctl, ctrl2ORA7.ctl, ctrl3ORA7.ctl

지정된 FILE : config0RA7.ora
control_file = (/user/oracle7/dbs/ctrl1ORA7.ctl,
/user/oracle7/dbs/ctrl2ORA7.ctl,
/user/oracle7/dbs/ctrl3ORA7.ctl)

4. ONLINE LOGFILE
내용 : TRANSACTION ID
COLUMN ADDRESS
NEW DATA

FILE NAME : log1ORA7.dbf, log2ORA7.dbf, log3ORA7.dbf

5. DATA FILE

DATAFILE NAME	TABLESPACE	비고
systORA7.dbf	SYSTEM	DATA DICTIONARY TABLE,CATALOG TABLE 와 VIEW DB TRIGGER,PACKAGE,PROCEDURE,FUNCTION
rbsORA7.dbf	RBS	ROLLBACK SEGMENT (r01,r02,r03,r04)
toolORA7.dbf	TOOLS	ORACLE TOOLS(FORMS,REPORTWRITER 등)의 SOURCE 를 DB 에 저장하기위해 필요한 TABLE, VIEW 및 INDEX
tempORA7.dbf	TEMP	각종 SORT 을 위한 TEMPORARY 영역
usrORA7.dbf	USERS	USER 데이타

TABLESPACE 구성시 유의사항

a. SYSTEM TABLESPACE 에 생성되는 OBJECTS(TABLE,VIEW, etc) 의 수를 최소화 시킨다. 만일 새로운 TABLE 을 다른 TABLESPACE 에 생성을 하더라도 SYSTEM TABLESPACE 에 SPACE 가 없으면 그 TABLE 에대한 새로운 정보를 입력하지 못하기 때문에 TABLE 을 생성하지 못한다.
   또한 NON-SYSTEM TABLESPACE 에 다른 OBJECT 를 생성하면 DB 를 관리하기가 용이해진다. SYSTEM TABLESPACE 보다 NON-SYSTEM TABLESPACE 를 RECOVER 하기가 더쉽고 MEDIA FAILURE, TABLESPACE RESIZING 혹은 DATABASE FILE LAYOUT 의 TUNING 등이 보다 쉬어진다.
b. I/O 를 분산하기위해 TABLE 과 연관된 INDEX 를 분리한다.
   여러개의 DRIVE 에 I/O 가 분산되도록 DISK STRIPING 을 사용한다.
c. 많은 TABLE 및 INDEX 에 DATA 가 입력되고 DROP 이 되면 TABLESPACE 에 FRAGMENT 가 상당히 발생하여 SPACE를 효율적으로 사용하지 못한다.
   따라서 충분한 INITIAL 값으로 TABLE 을 생성하므로서 FRAGMENT 를 감소 시킬수 있다.

DB CREATION 및 TABLESPACE 생성 문장(?/dbs/crdb0RA7.sql, crdb20RA7.sql)

```
-------------------------- crdb0RA7.sql -------------------------------
REM * Start the  instance (ORACLE_SID here must be set to ).
REM *
connect internal
startup nomount pfile=/usr3/oracle7/dbs/init0RA7_0.ora

REM * Create the  database.
REM * SYSTEM tablespace configuration guidelines:
REM *   General-Purpose ORACLE RDBMS                5Mb
REM *   Additional dictionary for applications  10-50Mb
REM * Redo Log File configuration guidelines:
REM *   Use 3+ redo log files to relieve ``cannot allocate new log...'' waits.
REM *   Use ~100Kb per redo log file per connection to reduce checkpoints.
```

```
REM *
create database "ORA7"
    maxinstances 8
    maxlogfiles  32
    character set "US7ASCII"
    datafile
 '/usr3/oracle7/dbs/ctrl1ORA7.dbf'      size   25M
    logfile
 '/usr3/oracle7/dbs/ctrl1ORA7.dbf'      size 500k,
 '/usr3/oracle7/dbs/ctrl1ORA7.dbf'      size 500k,
 '/usr3/oracle7/dbs/ctrl1ORA7.dbf'      size 500k;

disconnect

------------------------------ crdb2ORA7.sql -----------------------
connect internal

REM # install data dictionary views:
@/usr3/oracle7/rdbms/admin/catalog.sql

REM * Create additional rollback segment in SYSTEM before creating tablespace.
REM *
connect internal
create rollback segment r0 tablespace system
storage (initial 16k next 16k minextents 2 maxextents 20);

REM * Use ALTER ROLLBACK SEGMENT ONLINE to put r0 online without shutting
REM * down and restarting the database.
REM *
alter rollback segment r0 online;

REM * Create a tablespace for rollback segments.
REM * Rollback segment configuration guidelines:
REM *   1 rollback segments for every 4 concurrent xactions.
REM *   No more than 50 rollback segments.
REM *   All rollback segments the same size.
REM *   Between 2 and 4 homogeneously-sized extents per rollback segment.
REM * Attempt to keep rollback segments to 4 extents.
REM *
create tablespace rbs datafile
 '/usr3/oracle7/dbs/ctrl1ORA7.dbf' size   4M
default storage (
 initial   128k
 next   128k
 pctincrease    0
 minextents    2
```

```
);

REM * Create a tablespace for temporary segments.
REM * Temporary tablespace configuration guidelines:
REM *   Initial and next extent sizes = k * SORT_AREA_SIZE, k in {1,2,3,...}.
REM *
create tablespace temp datafile
 '/usr3/oracle7/dbs/ctrl1ORA7.dbf' size   550k
default storage (
 initial      256k
 next         256k
        pctincrease  0
        optimal      1M
);

REM * Create a tablespace for database tools.
REM *
create tablespace tools datafile
 '/usr3/oracle7/dbs/ctrl1ORA7.dbf'      size   15M;

REM * Create a tablespace for miscellaneous database user activity.
REM *
create tablespace users datafile
 '/usr3/oracle7/dbs/ctrl1ORA7.dbf'      size   1M;

REM * Create rollback segments.
REM *
create rollback segment r01 tablespace rbs;
create rollback segment r02 tablespace rbs;
create rollback segment r03 tablespace rbs;
create rollback segment r04 tablespace rbs;

REM * Use ALTER ROLLBACK SEGMENT ONLINE to put rollback segments online
REM * without shutting down and restarting the database.  Only put one
REM * of the rollback segments online at this time so that it will always
REM * be the one used.  When the user shuts down the database and starts
REM * it up with initSID.ora, all four will be brought online.
REM *
alter rollback segment r01 online;
REM * alter rollback segment r02 online;
REM * alter rollback segment r03 online;
REM * alter rollback segment r04 online;

REM * Since we've created and brought online 2 more rollback segments,
REM * we no longer need the second rollback segment in the SYSTEM tablespace.
alter rollback segment r0 offline;
```

```
drop rollback segment r0;

REM * Alter SYS and SYSTEM users.
REM *
alter user sys temporary tablespace temp;

alter user system default tablespace tools temporary tablespace temp;

REM * For each DBA user, run DBA synonyms SQL script.  Don't forget that EACH
REM * DBA USER created in the future needs dba_syn.sql run from its account.
REM *
connect system/manager
@/usr3/oracle7/rdbms/admin/catdbsyn.sql
```

ORACLE7 에서 USER 등록

Bulletin no : 10068

단계 1 USER 가 사용하고자하는 TABLESPACE 생성 한다. 만일 존재하는 TABLESPACE 를 사용하려면 이 단계는 생략한다.

```
$ sqldba lmode=Y
SQLDBA> CONNECT INTERNAL
      > CREATE TABLESPACE data_ts
        DATAFILE '/usr/oracle/dbs/data_ts.dbf' SIZE 50M;
```

단계 2 USER 를 생성하면서 DEFAULT TABLESPACE,TEMPORARY TABLESPACE 등을 지정하고 각 TABLESPACE에 USER 가 사용 할수 있는 QUOTA 를 할당 한다.
제한없이 사용하고자 할때는 UNLIMITED 로 한다.

```
SQLDBA > CREATE USER user1 IDENTIFIED BY user1_pass
         DEFAULT TABLESPACE data_ts
         TEMPORARY TABLESPACE temp
         QUOTA UNLIMITED ON data_ts
         QUOTA UNLIMITED ON temp;
```

단계 3 USER 에게 ROLE 을 GRANT 한다.(V6 에서는 USER 가 TABLE 생성 및 ACESS 를 하려면 RESOURCE, CONNECT 권한을 부여하였으나 V7 에서는 CONNECT ROLE 만 부여하여도 된다.)

```
SQLDBA > GRANT CONNECT TO user1;
```

단계 4 DB 에 CONNECT 하여 사용한다.

```
sqlplus user1/user1_pass
SQL > CREATE TABLE test(col1 VARCHAR2(2)
                        col2 NUMBER);
```

위와 같이 TABLESPACE 를 생략하고 TABLE 을 생성하는 경우 USER 의
DEFAULT TABLESPACE 인 data_ts 에 TABLE 이 생성된다.

ORACLE의 MEMORY 사용량에 대한 테스트

Bulletin no : 10175

==========
ORACLE 7.1 의 MEMORY 요구사항
==========

오라클의 메모리 사용량은 시스템에 따라서 다르지만 큰 차이는 없으므로 여기서는 SEQUENT 기종에서의 테스트 결과를 알아보도록 한다. 사용된 시스템은 각각 128MB(MAINMACHINE), 64MB(SUBMACHINE FOR REMOTE CONNECT) 의 MAIN MEMORY 를 갖는 SEQUENT 기종이다.
RDBMS VERSION 은 7.1.2 이고 SQL*NET TCP/IP 2.1 을 사용하였다.

TEST 절차 및 측정결과

1)listener 기동 : 약 800 KB 소요

2)ORACLE startup : 약 6 MB 소요
kernel back ground process 가 SMON, PMON, DBWR, LGWR, RECO, ARCH 등 총 6 개가 fork 될때의 경우이고, 만약 dispatcher 나 Shared server를 하나 더 띄우면 하나당 약 1MB 가 더 소요 된다.

3)sqlplus scott/tiger 경우 : 약 2MB 소요
이 경우는 sqlplus 명령어 자체와 Dedicated ORACLE Server 1개가 동시에 기동되어 소요되는 양을 합친값

4)sqlplus scott/tiger@t:MAINMACHINE:ORA7 경우 : 약 1.5 MB 소요
이 경우는 sqlplus 명령어 자체만이 소요하는 메모리 양을 측정하기 위한 것임. 위의 3) 경우와 조합하면 sqlplus 는 1.5 MB , Server는 0.5 MB 소요한다는 결론이 나오는데 Sever가 500 KB 만 소요한다는 것은 Pipe를 사 용하여 연결되기 때문이다.

5)SUBMACHINE 에서 sqlplus scott/tiger@MAINMACHINE 경우 : 0 MB
이 경우는 SUBMACHINE에서 MAINMACHINE으로 SQL*Net V2 MTS를 이용하여 연결 경우 인데 Main test machine 인 MAINMACHINE에서 아무 프로세서도 추가로 기동되지 않고 다만 기존의 이미 기동된 dispatcher 를 통해 연결된다.

결론적으로 MTS로 연결할 때에는 이미 기동된 dispatcher와 shared server가 이를 흡수하므로 하나의 dispatcher와 server 가 각각 20 명의 Client를 감당한다고 하면

dispatcher : 1MB/20 = 50KB,
shared server : 1MB/20 = 50KB

이므로 사용자 1명당 100KB 정도로 계산하면 된다. 사용자 수가 많아지면 SGA Size 조정이 필요하므로 이점을 감안하여 전체 메모리 소요량을 산정해 야 한다.

6)SUBMACHINE에서 sqlplus scott/tiger@MAINMACHINEd 경우: 약 1 MB 소요
이 경우는 SQL*Net V2를 이용하되 dedicated server를 사용하여 MAINMACHINE에 연결된 경우 이다.

Option 으로 "(SRVR=DEDICATED)" 를 (SID=) 다음에 추가하면 SQL*Net V1 처럼 사용할 수 있다.

전체 메모리 소요량 산정 예 : 동시 사용자 100 명을 가정

---

1)Multi-thread Shared Server 사용하여 연결 : Best case

- SGA : 30 MB
- Kernel Process : 약 7MB (listener 포함)
- dispatcher 및 shared server : 10 MB (100KB * 100명)
- 총 합계 : 약 47 MB

2)Dedicated Server 사용하여 연결 : Worst case

- SGA : 30 MB
- Kernel Process : 약 7 MB (listener 포함)
- Dedicated Server : 100MB (1MB * 100명)
- 총 합계 : 약 137 MB

위의 두 경우는 Client/Server 경우 Server의 메모리 소요량이고 Host base 인 경우에는 해당 Tool 의 메모리 소요량도 같이 계산해야 한다.

OS 명령으로 DATA FILE 을 삭제한 경우 - ORA-1157,1110

Bulletin no : 10088

DATABASE RECOVERY에 앞서 ORACLE INSTANCE(즉, ORACLE RDBMS)의 STARTUP 단계를 우선 살펴보기로 하자.

첫번째 단계로 INSTANCE를 START시키며, 여기서는 initORACLE_SID.ora 화일의 파라미터를 참조하여 SGA(SYSTEM GLOBAL AREA)를 할당하고 백그라운드 프로세스를 START 시킨다.

두번째 단계로 DATABASE의 MOUNT이며 파라미터 화일에 명시된 CONTROL FILE을 오픈한다. CONTROL FILE로부터 DATABASE NAME과 REDO LOG FILE의 이름을 읽는다.

세번째 단계로 CONTROL FILE내의 정보를 이용하여 모든 데이타 화일을 오픈한다.

```
SQLDBA> CONNECT INTERNAL;
Connected.
SQLDBA> STARTUP;
ORACLE instance started.
Database mounted.
Database opened.
Total System Global Area    1913196 bytes
              Fixed Size      27764 bytes
           Variable Size    1787128 bytes
         Database Buffers     65536 bytes
             Redo Buffers     32768 bytes
```

RDBMS의 STARTUP시 문제의 데이타 화일이 CONTROL FILE 정보에서는 존재하지만 실제로 O/S상에서는 존재하지 않으므로 DATABASE OPEN 단계에서 삭제된 데이타 화일을 OPEN 할 수 없다. 따라서 다음과 같은 데이타 화일 오픈에 관련된 에러가 발생된다 :

```
SQLDBA> STARTUP;
        ORACLE instance started
        Database mounted
        ORA-01157 : cannot identify data file 11 - file not found
        ORA-01110 : data file 11 : '/user1/oracle7/dbs/user2.dbf'
        Attempting to dismount database .... Database dismounted
        Attempting to shutdown instance .... ORACLE instance shut down
```

DATABASE OPEN 단계에서 CONTROL FILE에서는 ORA-1157 에러에서 나타난 11번 데이타 화일이 존재하는 것으로 인식하지만 실제로 O/S상의 데이타 파일
(ORA-1110 에러에 명시된 '/user1/oracle7/dbs/user2.dbf' 화일)이 삭제된 상태이다.

이러한 경우에는 DATABASE STARTUP시 STARTUP MOUNT 단계까지 실행한 후, 문제의 데이타 화일을 OFFLINE 시킨 다 데이타베이스를 오픈한다. 단 데이타베이스 오픈이 정상적으로 수행되면 문제가 발생한 데이타 화일을 포함하고 있는 TABLESPACE를 DROP하지 않을 경우에는 DATABASE STARTUP시 항상 데이타 화일의 오픈 단계에서 에러가 발생된다. 따라서, 문제의 데이타 화일의 OFFLINE과 TABLESPACE의 DROP전에 반드시 해당 TABLESPACE를 사용하고 있는 USER의 데이타 백업을 수행해야 한다.

데이타 화일의 OFFLINE과 관련된 명령은 다음과 같다.
SQLDBA를 COMMAND LINE MODE로 기동시킨다.

```
$ sqldba lmode=y
SQLDBA> CONNECT INTERNAL;
SQLDBA> STARTUP MOUNT;
        ORACLE instance started.
        Database mounted.
SQLDBA> ALTER DATABASE DATAFILE '/user1/oracle7/dbs/user2.dbf'
        OFFLINE DROP;
        Statement processed.
SQLDBA> ALTER DATABASE OPEN;
        Statement processed.
SQLDBA> DROP TABLESPACE tablespace_name INCLUDING CONTENTS;
        Statement
```

정상적으로 DATABASE가 오픈된 후 CONTROL FILE로 부터의 데이타베이스 정보를 갖는 DATA DICTIONARY TABLE인 V$DATAFILE(SYS USER에서 액세스 가능)의 내용과 데이타베이스 화일에 관한 정보를 가지고 있는 DATA DICTIONARY VIEW인 DBA_DATA_FILES(SYSTEM USER)을 조회하면 아래와 같은 내용을 확인할 수 있다 :

(1) SQL> SELECT * FROM V$DATAFILE ;

FILE#	STATUS	NAME
9	ONLINE	/user1/oracle7/dbs/tools.dbf
10	ONLINE	/user1/oracle7/dbs/user1.dbf
11	RECOVER	/user1/oracle7/dbs/user2.dbf

(2) SQL> SELECT * FROM DBA_DATA_FILES ;

FILE_NAME	FILE_ID	TABLESPACE_NAME	STATUS
/user1/oracle7/dbs/tools.dbf	9	TOOLS	AVAILABLE
/user1/oracle7/dbs/user1.dbf	10	TEST	AVAILABLE
/user1/oracle7/dbs/user2.dbf	11	TEST	AVAILABLE

OUTER JOIN의 UIDE LINE

Bulletin no : 10398

---

1. 개념

1.1 Outer Join의 정의

다음의 용어에 대해 우선 살펴보자 :

'outer-join column' - a column reference followed by the
symbol (+), e.g. EMPNO(+) and
DEPT.DEPTNO(+) are outer join columns

'simple predicate' - a logical expression containing no
AND's, OR's, or NOT's (usually a
simple relation such as A = B)

'outer join predicate' - a simple predicate containing one
or more outer join columns

2. OUTER JOIN 사용법 -RULES

An outer join predicate may only contain outer join columns from one table (in other words, all outer join columns in a single outer join predicate must belong to the same table). This means, for example, that the following statement is illegal :-

```
EMP.EMPNO(+) = DEPT.DEPTNO(+)  -  outer join columns from
                                  two tables
```

Also, if a column in a predicate is an outer join column, then all columns from the same table must be outer join columns in that predicate. This means, for example, that the following statement is illegal :-

```
EMP.SAL + EMP.COMM(+) = SALGRADE.HIGH  - mixed columns from
                                         one table
```

In a predicate, the table referenced with a (+) is directly 'outer joined' to all other tables in the predicate. It is indirectly 'outer joined' to any tables to which these other tables are themselves 'outer joined'. A predicate may not be directly or indirectly 'outer joined' to itself. This means, for example, that the following combination of predictes is illegal :-

```
      EMP.EMPNO(+)    = PERS.EMPNO
AND   PERS.DEPTNO(+) = DEPT.DEPTNO
AND   DEPT.JOB(+)     = EMP.JOB          - circular outer
                                           join relationship
```

3. OUTER JOIN EXECUTION

For a given table, T, there may be both outer join and non-outer join predicates. Execution occurs (conceptually) as follows :-

1. The result of joining all tables mentioned in table T's outer join predicates is formed (by recursive application of this algorithm).

2. For each row of the result, a set of composite rows is formed, each consisting of the original row in the result joined to a row in table T for which the composite row satisfies all of table T's outer join predicates.

3. If a set of composite rows is the null set, a composite row is created consisting of the original row in the result joined to a row similar to those in table T, but with all values set to null.

4. Rows that do not pass the non-outer join predicates are removed.

This may be summarised as follows. Outer join predicates (those with (+) after a column of table T), are evaluated BEFORE table T is augmented with a null row. The null row is added only if there are NO rows in table T that satisfy the outer join predicates. Non-outer join predicates are evaluated AFTER table T is augmented with a null row (if needed)

4 OUTER JOIN - RECOMMENDATIONS

Certain types of outer joins in complicated logical expressions may not be well formed. In general, outer join columns in predicates that are branches of an OR should be avoided. Inconsistancies between the branches of the OR can result in an ambiguous query, and this may not be detected. It is best to confine outer join columns to the top level of the 'where' clause, or to nested AND's only.

5 OUTER JOIN - ILLUSTRATIVE EXAMPLES

5.1 Simple Outer Join

```
SELECT ENAME, LOC
FROM   DEPT, EMP
WHERE  DEPT.DEPTNO = EMP.DEPTNO(+)
```

The predicate is evaluated BEFORE null augmentation. If there is a DEPT row for which there are no EMP rows, then a null EMP row is concatenated to the DEPT row.

5.2 Outer Join With Simple Post-Join Predicates

```
SELECT ENAME, LOC
FROM   DEPT, EMP
WHERE  DEPT.DEPTNO = EMP.DEPTNO(+)
AND    EMP.DEPTNO IS NULL
```

The second simple predicate is avaluated AFTER null augmentation, since there is no (+), removing rows which were not the result of null augmentation and hence leaving only DEPT rows for which there was no corresponding EMP row.

5.3 Outer Join With Additional Pre-Join Predicates

```
SELECT ENAME, LOC
FROM   DEPT, EMP
WHERE  DEPT.DEPTNO = EMP.DEPTNO(+)
AND    'CLERK'     = EMP.JOB(+)
AND    EMP.DEPTNO IS NULL
```

The predicate on EMP.JOB is evaluated at the same time as the one on EMP.DEPTNO - before null augmentation. As a result, a null row is augmented to any DEPT row for which there are no corresponding clerks's in the EMP table. Therefore, this query displays departments containing no clerks.

Note that it the (+) were omitted from the EMP.JOB predicate, no rows would be returned. In this case, both the EMP.JOB and EMP.DEPTNO IS NULL predicates are evaluated AFETR the outer join, and there can be no rows for which both are true.

PARALLEL LOAD - SPACE USAGE

Bulletin no : 10052

I have put together a short summary of how Parallel Load handles storage allocation in 7.1 which I am going to send to a customer.
This could also quickly turn into a bulletin or PR entry (if one doesn't already exist).
I'd appreciate if people could review and give corrections/enhancements.
Here is how storage allocation works with parallel loads.

For each parallel loader session, Oracle

1) creates a temporary segment with the INITIAL and NEXT from the table that is being loaded, and in the datafile designated by the FILE option.

2) When a load is complete, the final extent used by that parallel load is trimmed back to the last block which it used.

3) Lastly, the extents belonging to the temporary segments are then converted to extents belonging to the table being loaded.

This creates a couple of problems:

* If your INITIAL and NEXT are different for the table, then you will end up with multiple extents sized the same as INITIAL, which doesn't follow general table growth patterns.

* Extent sizes are not predictable for the last extent of each load. This can lead to tablespace fragmentation.

What to do about this?

* Size INITIAL and NEXT to be the same.
  This will eliminate the first problem listed.

* Oracle 7.2 will have a feature in to specify the storage parameters used by the parallel load (bug 187319):

```
OPTIONS (STORAGE=(MINEXTENTS  n
                  MAXEXTENTS  n
                  INITIAL     n  [K¦M]
                  NEXT        n  [K¦M]
                  PCTINCREASE n))
```

* Enhancement Request 236552 has been filed to allow the user to select no trimming of the extents.

(V7.3) PARTITION VIEWS IN ORACLE7 RELEASE 7.3

Bulletin no : 10234

Introduction

Partition views are a new feature in Release 7.3 design to provide better support for very large tables that are commonly found in data warehousing environments.

The partition view feature offers two primary benefits:
- improved managability and availability of large tables
- improved performance for table scans on certain queries of these large tables

What is a partion view?

An example partition view is:

```
create view sales as
  select * from jan_sales
union all
  select * from feb_sales
...
union all
  select * from dec_sales
```

Each of the base tables (the monthly sales tables) *must* be identical in terms of column names, column datatypes, and indexes. These tables *must* also have a CHECK constraint on its partitioning column (thus, the jan_sales table must have a check constraint on the date column which constrains the date to fall between Jan 1 and Jan 31). All of these base tables, indexes, constraints as well as the UNION-ALL view definition are be created by the DBA.

Managability and availability benefits

A partition view greatly simplifies the administration of very large tables.

Consider the example of of a data warehousing containing a large 'sales' table. Once per month, the DBA loads all of new sales data into this table. Thus, the DBA would need to drop all of the indexes on the sales, load the new data, and rebuild the indexes. Since the sales table is very large, this could be a lengthy operation. Morever, the sales table is (for most practical DSS application) is not available while these load and index operations are occuring.

Using the partition view feature, the DBA could load the new month's data into a separate partition and build indexes on this new partition without impacting the original partition view. Then, after the new partition is entirely built and indexed, the DBA could recreate the UNION-ALL view to include the new partition. The sales partition view is unavailable for a very short length of time ... only while the UNION-ALL view is being built. Moreover, because the indexes are much smaller, the length of time to load and index a new month's worth of data is dramatically decreased.

Performance benefits of partition views

Any UNION-ALL view (even in earlier releases of Oracle7) can reap the aforementioned managability benefits; however, unless the UNION-ALL view offers reasonable query performance, the managability benefits are useless.
The enhancement in Release 7.3 is to insure that the query performance on UNION-ALL views will at least equivalent to (and, in many cases, much better than) single-table access. Note that these performance enhancements are only effective when all of the partitions have the appropriate CHECK constraints and when all of the partitions have
identical column definitions and indexes.

There are two basic performance enhancements for partition views:
  partition elimination
  parallel execution of UNION-ALL views
Certain queries may not require data from all of the partitions of a view partition. For example, consider the following query:
```
  select sum(revenues) from sales
  where sales_date between '15-JAN-96' and '15-FEB-96';
```

With 7.3's new support for partition views, Oracle will evaluate the above query using *only* the January partition and the February partition; the remaining ten partitions will not be accessed. This feature is commonly called 'partition elimination'. Partition elimination is only effective when querying based on the partitioning column; in this example, the partitioning column is the
sales_date column. But the performance savings can be significant. In the previous example, the partition view feature results in ten of the twelve partitions being eliminated from query processing. This could represent six-fold performance gain.

An additional enhancement in 7.3 is the parallel execution of UNION-ALL views. All queries on UNION-ALL views can be executed in parallel (when using the Parallel Query Option). It is *very*
important to note that the partitio ning scheme is absolutely independent of the degree of parallelism (this starkly contrasts with many of our competitior's parallel query architecture, in which the physical data partitioning determines the degree of parallelism). Oracle will dynamically distribute the data of a UNION ALL view across all parallel query processes, and partition elimintation will not impact the degree of parallelism.

Limitations of Partition Views

Partition views do not support DML operations. For this reason, partition views are most appropriate for read-only applications (such as data warehouses).

Conclusion

Partition views can be very effective for handling very large tables in data warehousing environments. The managability of these large tables is vastly improved, with significant performance improvements for many queries.

PRIMARY KEY 재생성 방법

Bulletin no : 10232

---

1. PRIMARY KEY DROP

```
SQL> alter table EMP drop primary key;
```

2. PRIMARY KEY 생성

```
SQL> alter table EMP add constraint EMP_PK primary key(EMPNO)
     using index storage(initial 1m next 1m pctincrease 0)
     tablespace USERS;
```

PRIMARY KEY를 REFERENCE 하는 FOREIGN KEY 찾는 SQL

Bulletin no : 10314

```
$ sqlplus sys/manager(SYS user 로 login 함)
sql> select  c.name  constraint_name
     from dba_objects a,
          cdef$       b,
          con$        c
    where a.object_name   = 'TABLE_NAME'
      and a.object_id = b.robj#
      and b.con#      = c.con#
/
```

PROC 의 실행화일 크기를 줄이는 방법

Bulletin no : 10242

- Oracle Server 7.2 이상에서는 Shared Library 를 제공하므로 이것을 사용하여 Compile 하면 실행화일의 크기를 줄일수 있다.

[실행방법]

1. Pro*C 2.X 의 경우

```
$ cd $ORACLE_HOME/proc/demo
$ make -f proc.mk sample1 ORA_CLIENT_LIB=shared
  또는
$ make -f proc.mk EXE=xyz OBJS="xyz.o" ORA_CLIENT_LIB=shared
```

2. Pro*C 1.6 의 경우

```
$ cd $ORCACLE_HOME/proc16/demo
$ make -f proc16.mk sample1 ORA_CLIENT_LIB=shared
```

ORACLE SERVER AND UNIX TRANSACTION PROCESSING(TP) MONITOR-2

Bulletin no : 10022

This file contains commonly asked questions about Oracle7 Server and UNIX Transaction Processing Monitors (TPMs). The topics covered in this article are

- o Oracle Parallel Server and TP Monitors
- o Oracle and DCE-based TP Monitors
- o Other commonly asked questions

The questions answered in part 3 provide additional detail to the information provided in part 1.

Oracle Parallel Server and TP Monitors

How does Oracle Parallel Server (OPS) work with TP Monitors?

If you are using Oracle-managed transactions, there are no special considerations. But if you are using TPM-managed transactions, and thus need to use the XA interface, then Oracle requires release 7.1.3 or later and a special version of the Distributed Lock Manager, called the session-based lock manager. This version of the DLM is not yet available for all platforms. To understand this restriction, let's take a look at one of the technical details of XA.

The XA specification requires that the Resource Manager be able to move a transaction from one process to another, and even to be able to commit in a separate process. In Oracle, transactions are attached to sessions, so that means that we also have to be able to move sessions. Therefore, the session/transaction can't have any state which is tied to a particular process. The first generation distributed lock managers were all built to use the process id as the lock owner, which doesn't work for locks which need to move with the transaction.

Oracle and DCE-based TP Monitors

How does Oracle interface to the Encina TP monitor? To CICS/6000? I've heard that they require OSF DCE facilities in order to run?

Oracle interfaces to Encina and CICS/6000 just as it does to any other TP Monitor. The TP Monitor issues XA commands to control transactions, and Oracle executes the commands. Encina

and CICS/6000 do use DCE features for their own operation. However, this use is transparent to the Oracle Server.

What DCE facilities can Oracle products take advantage of when working with a DCE-based TP Monitor?

The two most commonly mentioned DCE features which might be useful to Oracle users are multi-threading and security. We look at these in the subsequent questions in this section.

Encina documentation suggests that a Resource Manager such as Oracle can be either single-threaded or multi-threaded? Which way is Oracle XA implemented?

The Oracle XA implementation is single-threaded, as is any Oracle client.
Within a single process, at most one thread can access Oracle at a time.

Does that mean that only a single Encina application can access an instance of Oracle transactionally at any given moment?

No. Oracle XA is only single-threaded within a single application server process. Multiple applications can access Oracle simultaneously using XA by using different application processes. Encina allows

(1) serial reuse of a single server by different clients. There are two options for this. The server can use long term reservation but be defined to be in shared or concurrent access mode, which allows the server to be used by another client as soon as an RPC completes. Alternatively, the server can use default reservation and exclusive mode, which allows the server to be used by another client as soon as the current transaction ends.

(2) concurrent execution by multiple servers, even if they are accessing the same Oracle database. These may be executing the same or different procedures.

These two features should let you get as much concurrency as you need.

Why isn't the Oracle XA library multi-threaded?

The XA specification specifically states that its use of the phrase "thread of control" means a process. If an RM were to multi-thread its XA, it would be in violation of the specification. This restriction was put place in because at the time the specification was written, there were numerous thread packages: if the TM used one, the application another, and perhaps the RM yet a third, there's no way it could work. As threads standards settle down, the later versions of XA will probably relax this restriction.

Will Oracle change if the XA specification changes?

Very likely. The exact time frame will of course depend on the priority of all work items at that time.

Does Oracle use DCE security via the TP Monitors?

The integrity of the connection between a DCE TP Monitor client and DCE TP Monitor server is protected by the DCE security functionality. Theoretically, the TP Monitor could make the DCE-protected client security information available to Oracle. Unfortunately, there's no standard way for a TP Monitor to pass security information information to a Resource Manager such as Oracle. Oracle is leading an effort to extend the X/Open model to allow use of the security information provided by the Monitor. In the meantime, the basic DCE security features such as encryption are useful within TP Monitors.

Effective use of DCE security would normally also mean that the security of the TP Monitor client be passed through the TP Monitor, through the Oracle client (application server), to the Oracle Server, and possibly on to other Oracle Servers through database links. The ability to transfer security information to other processes, called delegation, is missing in DCE version 1.0. DCE version 1.1, expected to emerge in late 1994, has some delegation features. Oracle is examining these features to see how they might be used.

Are there any special considerations for CICS/6000?

There are two:

(1) It is inefficient to run without XA. CICS/6000 is designed to use XA. It uses XA so that the CICS server can log on to Oracle when it starts, after which it makes that Oracle connection available to any transaction it executes. If you don't use XA, the CICS server does not itself log on to Oracle so each transaction has to log on and log off - a very expensive mode of operation. Also, it is very un-cics-like in that the application does the log{on,off} and also commits - in a mainframe CICS database program CICS would implicitly do these operations. Oracle does not recommend this mode because of the performance penalty.

(2) CICS servers are generic and dynamically load application modules.
In order for these modules to access the Oracle connection made by CICS, the applications must be built with a shared object version of the Oracle libraries. This is an installation option on platforms which support CICS/6000 and other products using its architecture such as CICS 9000.

Other commonly asked questions

What other Resource Managers can be included in an Oracle XA transaction?

Several other relational database vendors have an XA implementation available or in progress. There is an XA C-ISAM product from Gresham Telecomputing. There are also Resource Managers contained within some of the TP Monitors which can be coordinated in the same transaction. For example, CICS/6000 has VSAM files and other data stores, Encina has its RQS queuing system, and Tuxedo has its /Q queuing system.

What is Recoverable Queuing Service (RQS) and how does it interoperate with Oracle7 and Encina? What about /Q?

Recoverable Queuing Service is a feature provided by Encina which allows transactional, distributed queuing (enqueue/dequeue). Tuxedo has a similar product called /Q. Because these products are themselves coordinated by the TM component of the TP Monitor, their queue operations are atomically coordinated with with operations on XA Resource Managers such as Oracle7 Server. That is, they can atomically put something on one of their queues and commit an Oracle transaction, then at some later time dequeue an entry atomically with doing some other Oracle transaction. The queue system guarantees that the message will not be lost or transmitted twice.

Can I mix TP Monitor applications with standard Oracle7 Server applications?

Yes, you can have existing Oracle applications connected to the database with alongside TPM applications against the same database. The TPM does not manage the whole database, just those transactions which are started by the TPM. The Oracle Server will properly handle concurrency control between the transactions managed by itself and those managed by the TPM.

Is Oracle planning to change its tools to be more suitable for TP Monitors?

With Oracle Procedure Builder 1.5, to be available with CDE2, Oracle will provide a foreign function interface that allows you to dynamically set up PL/SQL calls that access C functions. In other words, you can access C routines in Windows DLLs from within your PL/SQL procedures. This will allow PL/SQL under Windows easy access to TP Monitor APIs.

Does Oracle7 Server itself use XA-compliant TPMs as the interface to foreign RMs?

No, for this purpose Oracle Server uses the SQL*Connect products or the new Transparent and Procedural Gateway products.

Does Oracle7 Server use XA to coordinate Oracle7-only distributed transactions?

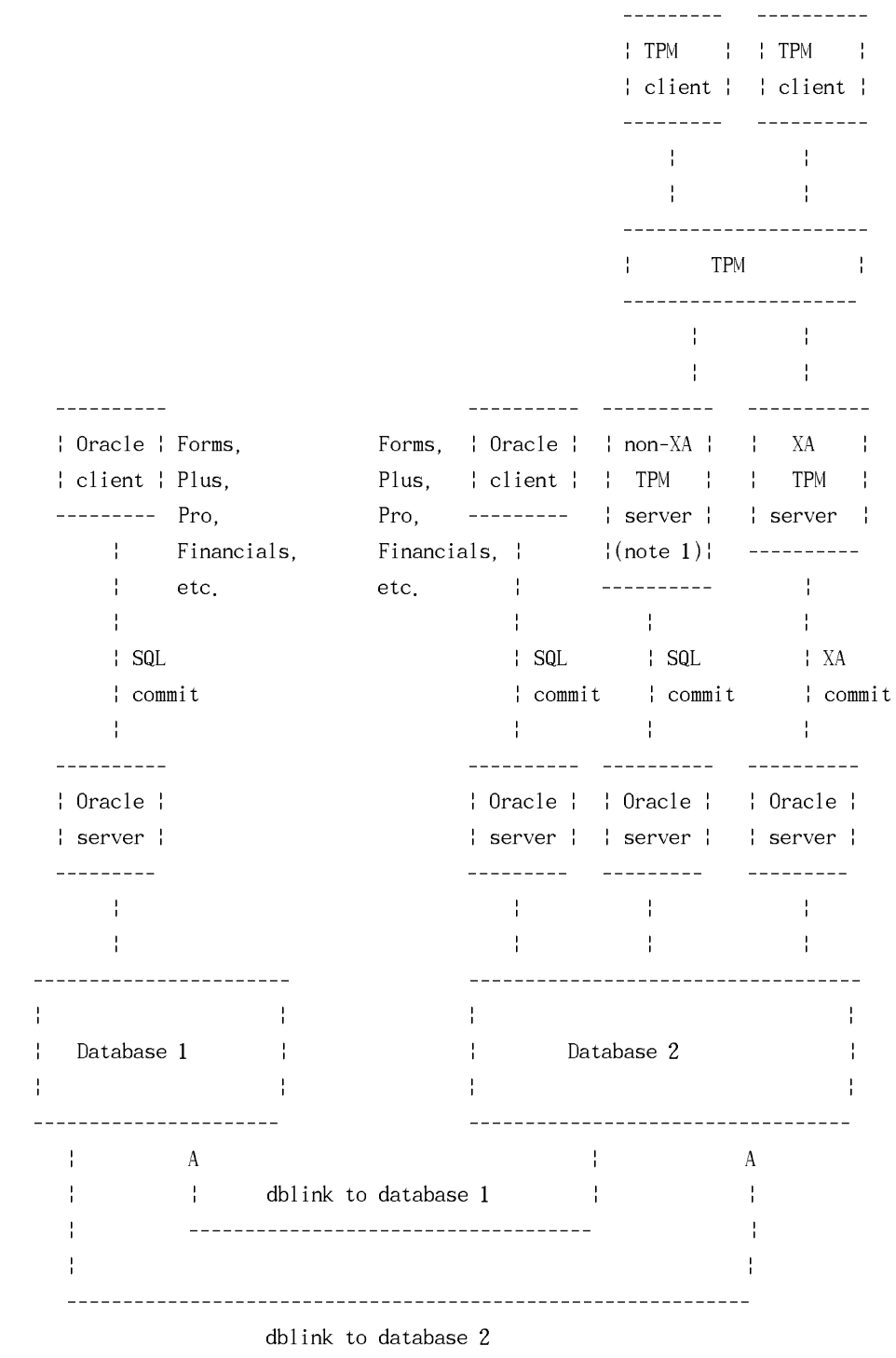
No, it uses an internal mechanism.

Can database links be used with XA?

If an Oracle7 database is running under XA, it can access other Oracle7 databases through database links, with some restrictions. First, the access to the other database must use SQL*Net V2 and be running MTS. Second, it must currently be to another Oracle7 database. Assuming those restrictions, the Oracle 7 database can do distributed update to another Oracle 7 database by using a database link, whether it is started by an Oracle application or a TP Monitor application. The TPM will see Oracle as only a single RM, but Oracle7 will propagate all the transaction commands to the other database, including the two-phase commit. If the transaction is started by a TP Monitor application and is using XA, it can also update

non-Oracle resources managed by the TPM. If it is started from an Oracle application, it can only include resources managed by Oracle.

Here's a sample configuration:



Note 1: Oracle will work having both XA and non-XA servers but some TPMs may have restrictions on this.

Are multiple direct connections possible from a Pro* program?

Using XA, you can not only specify multiple direct connections to Oracle7 databases, you can also update them both in the SAME transaction. The way to do this is to use a precompiler feature called a named database. When you use a named database, you qualify the SQL statement with the database name. For example, you write EXEC SQL AT dbname UPDATE emp ....

We have a complementary feature in the xa open string to let the user associate the name with a particular RM instance, called the DB clause. You will also want to use the SqlNet clause in the open string so you can give the two different SIDs. This clause does not require the use of the SQL*Net product, it is just a naming convention. For more information, see Oracle7 Server for UNIX Administrator's Reference Guide.

Some TP Monitors may not support having multiple Resource Mangers in the same server; check with the TPM vendor.

Is there any collateral available for XA or TP Monitors?

Oracle At Work	52684.0692
Oracle7 Server for UNIX Administrator's Reference Guide	#A10324-1
Guide to Oracle's Products and Services	#A10560
Oracle7 Server and CICS/6000	#A14200

Where can I get more information on the DTP model?

X/Open's address is

X/Open company Ltd (Publications)
P O Box 109
Penn
High Wycombe
Bucks HP10 8NP

Tel: +44 (0)494 813844
Fax: +44 (0)494 814989

Request

G307 Distributed Transaction Processing: Reference Model Version 2
X/Open Guide G307 ISBN 1-859120-19-9 28cm.44p.pbk.220g.11/93

This file contains commonly asked questions about Oracle Server and UNIX Transaction Processing Monitors (TPMs). The topics covered in this article are

- Performance with Oracle Server and TP monitors
- Performance using Oracle's XA Library

The questions answered in part 4 provide additional detail to the information provided in part 1.

Performance with Oracle Server and TP Monitors

I have heard that Transaction Processing Monitors (TPMs) will increase Oracle Server performance. Is this true?

Several hardware and TPM vendors have made the claim that TPMs will increase RDBMS performance. This claim is based on TPC-A benchmarks. The key point to understand about TPC-A is that it requires, for every transaction-per-second, ten times that many users to be connected. For example, to get 600 TPS, you need 6000 users. The next question will answer in more detail how the the three-tier architecture addresses this requirement, but first let's look more generally at what TP Monitors can and can't do to improve performance.

TP Monitors can provide better performance:

(1) When there are more than several hundred users connected.
This is because of the TP Monitor's role in the three-tier architecture, described in the next question. In this architecture, terminal handling is offloaded to one or more separate machines, freeing up those cycles to do database work. Note that this does *NOT* mean that Oracle itself runs faster, just that we've given it more CPU cycles to use.

(2) When, because of the high potential concurrency of requests, significant resource contention exists. Use of a TP Monitor can limit the degree of concurrency and thus reduce contention.

TP Monitors can not provide better performance:

(1) For existing applications. The applications must be designed to fit the TP Monitor architecture.

(2) For applications which are highly interactive in their use of the database. These applications put many messages through the transport system, and the TP Monitor is not as efficient as SQL*Net for point-to-point communication.

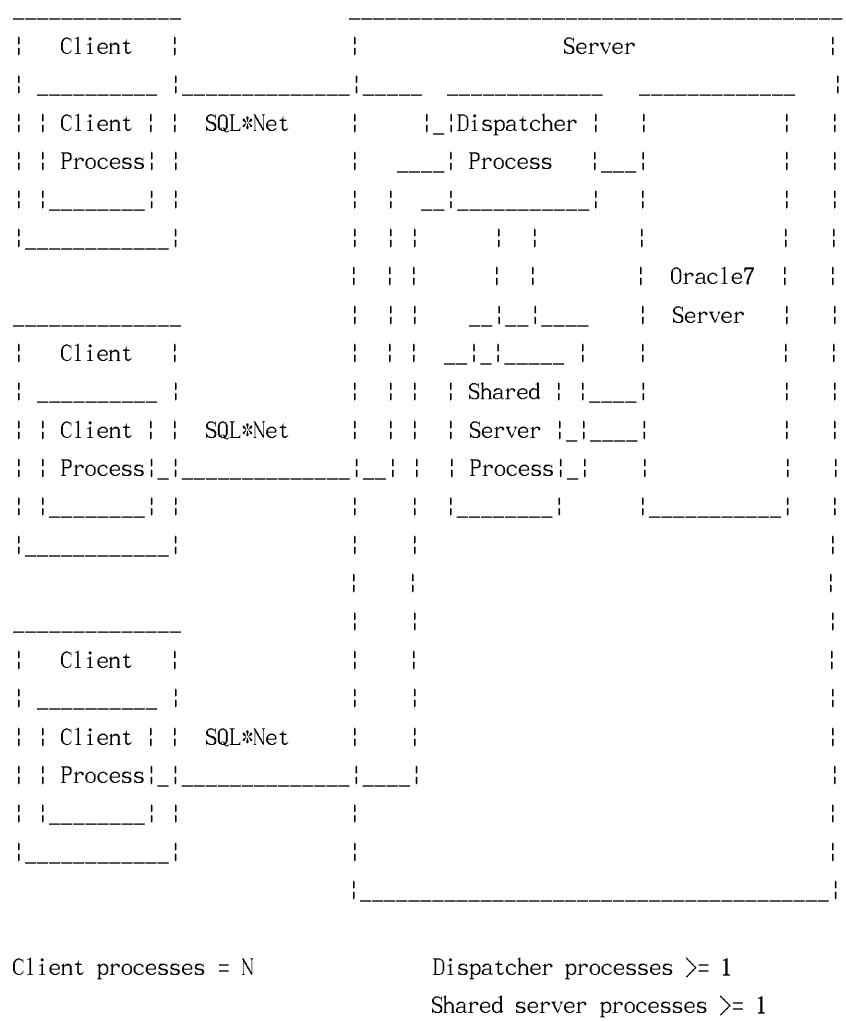
(3) For CPU intensive single-query decision support. When executing a single large command, Oracle query facilities work efficiently, especially with the use of Oracle Parallel Query, available in 7.1.

How does the three-tier solution help TPC-A, or other situations with thousands of on-line users?

The TPC-A test calls for a large number of users to produce a given result. In the high-end results we produced in June, 1992, for example, 6150 terminals were simulated to produce 618 TPC-A transactions. Thus, terminal concentration accounts for a large portion of the total processing time used.

First, let's look at how the Multi Threaded Server would work for this benchmark. I this case, there are many client processes, but only a few server processes, which handle client requests on a first-come first serve basis. When they are done with a request, they take another client's request.

ORACLE7 CLIENT/SERVER ARCHITECTURE WITH MULTI THREADED SERVER



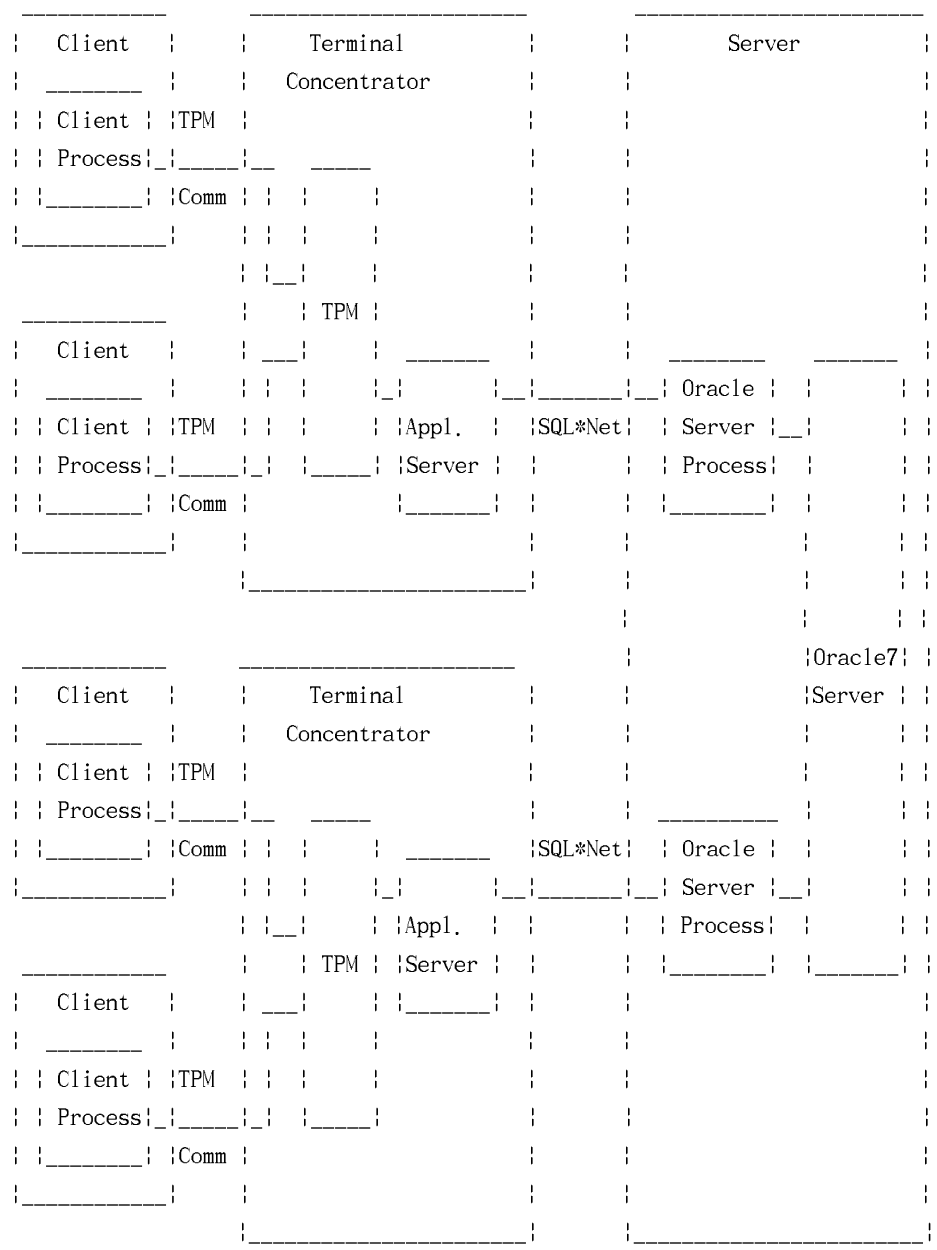
If there are 500 clients in this environment, there will be one or more dispatcher processes, dynamically tunable, and one or more shared server processes, dynamically tunable, on the server. The reduction in the total number of processes handled by the server system results in more processing time available for RDBMS activity. Thus higher RDBMS transaction throughput can be obtained on the server system.

But the problem for the TPC-A, and for certain large customer configurations, is not the only ability of the Oracle Server to process transactions, but also the ability of the operating system to handle huge numbers of incoming connections. There is one incoming connection for each client. Most UNIX

operating systems have a limit on how many such connections they can handle. Even if a particular operating system allows a large number of connections, each takes some amount of overhead to manage.

In order to service all 6150 terminals, we selected a 3-tier hardware environment where the middle tier, using a TPM, acted as a terminal concentrator. The high-end TPC-A architecture looked like the following. The Application Servers, which contain the Pro*C statements used to perform the transaction also run on the terminal concentrator machine in order to offload as much work from the database serve as possible. They send the compiled SQL over SQL*Net to the Oracle7 Server processes.

ORACLE7 TPS-A CLIENT/SERVER ARCHITECTURE



Clients = 6150	Terminal concentrators = 17	
	TP Monitor instances = 17	
	Application server processes = 17*8	Oracle Server processes = 17*8

The TPM is the software component of the terminal concentrator. In this role it offloads terminal handling from the the machine running Oracle Server. Since more than one terminal concentrator can be configured, whereas the database in this case had to run on a single machine, concentrator machines can be added until the performance of the back-end machine was optimized. This three-tier solution resulted in the outstanding transaction throughput announced with Oracle7 Server. Even with Oracle Parallel Server, it may pay to offload the terminal handling so that the cluster can be exclusively used for database operations.

Can you summarize the performance discussion for me?

Depending on the number of users required, different architectures may be used in a client/server environment to maximize performance:

1) For a small number of users, the traditional Oracle two-task architecture can be used. In this case, there is a one-to-one correspondence between client processes and server processes. It's simple, straightforward, and efficient.

2) For a large number of users, Multi Threaded Server might be a better approach. Although some tuning may be required, Multi Threaded Server can handle a relatively large number of users for each machine size compared to the traditional Oracle approach. Using this approach, customers will be able to handle many hundreds of users on many platforms. Furthermore, current Oracle applications can move to this environment without change.

3) For a very large number of users, where transactions are simple and terminal input concentration is the overriding performance issue, a 3-tier architecture incorporating a TPM may be useful. In this case, eterminal concentration is handled by the TPM in the middle tier. As you might expect, it is a more complex environment requiring more system management. For existing Oracle customers, significant Oracle application modifications will be required.

Oracle provides all of these choices.

Performance using Oracle's XA Library

==========

Are there any performance implications to using the XA library (in other words, to using TPM-managed transactions)?

(1) The XA library imposes some performance penalty. You should use TPM-managed transactions only if you actually need them. Even if you are getting the one-phase commit optimization, the code path is longer because we need to map back and forth between external formats and

internal ones. Also, prior to 7.1, XA requires you to release all cursors at the end of a transaction, which results in extra parsing. Even with shared cursors, there is time spent looking up the one you need and re-validating it. This has been improved for 7.1.

(2) If you need to use two-phase commit, this will incur additional cost since extra I/Os are required. If you do need 2PC, you need to account for that when sizing the application.

(3) Although some TPMs allow parallel execution of services (such as Tuxedo's "tpacall"), this will not normally enhance performance unless different resource managers are being used. In fact, Oracle Server must serialize accesses to the same transaction by the same Oracle instance, and the block/resume code will in fact degrade performance in that case compared to running the services sequentially.

RECOVERY 의 모든것
(DATABASE ,TABLESPACE, DATAFILE)

Bulletin no : 10326

---

(3) DATABASE RECOVERY
==========

. Complete,Incomplete recovery 가 가능

. Offline recovery 만 가능하다 (즉 database가 recovery 시 open 되어 있지 않아야 함.)
. SYSTEM tablesapce 의 recovery 도 가능하다.

. recovery 시 database access가 불가능하고 복구시간은 적용되어야 하는 redo 의 양에 dependent 하다.

Complete recovery

1 % cp backup_file oracle_file_directory

2 SQLDBA) startup restrict mount

3 SQLDBA) alter database rename file 'old' to 'new'

4 Recovery를 위한 모든 datafile 이 online 인지 check 한다.
  SQLDBA) select file#,status,name from V$DATAFILE;
  SQLDBA) alter database datafile 'filename' online;

5 RECOVER DATABASE ;

6 alter database open;

Incomplete recovery

(Incomplete recovery 시 datafile 이 offline 되어 있으면 복구가 불가능하다)

5 RECOVER DATABASE UNTIL TIME '1996-10-10:10:37:00';

6 ALTER DATABASE OPEN RESETLOGS;

(2) TABLESPACE RECOVERY
==========

RECOVER TABLESPACE tablespace_name [,tablespace_name..]

. 이 방법은 tablespace list 에 등록된 모든 data file 의 media recovery 를 위한 command 이다.
Oracle 은 tablespace 가 갖는 datafile 이 어떤것인가를 아는 것은 반드시 database 가 open 되어있을 경우이므로 이 tablespace recovery command는 database 가 open되었을때 비로소 가능하다.

. command 를 실행하기 위해 반드시 tablespace는 offline 되어 있어야 한다.

. offline 되어지지 않는 (예를 들면 SYSTEM) tablespace는 recovery 가 불가능하다.

. incomplete recovery 는 불가능하다 .

복구 순서
=========
1 SQLDBA) alter tablespace ts_name offline;

2 recovery 가 필요한 tablesapce의 datafile을 restore 한다.

3 lost 되었거나 손상된 datafile 이 invalid 일경우는 control file 에 datafile 의 위치가 변경되어야 한다.
SQLDBA) alter database rename file 'old' to 'new';

4 SQLDBA) RECOVER TABLESPACE ts_name;

5 alter tablespace ts_name online;

(3) DATAFILE RECOVERY
==========

RECOVERY DATAFILE 'filename' ['filename]

. media recovery lock 만 datafile 에 얻을 수 있다면 online,offline recovery 가 모두 가능하다.

. On line recovery시 (database 가 임의의 instance 에 의해 open 되어 있다면) datafile recovery는 offline 되어있는 datafile 만 recovery 가 가능하다.

.Incomplete recovery 는 불가능하다.

복구순서

========

+++Online Recovery

1 SQLDBA) startup mount

2 SQLDBA) alter database datafile 'filename' offline;

3 SQLDBA) alter database open

4 SQLDBA) alter database rename 'old' to 'new'

5 SQLDBA) RECOVER DATAFILE 'datafile'

6 SQLDBA) alter database datafile 'filename' online

+++Offline Recovery

1 SQLDBA) alter databse rename 'old_filename' to 'new'

2 SQLDBA) startup restrict mount

3 SQLDBA) alter database datafile 'filename' online
SQLDBA) alter database datafile 'filename' offline
양쪽 모두 가능하다.

4 SQLDBA) RECOVER DATAFILE 'datafile'

5 SQLDBA) alter databse datafile 'filename' online;

6 SQLDBA) alter database open

REDO LOGFILE SIZE의 변경 또는 추가

Bulletin no : 10292

---

* Oracle에서 사용중인 REDO Logfile은 V7에서는 Default로 Size가 512K인 3개의 REDO Logfile이 생긴다.

DB를 이용하는 Application의 특성에 따라 REDO Logfile의 수를 늘리거나 REDO Logfile의Size를 크게 할 필요가 있다. REDO Logfile의 Size를 변경 하거나 수를 늘리는 것은 아래와 같은 방법을 이용한다.

- REDO Logfile의 Size 변경

(1) 먼저 ONLINE중인 DB는 Shutdown후에 Startup Mount로 기동 시킨다.

```
SQLDBA> startup mount
ORACLE instance started.
Database mounted.
```

(2) V$Log와 V$Logfile을 이용해 느 것이 Current Log인지 또한 각 Logfile의 Datafile명인지를 확인한다.

```
SQLDBA> select * from v$log;

GROUP#  THREAD#  SEQUENCE#  BYTES   MEMBERS  ARC STATUS   FIRST_CHAN FIRST_TIME
------  -------  ---------  ------  -------- --- -------- ---------- -----------------
     1        1         55  512000        1  NO   CURRENT     65081  2/06/95 19:32:57
     2        1         53  512000        1  NO  INACTIVE     64111  1/21/95 11:06:14
     3        1         54  512000        1  NO  INACTIVE      64801  1/21/95 11:41:44
3 rows selected.

SQLDBA> select * from v$logfile;

  GROUP# STATUS    MEMBER
-------- -------  ------------------------------------------------
      1             /usr4/ora713/dbs/log1ORA713.dbf
      2             /usr4/ora713/dbs/log2ORA713.dbf
      3   STALE    /usr4/ora713/dbs/log3ORA713.dbf
3 rows selected.
```

(3) V$Logfile에서 확인된 Inactive Logfile을 먼저 아래의 Command를 이용해 Logfile을 Drop한다. 정상적으로 Drop되면 Host Command를 이용해 Datafile을 Remove한다.

```
SQLDBA> alter database drop logfile
```

```
          '/usr4/ora713/dbs/log2ORA713.dbf';
Statement processed.

SQLDBA> !rm /usr4/ora713/dbs/log2ORA713.dbf
```

(4) Oracle V7에서는 Default로 3개의 REDO Logfile이 생기는데 최소한 2개 이상의 REDO Logfile이 필요하다. 즉 위의 (3)과 같이 2번째 Logfile을 Drop 하였으므로 Logfile은2개만 남아 있는데 1개를 또 Drop하고자 하니 아래와 같이 Ora-1567 & Ora-1532 Error가 발생한다. 이때는 다음과 같이 Logfile의 Size를 크게하여 이미 Drop된 2번 Logfile을 Add한 후 Drop하여야 한다.

```
SQLDBA>  alter database drop logfile
          '/usr4/ora713/dbs/log3ORA713.dbf';
ORA-01567:dropping log 3 would leave less than 2 log files in
            thread 1
ORA-00312:online log 3 thread 1:'/usr4/ora713/dbs/log3ORA713.dbf'
```

(5) Drop한 Logfile을 추가한 후에 Drop 한다. Logfile Add시 여기에서는 3m로 정의하여 크기를 조정하였다.

```
SQLDBA> alter database add logfile
          '/usr4/ora713/dbs/log2ORA713.dbf' size 3M;
Statement processed.

SQLDBA> alter database drop logfile
          '/usr4/ora713/dbs/log3ORA713.dbf';
Statement processed.

SQLDBA> !rm /usr4/ora713/dbs/log3ORA713.dbf
```

(6) (5)에서 Drop된 3번 Logfile을 Add한다.

```
SQLDBA> alter database add logfile
          '/usr4/ora713/dbs/log3ORA713.dbf' size 3M;
Statement processed.
```

(7) V$Log에서 Current Logfile은 바로 Drop할 수 없다. 이를 Drop하기 위해서는 현재의 Current Logfile을 Inactive로 하여야 한다. Inactive로 하려면 'Alter System Switch Logfile' Command를 이용하여야 하며 이는 DB가 Open되어 있어야 한다.

```
SQLDBA> alter system switch logfile;
ORA-01109: database not open

SQLDBA> alter database open;
Statement processed.

SQLDBA> alter system switch logfile;
Statement processed.
```

(8) 위의 (7)번 작업으로 Current Logfile을 Inactive한 상태로 하였으며 이를 Drop하기 위하여DB를 다시 Alter Database Close를 이용해 DB를 Close 한다. Close후에는 (5)번, (6)번 작업과 같이 Logfile을 Drop하고 Datafile의 Remove후 Logfile의 Size를 변경하여 Add한다.

```
SQLDBA> alter database close;
Statement processed.

SQLDBA> alter database drop logfile
          '/usr4/ora713/dbs/log1ORA713.dbf';
Statement processed.

SQLDBA> !rm /usr4/ora713/dbs/log1ORA713.dbf

SQLDBA> alter database add  logfile
          '/usr4/ora713/dbs/log1ORA713.dbf' size 3M;
Statement processed.
SQLDBA> alter  database add  logfile
          '/usr4/ora713/dbs/log4ORA713.dbf' size 3M;
Statement processed.
```

필요한 Logfile의 Size를 모두 변경하고 나면 DB를 Normal Shutdown후 Startup하여 사용하면 된다.

(9) (1)에서 (8) 까지는 기존의 Logfile을 Drop후 새로이 만드는 것에 관한 설명이며 이외 기존에 3개 Logfile을 4개로 변경 할 경우에는 아래와 같이 Logfile을 필요한 갯수만큼 Add하면된다.

```
SQLDBA> alter  database add  logfile
          '/usr4/ora713/dbs/log4ORA713.dbf' size 3M;
Statement processed.
```

ROLLBACK SEGMENT 관리

Bulletin no : 10089

롤백 세그먼트는 다른 세그먼트처럼 익스텐트로 구성되어 있다. 롤백 세그먼트는 이들 익스텐트를 Circular 하게 이용한다. 하나의 트랜잭션은 롤백 세그먼트의 Current Pointer 위치에 레코드를 기록하고 Current Pointer를 레코드 크기만큼 증가시킨다. 롤백 세그먼트에 레코드가 기록되는 현재 위치를 Head 라고 부르기로 한다. 그리고 롤백 세그먼트 상에서 가장 오래된 트랜잭션 레코드의 시작 위치를 Tail 이라고 부르기로 한다.

트랜잭션들이 필요로 할 때 마다 롤백 세그먼트를 사용할 수 있도록 적절한 갯수의 롤백 세그먼트를 유지해야 하며 각각의 롤백 세그먼트는 트랜잭션 부하를 감당하도록 충분한 크기를 가져야한다.

〈롤백 세그먼트의 갯수와 크기〉

롤백 세그먼트가 사용되는 방법은 다음과 같다.

1. 하나의 트랜잭션은 하나의 롤백 세그먼트 만을 사용한다.
2. 롤백 세그먼트의 Head는 Tail 에 의해서 사용되는 익스텐트를 침범하지 는다.
3. 링 형으로 구성된 롤백 세그먼트 안에 더 이상 할당할 익스텐트가 없으면 새로운 익스텐트가 할당되어 링 안에 포함된다.
   따라서 롤백 세그먼트의 크기를 적절히 유지하는 것은 매우 중요하다.
   롤백 세그먼트의 적절한 크기는 데이타베이스에서 수행되는 트랜잭션의 활동 상태에 따라서 달라진다. 필요한 롤백 세그먼트의 갯수는 현재의 롤백 세그먼트 사용 상태를 모니터 해서 결정한다.

롤백 세그먼트의 적절한 크기와 갯수를 결정하는 것은 다음 테스트과정을거쳐서 결정한다.

1. 롤백 세그먼트를 저장할 테이블 스페이스 생성
2. 테스트할 갯수의 롤백 세그먼트를 1)에서 만든 테이블 스페이스에 생성
3. 롤백 세그먼트의 익스텐트 크기는 모두 동일하게 설정. 스텐트 크기는 세그먼트가 최대로 확장되었을 때 10~30 개 정도가 되도록 한다.
4. 롤백 세그먼트의 Minextent 는 2 이상.
5. 테스트할 롤백 세그먼트의 Status 를 In Use 상태로 변경하여 활성화 시킨다
6. 애플리케이션을 실행한다
7. 롤백 세그먼트 Contention을 측정한다.
8. 롤백 세그먼트의 최대 확장상태를 측정한다.

테스트 과정에서 가장 큰 상태가 될 때의 롤백 세그먼트 크기를 Minimum Coverage Size 라고 하자. 만약 Contention 이 발생하면 롤백 세그먼트의 갯수를 늘려가면서 테스트를 반 한다.

<익스텐트의 갯수와 크기>

롤백 세그먼트의 익스텐트 크기는 모두 같고 롤백 세그먼트가 포함된 테이블 스페이스의 크기는 익스텐트 크기의 정수배인 것이 바람직하다. 롤백 세그먼트의 일반적인 크기는 20개 정도의 익스텐트가 적합하다.

<오라클7 에서의 기능>

오라클7에서는 롤백 세그먼트의 pctincrease 는 항상 0이다. optimal 값을지정하면 오라클은 롤백 세그먼트를 항상 이 크기 이하로 유지하려고 하게 된다. 이와같은 Shrinking은 롤백 세그먼트의 Head 가 하나의 익스텐트에서 다음 익스텐트로 어갈 때 일어난다. 오라클은 가장 오래된 비활성 상태의 익스텐트 부터 제거하여 Shrink 한다.

롤백 세그먼트 모니터에서 롤백 세그먼트에 할당되었던 최대 익스텐트와 Optimal, Shrink 횟수 등이 나타나는데 여기서 적절한 Optimal 값을 결정 할 수 있다.

오라클7 에서는 동적으로 롤백 세그먼트의 Online/Offline이 가능하다. 롤백 세그먼트가 처음 생성되었을 때는 Offline 상태이므로 이것을 Online 상태로 만들려면 Alter Rollback Segment 명령을 Online 옵션으로 사용하거나 init.ora 화일에 해당 롤백 세그먼트를 등록하고 데이타베이스를 Restartup 하면 된다. 반대로 Alter Rollback Segment 명령을 이용하여 Offline 시킬 수도 있다. Offline 된 롤백 세그먼트를 Online 시키려면 Alter Rollback Segment 를 이용하거나 init.ora 의 rollback_segments 에 지정을 고 데이타베이스를 Restartup 해야 한다.

<롤백 테이블 스페이스의 디자인>

트랜잭션에 의해서 발생되는 롤백 정보의 크기는 다음의 스크립트를 이용하여 추정할 수 있다.

```
set  feedback off
set termout  off
column name format A40
define undo_overhead=54
DROP TABLE undo$begin;
DROP TABLE undo$end;
CREATE TABLE undo$begin ( writes number );
CREATE TABLE undo$end ( writes number );
INSERT INTO undo$begin
     SELECT sum(writes) FROM v$rollstat;
set termout on
set feedback on
UPDATE test1 SET col1=99 WHERE col3 ='Y';

set termout off
set feedback off
INSERT INTO undo$send
     SELECT sum(writes) FROM v$rollstat;
set termout on
```

```
set  feedback on
SELECT  ( ( e.writes - b.writes) - &undo_overhead)
" number of bytes generates"
     FROM undo$begin b, undo$end e;
set termout off
set feedback off
DROP TABLE undo$begin;
DROP TABLE undo$end;
```

위의 스크립트가 출력하는 결과는 트랜잭션 동안 발생하는 UNDO 이다.
따라서 데이타베이스에 다른 트랜잭션이 없다면 이 값은 정확한 값을 보여준다.

아래의 스크 트는 활성화된 트랜잭션을 실행하는 유저와 각 트랜잭션이 사용중인 롤백 세그먼트를 보여준다.

```
SELECT r.name "ROLLBACK SEGMENT NAME ",
               l.pid "ORACLE PID",
               s.pid "SYSTEM PID ",
               NVL ( p.username , 'NO TRANSACTION'),
               p.terminal
FROM v$lock l, v$process p, v$rollname r
WHERE  l.pid = p.pid(+)
AND TRUNC (l.id1(+)/65536) = r.usn
AND l.type(+) = 'TX'
AND l.lmode(+) = 6
ORDER BY r.Name
```

여러가지 경우에 대해서 알아보자.

1. 보통의 트 잭션인 경우

롤백 테이블 스페이스의 크기는 앞에서 결정한 롤백 세그먼트들이 Minimun Coverage Size인 경우와 거의 같은 크기로 설정한다. 롤백 세그먼트를 생성 할 때 필요한 크기를 모두 할당 받도록 한다. 따라서 롤백 세그먼트를 만들고 나면 테이블 스페이스에 더 이상의 빈공간은 남지 않게 된다.

2. 매우 크고 자주 발생하는 트랜잭션인 경우

이 경우에는 다른 롤백 세그먼트들보다 큰 롤백 세그먼트를 하나 만들어서 큰 트랜잭션이 사용하도록 한다. 예를 들어 가장 큰 트랜잭션의 크기가 45M 이고 minimum coverage size 가 10M 라면 큰 롤백 세그먼트의 크기는 55M 가 된다. 다른 롤백 세그먼트들은 Minimum Coverage Size 로 생성된다. 큰 롤백 세그먼트를 사용하고자 할 때에는 Set Transaction Use Rollback Segment 를 사용하여 지정한다. 만약 큰 트랜잭션이 여러개가 동시에 발생하면 큰 롤백 세그먼트도 여러개가 필요하다.

3. 매우 크지만 자주 발생하지는 않는 트랜잭션인 경우

이 경우에는 큰 트랜잭션을 위한 큰 롤백 세그먼트를 하나 만들되 필요한 경우에만 만들어서 사용하고 사용이 끝나면 삭제하는 방법을 사용한다. 아니면 아예 별도의 롤백 테이블 스페이스와 큰 롤백 세그먼트를 만들어서 작업중에 사용하고 작업이 끝나면 삭제하여도 된다.

실제로는 위와 3가지 경우가 복합되어 발생하므로 상황에 따른 적절한 대응이 필요하다.

〈오라클7〉

오라클7에서는 여러가 기능이 추가되어서 롤백 세그먼트의 설계가 달라진다. Optimal 값을 설정하면 좋은 점도 있지만 그 값이 너무 작다면 익스텐트의 할당과 Deallocation 이 빈번히 발생하여 퍼포먼스의 저하를 가져올 수도 있다. 그리고 언제 롤백 세그먼트가 Optimal 크기로 돌아 지를 예측할 수 없다는 점도 문제이다.

1. 보통의 트랜잭션인 경우

V6 의 경우와 같다. 단지 테이블 스페이스에 여유공간을 둘 경우 Optimal 을 설정하여 초기에 설정한 값을 유지하도록 한다. Optimal 값은 Minimum Coverage Size 보다 작게 주면 퍼포먼스가 저하되므로 유의하도록 한다.

2. 매우 크고 자주 발생하는 트랜잭션인 경우

여기서 자주 발생한다는 것은 모든 롤백 세그먼트가 optimal 크기로 돌아가기 전에 또다시 큰 트랜잭션이 발생한다는 의미이다. (큰 트랜잭션이란 이것이 발생시키는 롤백 정보를 담을 롤백 세그먼트를 만들 수 없는 것을 말한다. 이 경우에는 Optimal 은 해결방법이 되지 못한다. 해결방법으로는 우선 롤백 세그먼트의 갯수를 줄여서 각각의 롤백 세그먼트가 충분히 커질 수 있도록 하는 것이다. 하지만 이렇게 하면 Contention 이 일어 서 퍼포먼스가 저하될 수 있다. 다른 방법은 큰 롤백 세그먼트를 만들어서 Set Transaction Use Rollback Segment 명령으로 해당 트랜잭션이 이것을 지정하여 사용하도록 하는 방법이다. 하지만 이 방법은 애플리케이션에서 롤백 세그먼트를 지정하는 명령을 사용할 수 있어야만 사용 가능하다.

3. 매우 크지만 자주 발생하지 않는 트랜잭션인 경우

롤백 테이블 스페이스를 롤백 세그먼트들의 Minimum Coverage Size를 큰 트랜잭션의 정보를 담을 만한 크기로 만든다. 그리고 모든 롤백 세그먼트의 Optimal 을 Minimum Coverage Size 로 설정한다. 그렇게 하면 큰 트랜잭션이 어느 롤백 세그먼트에 할당되어도 완전한 처리가 가능해진다.

다음의 큰 트랜잭션이 들어올 때 쯤이면 optimal 설정에 의해서 필요한 공간을 항상 보유하고 있게 된다. 하지만 optimal 설정에 따른 약간 퍼포먼스 저하가 발생할 수도 있다.

ROLLBACK SEGMENT 크기를 증가하여 재생성 하는 방법

Bulletin no : 10072

ROLLBACK SEGMENT 는 LONG TRANSACTION 을 수행하면 크게되는 경우가 있다. 확장된 ROLLBACK SEGMENT 는 줄어들지 않게되므로(OPTIMAL을 지정하지 않는 경우) 주기적으로 재생성 하여 TABLESPACE 의 FREESPACE 영역을 확보한다.

1. SQLDBA 기동한다.

```
$ sqldba lmode=y
sqldba > connect internal
```

2. ROLLBACK SEGMENT 를 OFFLINE 시킨다. ROLLBACK SEGMENT 를 OFFLINE 시키기위해서는 그 ROLLBACK SEGMENT 가 INACTIVE상태여야 하므로 모든 TRANSACTION 을 종료 하고 ROLLBACK SEGMENT r01-r04를 OFFLINE 시켜야 한다.

```
sqldba > alter rollback segment r01 offline;
sqldba > alter rollback segment r02 offline;
      ** r03 ~ r04 도 같은 방법으로 OFFLINE 함.
```

3. ROLLBACK SEGMENT 를 DROP 한다.

```
SQLDBA > drop rollback segment r01;
SQLDBA > drop rollback segment r02;
      ** r03 ~ r04 도 같은 방법으로 DROP 함.
```

4. ROLLBACK SEGMENT 를 생성한다. SYSTEM ROLLBACK 이외에 모든 ROLLBACK SEGMENT 를 DROP 한경우는 SYSTEM TABLESPACE 에 ROLLBACK SEGMENT 를 한개 이상 생성후에 NON-SYSTEM TABLESPACE 에 ROLLBACK SEGMENT 를 생성해야한다.

```
SQLDBA > create rollback segment r0;
SQLDBA > alter rollback segment r0 online;

SQLDBA > create rollback segment r01  tablespace rbs
         storage(initial 512k next 512k minextents 2);
SQLDBA > create rollback segment r02  tablespace rbs
         storage(initial 512k next 512k minextents 2);
      ** r03 ~ r04 도 같은 방법으로 생성 함.
```

5. 데이타베이스를 재 동 한다.

```
SQLDBA > shutdown
SQLDBA > startup
```

RBS TABLESPACE 를 재생성 후 ROLLBACK SEGMENT 를 생성하는 방법

---

6. RBS TABLESPACE를 DROP 하기 위해서는 1-2 과 같이 ROLLBACK SEGMENT를 전부 OFFLINE를 한다.
   ROLLBACK SEGMENT 를 OFFLINE할수 있는 다른 방법은
   ORACLE_HOME/dbs/init.ora 화일의 rollback_segments=r01..
   부분을 #으로 COMMENT 처리하고 DB를 다시 startup 시키면 된다.

7. RBS TABLESPACE를 DROP 한다.

```
SQLDBA> drop tablespace RBS including contents;
SQLDBA> ! rm /oracle/dbs/rbs.dbf
```

8. RBS TABLESPACE 를 다시 생성한다.

```
SQLDBA> create tablespace  RBS datafile '/../oracle/dbs/rbsORA7.dbf'
        size 50M default storage(initial 512k next 512k);
```

9. ROLLBACK SEGMENT r0, r01-r04 를 생성 한다.

```
SQLDBA> create rollback segment r0;
SQLDBA> alter rollback segment r0 online;

SQLDBA> create rollback segment r01 tablespace RBS;
      ** r02 - r04 도 같은 방법으로 생성한다.
```

10. 1 에서 COMMENT 처리했던 부분을 다음과 같이 풀고 DB 를 다시 startup 한다.

```
rollback_segments=r01,r02,r03,r04

SQLDBA> shutdown
SQLDBA> startup
```

ORA-1562 조치 방법

RBS TABLESPACE 영역이 부족한경우 8 번 과 같이 TABLESPACE를 더큰 크기로 다시 생성 할수 있지만 DATAFILE을 추가하여 영역을 확장하는 방법이 있다.

11. RBS TABLESPACE 영역 확장

```
SQLDBA> alter tablespace RBS add datafile '/oracle/dbs/rbs2ORA7.dbf'
        size 50M;
```

ROLLBACK SEGMENT의 상태 해결

Bulletin no : 10084

---

* 롤백세그먼트의 정보를 보여주는 dba_rollback_segs를 보면 Status 컬럼이 있다. 일반적으로 이 컬럼은 Online/Offline 값을 갖지만 롤백 세그먼트가 손상된 경우에는 Needs Recovery 값을 갖게 된다.

* Needs Recovery 상태의 롤백 세그먼트는 Online/Offline 으로 변경하거나 init.Ora 화일의 Rollback_segments 라인에서 제거하는 것만으로는 문제가 해결되지 않는다. Needs Recovery 상태가 생기는 이유는 롤백 과정에서 어떤 이유에서든 문제가 생겨서 실패하기 때문이다. 롤백은 사용자가 Rollback 명령을 내린 경우와 Shutdown Abort 후 Startup 시에 자동적으로 수행되는 Instance Recovery 시에 일어난다.

* 롤백이 일어나면 수정된 데이타 블럭에 기록된 결과는 취소되어야 하지만 해당 데이타 블럭에 접근이 불가능하여 롤백 작업을 하지 못하면 롤백 세그먼트의 상태가 Needs Recovery 상태로 되는 것이다.

* 데이타 블럭에 접근이 불가능한 경우로는 다음과 같은 예가 있다.

1. 테이블 스페이스나 데이타 화일이 Offline 이거나 없는 경우
2. 해당 데이타 블럭이 속한 Object가 손상된 경우
3. 롤백 세그먼트의 데이타 블럭이 손상된 경우

* 이 문제를 해결하는 방법은 다음과 같다.

1. 모든 테이블 스페이스와 데이타 화일이 online 상태인지를 확인한다.
이것은 DBA_DATA_FILES 와 DBA_TABLESPACES 의 STATUS 컬럼을 조회하면 알 수 있다.

2. 다음 문장을 init.ora 화일에 추가한다.

```
event = "10015 trance name context forever, level 10"
```

위의 Event는 롤백하는 트랜잭션과 Object에 관한 정보를 담은 Trace File을 생성시켜 준다.

3. 손상된 롤백 세그먼트를 init.ora 의 Rollback_segments 라인에서 삭제한다.

4. 데이타베이스를 Shutdown (가능한 Normal, Immediate 를 사용하고, 안될 경우 Abort 를 사용하고 다시 Startup 한다.

5. USER_DEMP_DEST에 지정된 디렉토리에 Trace 화일이 생성되었는지 확인한다.
6. Trace 화일을 보면 Error Recovery tx(#,#) Object # 와 같은 메세지가 있을 것이다. 여기서 tx(#,#)은 트랜잭션의 정보를 가리키고 Object # 는 DBA_OBJECTS 의 OBJECT_ID 와 같다.

7. 다음의 Query를 이용하여 어떠한 Object에 대해서 Recovery 가 수행되는 가를 확인한다.

```
select owner, object_name, object_type, status
from dba_objects
where object_id=;
```

8. 출력된 Object를 Drop 한다.

9. init.ora 에서 삭제했던 롤백 세그먼트를 다시 Rollback_segments 라인에 넣고 Event를 삭제한 다음 데이타베이스를 Shutdown 하고 Startup 한다.

ROW CHAINING 과 ROW MIGRATION

Bulletin no : 10381

Row chaining 은 단일 테이블 상의 특정 Row의 길이가 증가해서 더 이상 동일한 데이타 블럭에 들어갈수 없을때 발생한다. 이때 RDBMS는 또 다른 데이타 블럭을 찾는다. 이 데이타 블럭은 원래 블럭 연결되어 있다. 이 경우 데이타 블럭이 하나의 I/O 작업과 동일한 양을 수행하기 위해 두 개의 I/O를 사용해야 한다는 점이다. 이 상황은 여러분의 데이터베이스 성능을 빠르게 약화시킬 것이다.

Data Block상의 하나의 Row는 길이가 증가하면서 갱신되며, Block의 Freespace가 0%일 때, Row는 Migration을 일으킨다. 물론, 전체 Row가 들어갈 만한 크기의 새로운 Block에 Row에 대한 Data가 Migration된다.

이경우 ORACLE은 Row에 대한 정보를 가져오기 위해 하나 이상의 Data Block을 반드시 읽어야 하므로 I/O Perfmance는 감소한다.

<해결안>

1) ANALYZE Command를 통해 Chaining과 Migrating의 횟수를 조사한다.

```
analyze table  emp list chained rows;
```

1. 데이터 열을 CHAINED_ROWS 테이블의 ROWID를 사용하여 원래 테이블과 같은 행 구조를 가진 중간 테이블(intermediate table)로 이동시킨다.
2. 옮겨진 데이터 열을 CHAINED_ROWS 테이블의 ROWID를 사용하여 삭제 한다.
3. 중간 테이블로부터 열들을 다시 원래 테이블로 삽입한다.
4. 중간 테이블을 버린다.
5. CHAINED_ROWS 테이블의 레코드를 삭제한다.

이 과정이 수행되고 나면 analyze 명령은 다시 수행되야 한다. row가 다시 CHAINED_ROWS 테이블에 쌓이면 어떤 블럭에도 전체row 가 들어갈 충분한 공간이 없기 때문이다. 이것은 한 데이타 블럭의 한 row 가 너무 길어서 이거나 테이블의 PCTFREE 가 적절하지 못하기 때문이다. 전자의 경우는 chaine 현상이 일어날수 밖에 없고 후자의 경우 다음과 같이 PCTFREE 를 수정한다.

1. 테이블에 대한 더 나은 퍼센트 프리 요소(percent free factor)를 결정한다.
2. 전체 테이블을 그 모든 의존관계(예를 들면, 인덱스, 그렌트(grants), 약조건들 등)와 함께 export한다.
3. 원래 테이블을 버린다.
4. 새로운 사양으로 다시 만든다.
5. 테이블을 import한다.

ROWID에 대한 이해

Bulletin no : 10308

Fast and Dangerous

The Misunderstood Oracle ROWID By Bert Scalzo, Ph.D.

Oracle ROWID는 중요하고 powerful한 SQL programming tool이다.
그러나 언제, 어떻게 안전하게 사용되어야 하는지에 대해 다소 잘못 이해된 점들이 있다.

정상적으로 올바른 환경에서의 ROWID의 사용은 SQL이나 PL/SQL CODE의 실행을 보다 빠르게 하여 준다.
그러나 주의할 것은 ROWID는 당신의 PROGRAM을 특별하게 동작하게하거나 아니면, 전혀 엉뚱한 동작을 하게 만들수 있다는 것이다.
그러므로 ROWID의 사용은 특별한 주의를 기울여야 한다.

Just What Is It Exactly ?

Oracle ROWID는 무엇인가 ? Oracle table의 모든 Row는 ROWID라고 하는 물리적인 Address로 assign된다.
Non-clustered table 에 대해서는 이러한 address는 unique하다.
Clustered table에 대해서는 서로다른 table에서 추출된 row들은 동일한 data block내에 존재하며, 정확히 같은 ROWID를 가지게 될것이다.

누군가가 ROWID를 사용하여 한 table안의 서로 다른 column에 대하여 설계 하고, 제한하는 기능들을 사용한다하더라도 이것은 pseudo-column이다.Oracle의 pseudo column은 table의 실제적인 column은 아니므로 table의describe을 통해서는 보이지 않지만 매우 유용하게 사용된다.

Pseudo column의 다른 예는 CURRVAL, NEXTVAL (sequence number generator)와 LEVEL (for hierarchical queries), ROWNUM (a row's relative number) 같은 것들이 있다.

내부적으로 ROWID는 binary 값을 가진다. 그러나 외부적으로 Display 될때에는 VARCHAR2 type의 column에 저장된 3 part의 hexdecimal string으로 표현된다.

첫번째 부분(position 1 through 8) 은 block ID를 나타낸다.
두번째 부분(position 10 through 13) 은 block 내부에서 row의 sequence number를 나타낸다.
세번째 부분(position 15 through 18) 은 data file ID를 나타낸다.

SUBSTR function을 사용하여 ROWID를 구분하여 SELECT하는 구분을 보면,

```
SELECT ROWID
        SUBSTR(ROWID,  15,  4)  AS  "FILE",
        SUBSTR(ROWID,    1,  8)  AS  "BLOCK",
        SUBSTR(ROWID,  10,  4)  AS  "ROW"
FROM    dual;
```

위의 SELECT 구문은 다음과 같은 ROWID에 대하여 아래의 결과를 출력할 것이다.

```
ROWIDFILEBLOCKROW
------------------------------------------
0000033D.0000.000100010000033D0000

ROWID           0000033D.0000.0001
FILE            0001
BLOCK           0000033D
ROW             0000
```

위의 예에서 보듯이 ROW의 값은 BLOCK 내에서의 첫번째 ROW이다.(BLOCK 내에서의 첫번째 ROW의 OFFSET은 ZERO이기 때문.) 더 나아가서 이 ROW는 한 DATA FILE내의 829 BLOCK 내에 위치하고 있다. 이것이 이 ROW의 PHYSICAL ADDRESS 이다.

It's Fast!

ROWID를 사용하여 table의 row를 access하는 것은 빠르다. 사실상 rule-based optimizer에서는 ROWID를 이용하여 access하는 것이 가장 상위로 rank 된 access 방법이다.

Oracle Application Developers Guide에 따르면 access path 선택의 rule은 Figure 1에 기술된 순서와 같다.

RANK	ACCESS PATH
1	Single row by ROWID
2	Single row by cluster join
3	Single row by hash cluster key with unique or primary key
4	Single row by unique or primary key
5	Cluster join
6	Hash cluster key
7	Indexed cluster key
8	Composite key
9	Single-column indexes
10	Bounded range search on indexed columns
11	Unbounded range search on indexed columns
12	Sort-merge join
13	MAX or MIN of indexed column

14	ORDER BY on indexed columns
15	Full table scan

Figure 1 : The rules for access path selection.

보신 바와 같이 가장 첫번째의 (빠른) access 방법은 ROWID를 사용하는 것이다.
이러한 access 방법은 아래의 사항이 만족되어 질때에만 사용가능하다.

1. WHERE 절에서 ROWID로 row를 선택할수 있도록 기술되거나

2. CURRENT OF CURSOR 절로 기술되어야 한다.

위의 사항에 해당한다면 Oracle Server는 해당하는 physical address를 가지고 data row를 access 할 것이다. 결과는 가장 빠르게 생성될 것이다. 그러나 빠른 access 가 가능한 만큼 허점도 가지고 있는데, ROWID가 변화할 수 있다는 것이 그것이다.

Ch... Ch... Changes

row의 address는 서로 다른 환경하에서는 변화할수 있다. 예를들면, import 직후의 export는 일반적으로 서로 다른 ROWID를 생성한다. 그러 이 예는 전혀 해가 없는 예이다. 왜냐하면 ROWID가 user의 transaction의 영역 밖에서 일어났기 때문이며, 이러한 변화는 전혀 고려할 만한 사항이 아니다.

반면에 어떤 user가 DML 작업을 하는 도중에 ROWID가 변화 하였다고 생각해보자.
만약 user의 program이 전체적인 범위에서 ROWID의 consistence가 유지되어야 한다고 가정해보면, 이것은 정말 큰 문제가 될 것이다.

ROWID는 단지 transaction의 length 동안만 consistence의 유지가 보장된다.
한 transaction은 connect 되는 순간부터 commit이나 rollback을 만나거나, 또 다른 transaction 이 시작되는 시간까지 유지된다. 이러한 이유 때문에 user program 내부에 하나 이상의 transaction이 존재한다면, ROWID 가 변화할수도 있다.

An Example

이러한 문제의 예로는 Figure 2. 에서 보이는 transfer_all procedure를 들 수 있다.

```
PROCEDURE transfer_all
        (commit_after_in IN INTEGER := 100,
         requery_after_in IN INTEGER := 1000)
IS
        more_data BOOLEAN := TRUE;
        commit_count INTEGER := 0;

        CURSOR trans_cur
        IS
                SELECT ROWID FROM transfer_table
                WHERE transfer_indictor = 'U'
```

```
                AND ROWNUM <= requery_after_in;

BEGIN
        WHILE more_data
        LOOP
                more_data := FALSE;
                FOR trans_rec IN trans_cur
                LOOP
                        more_data := TRUE;

                        INSERT INTO master_table
                        SELECT  *
                        FROM     transfer_table
                        WHERE ROWID = trans_rec.ROWID;

                        UPDATE transfer_table
                        SET        transfer_indicator = 'T'
                        WHERE   ROWID= trans_rec.ROWID;

                        commit_count := commit_count + 1;
                        IF commit_count = commit_after_in
                        THEN
                                COMMIT;
                                commit_count := 0;
                        END IF;
                END LOOP;
        END LOOP;
END;
```

위의 Procedure는 몇가지 제한조건을 가지고 ROWID를 select 하여 TRANS_CUR 라는 CURSOR를 생성한다. 이 Cursor로 Return 되는 Row는 For Loop 안에서 사용된다. 이 Loop 내에서는 current cursor record의 Rowid를 reference 하는 INSERT 문장과 UPDATE DML command 가 사용되고 있다. 또한 이 cursor For loop 안에서는 argument로 넘겨받은 interval 변수에 따라 Commit 문이 실행된다. Commit 문이 실행되면, cursor에 의해 사용되던 ROWID는 변화할수 있게 된다.

이러한 상황이 single-user의 환경에서 발생하는 것이라면 아무런 문제가 없으나, Multi-user의 concurrent transaction 환경이라면 많은 문제가 발생할것이다.

. 다른 user 가 당신이 사용하고 있는 ROWID를 invalidate 하게 하는 UPDATE나 INSERT, DELETE 작업을 수행할수도 있다.

. 또 한가지는 INSERT나 length를 줄이는 UPDATE 작업 등에서 내부적으로 automatic 하게 ROWID 를 변경할수도 있다.

이러한 문제는 잘 드러나지 않을 뿐만 아니라 Error를 detecct 하기도 힘들뿐만 아니라, 거의 Debug 가 불가능한다.

Conclusion

Rowid는 엄격한 제한조건을 가지고 주의깊게 사용되어야 한다. 저자의 생각으로는 작업되는 row에 locking을 하지 않고는 ROWID를 사용하는 sqlstatement를 사용하여서는 안된다.

Oracle 7 Server Concept Manual에서 언급된 사항은 아래와 같다.
" DML 문장에서 사용되는 ROWID는 그것이 변화되지 않을 것이라는 사실이 검증되고, 보장되어야 한다. 이렇게 변화하지 않기 위해서는 사용되는 Row는 locked되어야 한다."

Oracle Performance Tuning [Corrigan and Gurry, O'Reilly & Associates, 1993]에서는 아래와 같이 언급하고 있다.
" 당신이 Record를 Query 하거나, update하기위하여 query를 수행할 때, 선택된 record가 다른 process로 부터 보존되고, record의 rowid는 당신의 환경 밖에서 변화되어야 한다."

Transfer_all procedure의 경우 error를 fix 하기 위하여 2가지가 변화 (또는 추가)되어야 한다.

첫번째로는 cursor의 선언 부분에

" FOR UPDATE OF ROWID " 가 추가 어야 하며,

두번째로는 FOR loop의 밖으로 COMMIT statement를 옮겨주어야 한다.

이러한 변화가 program을 다소 non-flexible하게 만드는 것처럼 보일지도 모르지만, 확실하게 program 을 reliable 하게 만들어 준다.

IMPLEMENTING DATA REPLICATION

Bulletin no : 10375

Here's how to move beyond the basics to implement an update-anywhere symmetric-replication system, including a simple macro you can download.

Read this article as it originally appeared in the July/August 1996 issue of Oracle Magazine.

Listing 1: A simple Microsoft Word macro you can use to build an Oracle symmetric-replication environment (reprinted with permission from Animated Learning).

```
Dim Shared surrogateid$
 REM USERID FOR SURROGATE SYS USER
Dim Shared surrogatepassword$
 REM PASSWORD FOR SURROGATE SYS USER
Dim Shared repadminid$
 REM USERID FOR REPADMIN USER
Dim Shared repadminpassword$
 REM PASSWORD FOR REPADMIN USER
Dim Shared schemanum
 REM NUMBER OF REPLICATED SCHEMAS
Dim Shared schemaids$(2)
 REM ARRAY OF REPLICATED SCHEMA IDS
Dim Shared schemapasswords$(2)
 REM ARRAY OF REPLICATED SCHEMA PASSWORDS
Dim Shared masterdefname$
 REM MASTER DEF SITE NAME
Dim Shared masterdefsyspassword$
 REM MASTER DEF SITE SYS PASSWORD
Dim Shared mastername$
 REM MASTER SITE NAME
Dim Shared mastersyspassword$
 REM MASTER SITE SYS PASSWORD
Dim Shared snapnum
 REM NUMBER OF SNAPSHOT SITES
Dim Shared snapnames$(9)
 REM ARRAY OF SNAPSHOT SITE NAMES
Dim Shared snapsyspasswords$(9)
 REM ARRAY OF SNAPSHOT SITE SYS PASSWORDS
Dim Shared snapmasters$(9)
 REM ARRAY OF SNAPSHOT MASTER SITES
```

```
Sub MAIN

REM BUILDS A SQL COMMAND SCRIPT YOU CAN USE TO GENERATE AN
REM ORACLE SYMMETRIC REPLICATION ENVIRONMENT. THE SCRIPT
REM IS OUTPUT TO THE CURRENT OPEN MICROSOFT WORD DOCUMENT.

REM GET MASTER DEFINITION DATABASE NAME
masterdefname$ = InputBox$(\
 "Enter the master definition site name " + \
 "(e.g ., SALES.ACME.COM):")

REM GET SYS PASSWORD FOR MASTER DEFINITION SITE
masterdefsyspassword$ = InputBox$(\
 "Enter SYS password as master definition site:")

REM GET INFORMATION ABOUT MASTER SITE
button1 = MsgBox("Create one other master site?", 292)
If button1 = - 1 Then
 mastername$ = InputBox$(\
  "Enter the master site name (e.g ., SALES.ACME.COM):")
 mastersyspassword$ = InputBox$(\
  "Enter corresponding SYS password for master site:")
End If

REM GET INFORMATION ABOUT SNAPSHOT SITES
button2 = MsgBox("Create snapshot sites?", 292)
If button2 = - 1 Then
 snapnum = Val(InputBox$(\
  "Enter the number of snapshot sites to create " + \
  "(max = 10):"))
 For count = 0 To snapnum - 1
  snapnames$(count) = InputBox$(\
   "Enter the snapshot site name (e.g., SALES.ACME.COM):")
  snapsyspasswords$(count) = InputBox$(\
   "Enter corresponding SYS password for snapshot site:")
  snapmasters$(count) = InputBox$(\
   "Enter corresponding master site of snapshot site " + \
   "(e.g ., SALES.ACME .COM):",, masterdefname$)
 Next
End If

REM GET SURROGATE SYS USER ID
surrogateid$ = InputBox$(\
 "Enter a name for the surrogate SYS account at " + \
 "all master sites:",, "remotesys")
```

```
REM GET SURROGATE SYS PASSWORD
surrogatepassword$ = InputBox$(\
 "Enter a password for the surrogate SYS accounts:")

REM GET REPLICATION ADMINISTRATOR NAME
repadminid$ = InputBox$(\
 "Enter a name for the replication administrator " + \
 "account:",, "repadmin")

REM GET REPLICATION ADMINISTRATOR PASSWORD
repadminpassword$ = InputBox$(\
 "Enter a password for the replication " + \
 "administrator account:")

REM PROMPT FOR NUMBER OF REPLICATED SCHEMAS
schemanum = Val(InputBox$(\
 "How many replicated schemas do you want to create " + \
 "(maximum = 3)?:"))

REM GET REPLICATED SCHEMA INFORMATION
For count = 0 To schemanum - 1
 schemaids$(count) = InputBox$("Enter replicated " + \
  "schema name:")
 schemapasswords$(count) = InputBox$(\
  "Enter corresponding replicated schema password:")
Next

REM BUILD A CORRESPONDING SQL COMMAND SCRIPT IN CURRENT
REM OPEN DOCUMENT

REM CREATE ACCOUNTS, SCHEMAS, AND LINKS AT MASTER
REM DEFINITION SITE
Insert "REM MASTER DEFINITION SITE BUILD"
InsertPara
CreateAccounts(masterdefname$, masterdefsyspassword$)
CreateSchemas(masterdefname$, masterdefsyspassword$)
If button1 = - 1 Then
 CreateLinks(masterdefname$, mastername$, 1)
End If

REM CREATE ACCOUNTS, SCHEMAS, AND LINKS AT MASTER SITE
If button1 = - 1 Then
 Insert "REM MASTER SITE BUILD"
 InsertPara
 CreateAccounts(mastername$, mastersyspassword$)
 CreateSchemas(mastername$, mastersyspassword$)
```

```
 CreateLinks(mastername$, masterdefname$, 1)
End If

REM CREATE ACCOUNTS, SCHEMAS, AND LINKS AT SNAPSHOT SITES
If button2 = - 1 Then
 Insert "REM SNAPSHOT SITE BUILD(S)"
 InsertPara
 For count = 0 To snapnum - 1
  CreateSchemas(snapnames$(count), \
   snapsyspasswords$(count))
  CreateLinks(snapnames$(count), snapmasters$(count), 0)
 Next
End If
End Sub

Sub CreateAccounts(site$, syspassword$)
REM SUBROUTINE TO CREATE ACCOUNTS AT GIVEN SITE
Insert "REM Create accounts at master site " + site$
InsertPara
Insert "REM Connect as SYS"
InsertPara
Insert "CONNECT sys/" + syspassword$ + "@" + site$ + ";"
InsertPara
InsertPara
Insert "REM Create surrogate SYS account"
InsertPara
Insert "CREATE USER " + surrogateid$ + \
 " IDENTIFIED BY " + surrogatepassword$ + ";"
InsertPara
Insert "EXECUTE dbms_repcat_auth.grant_surrogate_repcat" + \
 "('"   + surrogateid$ + "');"
InsertPara
InsertPara
Insert "REM Create replication administration account"
InsertPara
Insert "CREATE USER " + repadminid$ + " IDENTIFIED BY " + \
 repadminpassword$ + ";"
InsertPara
Insert "EXECUTE dbms_repcat_admin.grant_admin_any_repschema" + \
 "(userid=>'" + repadminid$ + "');"
InsertPara
InsertPara
End Sub

Sub CreateSchemas(site$, syspassword$)
REM SUBROUTINE TO CREATE REPLICATED SCHEMAS AT GIVEN SITE
```

```
Insert "REM Create replicated schemas at " + site$
InsertPara
Insert "REM Connect as SYS"
InsertPara
Insert "CONNECT sys/" + syspassword$ + "@" + site$ + ";"
InsertPara
InsertPara
For count = 0 To schemanum - 1
 Insert "CREATE USER " + schemaids$(count) + \
   " IDENTIFIED BY " + schemapasswords$(count) + ";"
 InsertPara
 Insert "EXECUTE dbms_repcat_admin.grant_admin_repschema" + \
  "(userid =>'" + schemaids$(count) + "');"
 InsertPara
 Insert "GRANT EXECUTE ON dbms_defer to " + \
  schemaids$(count) + ";"
 InsertPara
Next
InsertPara
End Sub

Sub CreateLinks(site1$, site2$, master)
REM SUBROUTINE TO CREATE NECESSARY DATABASE LINKS FROM
REM SITE1 TO SITE2

If master = 1 Then
 Insert "REM Create master-to-master database links at " + \
  site1$
 InsertPara
 Insert "REM Create link for surrogate SYS account"
 InsertPara
 Insert "CONNECT " + surrogateid$ + "/" + \
  surrogatepassword$ + "@" + site1$ + ";"
 InsertPara
 Insert "CREATE DATABASE LINK " + site2$
 InsertPara
 Insert " CONNECT TO " + surrogateid$ + " IDENTIFIED BY " + \
  surrogatepassword$ + ";"
 InsertPara
 InsertPara
 Insert "REM Create link for replication " + \
    "administrator account"
 InsertPara
 Insert "CONNECT " + repadminid$ + "/" + repadminpassword$ \
  + "@" + site1$ + ";"
 InsertPara
```

```
 Insert "CREATE DATABASE LINK " + site2$
 InsertPara
 Insert " CONNECT TO " + repadminid$ + " IDENTIFIED BY " + \
  repadminpassword$ + ";"
 InsertPara
 InsertPara
End If
For count = 0 To schemanum - 1
 Insert "REM Create link for " + schemaids$(count)
 InsertPara
 Insert "CONNECT " + schemaids$(count) + "/" + \
  schemapasswords$(count) + "@" + site1$ + ";"
 InsertPara
 Insert "CREATE DATABASE LINK " + site2$
 InsertPara
 Insert " CONNECT TO " + schemaids$(count) + \
  " IDENTIFIED BY " + schemapasswords$(count) + ";"
 InsertPara
 InsertPara
Next
End Sub
```

Steve Bobrowski is president of Animated Learning (http://www.animatedlearning.com), a software company that develops multimedia courseware to teach people about client/server technology. Read this article as it originally appeared in the July/August 1996 issue of Oracle Magazine.

SHARED MEMORY 문제

Bulletin no : 10305

---

1. 왜 Problem 이 생기나?

Oracle 은 Process와 SGA(System Global Area) 간의 Communication를 위해 Shared Memory 와 Semaphore 를 사용한다.

Oracle Instance 가 뜰 때 SGA를 Create 하기 위해 Main Memory 의 임의의 부분을 할당하는데 이때 Shared Memory Semaphore 가 적절하지 않으면 이에 관련한 Error가 발생한다.

2. 해결 안

SGA는 Shared Memory 안에 생기므로 Shared Memory 는 각 Process에게 사용 가능해야 한다.
Shared memory 와 Semaphore parameter 는

- SHMMAX = 1개의 shared memory segment 의 maximum size
- SHMMIN = 1개의 shared memory segment 의 minimum size
- SHMMNI = shared memory identifier 의 숫자
- SHMSEG = 1개의 process에 attach되는 shared memory segment의 maximum개수
- SEMMNS = system의 semaphore 개수
- SEMMNI = 시스템에서 identifier 를 setting하는 semaphore 수

* 추천하는 Semaphore와 Shared Memory Parameter

Operating System	Shared Memory Parameters	Semaphore
Sun OS		
	SHMSIZE = 32768	SEMMNS = 200
	SHMMNI = 50	SEMMNI=50
Solaris		
	SHMMAX = 8388608	SEMMNS = 200
	SHMSEG = 20	SEMMSL = 50
	SHMMNI = 100	SEMMNI = 70
HP/UX		
	SHMMAX = 0x4000000(64Mb)	SEMMNS = 128
	SHMSEG = 12	SEMMNI = 10
Digital Unix (DEC Alpha OSF/1)		
	SHMMAX = 4194304	SEMMNS = 60
	SHMSEG = 32	SEMMSL = 25
Ultrix		
	Use System Default	SEMNS
		SEMMSL = 5

AT&T Unix

	SHMMAX = RAM-Dependant	SEMMNS = 200
8 or 16Mb RAM		
	SHMMAX = 5 Mb	For All RAM
32 Mb RAM		
	SHMMAX = 8 Mb	Values
64 Mb RAM		
	SHMMAX = 16 Mb	
128 Mb RAM		
	SHMMAX = 32 Mb	
256 Mb RAM		
	SHMMAX = 64 Mb	
512 Mb RAM		
	SHMMAX = 128 Mb	
1024 Mb RAM		
	SHMMAX = 256 Mb	
2048 Mb RAM		
	SHMMAX = 512 Mb	
	SHMSEG = 6 for all RAM	Values
	SHMMIN = 1 for all RAM	Values
Dynix/PTX		
	SHMMAX = 11010048	SEMMNS = 200
	SHMSEG = 20	SEMMSL = 85
Other Parameter	NOFILES = 128	
DG/UX		
	SHMMAX = 4194304	SEMMNS = 200
	SHMSEG = 15	

Shared Memory 와 Semaphore Parameter 는 OS 의 Kernel Configuration File 에 반드시 지정되어야 하며,File 의 위치는 OS마다 차이가 있다.
현재의 Shared Memory 와 Semaphore Configuration 을 알기 위해서는 다음의 Command를 이용한다.

```
$ sysdef |more
```

* HP-UX (relevant sections only) 에서의 예:

```
     Semaphore 관련 Parameters
- maximum value for semaphores(semaem)= 16384
- Semaphore  map(semmap)= 4098
- number of semaphore identifiers(semmni) = 4096
- total number of semaphores in the system(semmns) = 8192
- number of semaphore undo structures(semmnu) = 1536
- semaphore undo entries per process(semume) = 512
- semaphore maximum value(semvmx) = 32767
```

Shared Memory 관련 Parameters

- maximum shared memory segment size in bytes(shmmax) = 536870912
- minimum shared memory segment size in bytes(shmmin) = 1
- maximum shared memory segments in system (shmmni) = 512
- maximum shared memory segments per process(shmseg) = 512

NOTE: SHMMAX는 현 system에 8개의instance가 수행될 수 있는 충분한 값이다.

* Shared memory 또는 semaphore parameters 를 변경하기 위해서는 ...

1. Oracle Instance를 Shut Down 한다.
2. OS의 Kernel Configuration File이 있는 곳으로 간다.
3. System Utility 또는 Editor를 이용해서 필요한 값을 바꾼다.
   System Utility는 다음과 같다.

OS	Utility
HP/UX	SAM
SCO	SYSADMSH
AIX	SMIT
Solaris	ADMINTOOL

4. Kernel 을 Reconfigure 한다.
5. System을 Reboot 한다.
6. Oracle Instances 를 Startup시킨다.

[예제] Solaris 2.3/2.4 parameters and commands:

1. SQLDBA 에서 :

```
SQLDBA> shutdown
SQLDBA> exit
```

2. Superuser(root)로 login 하고 :

```
# cd /etc
```

3. /etc/system file 에 다음을 추가 한다:

```
set shmsys:shminfo_shmmax=8388608
set shmsys:shminfo_shmmin=1
set shmsys:shminfo_shmmni=100
set shmsys:shminfo_shmseg=20
set semsys:seminfo_semmns=200
set semsys:seminfo_semmni=70
```

4. Kernel을 reconfigure 한다.
```
# touch /reconfigure
```

5. Machine 을 reboot 한다.
```
#init 6
```

6. SQLDBA 에서 :
```
SQLDBA> startup
SQLDBA> exit
```

Oracle의 init.ora Parameters 에는 SGA 의 영향을 주는 Parameters가 있다. OS의 Shared Momory와 Semaphore Parameter에 연결된 이 Parameter의 Setting은 System과 Oracle의 Performance에 영향을 미친다.

SHARED POOL SIZE의 계산

Bulletin no : 10393

오라클 instance를 background process + SGA라고 말할때 처음 instance가 기동 되면 SGA가 할당된다.이때 이 SGA를 구성하는 메모리 영역은 크게 3부분으로 구성된다고 할수있다.- Shared pool, Redo log buffer, DB buffer cache.
여기서는 SGA를 구성하는 shared pool의 size를 시스템에 맞게 산출하는 방법에 대해서 알아본다.

Shared pool영역 구성은 MTS방식으로 접속된 session의 PGA,Shared SQL area 그리고 dynamic하게 할당되는 data structure로 구성된다.

1. Shared Pool

Shared pool은 ?/dbs/initSID.ora(parameter file)의 SHARED_POOL_SIZE라는parameter로 그 크기를 지정하는데 default값은 3.5M를 갖게 된다.일반적으로 shared pool이 얼마나 사용되는가 하는 문제는 application dependent하므로 각 application을 조사하는 게 필요하다.시스템에서 필요로 하는 크기를 검사하기 위해 아주 큰 크기로 parameter file에 지정하여 dynamic하게 할당되는 SGA가 충분히 큰 값을 갖게 한 후,검사가 끝난 다음 아래에서 계산된 size로 변경 해 주도록 한다.

2. 계산 공식

```
   Session당 최대메모리사용량(Max Session Memory)
*  동시 접속하는 User의
+  Shared SQL 영역으로 사용되는 메모리양
+  Shared PLSQL을 위해 사용하는 메모리 영역
+  최소 30%의 여유 공간

=  Minimum Shared Pool
```

3. 계산 예제

(1) 적당한 user session에 대한 session id를 찾 다.

```
SQLDBA> select sid from v$process p, v$session s
        2   where p.addr=s.paddr and s.username='SCOTT';

       SID
----------
        29
1 rows selected.
```

(2) 이 session id에 대한 maximum session memory를 찾는다.

```
SQLDBA> select value from v$sesstat s, v$statname n
        2   where s.statistic# = n.statistic#
        3      and n.name = 'session uga memory max' and sid=29;
  VALUE
----------
    273877
1 rows selected.
```

(3) Total shared SQL area를 구한다.

```
SQLDBA>  select sum(sharable_mem) from v$sqlarea;

SUM(SHARAB
------------------
          8936625
1 row selected.
```

(4) PLSQL sharable memory area를 구한다.

```
SQLDBA>  select sum(sharable_mem) from v$db_object_cache;

SUM(SHARAB
------------------
          4823537
1 row selected.
```

(5) Shared pool size를 계산한다.

```
      274K shared memory  *  400 users
+       9M Shared SQL Area
+       5M PLSQL Sharable Memory
+      60M Free Space (30%)

=     184M Shared Pool
```

이 예제에서는 Shared pool의 size는 184M가 적당하다고 할수있다.

4. Shared Memory부족 (ORA 4031)에 대한 대처

Ora-04031이 발생하는 원인은 2가지 이다.첫째 절대적으로 shared pool size가 작아서 나는 경우와, 둘째로 memory는 있으나 적재하려 하는 PL/SQL package가 너무커서 연속된 shared pool영역을 점유 하지 못하는 경우가 있다.만일 첫번의 경우 라면 적당한 계산 과정으로 계산하여 parameter file에서 SHARED_POOL_SIZE를 늘려주고 ,두 번째 경우라면 다음과 같은 방법으로 에러를 피해 갈 수 있다.- "Sys.dbms_shared_pool.keep" procedure사용.

[참고] 위 package를 사용하려면 ?/rdbms/admin/dbmspool.sql,prvtpool.plb를 수행시켜 package를 create시킨 후 사용한다.(자세한 사항은 bulletin 10095,Oracle7 Server Tuning 4장12를 참조한다.)

SNAPSHOT 생성방법

Bulletin no : 10201

OPERATION GUIDE FOR CREATE SNAPSHOT
==========

SQLDBA> shutdown

. initSID.ora file 에 다음의 3 parameter 를 setting 합니다.
SNAPSHOT_REFRESH_PROCESSES= 1
SNAPSHOT_REFRESH_INTERVAL=60
SNAPSHOT_REFRESH_KEEP_CONNECTION=FALSE

SQLDBA>startup

1.run script file(default :run script file)

~/rdbms/admin/catsnap.sql

2.run script file(for using stored pkg)
~/rdbms/admin/dbmssnap.sql

global_name=false

3.SQL> connect scott/tiger
SQL> create database link pu_lk
connect to scott identified by tiger
using 't:pusan:ORATJ';

4.SQLDBA> connect sys/manager;
SQLDBA> grant create snapshot to scott;
SQLDBA> connect scott/tiger;

5.SQL> connect scott/tiger
SQL> create snapshot emp_pu
pctfree 10 pctused 60
refresh fast
start with sysdate
next sysdate +1
as select * from emp@pu_lk

SQL> desc snap$_emp_pu

1 SQL>connect scott/tiger
SQL> create snapshot log
SQL> desc mlog$_emp

Manual operation test
==========

SQL> connect scott/tiger
SQL> update emp set ...
SQL> commit;

2 SQLDBA> connect internal;
SQLDBA> execute dbms_snapshot.refresh('scott.emp_pu','f');
SQLDBA> selct * from scott.snap$_emp_pu;